ZENVA

Develop a 3D Platformer with Godot

A PRACTICAL GUIDE - 1ST EDITION

Daniel Buckley

Pablo Farias

academy.zenva.com
youtube.com/c/zenva
linkedin.com/company/zenva

Table of Contents

Introduction

Welcome to the world of Godot! If you have ever wanted to bring your own digital worlds to life, you are in the right place. Game development is one of the most rewarding creative pursuits available today: it combines logic, art, sound, and design into a single interactive experience. Whether you are looking to build a career in the industry or simply want to create games as a hobby, this book is designed to be your companion on that journey.

Godot has rapidly become a favorite engine for developers of all skill levels. It is lightweight, open-source, and incredibly powerful. In this book, we will focus on practical application rather than dry theory. We want to get you making games as quickly as possible.

Over the course of this book, we will walk through the entire process of building a complete 3D platformer game from scratch. Starting with the absolute basics of the interface, we will move step by step toward complex systems like 3D environments, enemy artificial intelligence, and polished user interfaces. By the end, you will have a finished, playable game and the confidence to take your own ideas and turn them into reality.

How to use this Book

To get the most out of this material, we recommend an active approach. Game development is a skill best learned by doing.

Here are a few tips for success:

- **Follow along actively.** Do not just read the code; type it out. The act of typing helps muscle memory and forces you to pay attention to syntax and structure.
- **Experiment often.** Once you finish a section, try changing a few values. Make the player faster, change the color of the environment, or tweak the gravity. Breaking things is often the fastest way to learn how they work.
- **Use the reference material.** If you get stuck or your code does not work as expected, do not panic. The final chapter contains the full source code for reference. You can compare your work against it to find typos or missing steps.

We have structured this book to be a progressive guide. Each chapter builds upon the concepts introduced in the previous one. While it might be tempting to skip ahead to the “cool stuff”, we encourage you to move sequentially to ensure you have a solid foundation.

The blank canvas awaits. Let’s turn the page and set up our development environment.

Set up and Installation

Welcome to this guide on developing a 3D platformer game using the Godot engine. Together, we will explore various aspects of game development in Godot, including level setup, creating coins, enemies, a player controller, and more. By the end of this book, you will have a fully functional 3D platformer game.

Prerequisites

To ensure a smooth learning experience, a basic understanding of Godot and GDScript is recommended. You should be familiar with the following fundamental concepts:

- Navigating the Godot editor (Scene dock, Inspector, FileSystem dock)
- How nodes and scenes work (parent-child relationships, instancing)
- GDScript fundamentals (variables, types, functions, conditionals, `for` loops)
- Basic vector math and using `delta` for frame-independent movement
- Working with 3D models, materials, and cameras at a beginner level

If some of these concepts are new to you, we recommend the following resources to get up to speed:

- **Learn Godot From Scratch** (https://academy.zenva.com/product/intro-to-godot-ebook/)
- **Mini-Projects with Godot** (https://academy.zenva.com/product/godot-mini-projects-ebook/)

Alternatively, if you prefer video courses:

- **Intro to Godot 4 Game Development** (https://academy.zenva.com/product/intro-to-godot-4-game-development/)
- **Godot 4 Mini-Projects** (https://academy.zenva.com/product/godot-4-mini-projects/)

That said, do not worry if you are not fully proficient yet. We will walk through each step carefully as we work in the editor and write code together.

Project Files

You can download the assets required for this project, as well as the completed project files for reference, using the links below:

- **Asset Files**: (https://academy.zenva.com/wp-content/uploads/2026/02/Assets-Godot-3D-Platformer.zip)
- **Completed Project Files**: (https://academy.zenva.com/wp-content/uploads/2026/02/Complete-Project-Godot-3D-Platformer-v4.6.zip)

Third Party Asset Licenses

Some of the assets provided in this course are subject to third-party licenses:

Assets by Kenney

Models and HUD Heart

URL: https://kenney.nl/

License: Creative Commons CC0 (https://creativecommons.org/publicdomain/zero/1.0/)

Zenva-Created Assets

All other artwork and media assets included in the project files that are not listed above were created by **Zenva** and are released under:

Creative Commons CC0 (Public Domain)
https://creativecommons.org/publicdomain/zero/1.0/

Project Overview

Our 3D platformer project will consist of several key components that work together to create a complete gameplay loop.

The Player

The player is the core of the experience. We will create a scene that can be moved around the level and jump using keyboard inputs (arrow keys, WASD, etc.). To achieve this, we will:

- Set up input maps to handle control schemes.
- Implement movement logic and physics.
- Add procedural animations, such as bobbing up and down and rotating to face the direction of movement, to give the character life.

Enemies

To provide a challenge, we will introduce enemies represented by moving buzz saws. These will be designed to be modular and customizable:

- We will create a system to set their movement speed, direction, and travel distance.
- Upon contact, enemies will deal damage to the player.
- We will implement feedback systems, including sound effects and screen shake, to make impacts feel impactful.

Coins

Coins will serve as the primary collectible item. They encourage exploration and provide a score metric.

- Coins will disappear when collected, increasing the player's score and playing a sound effect.

- We will add visual flair by making them bob and rotate, ensuring they catch the player's eye.

End Flag

The end flag acts as the goal for each level. When the player reaches this object, they will be teleported to the next level, allowing for a seamless transition between different stages of the game.

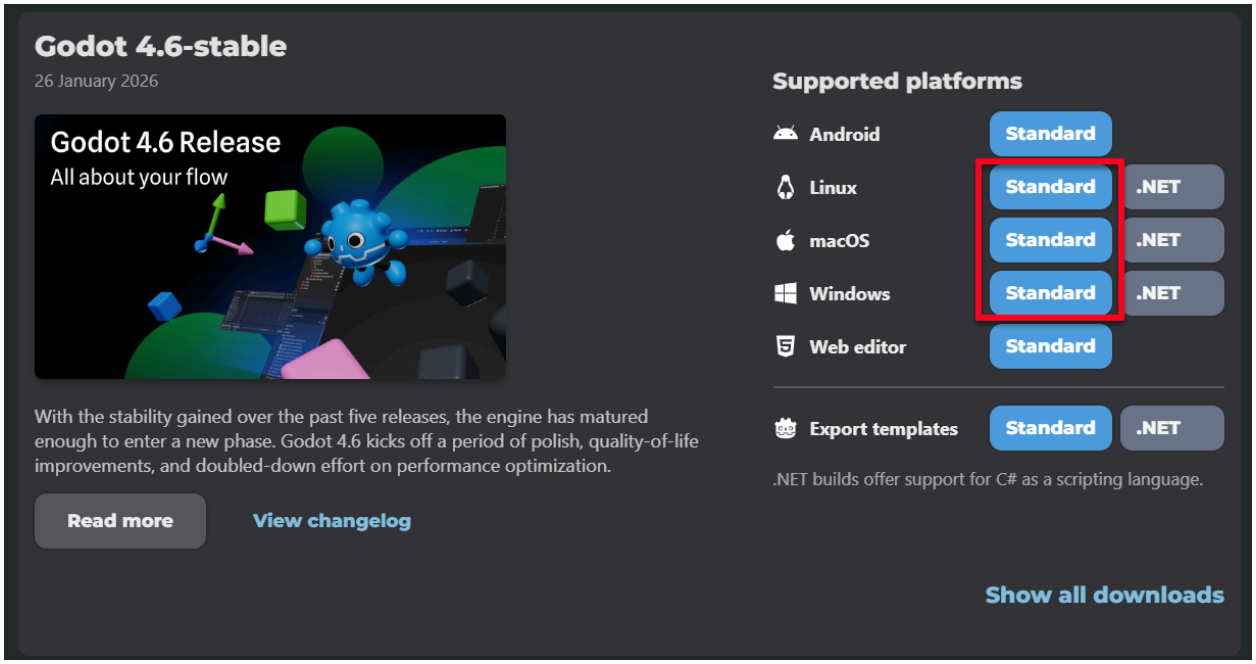
Godot Installation and Version

For this project, we will be using **Godot version 4.6**. This is the version used to create the course materials, and using it ensures that all code and assets work exactly as expected.

Please make sure **NOT** to use newer versions of the software than what we recommend, as changes in the engine could cause compatibility issues with the provided code.

How to Install Version 4.6

You can download the 4.6 version of the engine by visiting the official Godot download archive (https://godotengine.org/download/archive/4.6-stable/). On this page, select the Standard option that matches your operating system. Note that instructions provided in this ebook are intended for desktops only.



Once the file is downloaded, unzip it. Godot does not require a traditional installation process; simply double-click the application file to launch the engine.

Game Design and Project Setup

Before we dive into the practical aspects of creating the game, it is essential to outline the game's design, mechanics, and key elements. This chapter will walk you through the fundamental building blocks required to develop your 3D platformer game.

Game Overview

The game is a 3D platformer where the player's primary objectives are to collect coins and avoid enemies. The ultimate goal of each level is to reach the end flag, which teleports the player to the next level.

The image below illustrates the core gameplay elements we will build, including the player, platforms, collectibles, and the goal.

Player Character

The player character is the heart of the experience. We will implement several systems to make movement feel responsive and engaging:

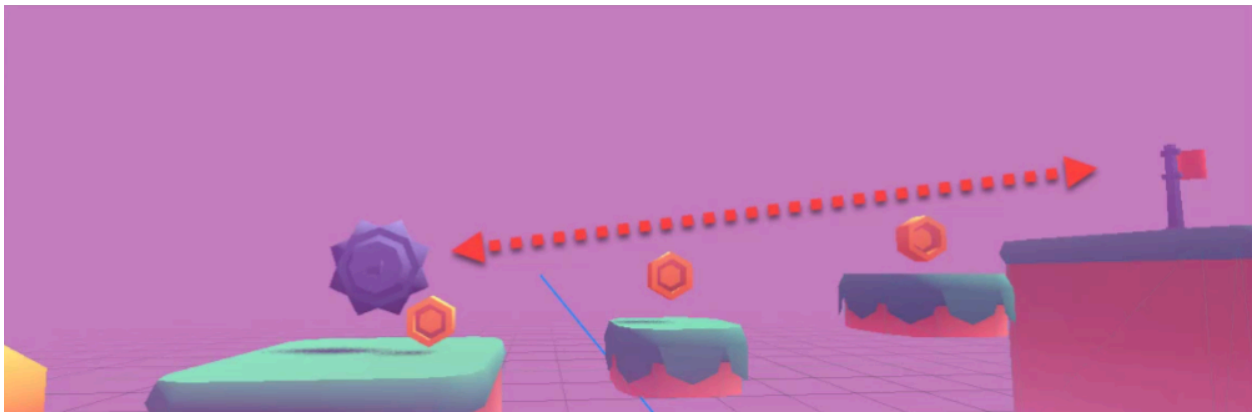
- **Movement:** The player can move using the keyboard and jump. We will define these inputs using Godot's input map system.

- **Visual Polish:** To simulate motion, the character will rotate to face the direction of movement and bob up and down.
- **Health System:** The player has a health variable that decreases when hit by enemies. If the health reaches zero, the player is sent back to the menu.

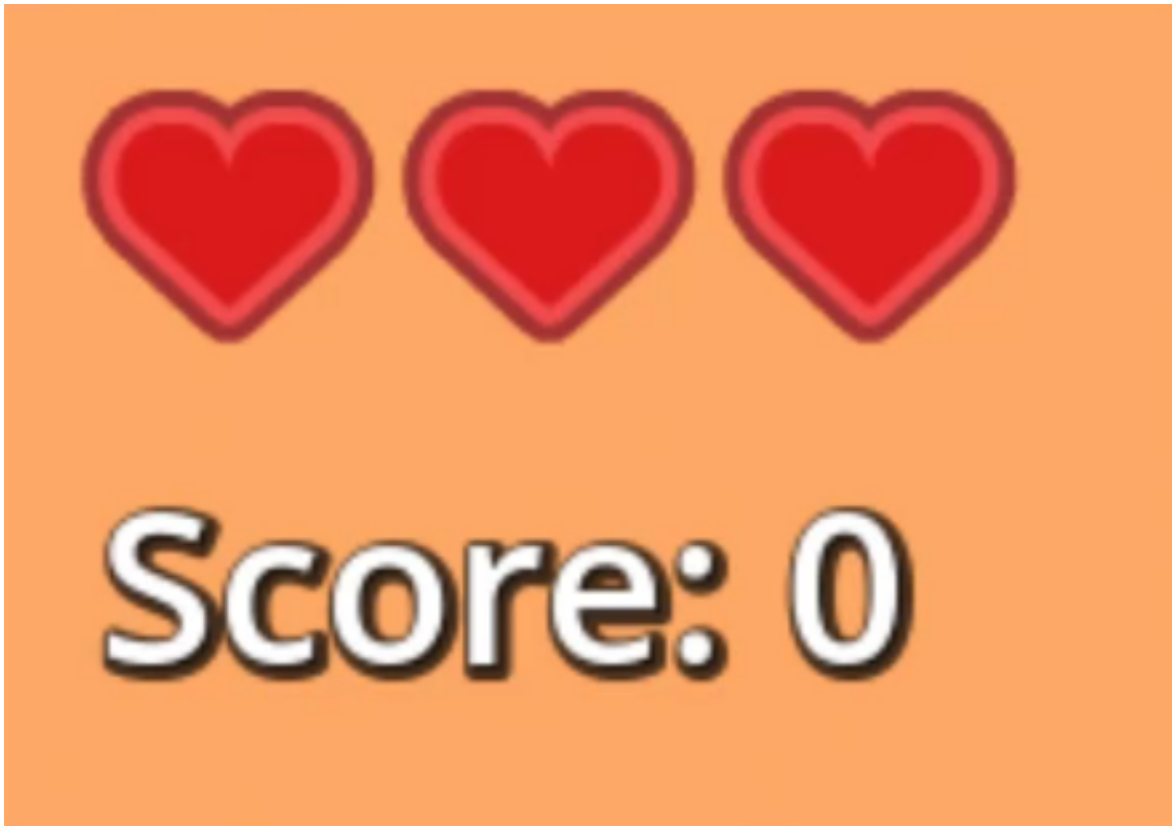
Enemies

To provide a challenge, we will introduce enemies in the form of spinning blades.

- **Behavior:** Enemies move from one point to another. We will design them so their movement direction, distance, and speed can be easily defined in the editor.



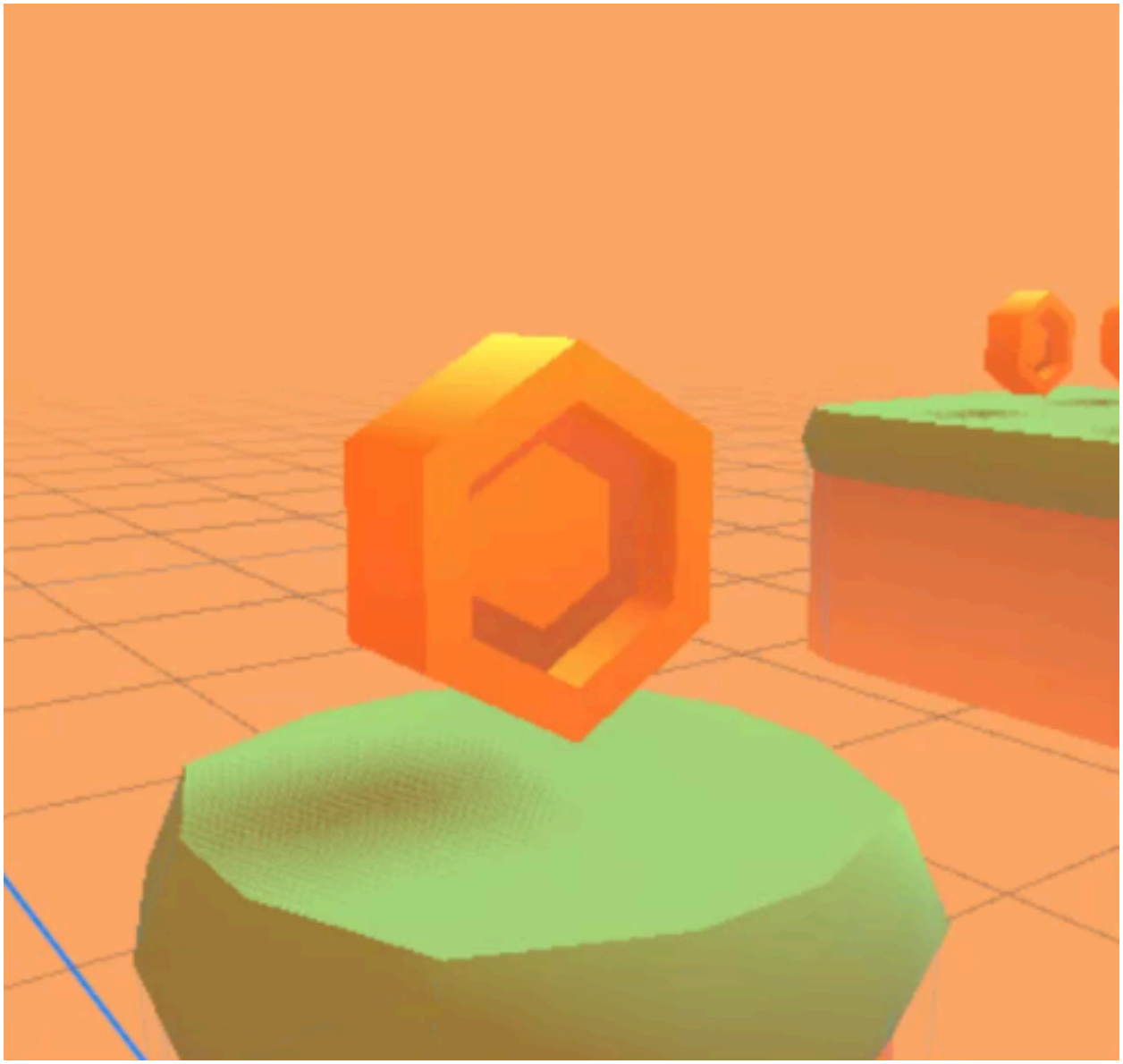
- **Damage Mechanics:** When an enemy hits the player, the following sequence occurs:
    1. The player's health variable decreases by one.
    2. The health UI updates to show the remaining health.
    3. A screen shake effect simulates the impact.
    4. A damage sound effect plays.



Coins

Coins serve as the primary collectible to encourage exploration.

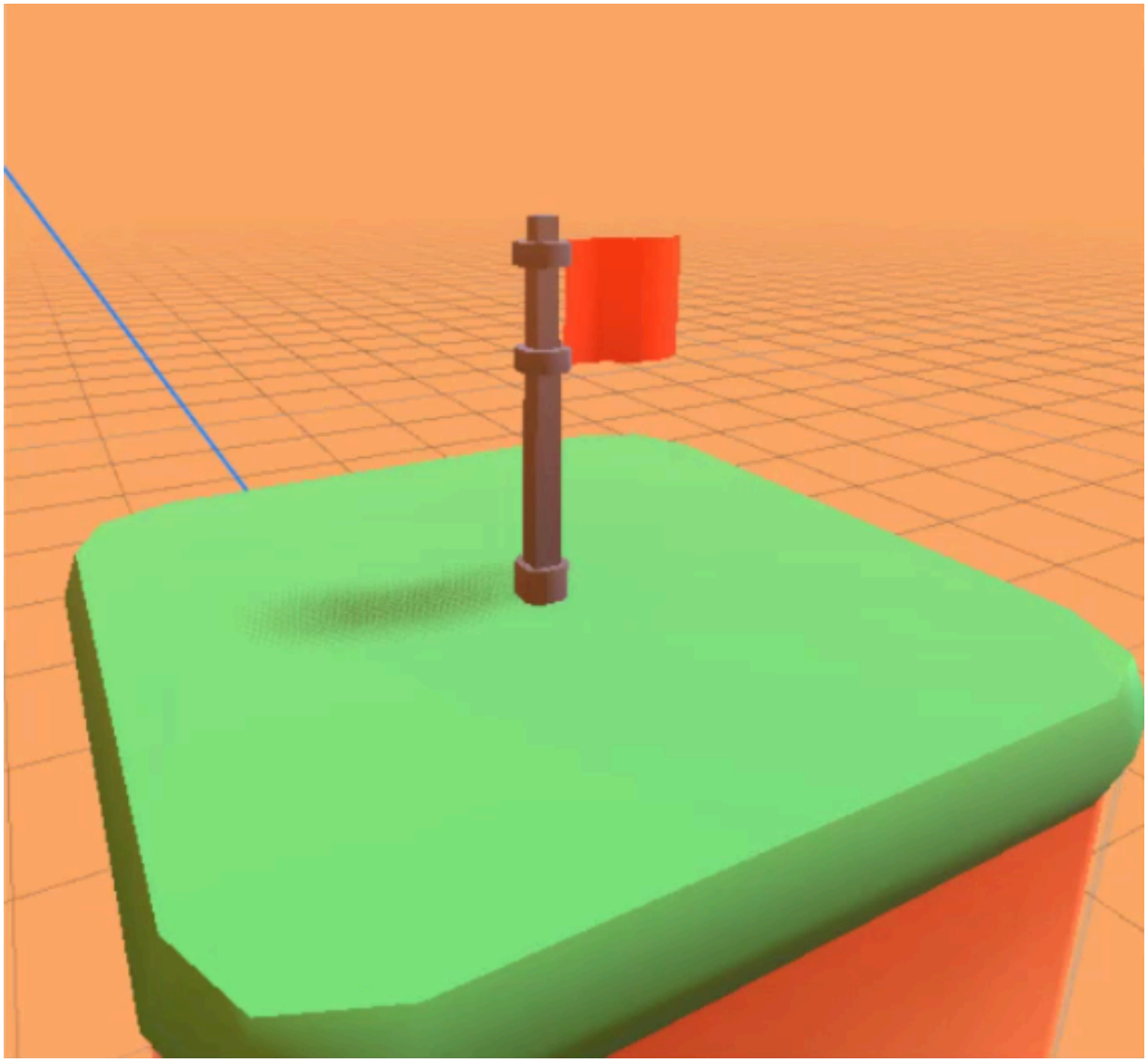
- **Collectibles:** Coins increase the player's score by one when picked up.
- **Animation:** To catch the player's eye, coins will rotate and bob up and down.
- **Sound Effect:** A satisfying sound effect plays when a coin is collected.



We will implement the scoring system using AutoLoad scripts (singletons). This allows the score to persist across levels, ensuring the player's progress is maintained throughout the game.

End Flag

The end flag marks the completion of a stage. It is a node placed at the end of each level that, when entered by the player, loads the next level.



Menu

Finally, we need a main menu to serve as the entry point for the game.

- **First Scene:** The menu is the first scene the player encounters.
- **Play Button:** Pressing this button loads the player into level one.
- **Quit Button:** Pressing this button closes the game application.

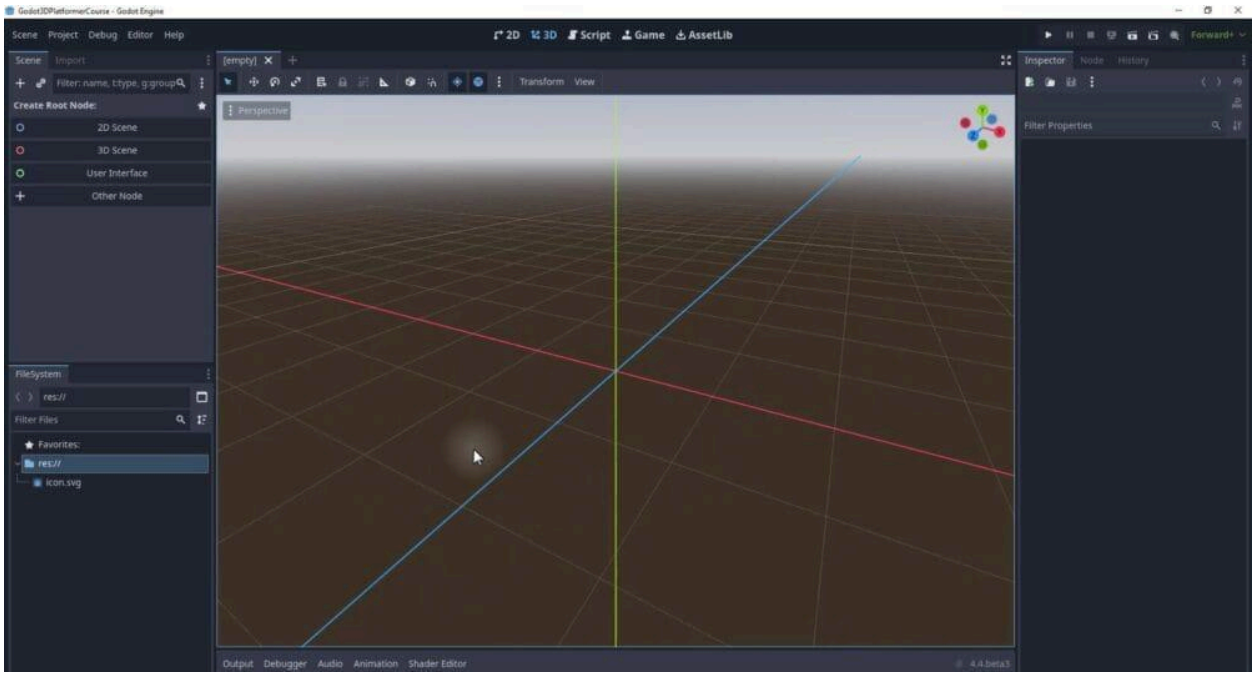
Now that we have a clear understanding of the game's design and mechanics, we can begin setting up the game in Godot.

Project Setup

In this section, we will walk through the initial steps of setting up your project, importing assets, and creating your first scene.

Setting Up the Project

Begin by opening Godot and creating a new project. The editor interface displays various docks, which you can rearrange to suit your preferences. For this guide, we will focus on the FileSystem and Inspector docks to manage our project structure and asset properties.



Importing Assets

Before we start building, we need to import our assets. These include audio clips, 3D models, and sprites for the user interface (UI). Ensure you have the asset pack mentioned in the previous chapter (refer to the Project Files section in Chapter 1).

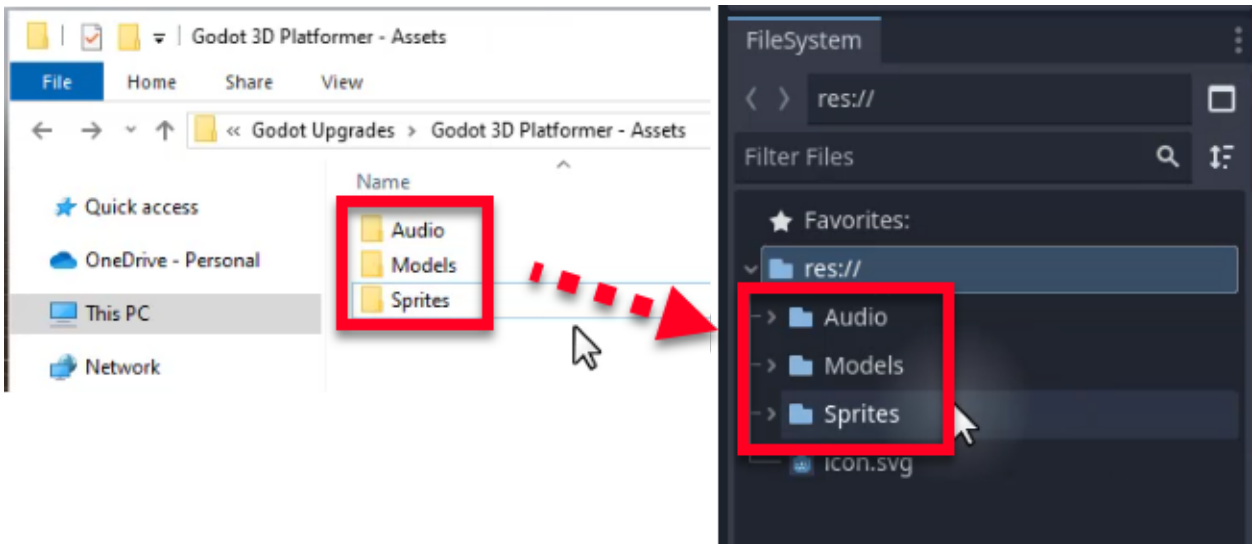
The asset pack contains the following:

- Audio files (e.g., coin sound effect, take damage sound effect)
- 3D models (e.g., character models, platformer models)

- Sprites (e.g., heart PNG, menu background)

To import these assets, follow these steps:

1. Extract the downloaded zip file to your computer.
2. Select all the assets and drag them into the FileSystem dock in Godot.
3. Once the import process is complete, you should see three folders in your project: `Audio`, `Models`, and `Sprites`.

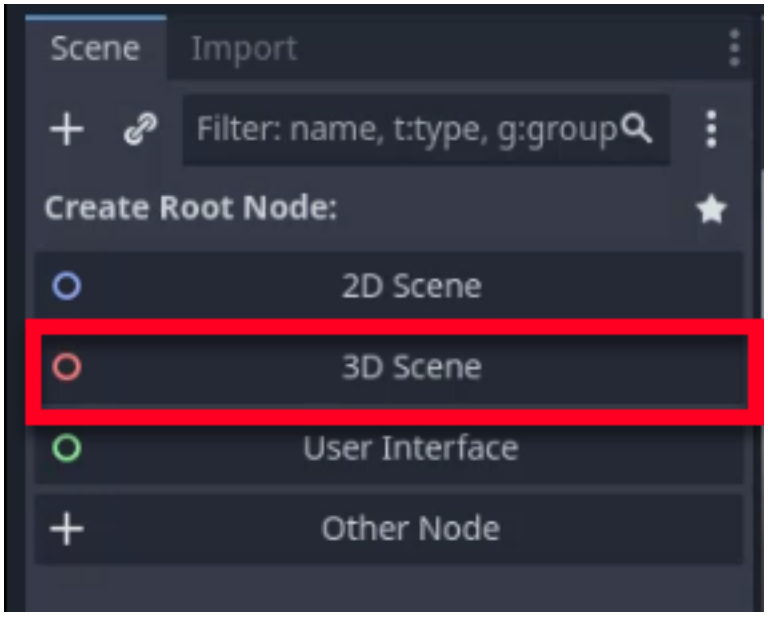


Feel free to explore the imported assets by clicking on the folders and previewing the contents. While you can use your own assets, using the provided files will help you follow along with the book exactly.

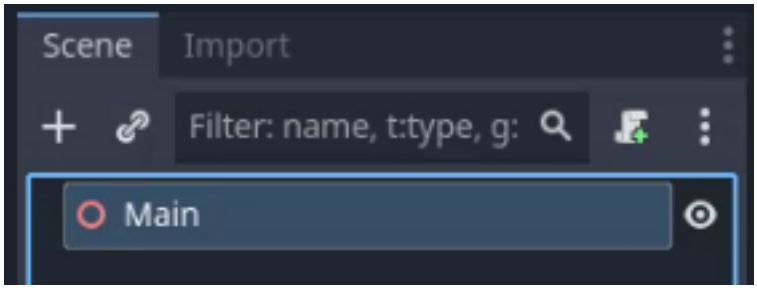
Creating the Base Scene

Now that we have our assets imported, let's create our base scene. This will serve as the first level of our game.

Click on the "New Scene" button in the top left corner of the editor. Since we are creating a 3D game, select "3D Scene" as the root node type.

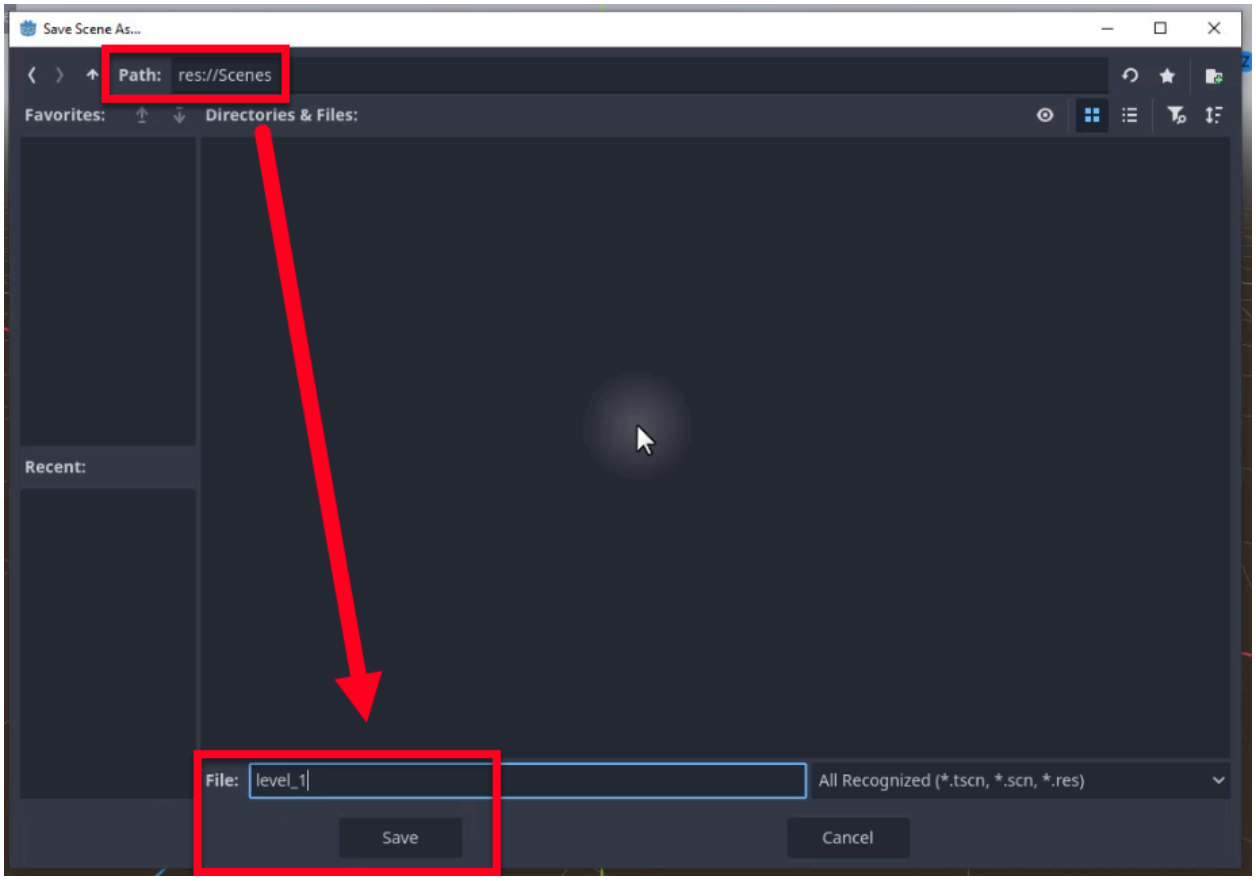


Rename the root node to “Main” for better organization.

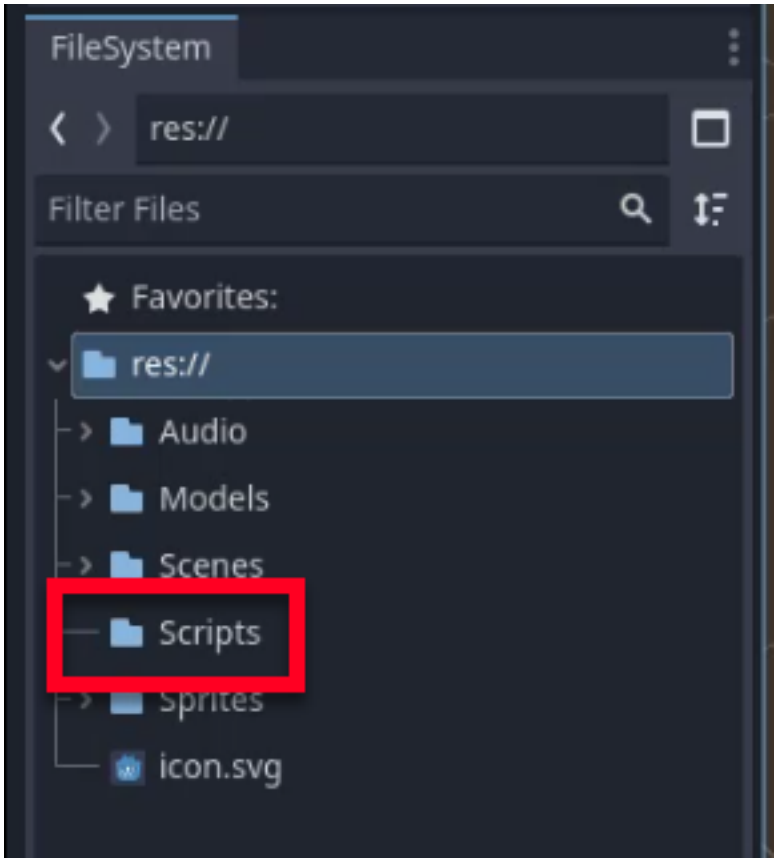


We need to save this scene into a dedicated folder. Save the scene by going to `Scene -> Save Scene` or pressing `Ctrl + S`.

Create a new folder named “Scenes” in the FileSystem dock and save the scene as “level_1” within this folder.



Additionally, let's create a folder for our scripts. Right-click on the `res://` folder, select "New Folder," and name it "Scripts." This folder will be used to store our scripts in the future.



Next Steps

With our base scene set up, we are now ready to start creating the game environment. In the next chapter, we will begin adding platforms and other visual elements to our scene. This will serve as the foundation for our player to interact with as we continue to develop our game.

Building the 3D Environment

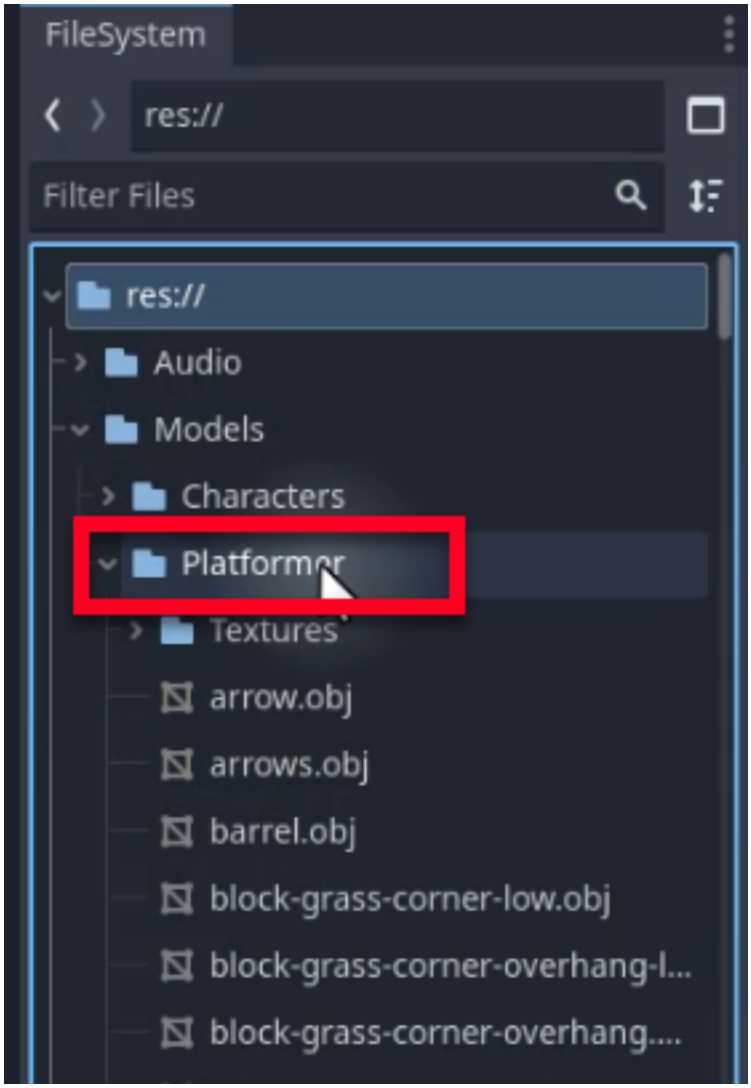
In this chapter, we will guide you through the process of creating a simple level for your player to navigate. By the end of this tutorial, you will have a basic understanding of how to set up platforms, configure lighting, and manage physics in your game environment. We will also explore how to enhance the visual appeal of the scene using the WorldEnvironment node to add sky, fog, and color adjustments.

Planning Your Level Design

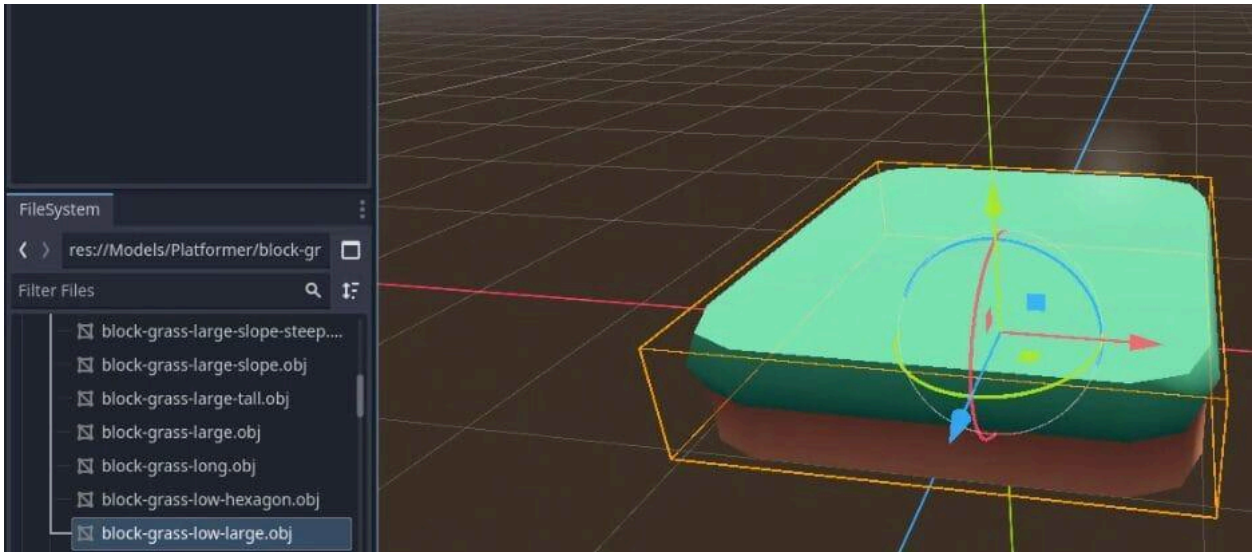
Before we dive into the technical aspects, let's plan what we want our level to look like. For this tutorial, we will create a central platform where the player spawns and a clockwise loop of platforms that get higher, leading to an end flag.

Setting Up the Initial Platform

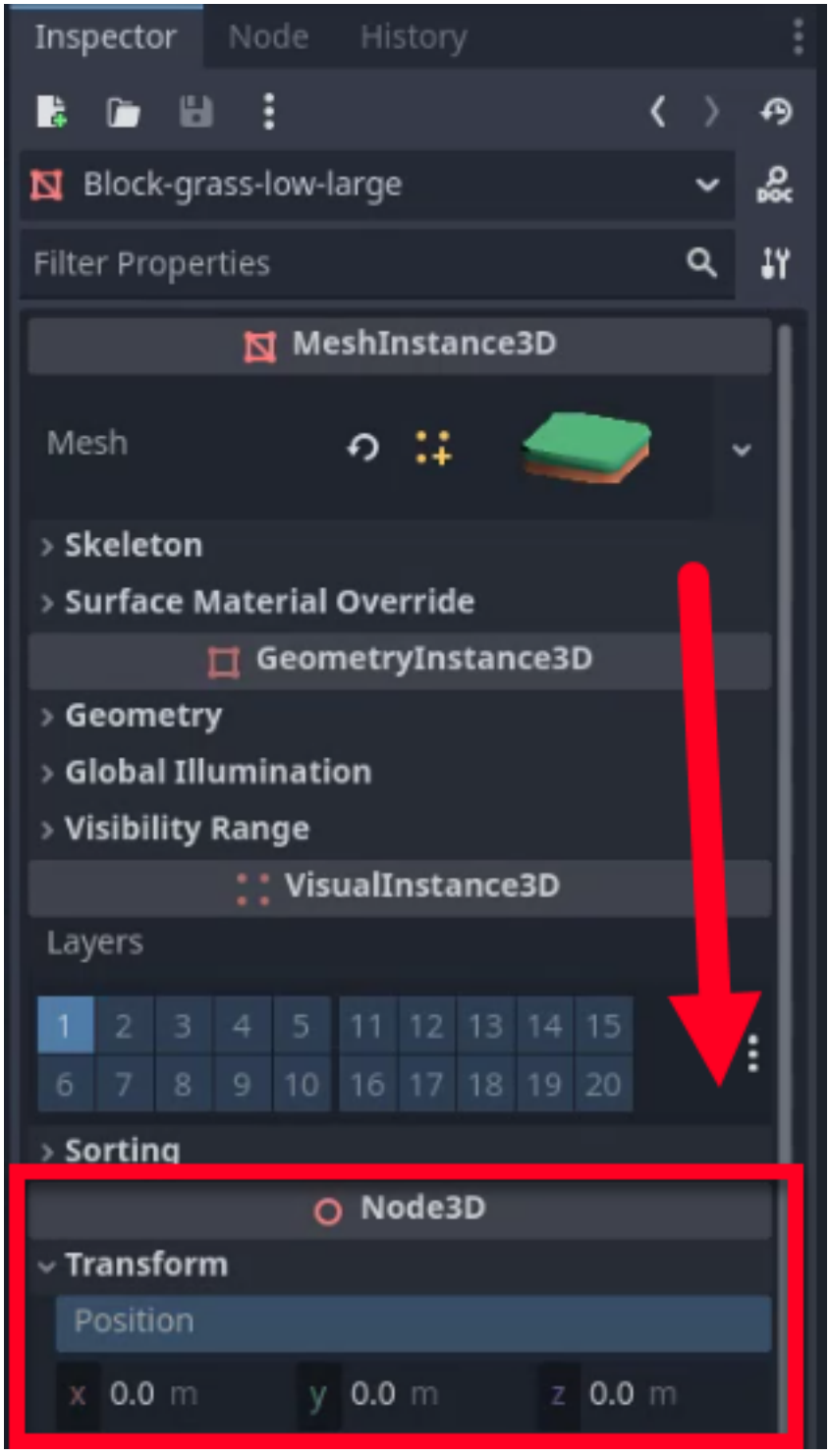
To begin building our world, we need to place our first platform. Open your models folder in the FileSystem dock and navigate to the `Models/Platformer` section.



Select a suitable platform model, such as `block-grass-low-large.obj`, and drag it into your scene.



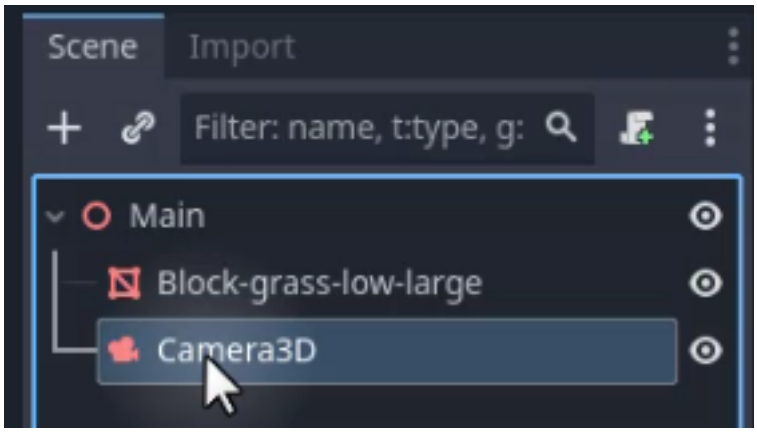
We want this platform to serve as the starting point. As such, center the platform in the world by setting its Position to `(0, 0, 0)` in the Inspector, as shown below.


Inspector
Node
History
Block-grass-low-large
Filter Properties
MeshInstance3D
Mesh
Skeleton
Surface Material Override
GeometryInstance3D
Geometry
Global Illumination
Visibility Range
VisualInstance3D
Layers
1 2 3 4 5 11 12 13 14 15
6 7 8 9 10 16 17 18 19 20
Sorting
Node3D
Transform
Position
x 0.0 m
y 0.0 m
z 0.0 m

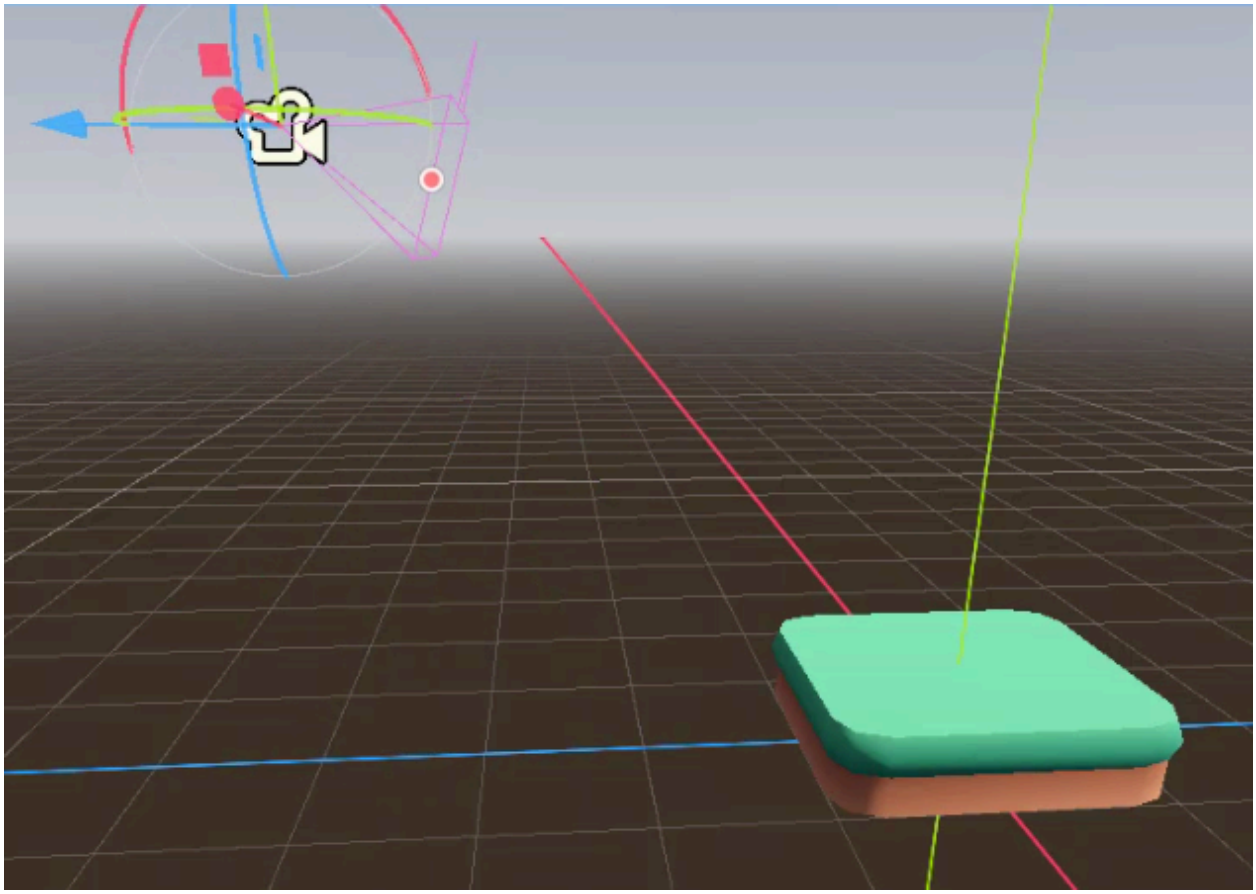
Adding a Camera

Without a camera, there is nothing to render the scene through when the game runs. We need to add a Camera3D node and position it so the player gets a clear view of the platforms.

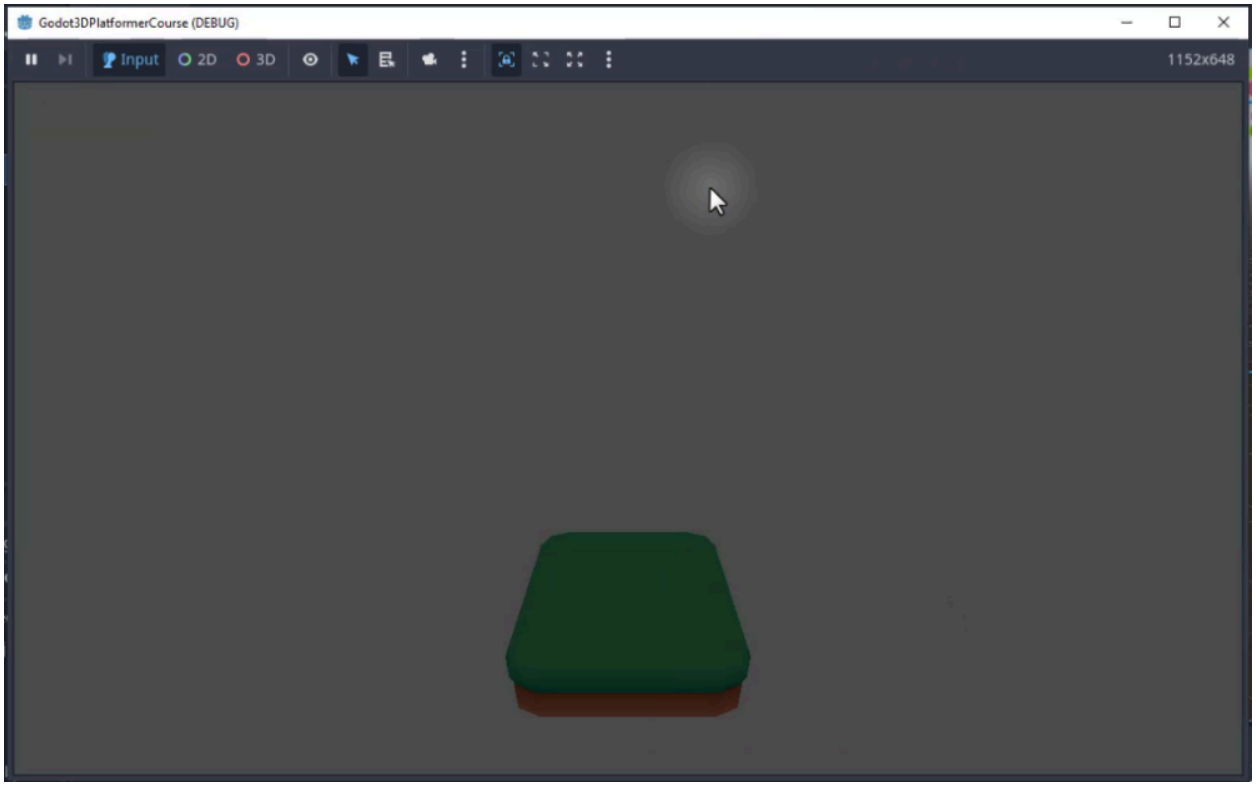
Create a new **Camera3D** node in your scene as a child of Main.



Once added, position the camera at an angle that provides a good view of the platform. You can use the editor's viewport to move and rotate the camera node until the preview looks correct.



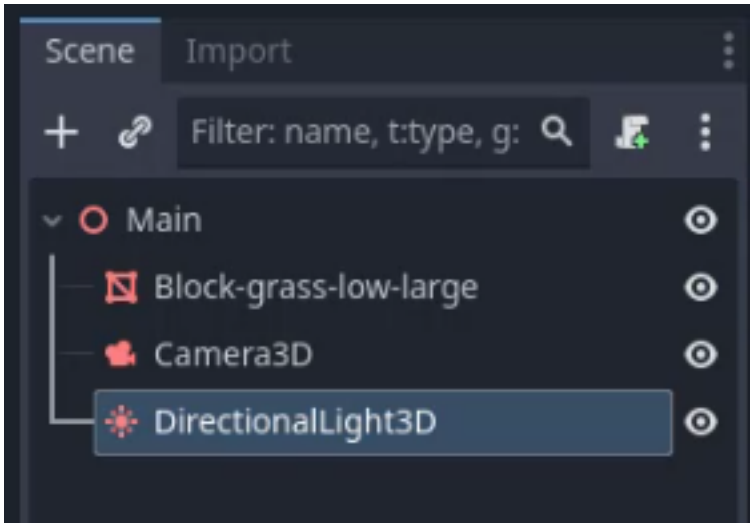
Once the camera is positioned, save your scene and press the Play button to ensure the camera is working correctly. You should see your platform rendered in the game window.



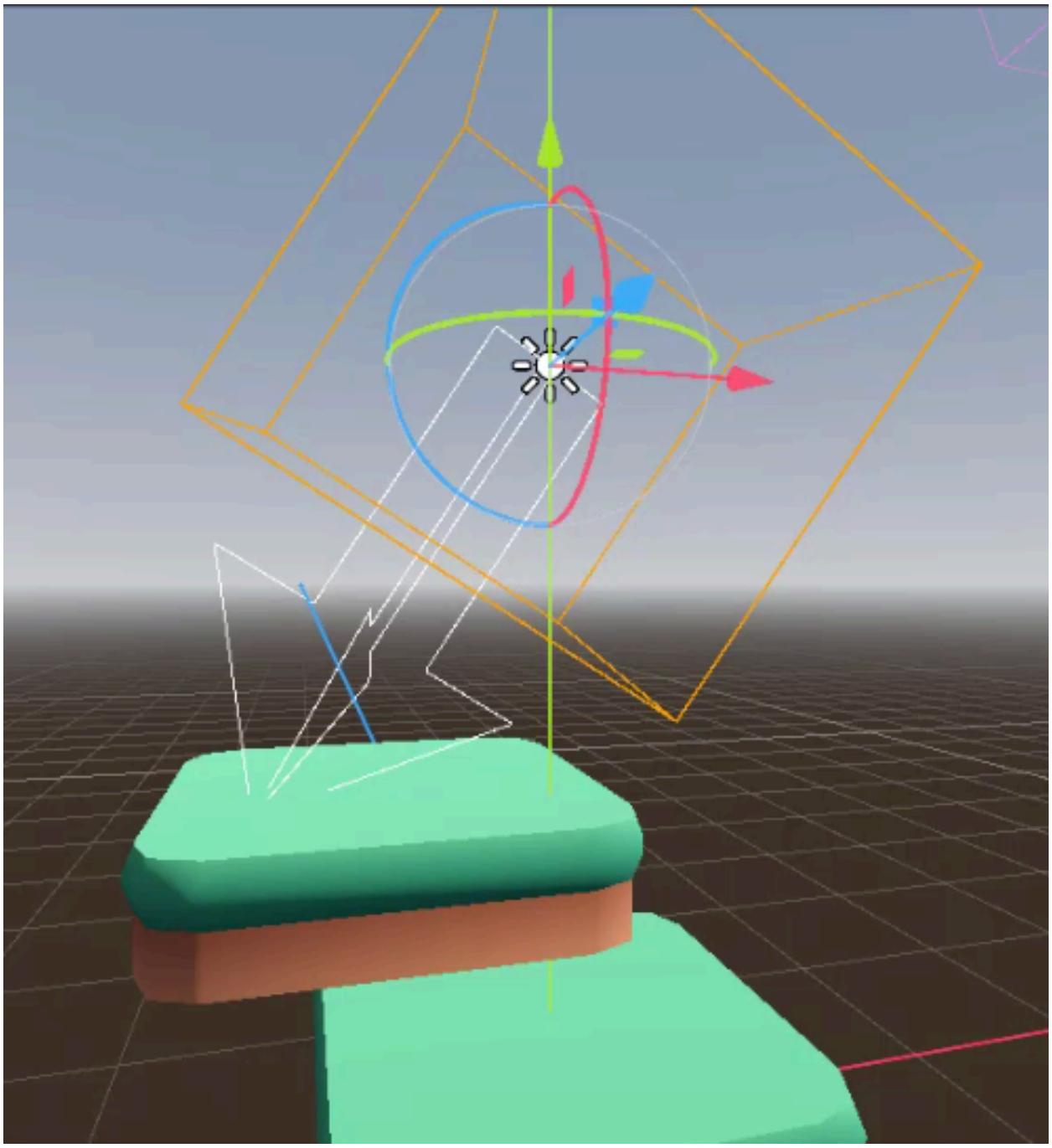
Configuring Lighting

A 3D scene without lighting looks flat and lifeless. To improve the visibility and aesthetics of our scene, we need to add a light source. A DirectionalLight3D node acts as the sun: it casts parallel rays across the entire scene, providing global illumination and shadows.

Let's add a new **DirectionalLight3D** node as a child of Main.

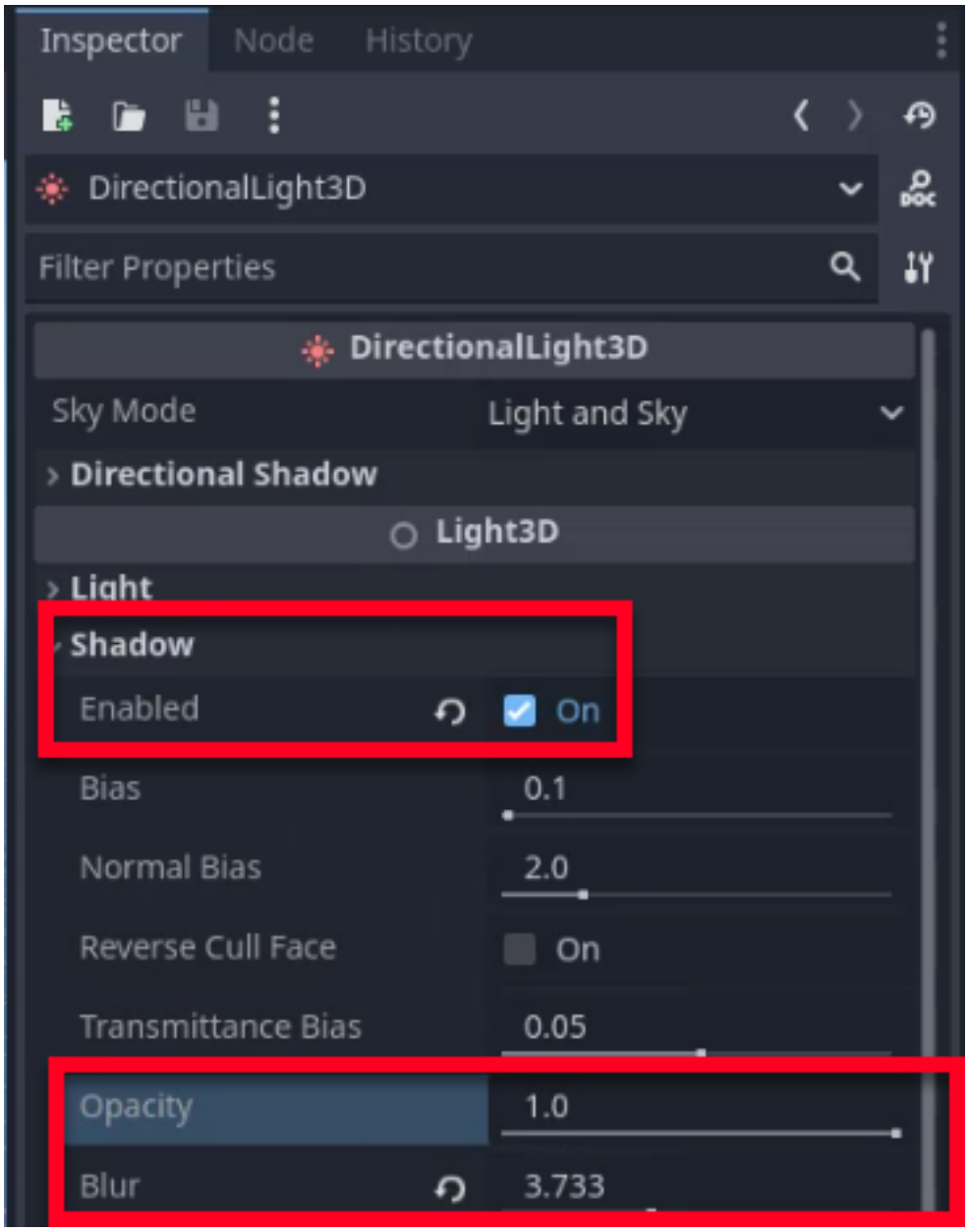


You can now adjust the light's rotation to change the time of day. Rotating the node changes the angle of the "sun," allowing you to simulate daytime, sunset, or nighttime. Feel free to pick an angle you like, as the exact angle won't affect our game logic.

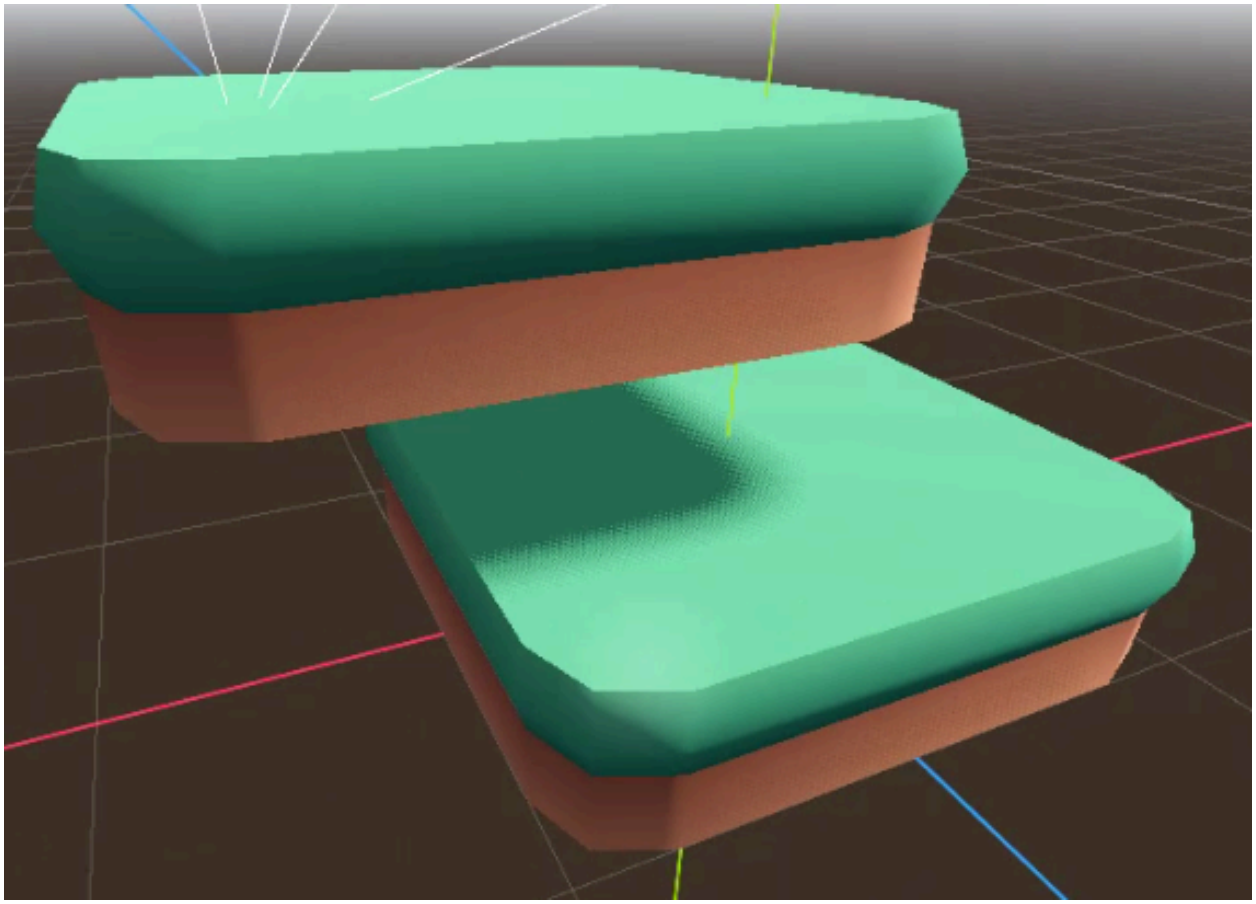


To add realism, enable shadows in the Inspector. You can adjust the blur and opacity of the shadows to achieve the desired effect. If needed, you can add

another platform to the scene temporarily to see how the shadows cast from one object to another.



The image below demonstrates how shadows appear when enabled.



Making Platforms Physical

Currently, our platforms are just visual meshes (MeshInstance3D). A MeshInstance3D only tells Godot what to draw; it has no physical presence. To ensure that the player can interact with them and stand on them, we need to give them physical properties by wrapping each mesh in a StaticBody3D with a collision shape.

Godot Insight: Physics body types

Godot provides three main physics body types, each suited to a different role:

StaticBody3D never moves on its own. It is the ideal choice for floors, walls, and platforms that the player stands on but that do not need to react to forces.

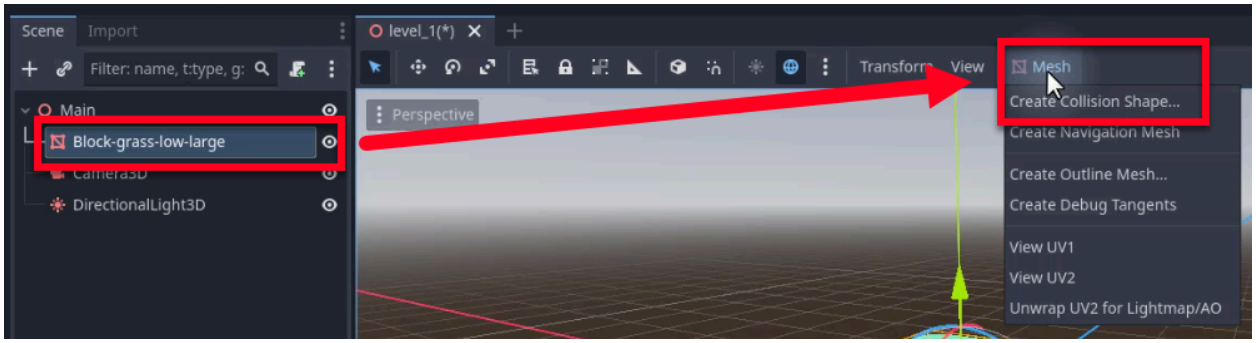
RigidBody3D is driven entirely by the physics engine. Use it for objects that should respond to gravity, collisions, and forces automatically, such as crates or rolling boulders.

CharacterBody3D is moved by your code. It provides helper methods like `move_and_slide()` for player characters and NPCs where you want full control over movement while still detecting collisions.

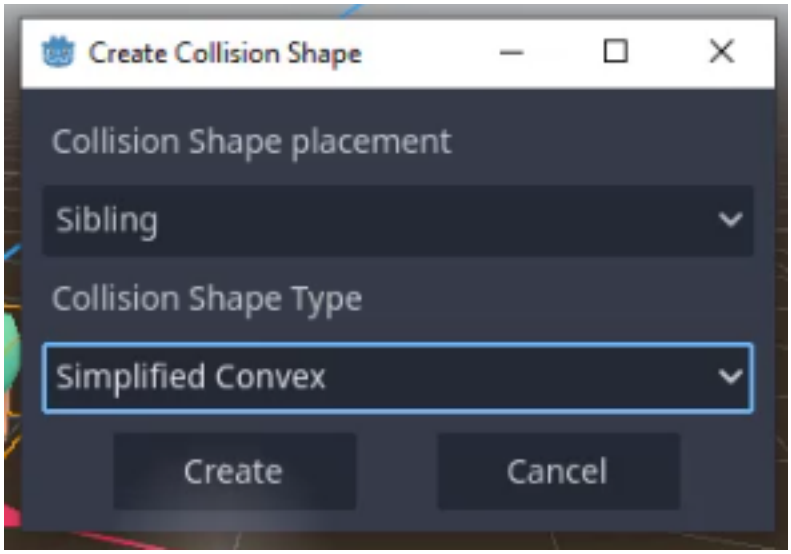
We will use StaticBody3D here because our platforms are immovable scenery. The player (a CharacterBody3D as you'll see later) will collide with them, but the platforms themselves never need to respond to physics.

Learn more: StaticBody3D (https://docs.godotengine.org/en/4.6/classes/class_staticbody3d.html)

Select the platform model in the scene. At the top of the 3D viewport, click on the **Mesh** button and choose **Create Collision Shape**.

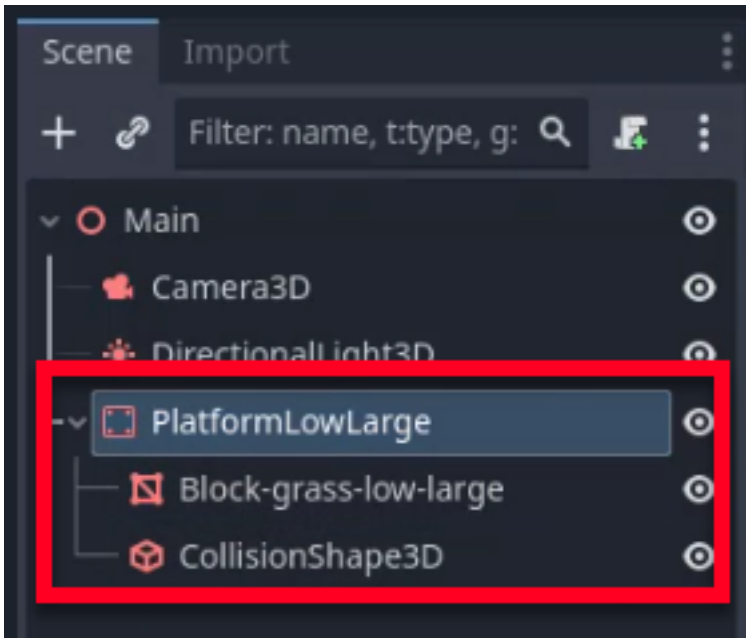


A dialog will appear. Select **Simplified Convex** as the collision shape type. This creates a collision shape that closely matches the mesh geometry but is optimized for physics performance.



After creating the shape, we need to organize the nodes correctly for physics:

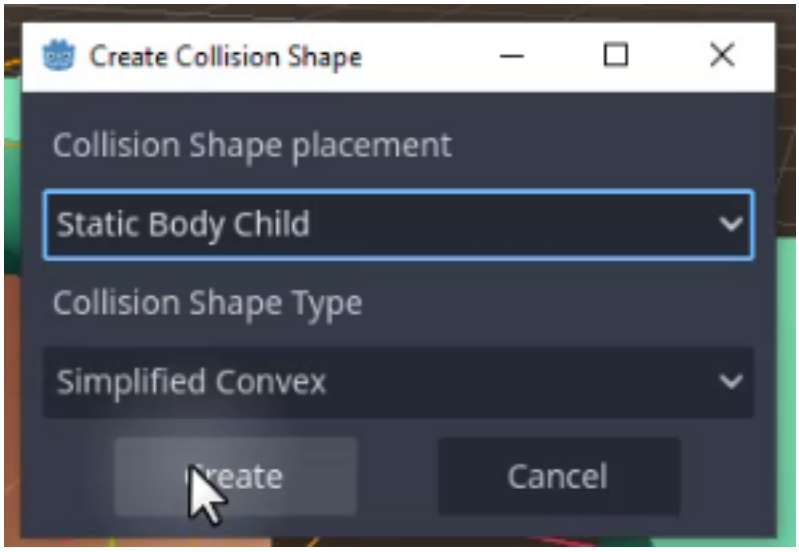
1. Create a new **StaticBody3D** node.
2. Drag the platform model (MeshInstance3D) and the newly created collider (CollisionShape3D) as children of this StaticBody3D node.
3. Rename the StaticBody3D to “PlatformLowLarge” for better organization.



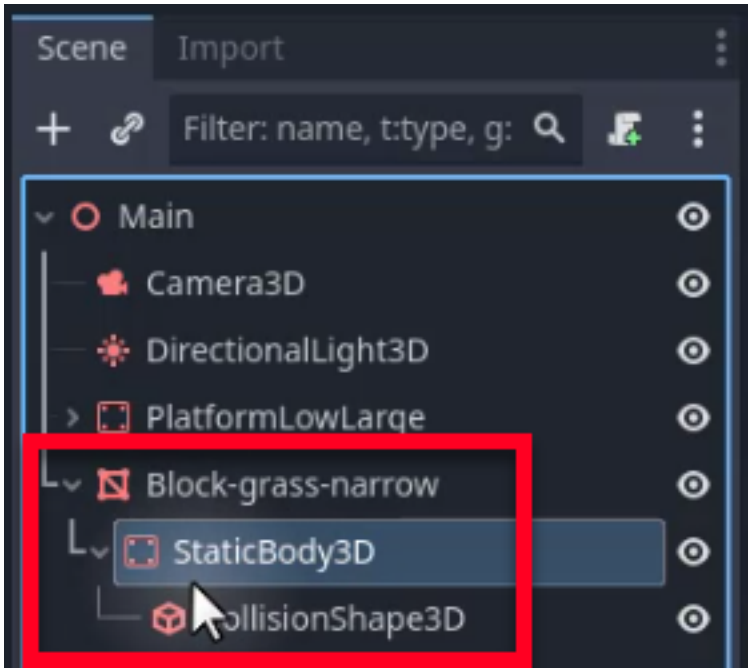
Alternative Method for Creating Platforms

There is a faster method to create platforms that automatically sets up the StaticBody3D and collider hierarchy.

1. Select the model and click on the **Mesh** button as done previously.
2. Choose **Create Collision Shape**.
3. In the dialog, we want to set the placement to **Static Body Child**.



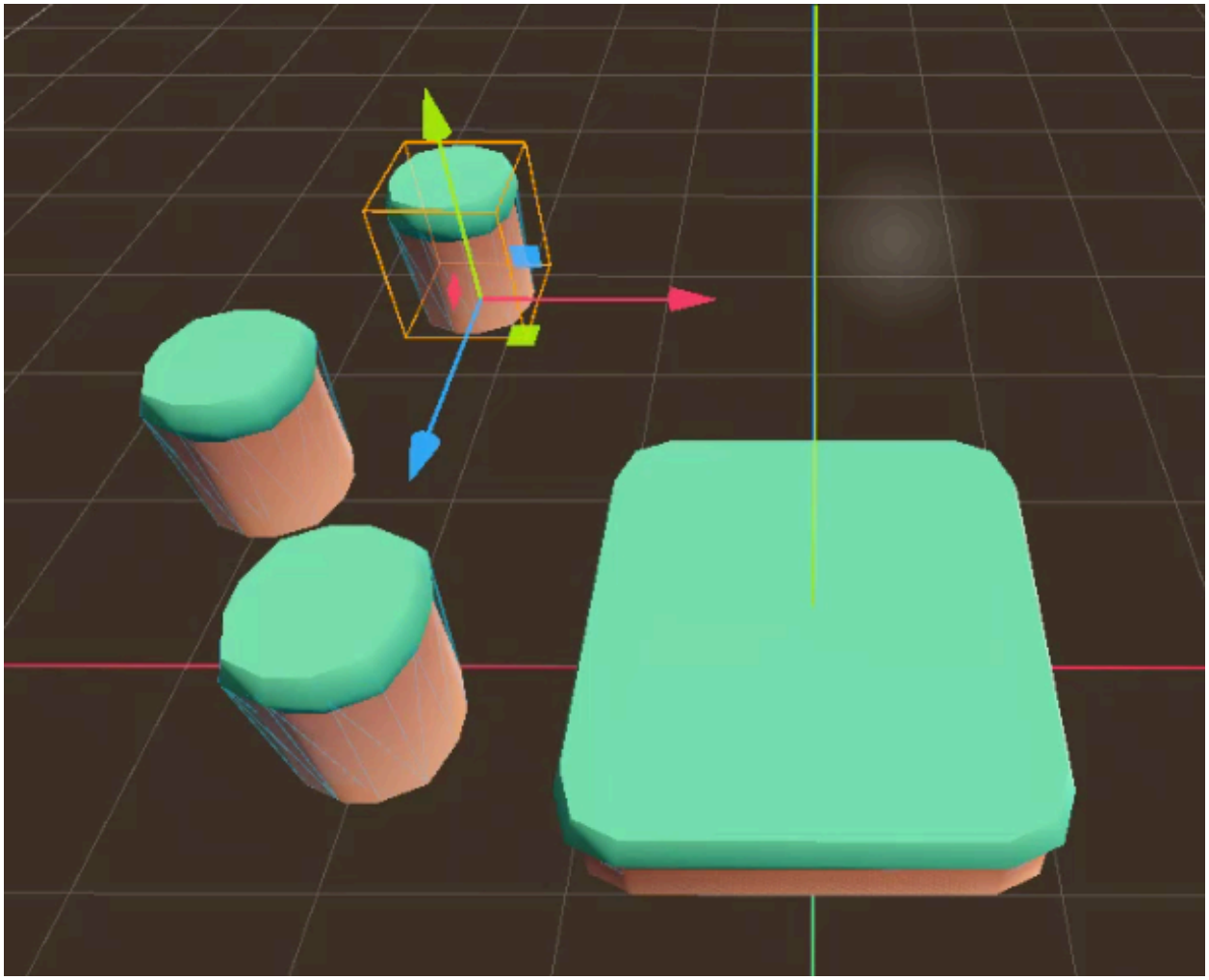
This method creates the StaticBody3D as a child of the model, streamlining the process.



Building Your Level

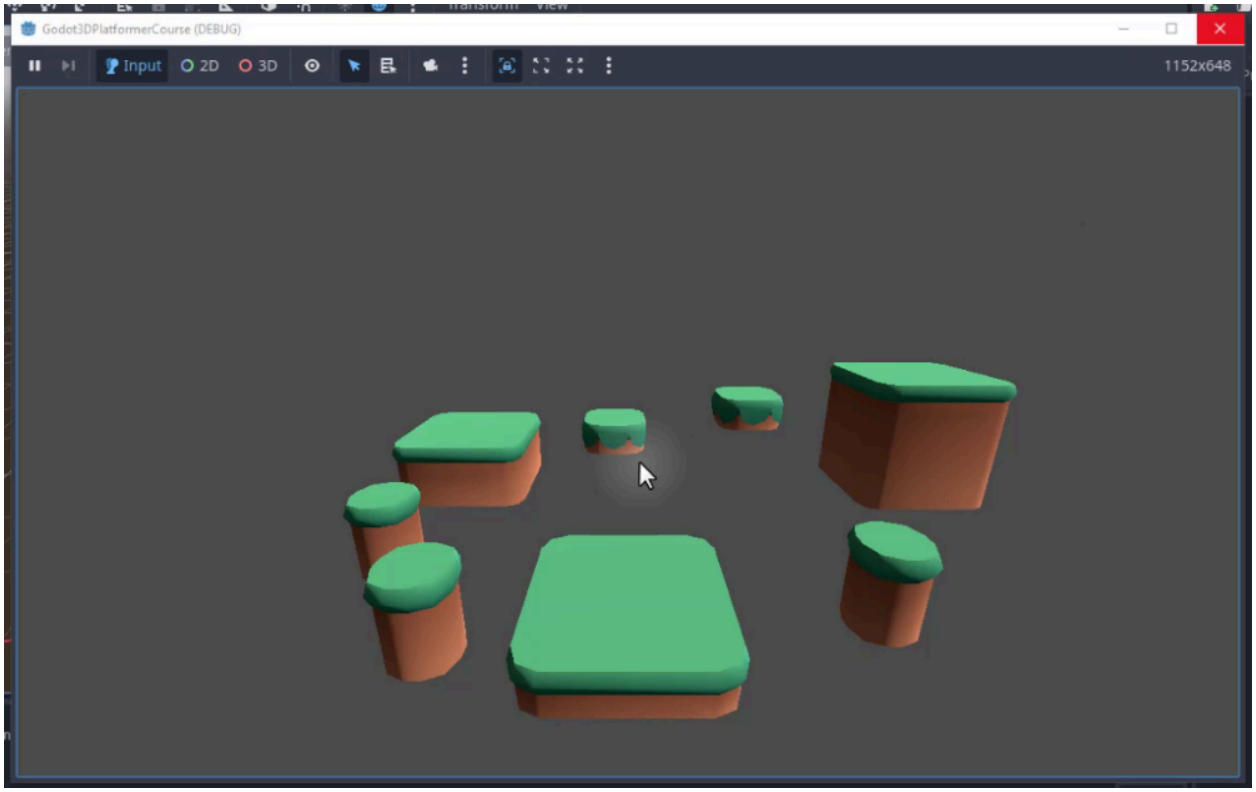
Now that you know how to create platforms, it is time to build your level.

Drag in more platforms and position them to create a path for the player. Use the methods outlined above to ensure each platform has a StaticBody3D and a collider. Experiment with different models and arrangements to create a unique level design.



Enhancing the Environment

By now, you should have a completed level design, though it might look a bit dull. To fix this, we will use the **WorldEnvironment** node.



The WorldEnvironment node in Godot is a powerful tool that allows you to modify various aspects of your game's visuals. With this node, you can add fog, adjust saturation, brightness, contrast, glow, ambient occlusion, reflections, skies, and more. This node is essential for creating an immersive and visually appealing game environment.

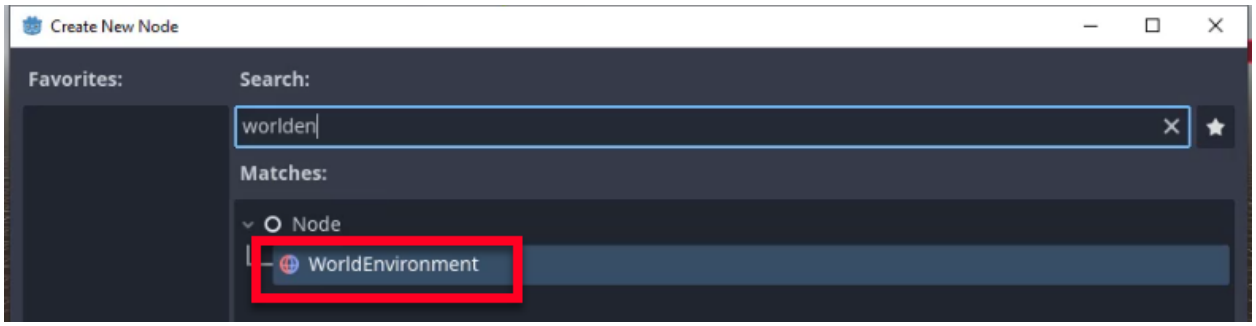
Godot Insight: WorldEnvironment and post-processing

WorldEnvironment acts as a global settings hub for your scene's visual atmosphere. It holds an Environment resource that controls background rendering (sky, solid color), ambient lighting, fog, tonemapping, and screen-space effects like SSAO and glow. Only one WorldEnvironment can be active per scene, but a Camera3D can override it with its own Environment if needed.

Think of it as the "mood dial" for your game: a single node that turns a flat-looking scene into a polished, atmospheric world without touching any individual object.

Learn more: WorldEnvironment (https://docs.godotengine.org/en/4.6/classes/class_worldenvironment.html)

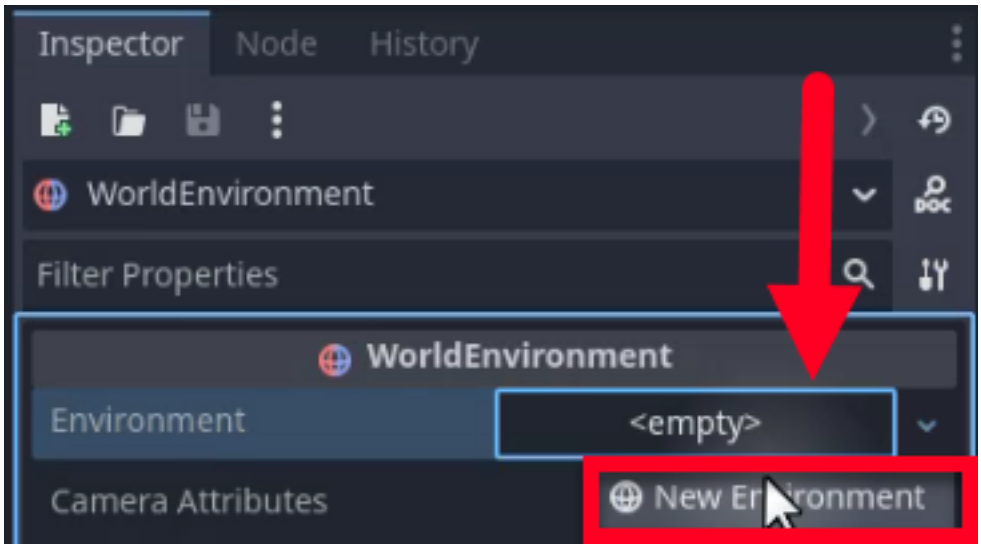
To begin, search for “WorldEnvironment” in the Create New Node dialog and add it to your scene.



Setting Up the Sky

Let’s set up a sky for our game to add depth and realism.

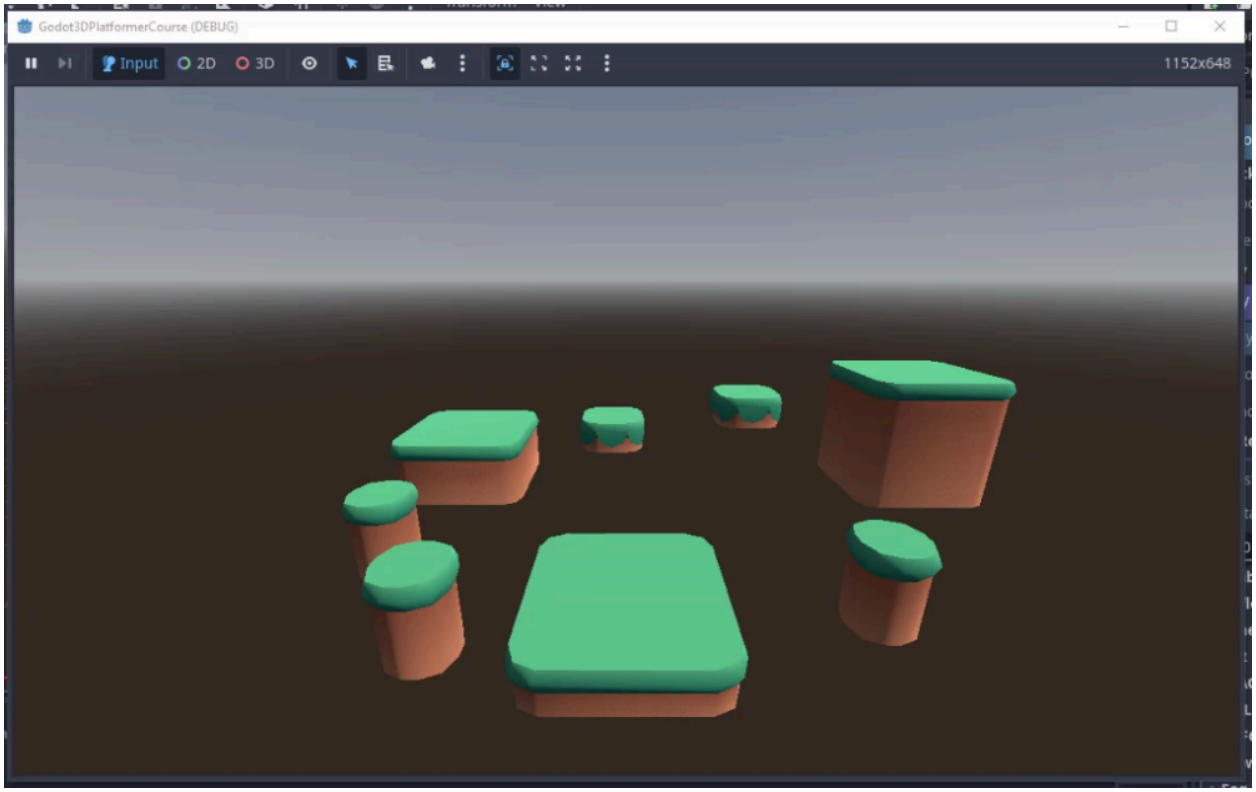
First, select the WorldEnvironment node. In the Inspector, we want to go to the **Environment** property and create a new **Environment** resource.



Expand the environment properties and navigate to the **Background** section. In this case, we want to change the **Mode** from Clear Color to **Sky**.

In the Sky section that appears, choose a new sky resource and select **ProceduralSkyMaterial**.

By following these steps, you will have a basic sky in your game.

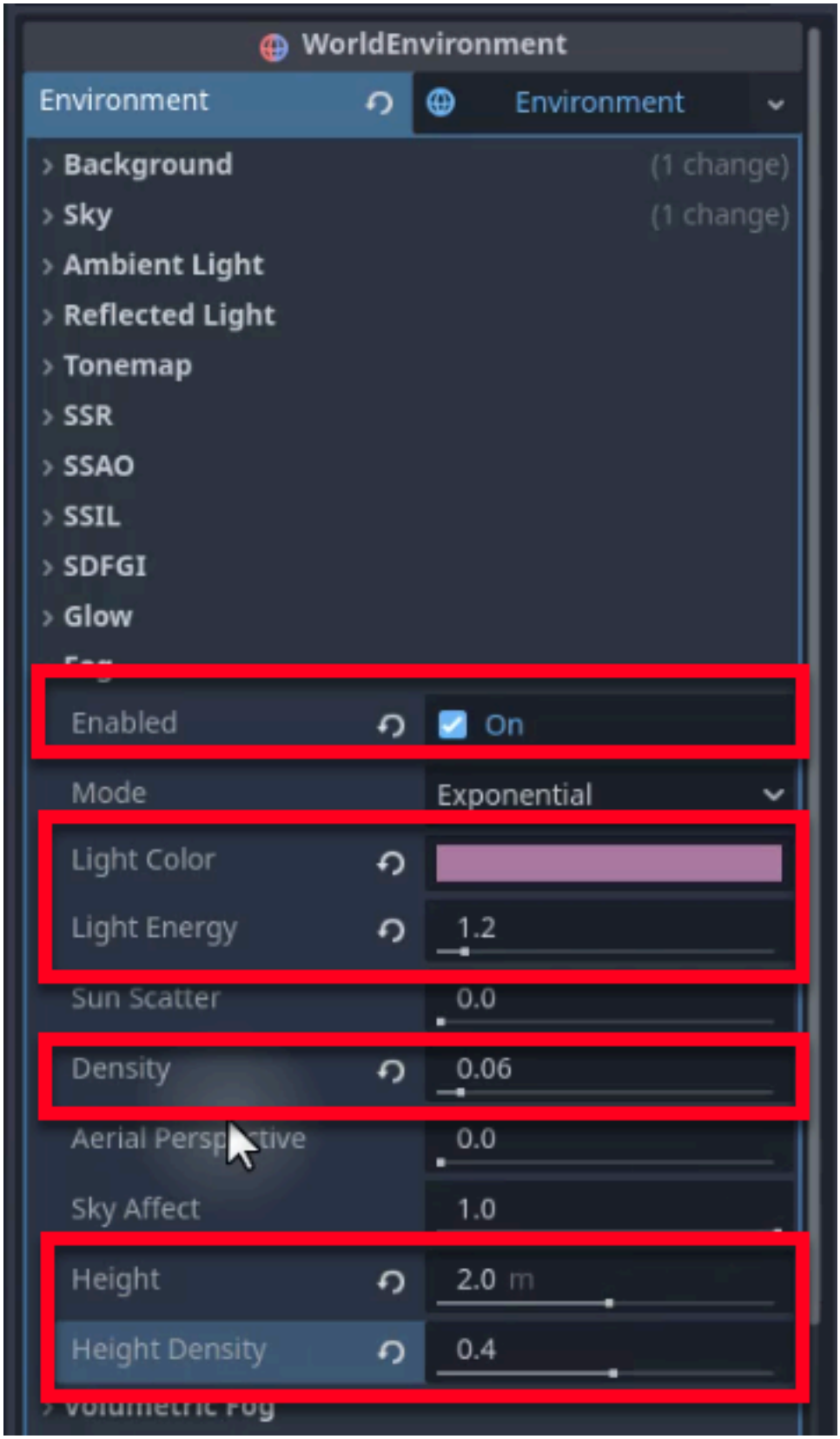


Adding Fog

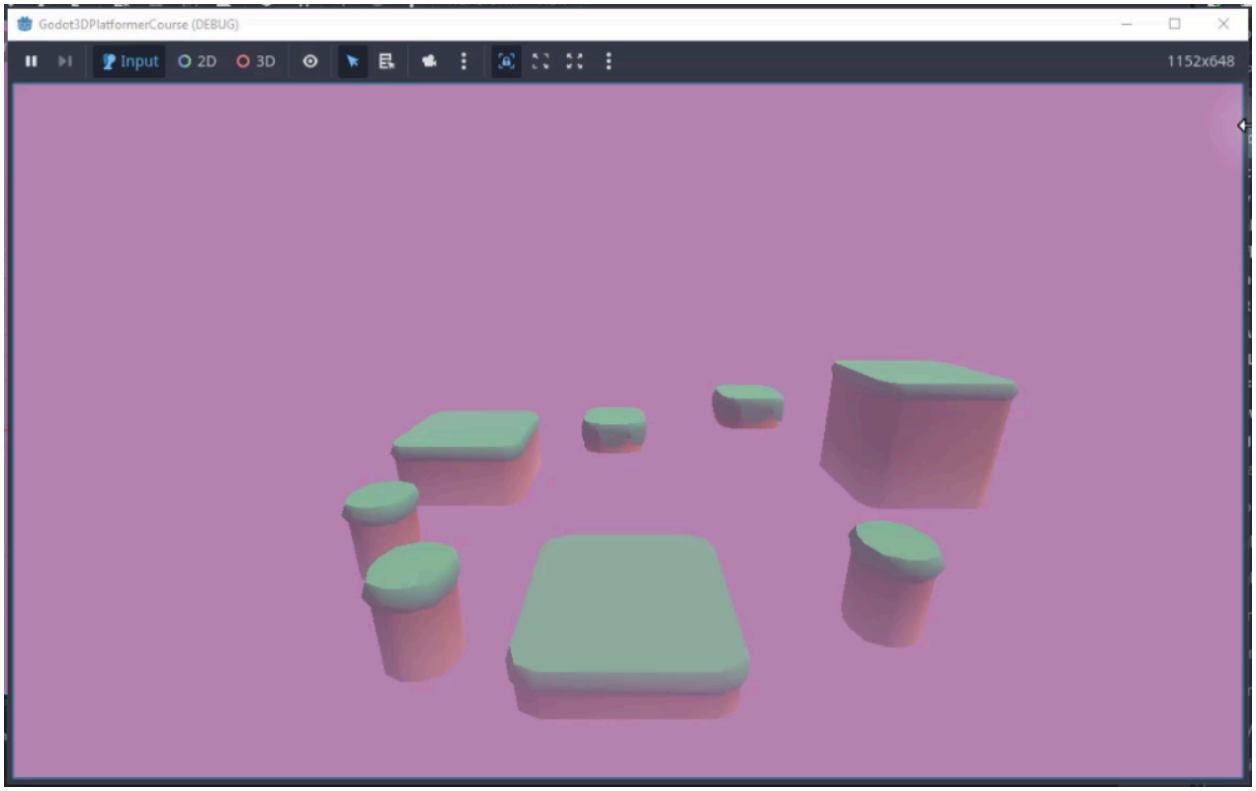
Fog can enhance the atmosphere of your game by adding a layer of depth and color.

In the WorldEnvironment node, scroll down to the **Fog** section and enable it. We will configure the following settings:

1. **Light Color**: Choose a desired fog color. For this level, let's choose a pinkish-purple color.
2. **Light Energy**: Increase this to make the fog brighter. A value of `1.2` is a good starting point.
3. **Density**: This controls the thickness of the fog. A value of `0.06` works well for a subtle effect.
4. **Height Fog**: Enable this and set the **Height** to `2m` and **Height Density** to `0.4`. This creates a gradual fog effect along the Y-axis.


WorldEnvironment
Environment
Environment
Background
(1 change)
Sky
(1 change)
Ambient Light
Reflected Light
Tonemap
SSR
SSAO
SSIL
SDFGI
Glow
Enabled
On
Mode
Exponential
Light Color
Light Energy
1.2
Sun Scatter
0.0
Density
0.06
Aerial Perspective
0.0
Sky Affect
1.0
Height
2.0 m
Height Density
0.4

Our level should already look significantly better with the fog applied.

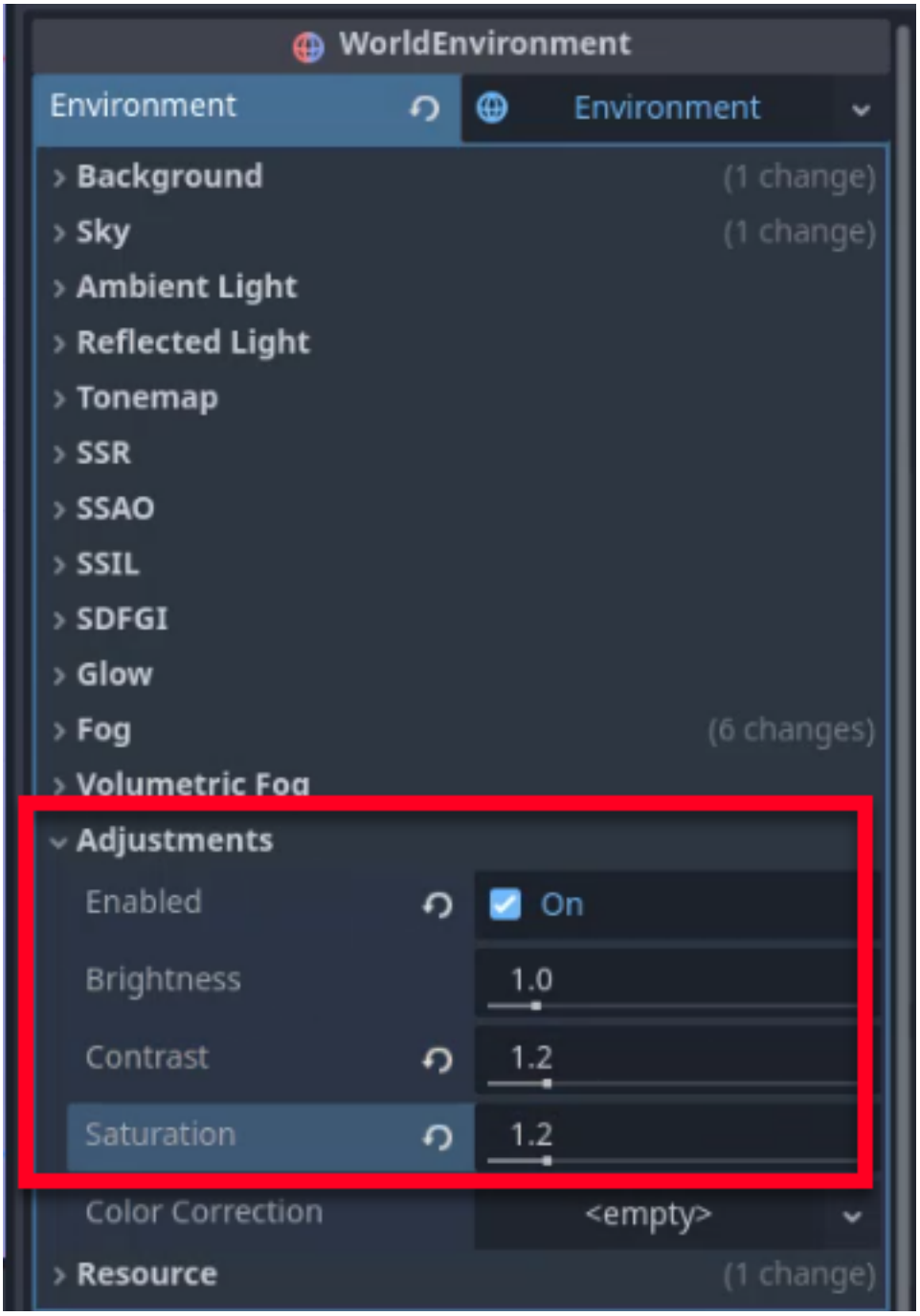


Adjusting Colors

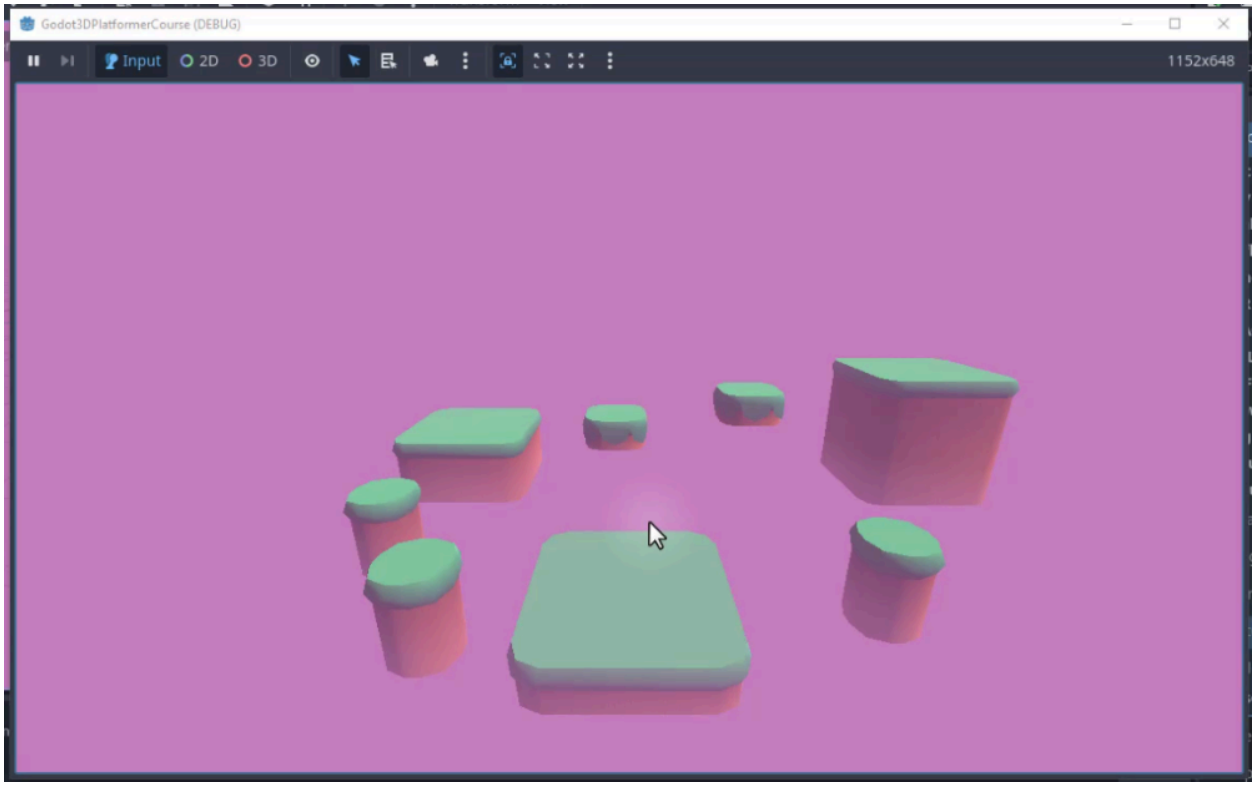
To make your game's visuals pop, you can adjust the brightness, contrast, and saturation.

In the WorldEnvironment node, scroll down to the **Adjustments** section and enable it.

1. Increase the **Contrast** to about `1.2` for a more vibrant look.
2. Increase the **Saturation** to `1.2` as well to make the colors more vivid.



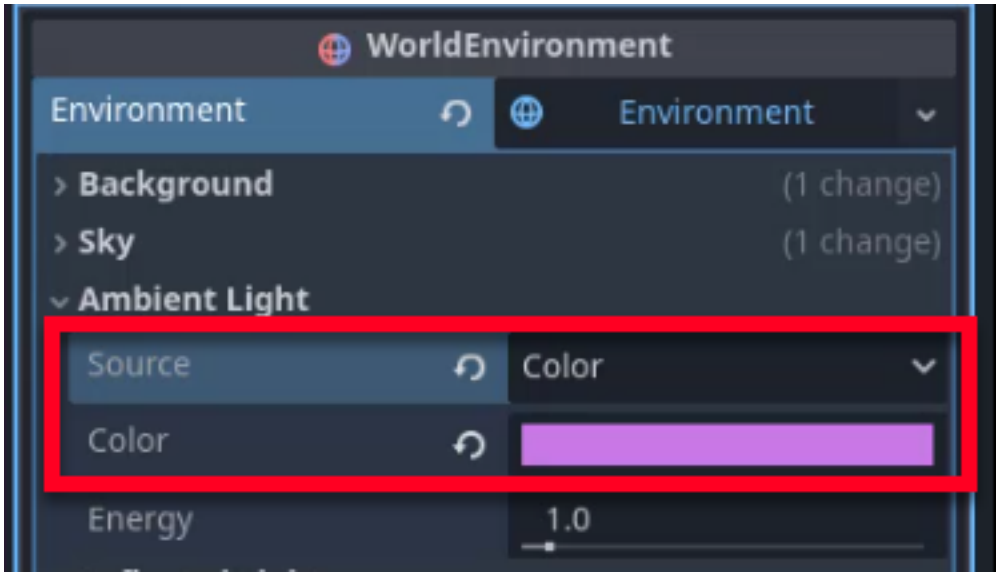
Once again, we can see visible changes in our level.



Ambient Light

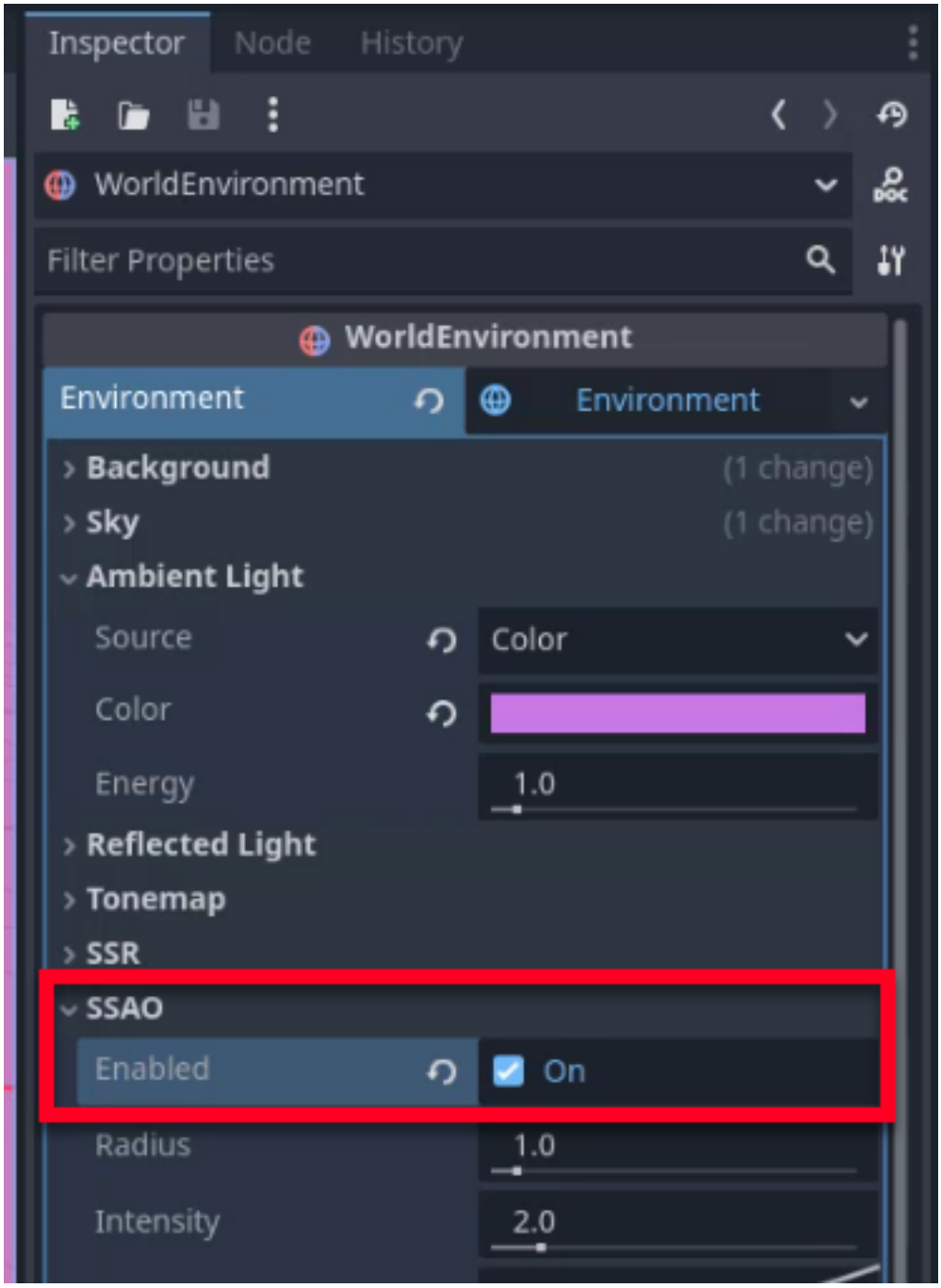
Ambient light defines the color of objects that are not directly hit by the sun. Adjusting ambient light can greatly enhance the overall look of your game.

In the WorldEnvironment node, scroll down to the **Ambient Light** section. Change the **Source** from Background to **Color**, and adjust the color to a purplish tint to match the fog and sky.

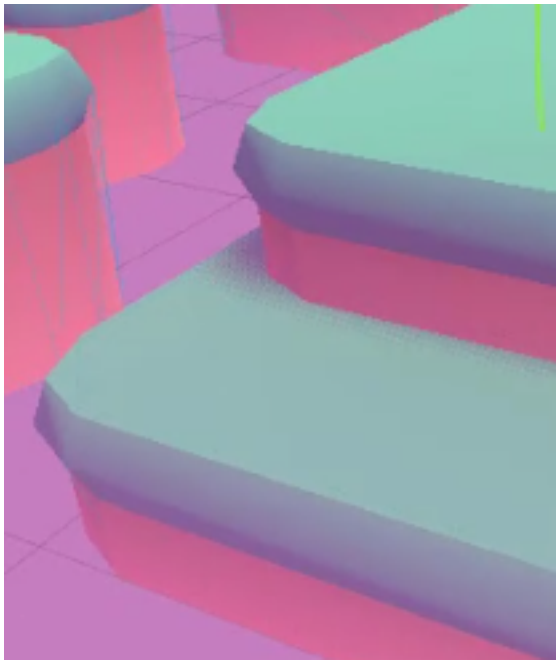


Screen Space Ambient Occlusion (SSAO)

Screen Space Ambient Occlusion (SSAO) adds shadows where two models intersect, creating a more realistic lighting effect. In the WorldEnvironment node, scroll down to the **SSAO** section and enable it.



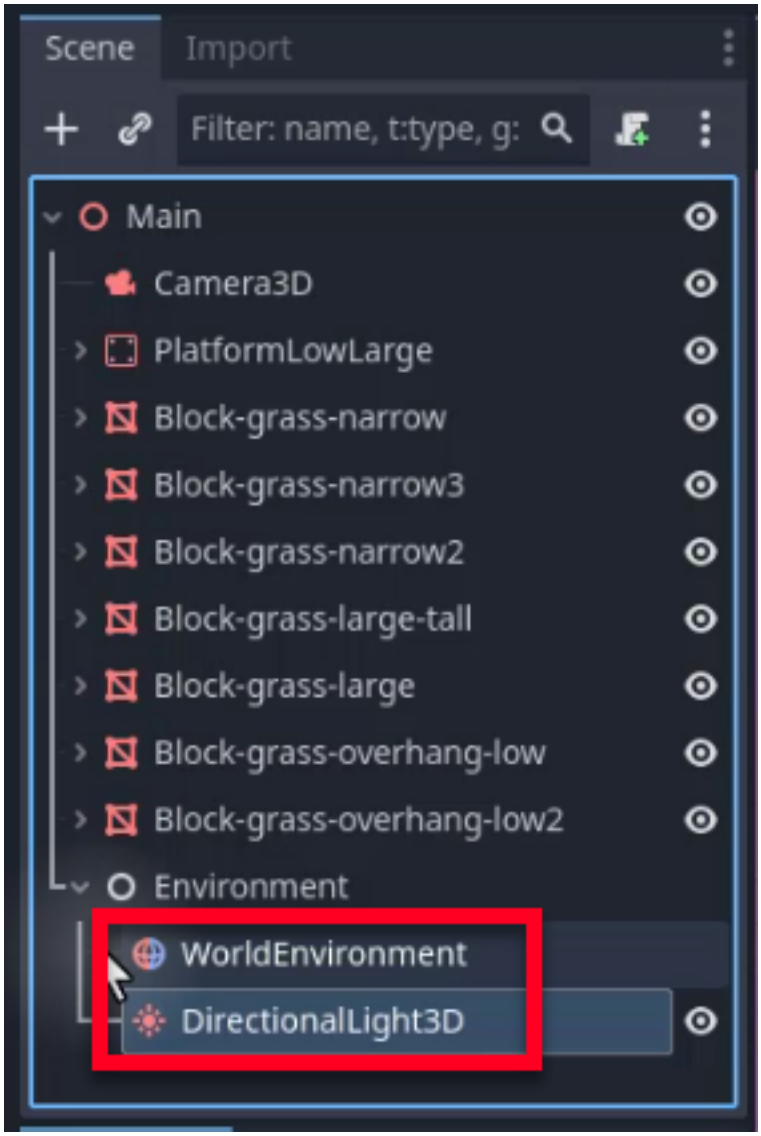
You can observe the differences in the corners where two models intersect to see the effect in action.



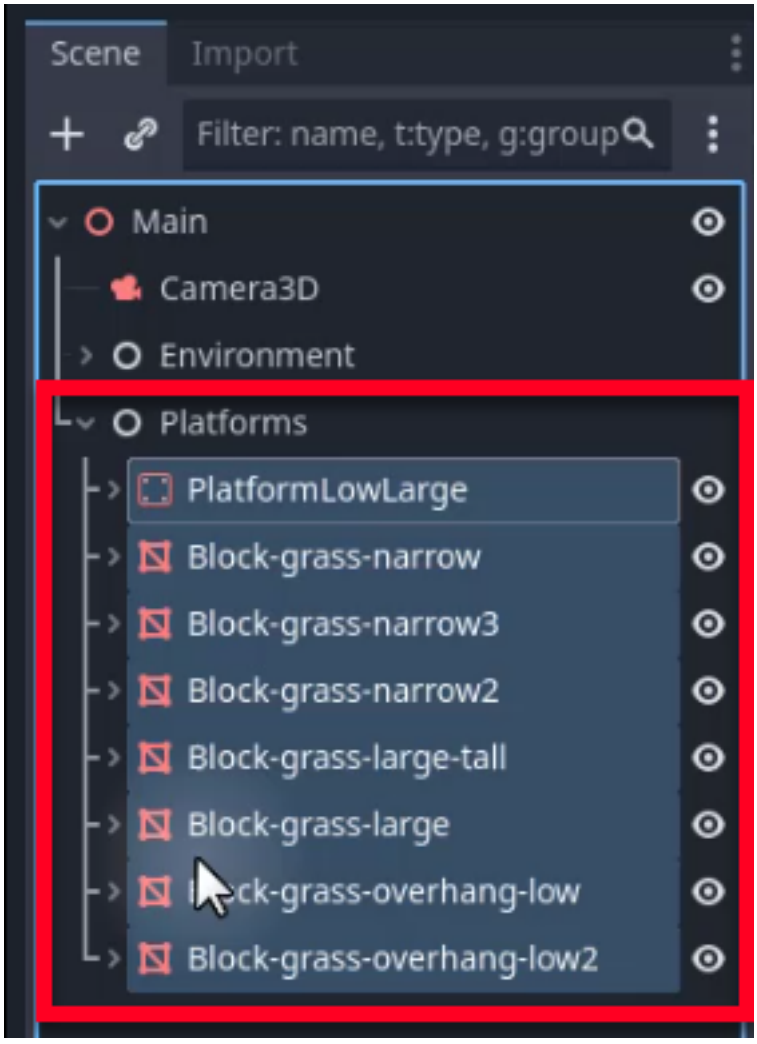
Organizing Your Scene

As scenes grow, a flat list of nodes becomes difficult to navigate. Using base nodes as organizational folders keeps your scene tree manageable, especially once you start adding enemies, coins, and other game objects in later chapters.

We need to create a new `Node3D` (or simply `Node`) and name it “Environment”. Drag the WorldEnvironment and DirectionalLight nodes into this node to make them children. Now, in your hierarchy, you will see this base node serve as a metaphorical folder that you can collapse and expand as needed.



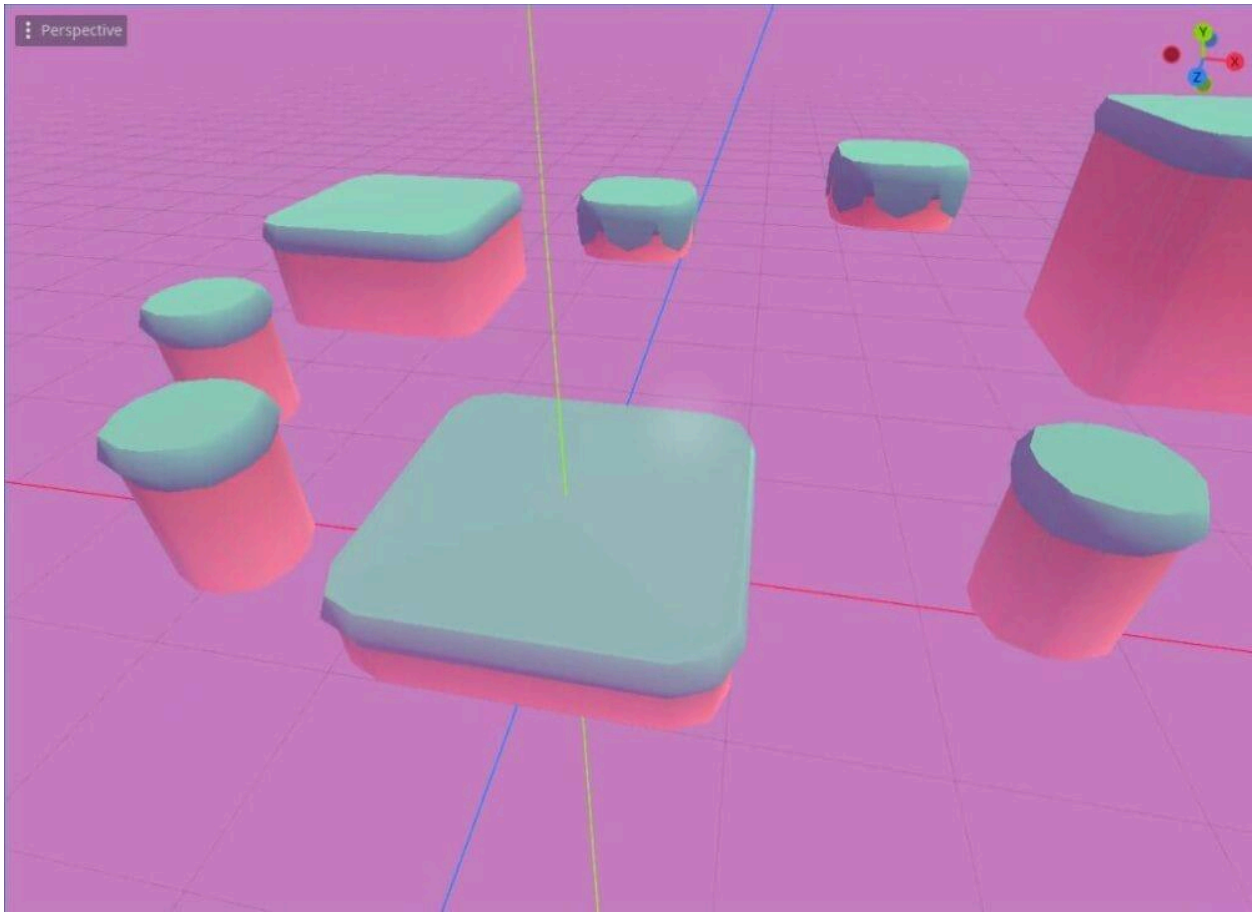
Let's create another base node for your platforms (called "Platforms") and drag all platform nodes into this folder.



Organizing your scenes like this makes it easier to manage, especially as you add more elements (like enemies and coins later in our case).

Conclusion

Congratulations! You have successfully built a 3D environment and enhanced its visuals using the WorldEnvironment node. By adding a sky, fog, adjusting colors, and organizing your scene, you have created a more immersive and visually appealing game environment.



In the next chapter, we will begin setting up our player character.

Player Configuration and Input

In this chapter, we will walk through the process of creating and configuring your player character, which will serve as the heart of your game. We will also configure the input map, ensuring that keyboard commands are correctly translated into in-game actions.

Creating the Player Scene

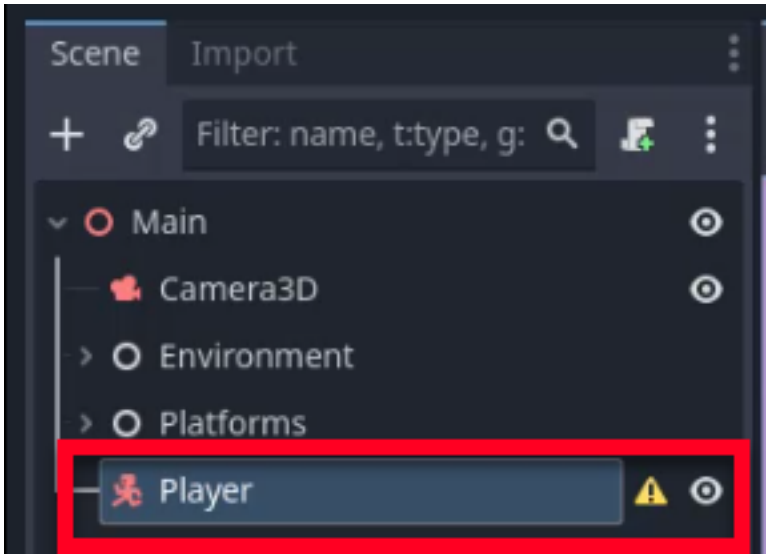
To begin, we need to create a root node for our player. We will use a `CharacterBody3D` node, which is essential for characters as it provides built-in physics, velocity settings for movement, collision detection, and various other functionalities that simplify character creation.

Godot Insight: Why CharacterBody3D?

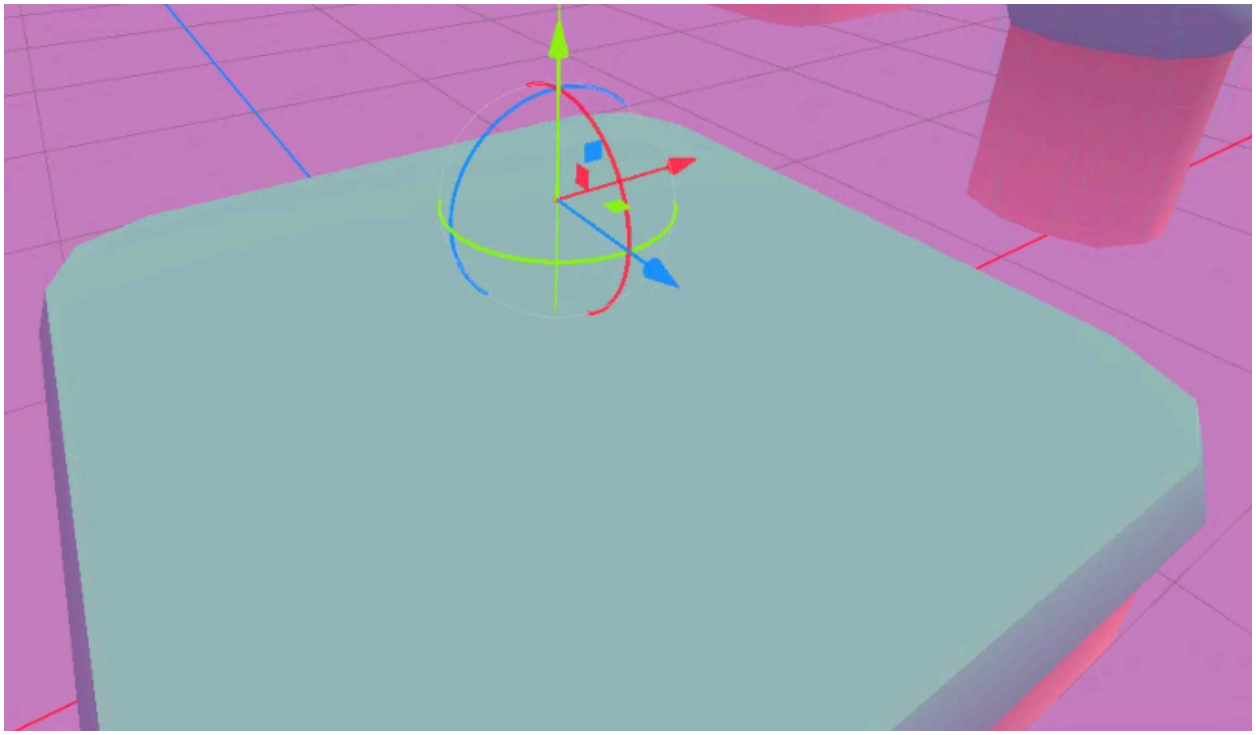
Godot offers three physics body types, and picking the right one matters. A RigidBody3D is fully driven by the physics engine, which is great for crates or boulders but makes precise player control difficult. A StaticBody3D never moves, so it is only useful for walls and floors. CharacterBody3D sits in between: it lets your code control movement directly through `move_and_slide()`, while still detecting collisions with floors, walls, and ceilings. That combination of control and collision awareness makes it the standard choice for player characters and NPCs.

Learn more: CharacterBody3D (https://docs.godotengine.org/en/4.6/classes/class_characterbody3d.html)

In your `level_1` scene, create a new node of type `CharacterBody3D` and rename it to `Player` for clarity.



Adjust your player in the editor so that it approximately rests on the main platform, as shown below.

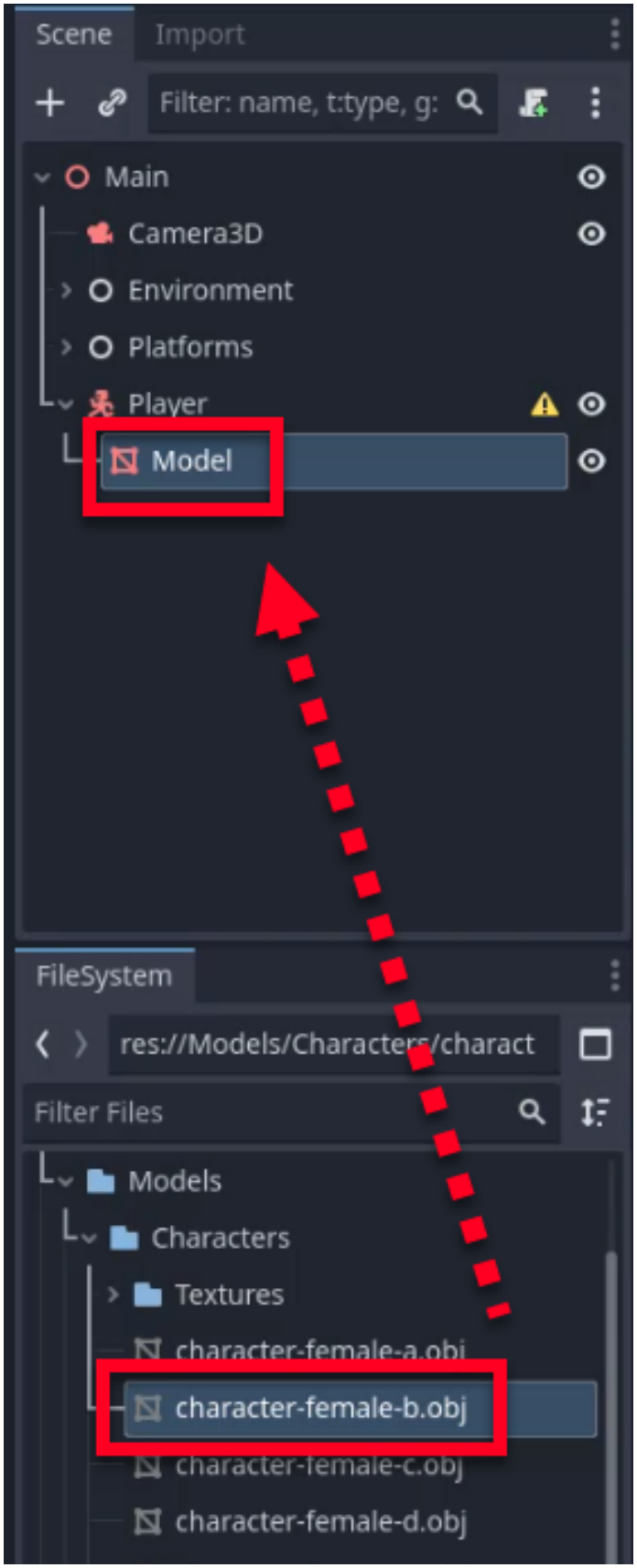


Adding a Visual Representation

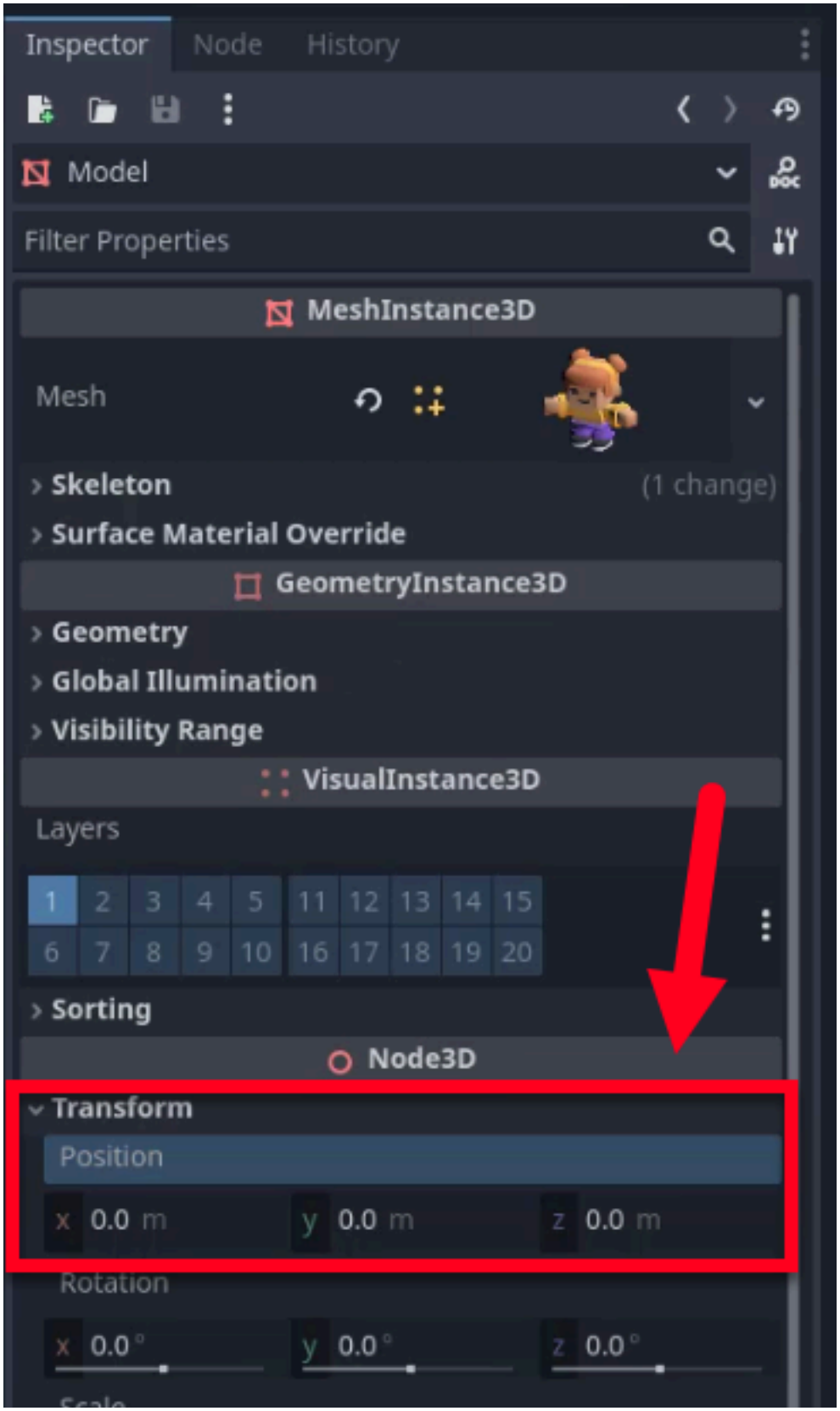
A CharacterBody3D is invisible by default: it only handles physics and collision. To give the player something to look at, we need to attach a 3D

model as a child node. This will be the character that players see moving through the world.

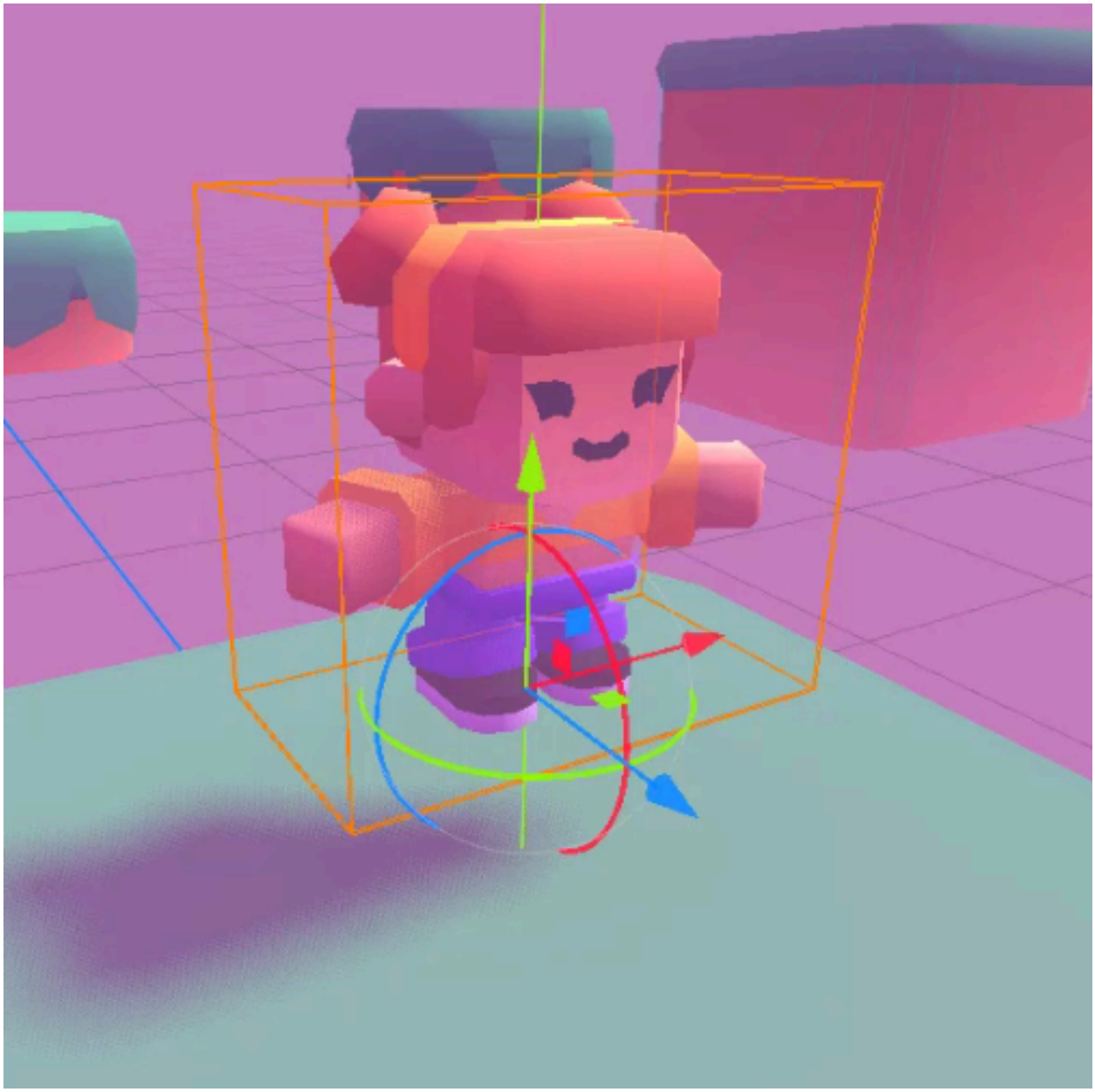
Navigate to your models folder in the FileSystem dock, specifically `Models/characters`. Drag and drop your chosen model (e.g., `character-female-b.obj`) into the scene as a child of the `Player` node. Once added, we can rename the model node to `Model`.


Scene
Import
Filter: name, t:type, g:
Main
Camera3D
Environment
Platforms
Player
Model
FileSystem
res://Models/Characters/charact
Filter Files
Models
Characters
Textures
character-female-a.obj
character-female-b.obj
character-female-c.obj
character-female-d.obj

To ensure the visual aligns perfectly with the physics body, set the model's transform position to `(0, 0, 0)` in the Inspector.


Inspector
Node
History
Model
Filter Properties
MeshInstance3D
Mesh
Skeleton
(1 change)
Surface Material Override
GeometryInstance3D
Geometry
Global Illumination
Visibility Range
VisualInstance3D
Layers
1 2 3 4 5 11 12 13 14 15
6 7 8 9 10 16 17 18 19 20
Sorting
Node3D
Transform
Position
x 0.0 m
y 0.0 m
z 0.0 m
Rotation
x 0.0 °
y 0.0 °
z 0.0 °

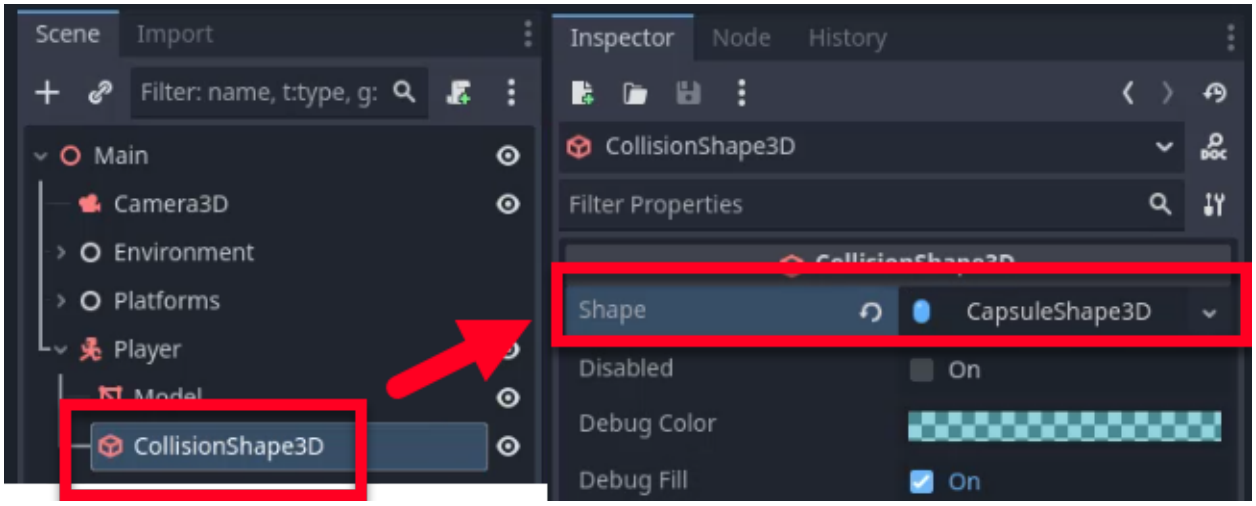
Our player now has a visual representation in the scene.



Adding a Collision Shape

The physics engine needs to know the shape of our player to calculate collisions. Without a collision shape, the CharacterBody3D has no physical presence, and the player would fall straight through the floor.

Right-click on the `Player` node and add a child node of type `CollisionShape3D`. In the Inspector, we want to set the **Shape** property to `New CapsuleShape3D`.

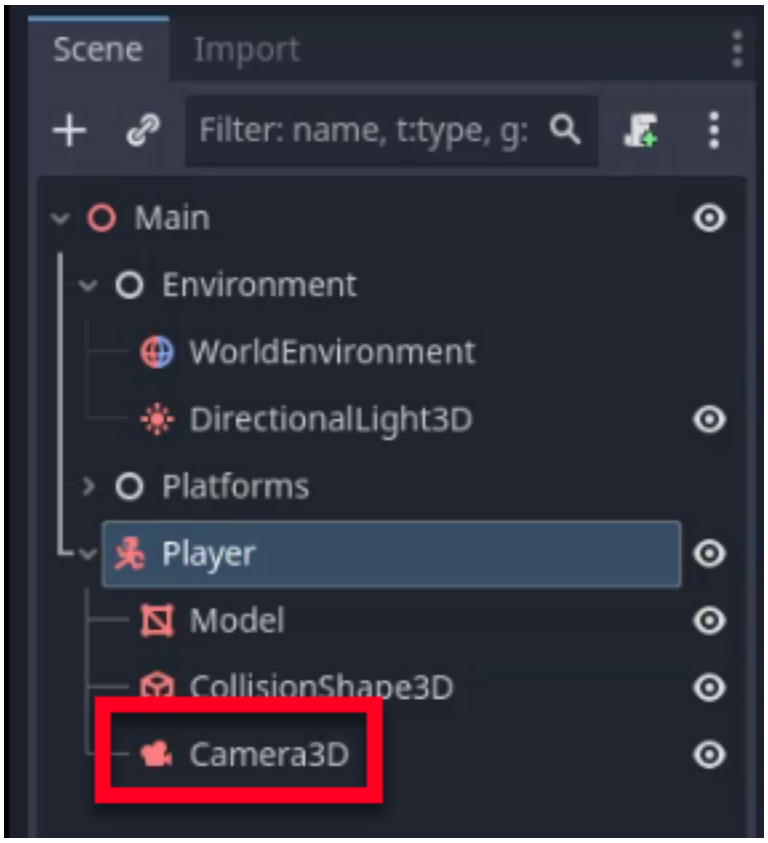


Adjust the radius and height of the capsule to fit the player model appropriately. We want to ensure the capsule is positioned so that the player's feet are at the bottom of the capsule. You can use the side view to fine-tune the positioning. If the view is obscured by a purple overlay, you may need to temporarily disable the fog in the world environment settings.

Setting Up the Camera

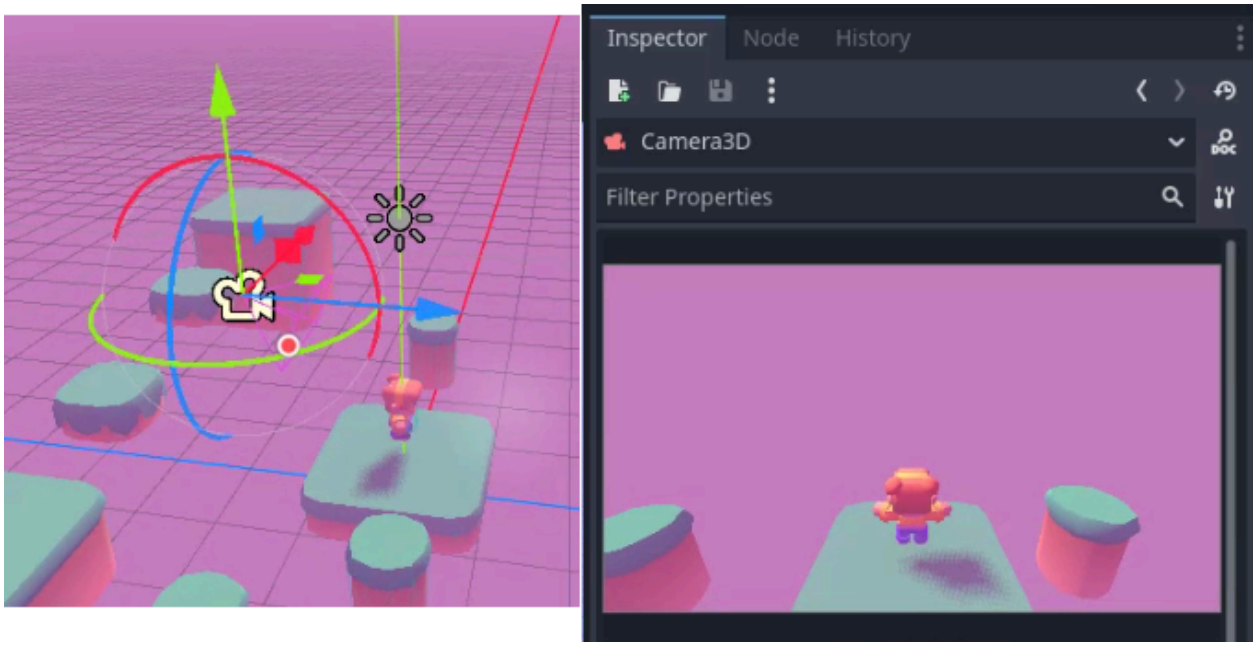
In Godot, a child node inherits the transform of its parent. By making the camera a child of the Player node, the camera will automatically follow the player wherever they go, without any extra code.

Drag the existing camera node in your scene to be a child of the `Player` node.

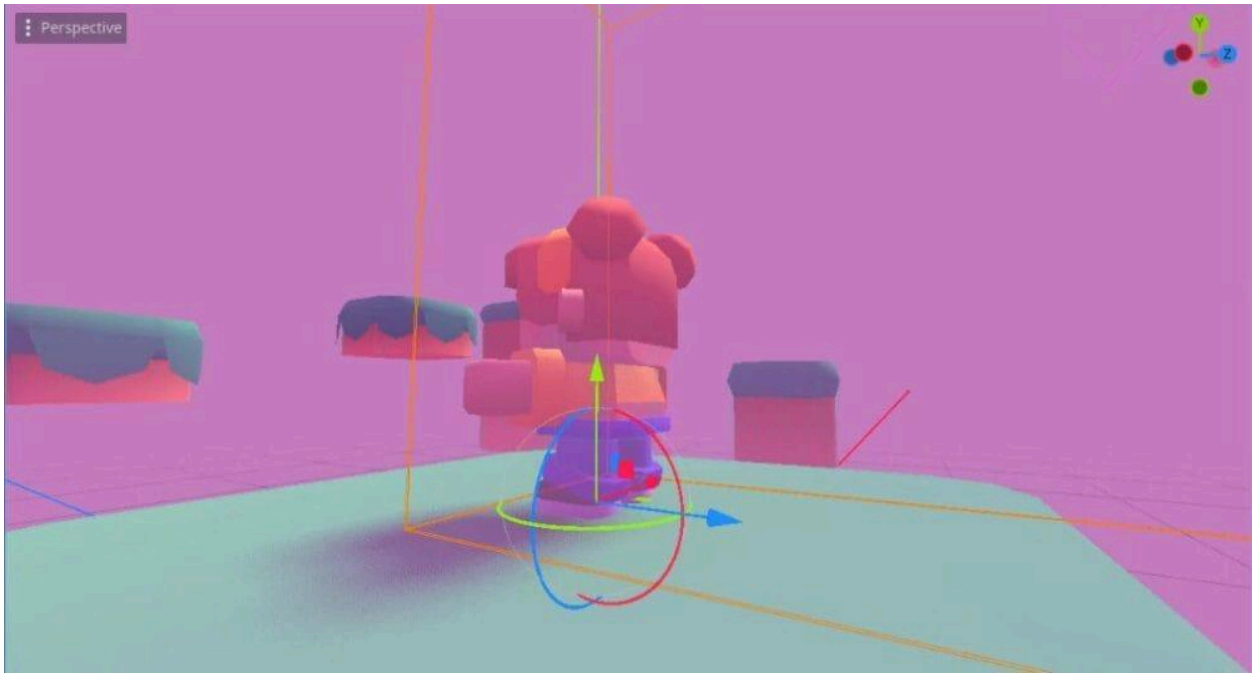


Now, adjust the camera's transform:

1. Rotate the camera around its Y-axis to `180` degrees so it faces the direction the player is moving.
2. Position the camera behind and slightly above the player for a proper third-person view.



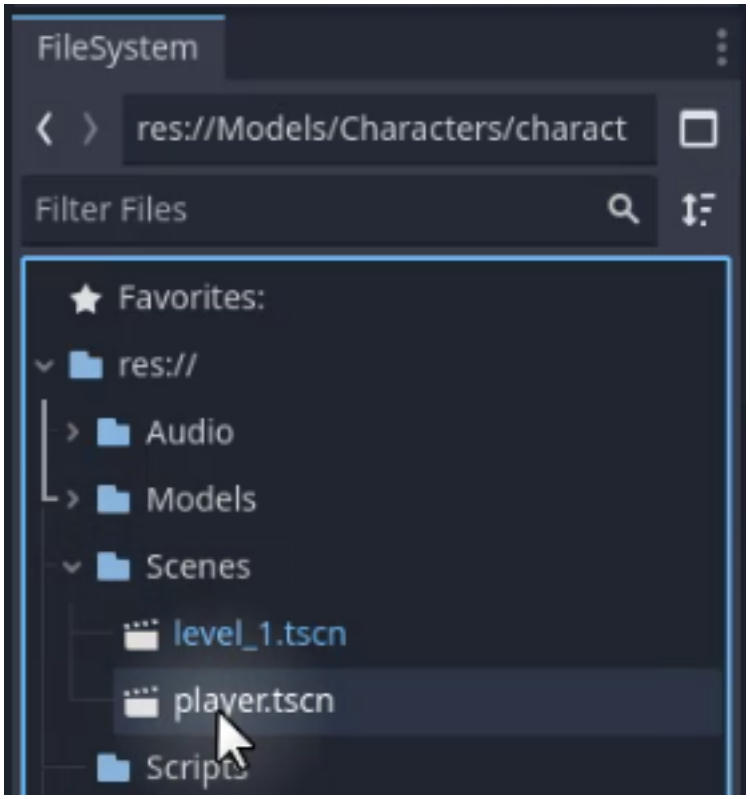
Finally, rotate the `Player` node itself around its Y-axis to `180` degrees so that it faces the level. Then, adjust the player's position to ensure it is standing firmly on the ground.



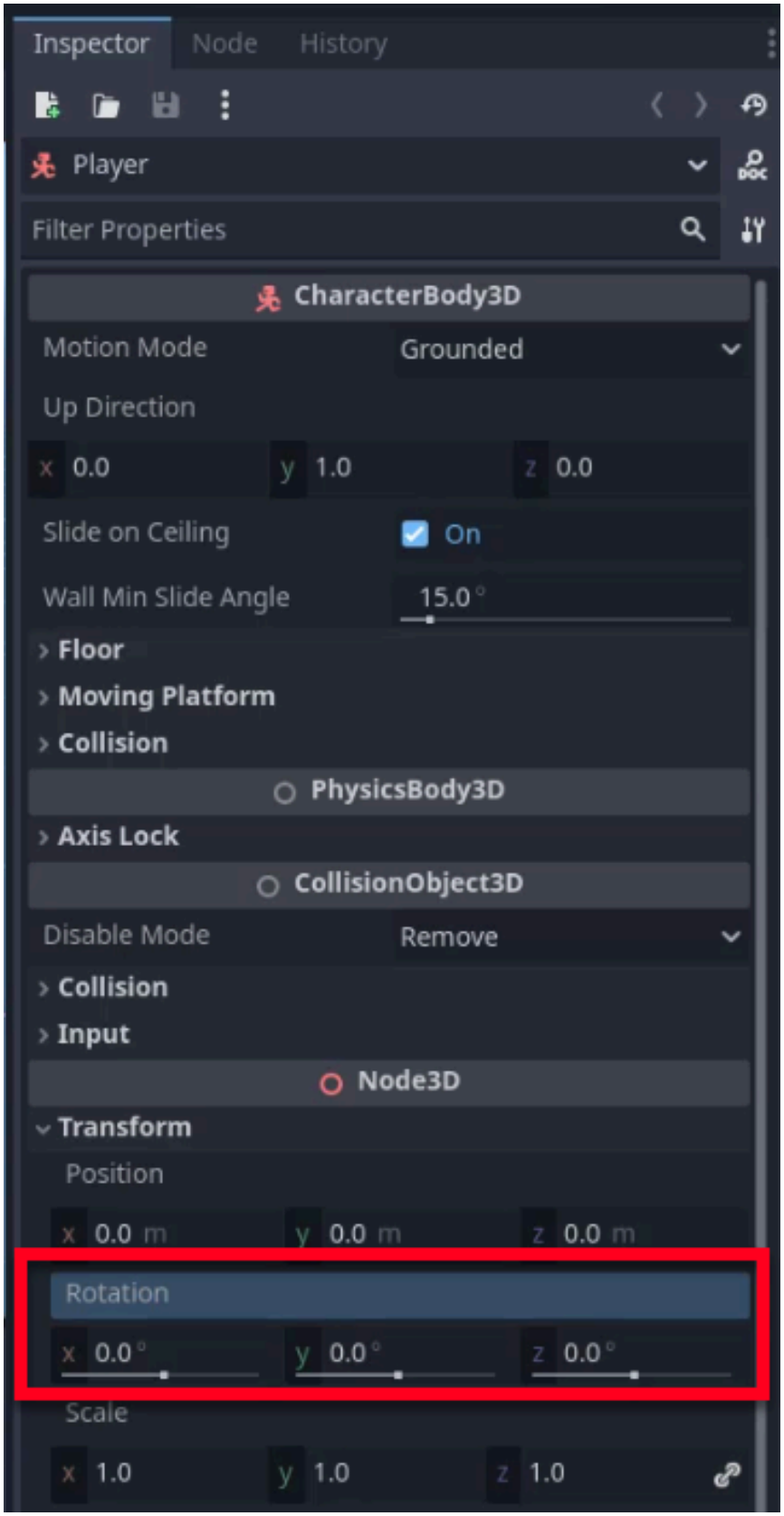
Saving the Player Scene

To reuse the player setup in multiple levels, we should save it as a separate scene.

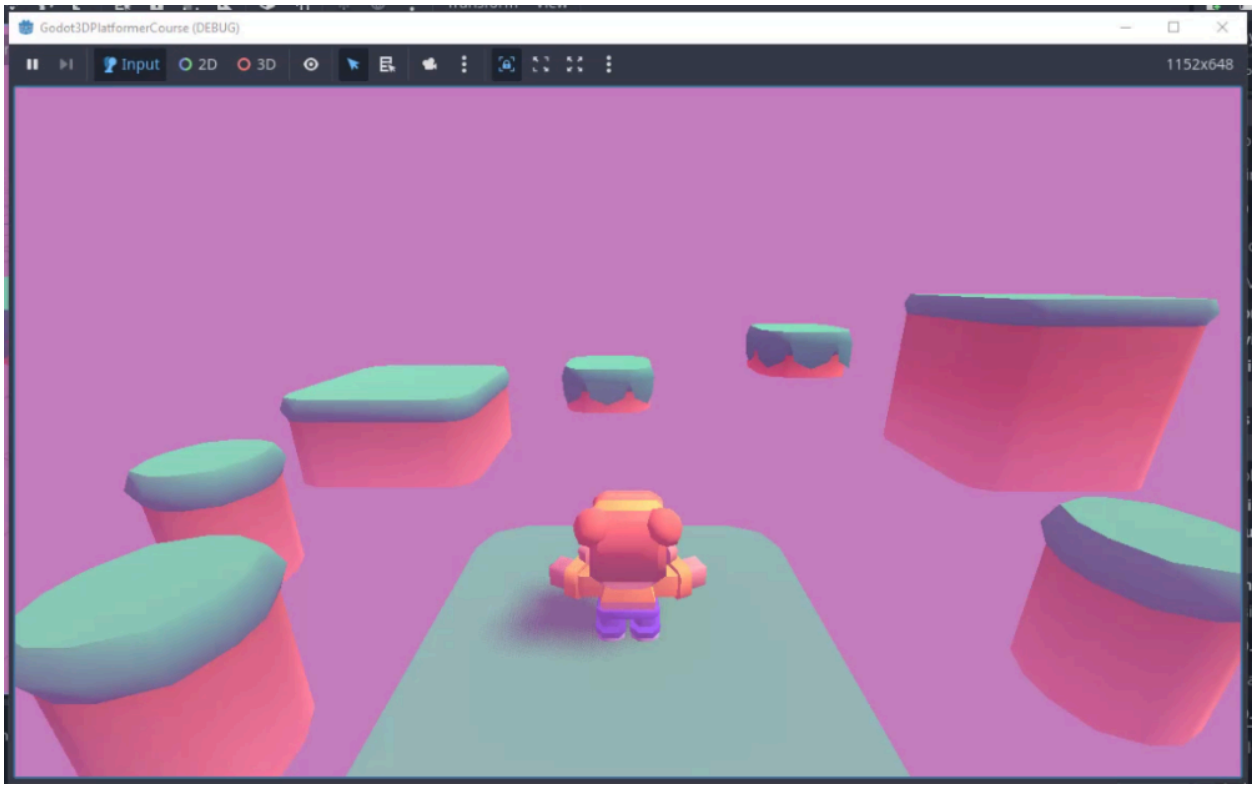
To do so quickly, drag the `Player` node from the Scene dock into your `Scenes` folder in the FileSystem dock. This will automatically save the branch as a new scene file. Name it `player.tscn`.



There is one adjustment we still have to make, so open the new `player.tscn` scene again. We want to ensure the player root node's rotation is set to zero, which prevents offset issues when instancing the player in other levels.


Inspector
Node
History
Player
Filter Properties
CharacterBody3D
Motion Mode
Grounded
Up Direction
x 0.0
y 1.0
z 0.0
Slide on Ceiling
On
Wall Min Slide Angle
15.0 °
Floor
Moving Platform
Collision
PhysicsBody3D
Axis Lock
CollisionObject3D
Disable Mode
Remove
Collision
Input
Node3D
Transform
Position
x 0.0 m
y 0.0 m
z 0.0 m
Rotation
x 0.0 °
y 0.0 °
z 0.0 °
Scale
x 1.0
y 1.0
z 1.0

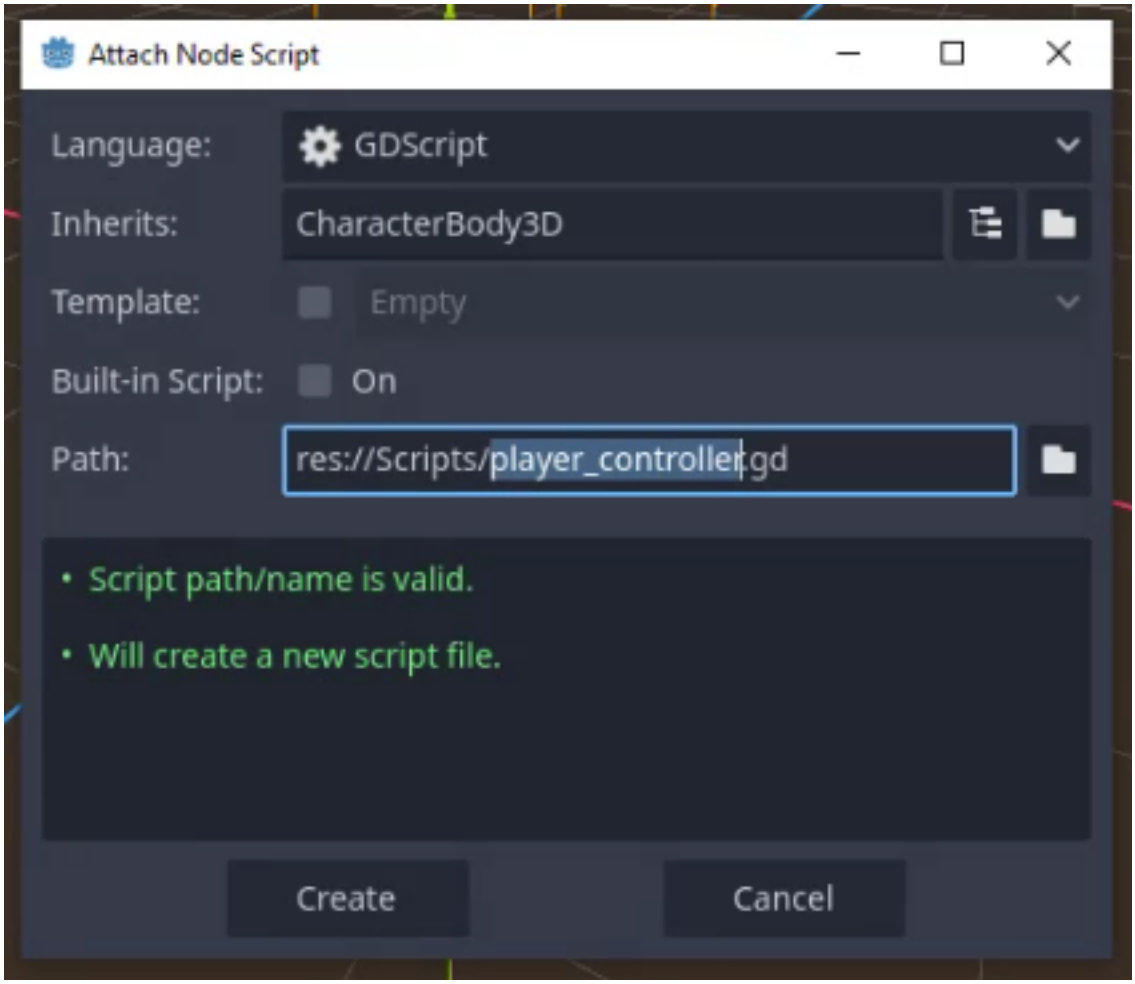
Back in the main level scene, you may need to adjust the rotation of the player instance to ensure it faces the level correctly again.



Adding a Script to the Player

Finally, we need to add a script to control the player's behavior.

1. Select the `Player` root node in the player scene.
2. In the Inspector, Click the "Attach Script" button and select "New Script".
3. Save the script in the `Scripts` folder and name it `player_controller.gd`.



Input Map Configuration

Before we start coding the movement logic, we need to set up the input map. Inputs are essential for mapping specific keys to corresponding movement actions in your game, such as moving forward with the 'W' key or jumping with the spacebar. Properly configuring these inputs ensures your script knows which keys trigger which actions.

In Godot, inputs are managed through **Actions**. Actions are what your script will listen for, rather than specific keys. For example, your code will listen for a 'jump' action instead of directly checking if the spacebar is pressed. This abstraction allows for greater flexibility, as you can associate multiple keys (like Spacebar and a gamepad button) with a single action.

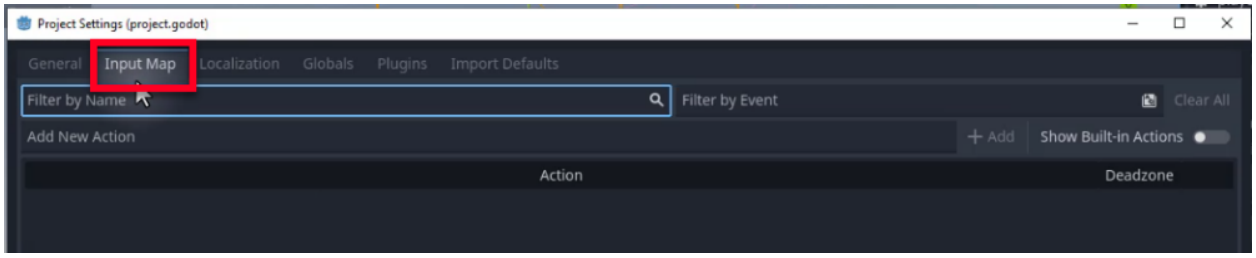
Godot Insight: The Input Map system

The Input Map is Godot's way of decoupling your game logic from specific hardware. Instead of hard-coding `KEY_SPACE` in your script, you define an action called "jump" and bind whatever keys or buttons you like to it. This means you can add gamepad support later without changing a single line of code, and players can remap controls without you writing a custom settings screen. In your scripts, you query actions with methods like `Input.is_action_pressed("jump")` and `Input.get_vector()`, which automatically handle all the bound keys behind the scenes.

Learn more: InputMap (https://docs.godotengine.org/en/4.6/classes/class_inputmap.html)

Setting Up Inputs

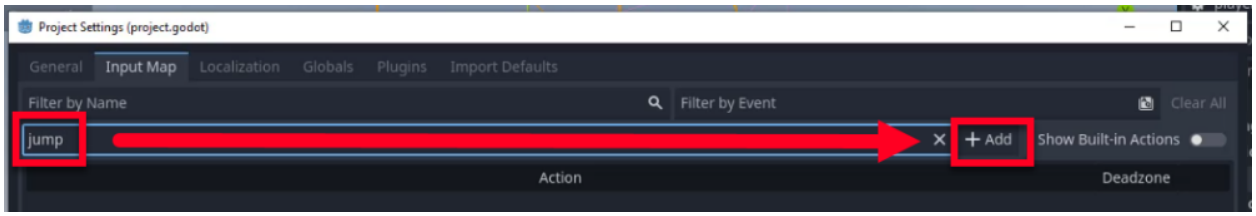
To set up your inputs, navigate to the top menu and select **Project** > **Project Settings**. In the window that appears, we want to select the **Input Map** tab.



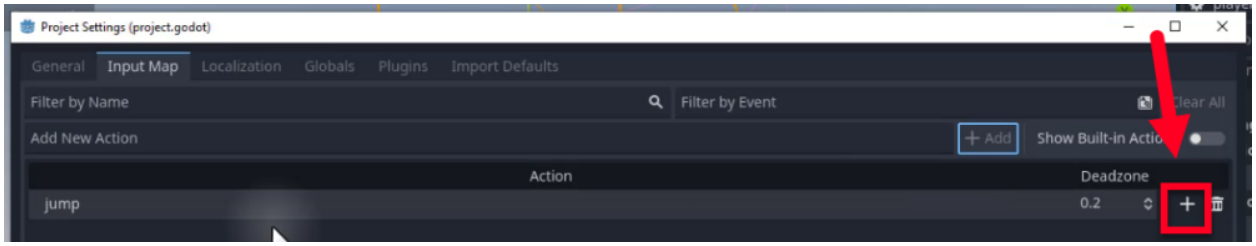
Creating Actions

We will now create actions and assign keys to them.

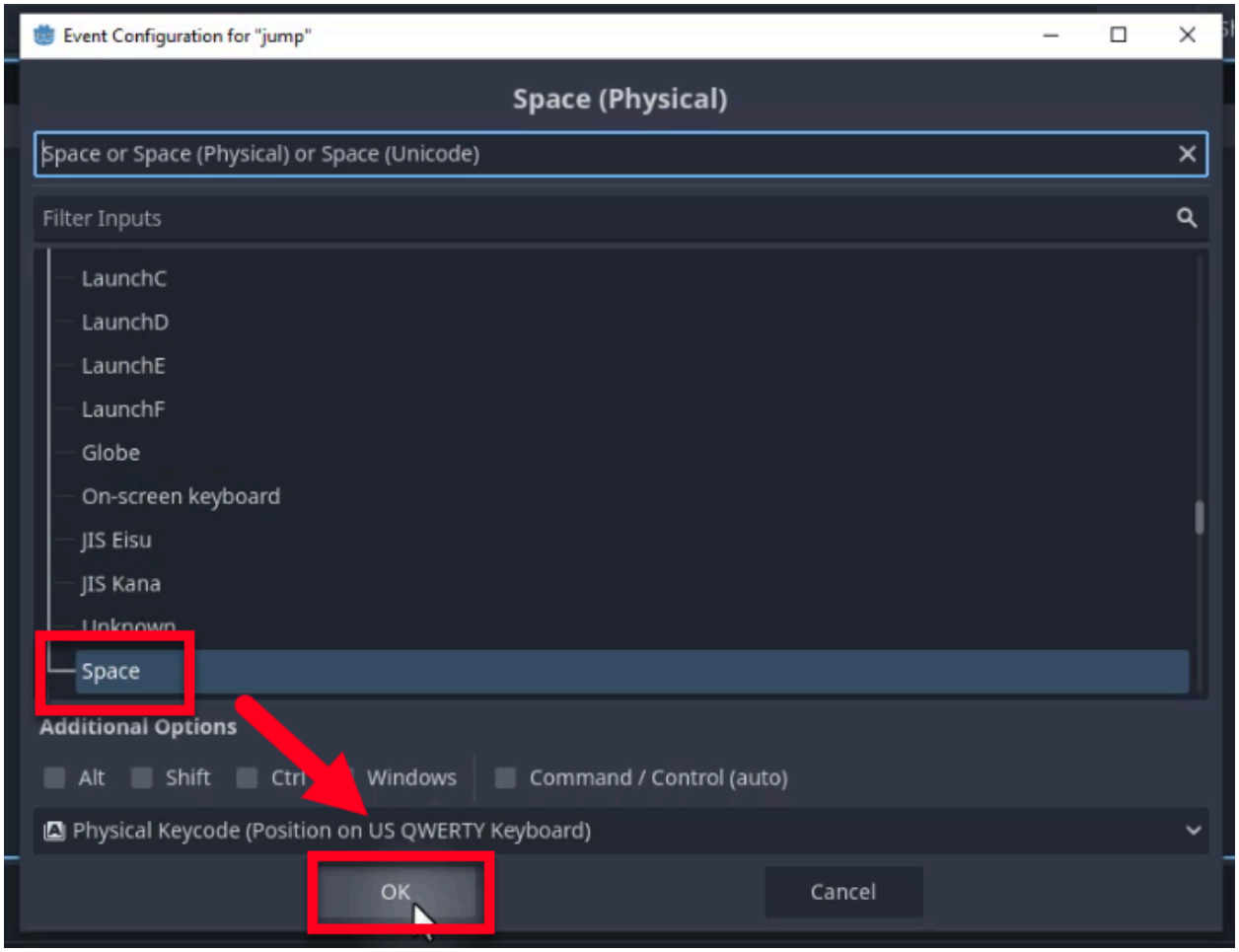
First, let's create a jump action. Type `jump` into the "Add New Action" field and click the **Add** button.



With the action created, we need to assign a key to it. To do so, click the **Plus** icon (+) next to the `jump` action to add a new event.



In the configuration window, press the Spacebar on your keyboard or search for “Space”. Select it and click **OK** to associate the spacebar with the ‘jump’ action.



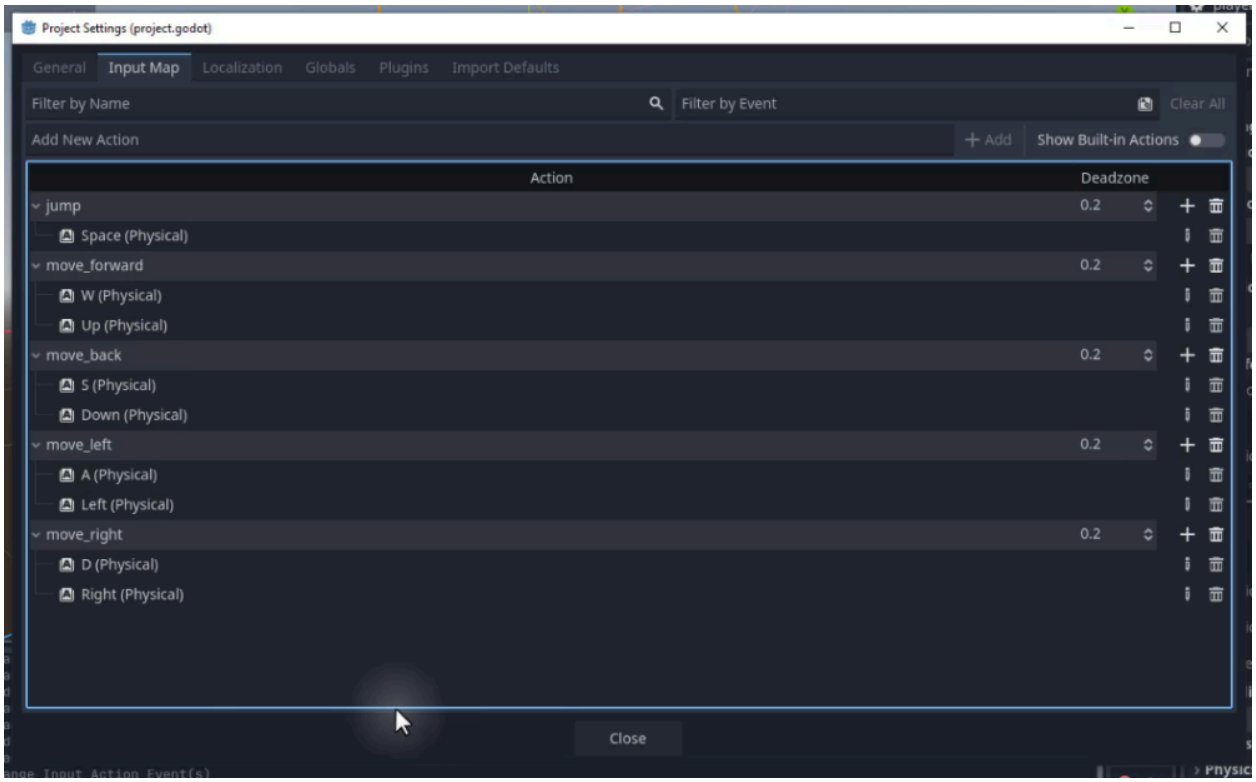
Adding Movement Actions

We also need actions for movement. Follow the same steps to create the following actions and assign the corresponding keys:

- **move_forward**: Associate with ‘W’ and the Up Arrow key.

- **move_back**: Associate with ‘S’ and the Down Arrow key.
- **move_left**: Associate with ‘A’ and the Left Arrow key.
- **move_right**: Associate with ‘D’ and the Right Arrow key.

Remember, you can add multiple keys to a single action. This is useful for supporting both WASD and Arrow keys simultaneously. Likewise, if you would like to, you can also map these actions to a gamepad - it will work out of the box without us having to make any code adjustments later in the project.



With our player scene created and our inputs configured, we are now ready to start scripting. In the next chapter, we will dive into the code and implement the logic that brings our player to life.

Player Movement and Animation

In this chapter, we will bring our player character to life by implementing movement, physics, and animations. We will begin by creating a `PlayerController` script to manage inputs, gravity, and jumping. Afterward, we will enhance the visual experience by adding procedural animations that make the character rotate towards their movement direction and bob up and down while walking.

The Player Controller Script

To get started, we need a script to handle the player's logic. Select your **Player** node in the scene tree. If a script does not already exist from the last chapter, create a new one named `player_controller.gd` and attach it to the Player node.

Defining Movement Variables

We will begin by defining a few essential variables that will control our player's movement capabilities:

- `move_speed`: Defines how many units the player moves per second.
- `jump_force`: Defines the upward force applied when the player jumps.
- `gravity`: Defines the downward force applied to the player to simulate weight.
- `camera`: Holds a reference to our camera node, which we will use later for movement orientation relative to the view.

Let's add the movement speed variable first:

```
extends CharacterBody3D

@export var move_speed : float = 3.0
```

The `@export` annotation allows us to modify this variable directly from the Godot Inspector, making it easier to tweak values without changing the code. This is one of GDScript's most useful features for game development:

you can fine-tune speed, jump force, and other values in real time without reopening the script.

Next, add the variables for jumping and gravity. Likewise, we'll want to be able to adjust them in the Inspector later:

```
@export var jump_force : float = 8.0
@export var gravity : float = 20.0
```

Finally, add the `camera` variable. We use the `@onready` annotation to ensure the node reference is assigned only after the node and its children are fully ready in the scene tree:

```
@onready var camera : Camera3D = $Camera3D
```

Setting Up the Physics Process

Godot provides a specific function for handling physics-related calculations called `_physics_process`. This function runs at a fixed frequency (typically 60 times per second), ensuring consistent physics behavior regardless of the frame rate.

Godot Insight: _physics_process vs _process

Godot gives you two main update loops. `_process(delta)` runs once per rendered frame, so its rate varies with your framerate. `_physics_process(delta)` runs at a fixed interval (60 Hz by default), which makes it the right place for anything involving physics: applying gravity, reading input for movement, and calling `move_and_slide()`. Using the fixed timestep means your character moves the same way whether the game runs at 30 FPS or 144 FPS.

As a rule of thumb: physics and movement go in `_physics_process`; visual effects like animations and UI updates go in `_process`.

Let's create the `_physics_process` function and outline the steps we need to implement:

```
func _physics_process(delta):
    # Apply gravity
    # Implement jump
    # Implement movement
```

Implementing Movement

We will start by implementing the basic movement logic. This involves capturing the player's input and translating it into movement in the game world.

First, we retrieve the input vector using the actions we defined in the Input Map:

```
var move_input : Vector2 = -Input.get_vector("move_right",
"move_left", "move_back", "move_forward")
```

Note that we negate the vector because in 3D space, "forward" is often represented as negative Z.

Next, we convert this 2D input vector (x, y) into a 3D direction vector (x, 0, z), as we do not want the player to move up or down based on arrow keys:

```
var move_dir : Vector3 = Vector3(move_input.x, 0, move_input.y)
```

Now, we apply this direction to the player's velocity.

```
velocity.x = move_dir.x
velocity.z = move_dir.z
```

Finally, we call `move_and_slide()`, a built-in function for `CharacterBody3D` that moves the body based on its velocity and handles collisions with walls and floors automatically. After this call, helper methods like `is_on_floor()` and `is_on_wall()` become available, which we will use shortly for jumping:

```
move_and_slide()
```

Applying Movement Speed

If you were to run the game now, the player would move, but very slowly (at 1 unit per second). To fix this, we need to incorporate our `move_speed` variable.

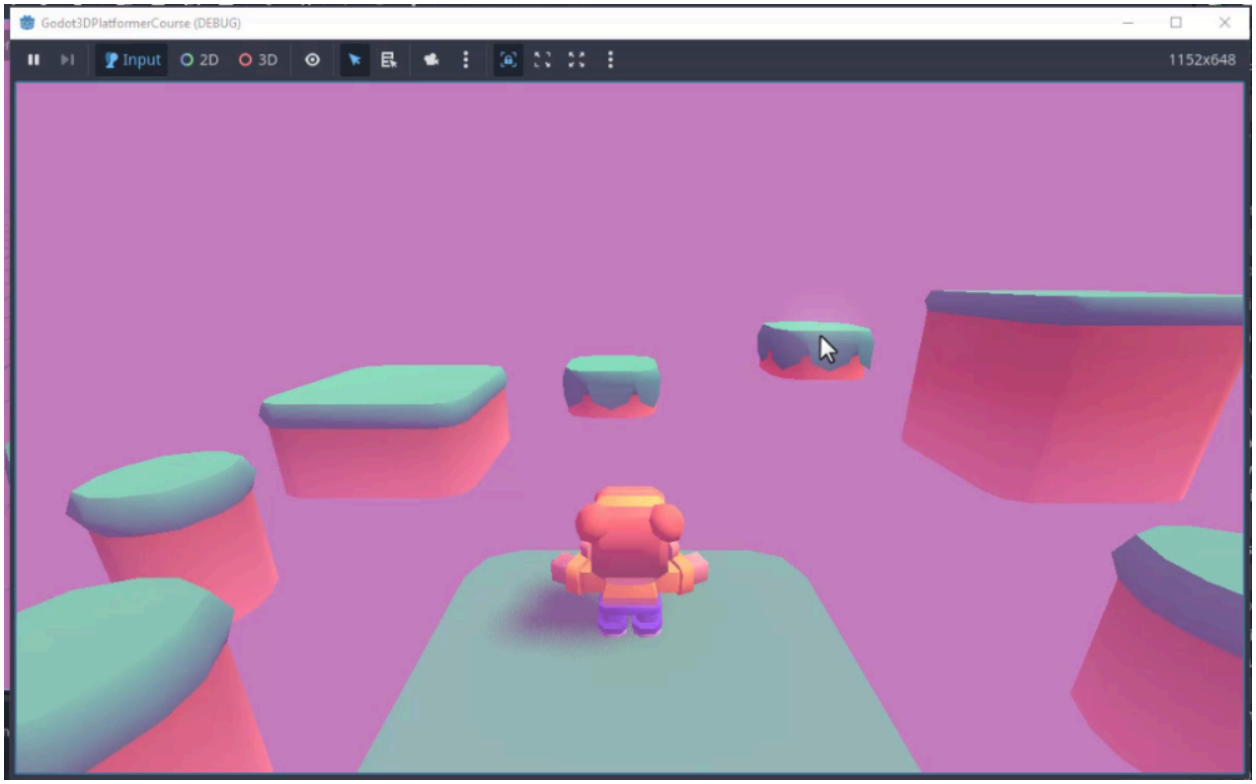
Update the velocity assignment lines to multiply the direction by the speed:

```
velocity.x = move_dir.x * move_speed
velocity.z = move_dir.z * move_speed
```

Your `_physics_process` function should now look like this:

```
func _physics_process(delta):
    # Apply gravity
    # Implement jump

    # Negate the vector because Godot's forward is -Z in 3D
    var move_input : Vector2 = -Input.get_vector("move_right",
"move_left", "move_back", "move_forward")
    var move_dir : Vector3 = Vector3(move_input.x, 0, move_input.y)

    velocity.x = move_dir.x * move_speed
    velocity.z = move_dir.z * move_speed

    move_and_slide()
```

You should now be able to move your player around the platform, as shown in the image below.



Gravity and Jumping

Currently, our player floats if they walk off a platform, and they cannot jump. Let's implement gravity and jumping to complete the physics controller.

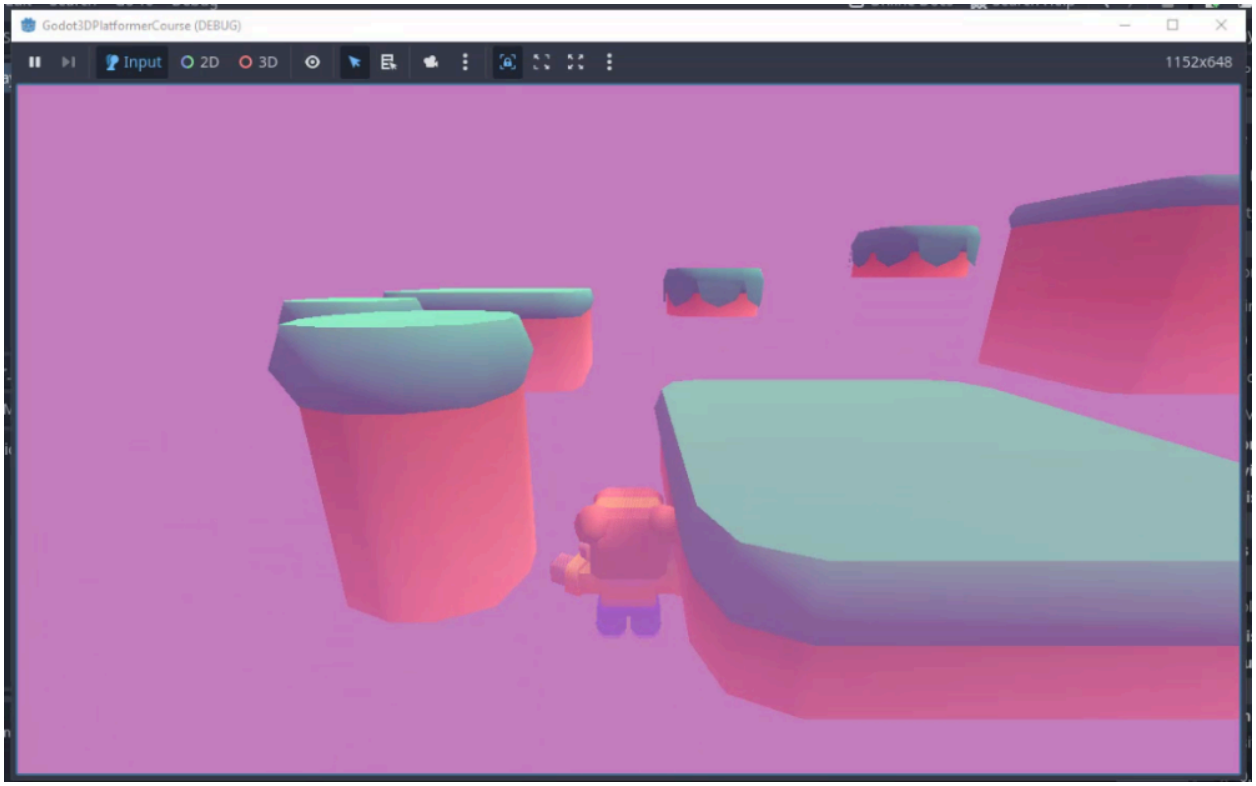
Implementing Gravity

To make the player fall, we need to apply a downward force to the Y-axis of our velocity. We will modify the `_physics_process` function to subtract gravity over time.

To add gravity, find the comment `# Apply gravity` and add the following code beneath it:

```
velocity.y -= gravity * delta
```

This line decreases the vertical velocity by the `gravity` value multiplied by `delta` (the time elapsed since the last frame), causing the player to accelerate downwards.



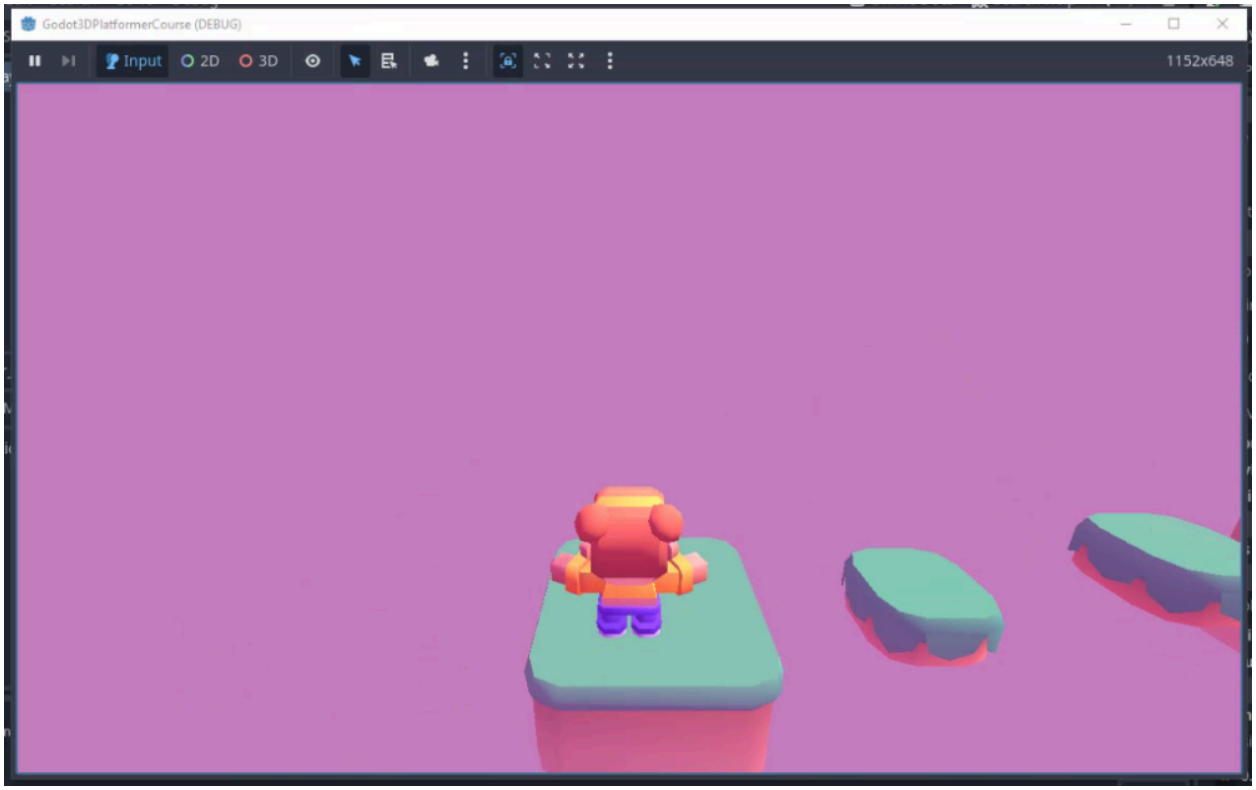
Implementing Jump

Next, we will add the ability to jump. We want the player to jump only when the jump button is pressed *and* they are currently standing on the floor.

Let's add the following code below the gravity implementation:

```
# jump
if Input.is_action_pressed("jump") and is_on_floor():
    velocity.y = jump_force
```

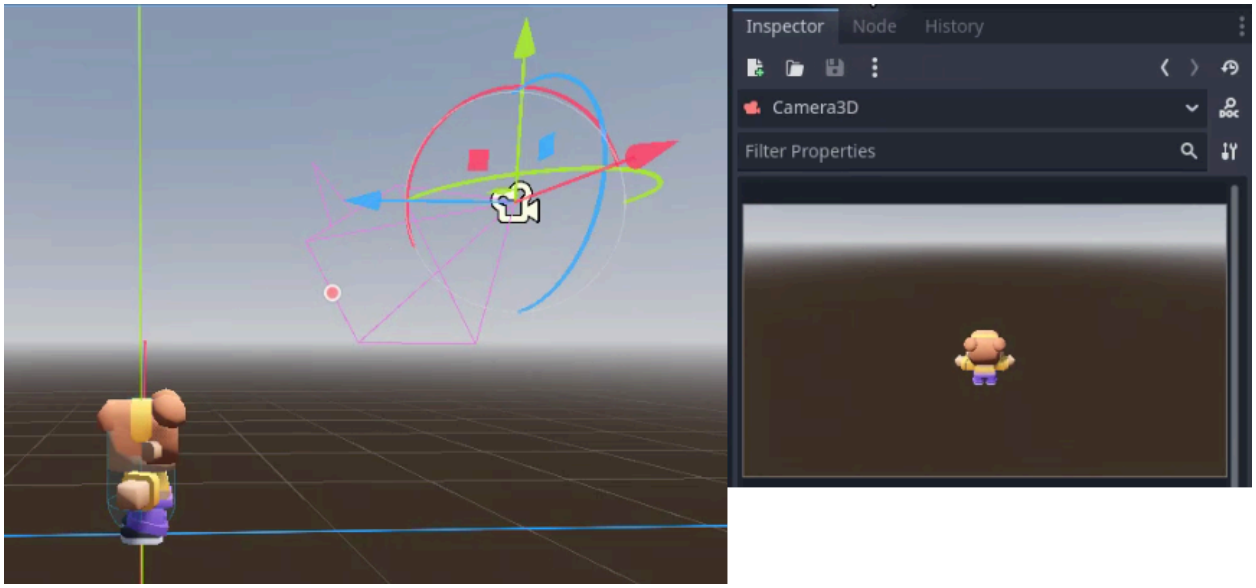
The `is_on_floor()` function is a helper provided by `CharacterBody3D` that returns `true` if the body collided with the floor during the last `move_and_slide()` call. If both conditions are met, we set the vertical velocity to `jump_force`, launching the player upwards.



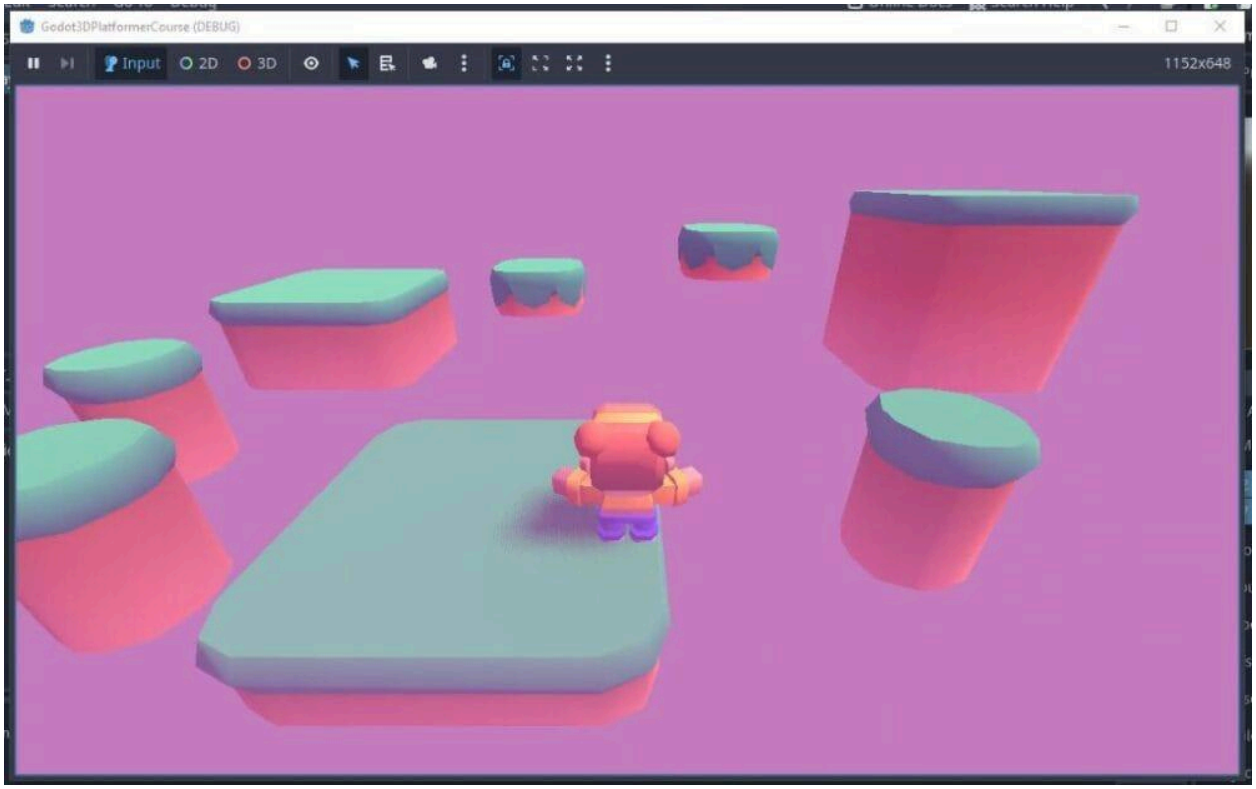
Adjusting the Camera

With movement working, you might notice that judging distances between platforms is difficult with the current camera angle. Let's adjust it to provide a better perspective.

Open the **Player** scene and select the **Camera3D** node. To improve visibility, manually adjust the rotation (tilt) of the camera downwards so the ground and platforms are more visible.



The goal is to help the player understand where they are jumping and judge distances correctly.



Player Visuals

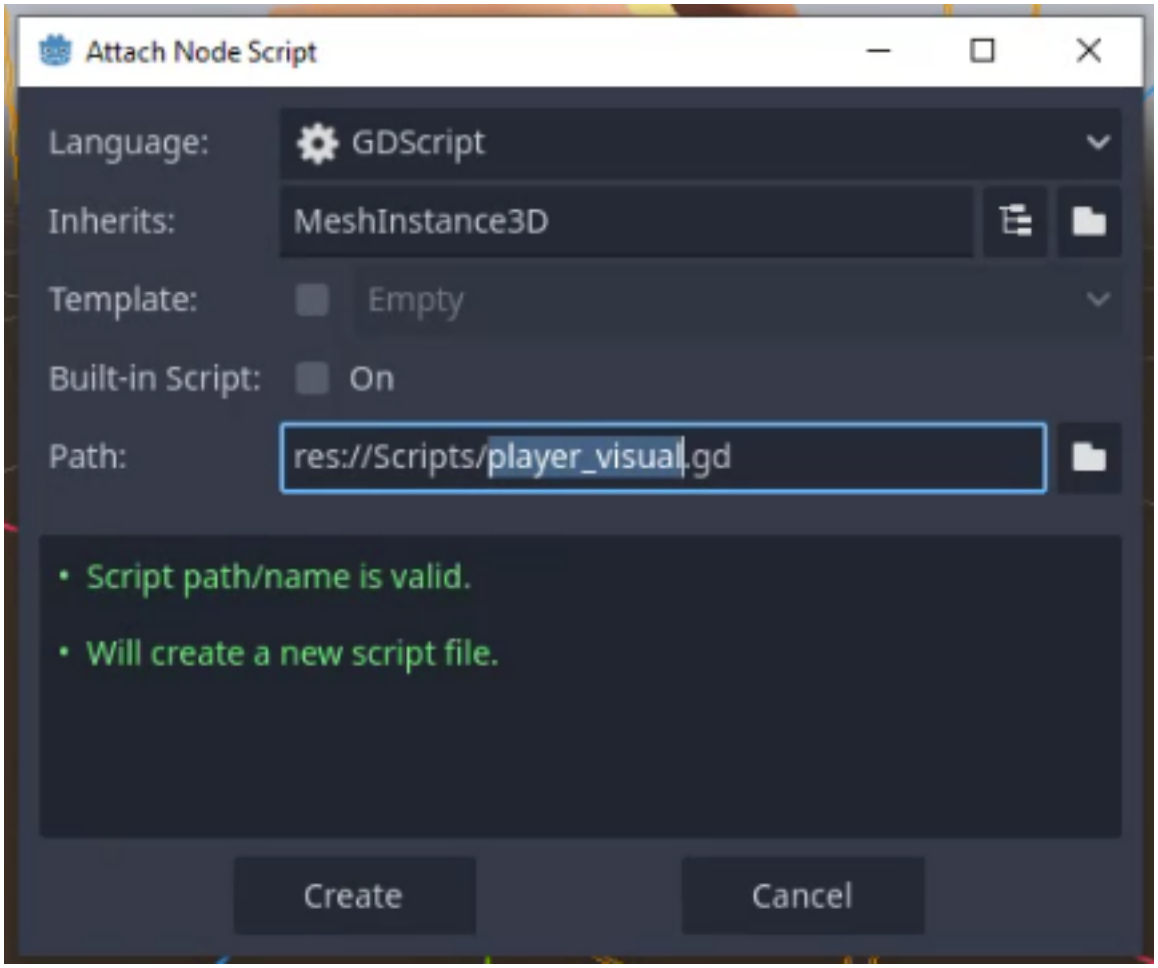
Now that our character moves correctly, we need to improve the visuals. Currently, the player model likely faces one direction regardless of movement, and it slides across the floor without any animation. We will implement two features to fix this:

1. **Smooth Rotation:** The character will rotate to face the direction of movement.
2. **Movement Bob:** The character will bob up and down slightly to simulate walking steps.

Setting Up the Visual Script

We will create a dedicated script for the player's model to handle these visual effects.

1. In the **Player** scene, select the **Model** node (the MeshInstance3D child).
2. Attach a new script named `player_visual.gd` and save it in the `Scripts` folder.



Defining Variables

In the new script, we need variables to control the rotation speed and reference the parent player node (to access velocity).

```
extends MeshInstance3D

@export var rotate_rate : float = 20.0
var target_y_rot : float = 0

@onready var player : CharacterBody3D = get_parent()
```

- `rotate_rate`: Controls how quickly the player rotates to face the new direction.
- `target_y_rot`: Stores the desired Y-axis rotation angle.

- `player`: A reference to the parent node, allowing us to read the player's velocity.

Creating the Process Loop

We will use the `_process` function (which runs every frame) to update our visuals. We will split the logic into two separate functions for clarity.

```
func _process(delta):
    _smooth_rotation(delta)
    _move_bob(delta)

func _smooth_rotation(delta):
    pass

func _move_bob(delta):
    pass
```

Implementing Smooth Rotation

To make the player face the direction they are moving, we can calculate the angle of the velocity vector.

Update the `_smooth_rotation` function as follows:

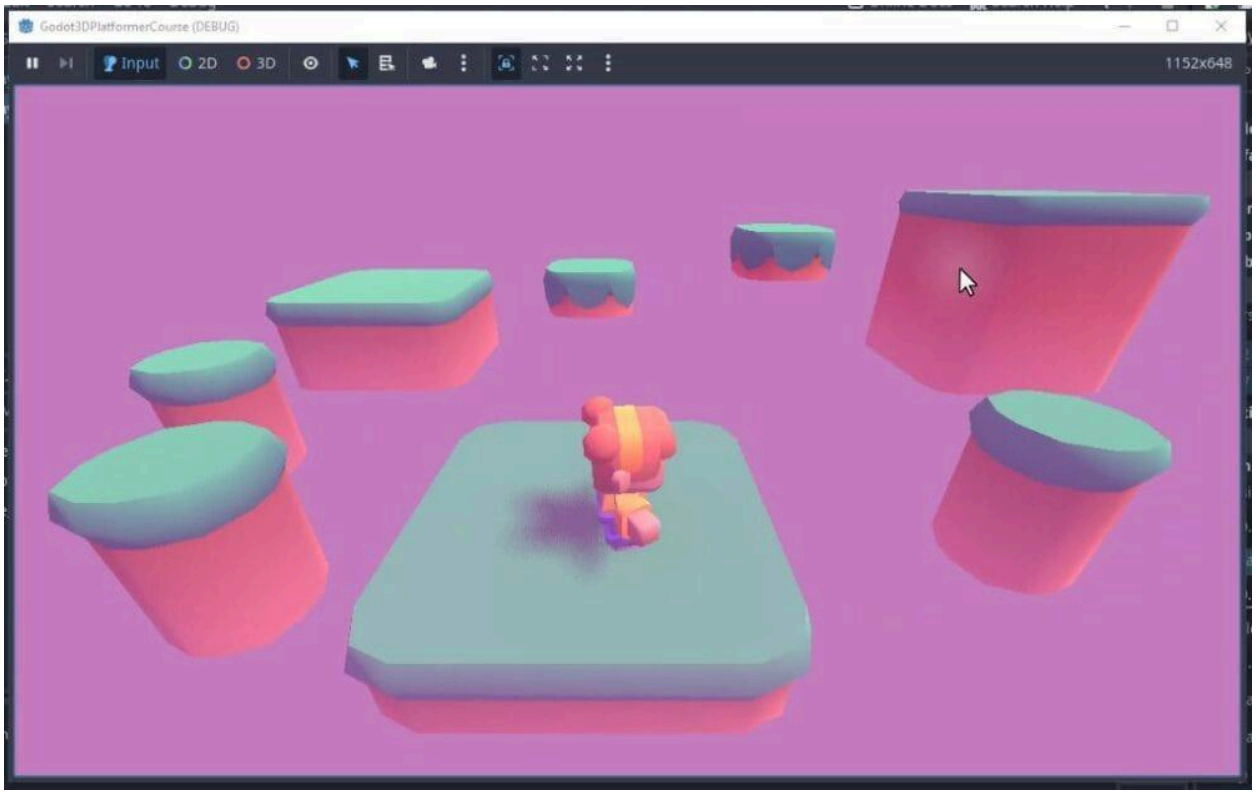
```
func _smooth_rotation(delta):
    # Negate velocity to match the model's forward orientation
    var vel = -player.velocity

    if vel.x != 0 or vel.z != 0:
        target_y_rot = atan2(vel.x, vel.z)

    # lerp_angle handles the wraparound at 360 degrees smoothly
    rotation.y = lerp_angle(rotation.y, target_y_rot, rotate_rate *
delta)
```

Here is what this code does:

1. We get the player's velocity. We negate it (`-player.velocity`) because the model's forward face is oriented differently than the engine's default forward vector.

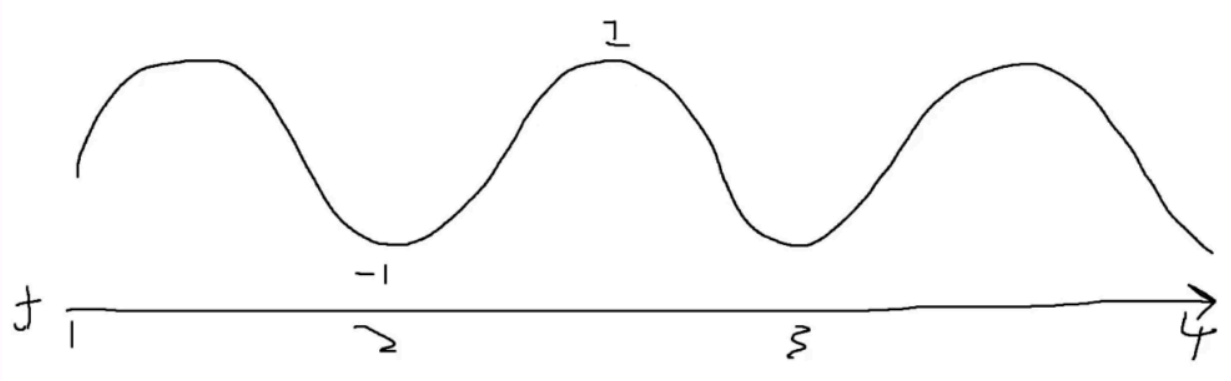
2. We check if the player is moving (`vel.x` or `vel.z` is not zero).
3. We use `atan2(x, z)` to calculate the angle in radians that corresponds to the movement vector.
4. We use `lerp_angle` to smoothly rotate the current `rotation.y` towards the `target_y_rot`. This prevents the character from snapping instantly to the new direction.

If you test the scene now, the player model should smoothly turn to face the direction you are moving.



Implementing Movement Bob

To simulate walking, we will scale the player model up and down slightly using a sine wave. A sine wave is a mathematical function that oscillates smoothly between -1 and 1 over time, making it perfect for looping animations like bobbing.



We will use Godot's system time to drive the sine wave. Update the `_move_bob` function as follows:

```gdscript
func _move_bob(delta):
    var move_speed = player.velocity.length()

    # Only bob when the player is moving on the ground
    if move_speed < 0.1 or not player.is_on_floor():
        scale.y = 1
        return

    var time = Time.get_unix_time_from_system()
    # Oscillate between 0.92 and 1.08 for a subtle squash-stretch
    var y_scale = 1 + (sin(time * 30) * 0.08)
    scale.y = y_scale
```

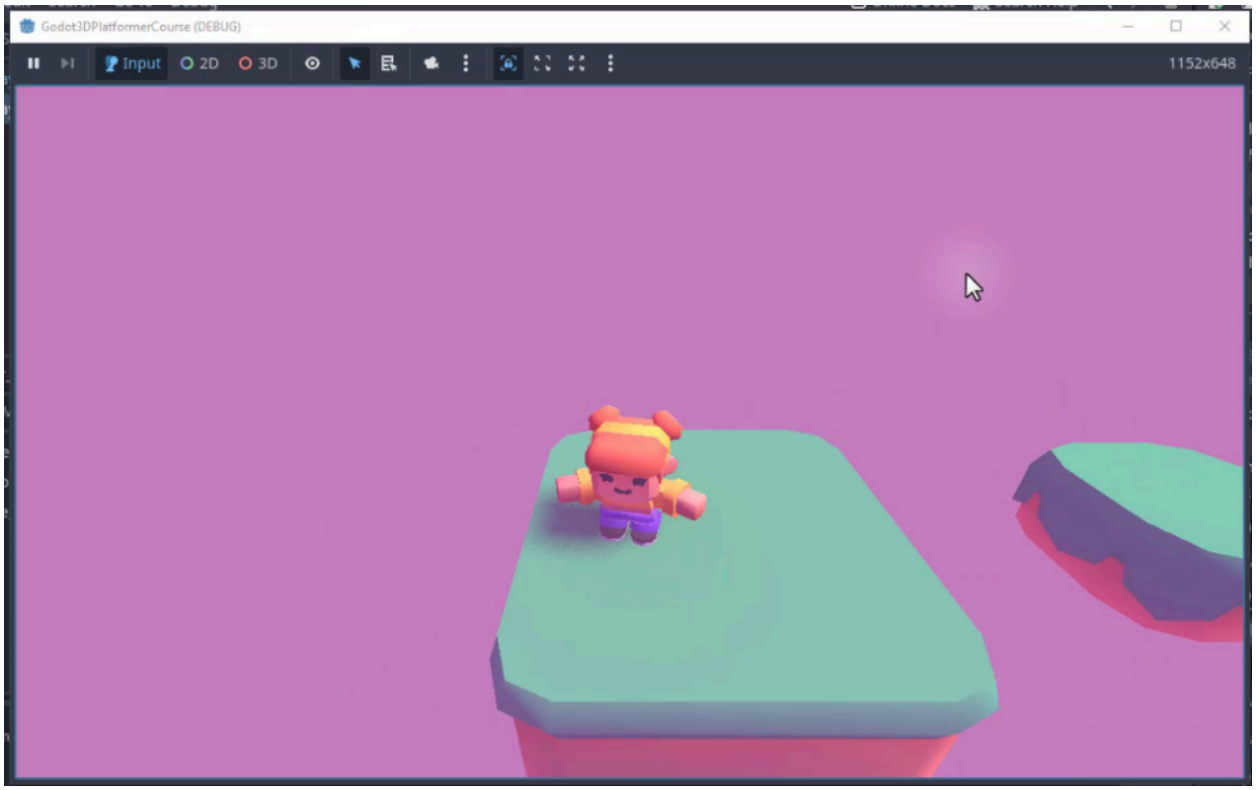
Let's break down this logic:

1. **Check Movement:** We calculate the speed. If the player is standing still (`move_speed < 0.1`) or is in the air (`not is_on_floor()`), we reset the scale to `1` and exit the function. This ensures the bobbing only happens when walking on the ground.
2. **Get Time:** We get the current time in seconds.
3. **Calculate Scale:** We use `sin(time * 30)`.
    - Multiplying time by `30` increases the **frequency** (how fast the bobbing is).
    - Multiplying the result by `0.08` decreases the **amplitude** (how strong the bobbing is).

    - Adding `1` ensures the scale oscillates around 1 (e.g., between 0.92 and 1.08) rather than around 0.
4. **Apply Scale:** We apply the result to `scale.y`.

Testing the Visuals

Run your scene again. When you move the player, you should see them rotate smoothly towards their destination and bob up and down rhythmically, adding a nice layer of polish to the movement.



With our player movement and visuals complete, we have a solid foundation for gameplay. In the next chapter, we will introduce enemies to provide a challenge for our player.

Enemy AI and Combat Mechanics

In this chapter, we will introduce a threat to our game: the enemy. We will create a spinning saw blade that patrols back and forth between two points. Upon contact with the player, this enemy will deal damage, eventually leading to a “Game Over” state if the player takes too many hits.

We will cover the creation of the enemy scene, scripting its movement and rotation logic, and implementing a health system for the player.

Creating the Enemy Node

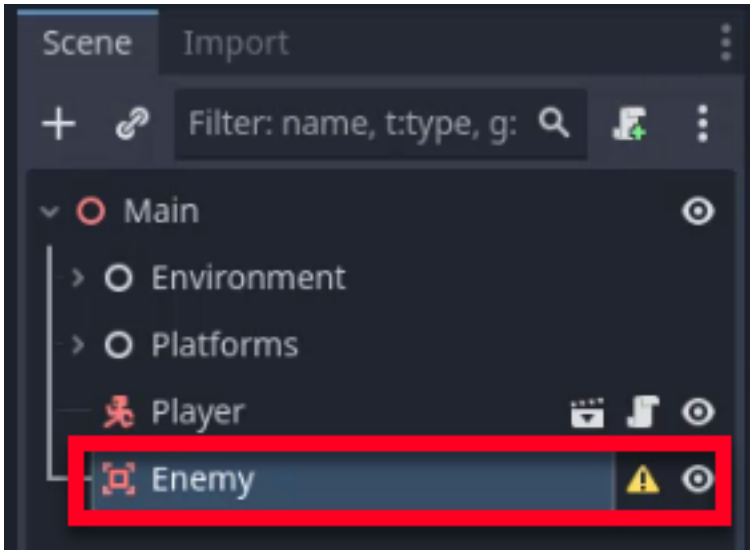
To begin, we need to create the enemy object. Unlike the player or platforms, the enemy does not need to be a solid physics body that blocks movement. Instead, it acts as a trigger that detects when the player enters its space. For this purpose, we will use an `Area3D` node.

Godot Insight: Area3D vs physics bodies

An Area3D does not block movement or participate in physics simulation. Its sole purpose is to detect when other collision objects enter or leave its region. This makes it perfect for triggers like damage zones, collectible pickups, and level exits. Internally, Area3D monitors overlapping bodies every physics step and fires the `body_entered` and `body_exited` signals when something crosses its boundary. You can also use it to override gravity or audio properties within a region.

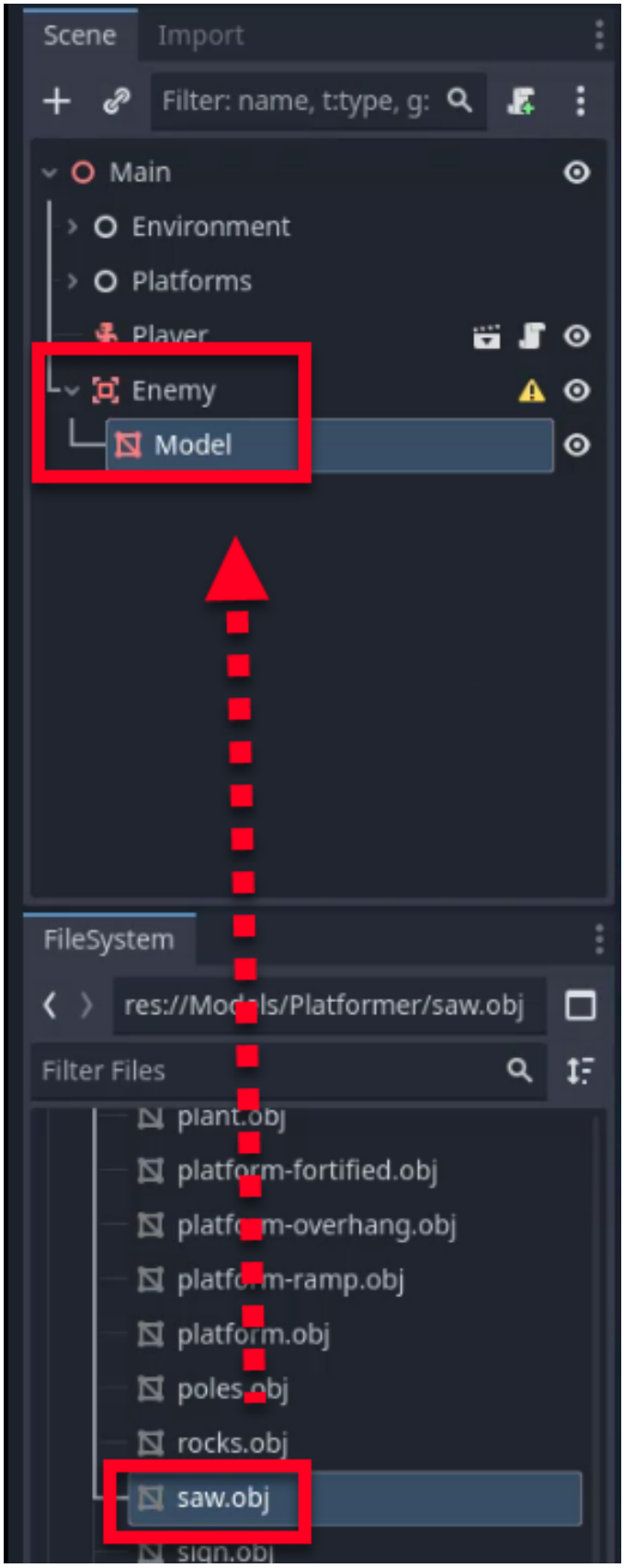
Learn more: Area3D (https://docs.godotengine.org/en/4.6/classes/class_area3d.html)

To begin, create a new `Area3D` node in your scene and rename it to `Enemy`.

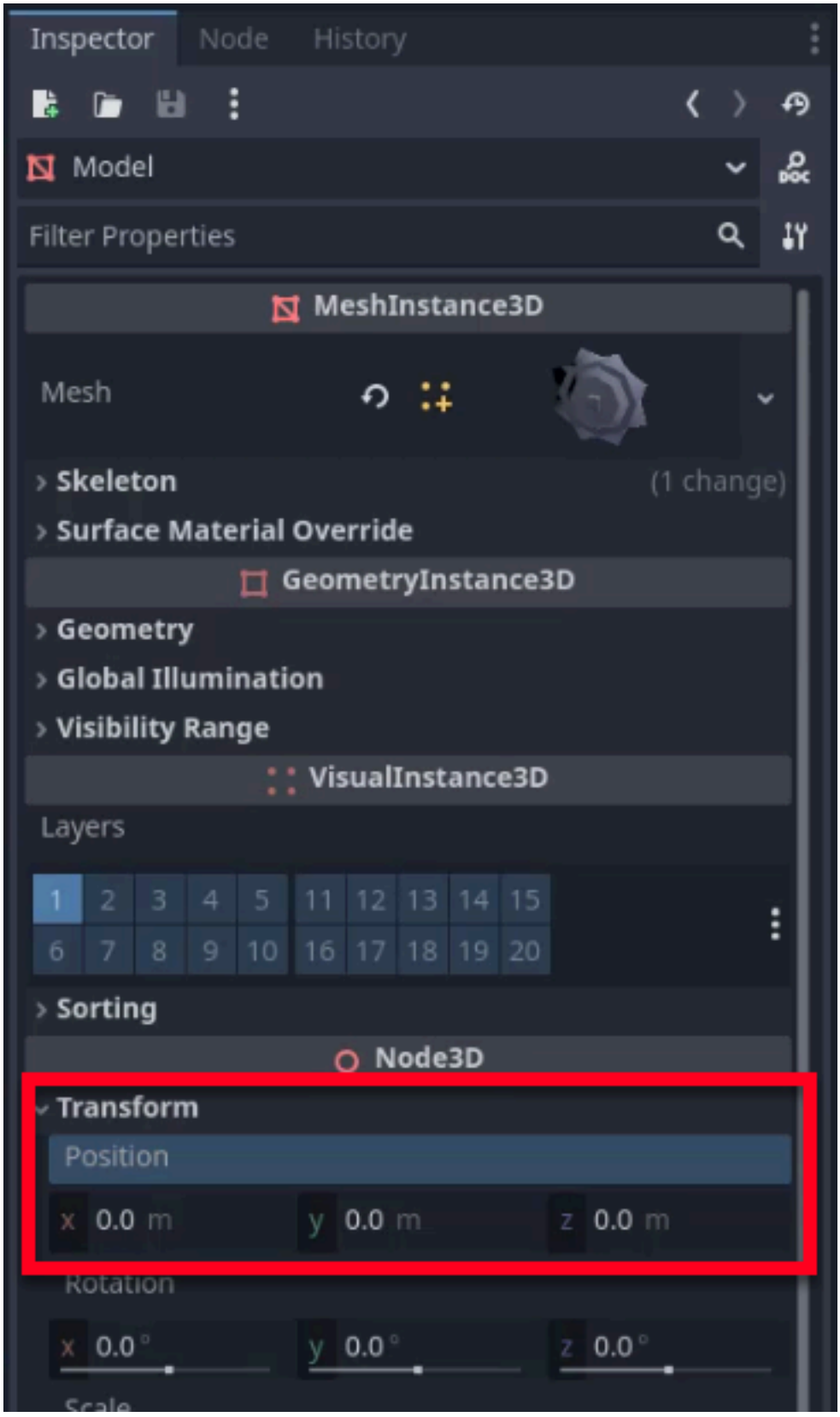


Adding the Visual Model

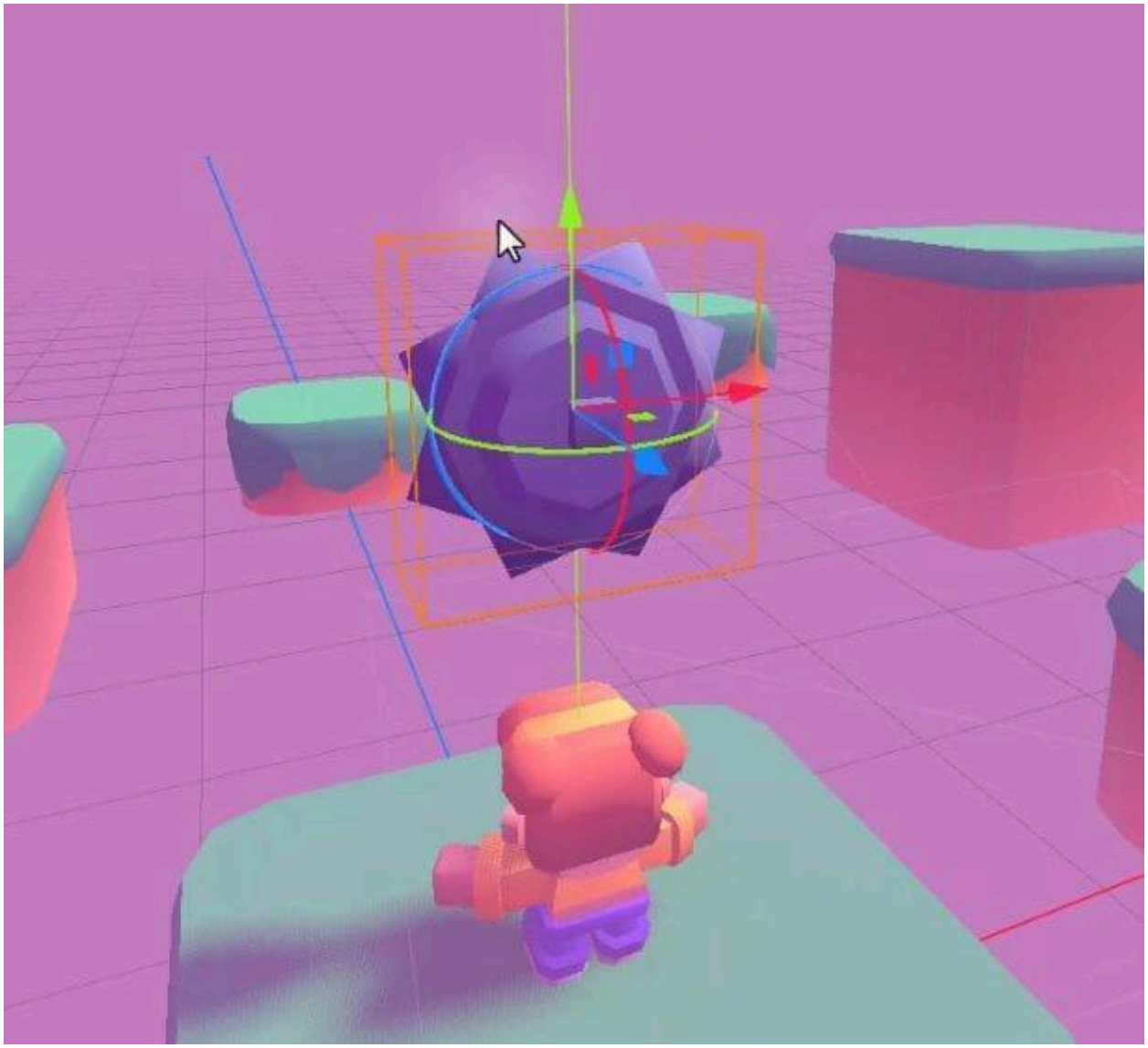
Next, we need a visual representation for our enemy. Navigate to the `Models` folder in the FileSystem dock, find the `saw` model (likely `saw.obj`), and add it as a child of the `Enemy` node. Once added, rename this child node to `Model` for clarity.


Scene
Import
Filter: name, t:type, g:
Main
Environment
Platforms
Player
Enemy
Model
FileSystem
res://Models/Platformer/saw.obj
Filter Files
plant.obj
platform-fortified.obj
platform-overhang.obj
platform-ramp.obj
platform.obj
poles.obj
rocks.obj
saw.obj
sign.obj

To ensure the model rotates around its center, set the position of the `Model` node to `(0, 0, 0)` in the Inspector.

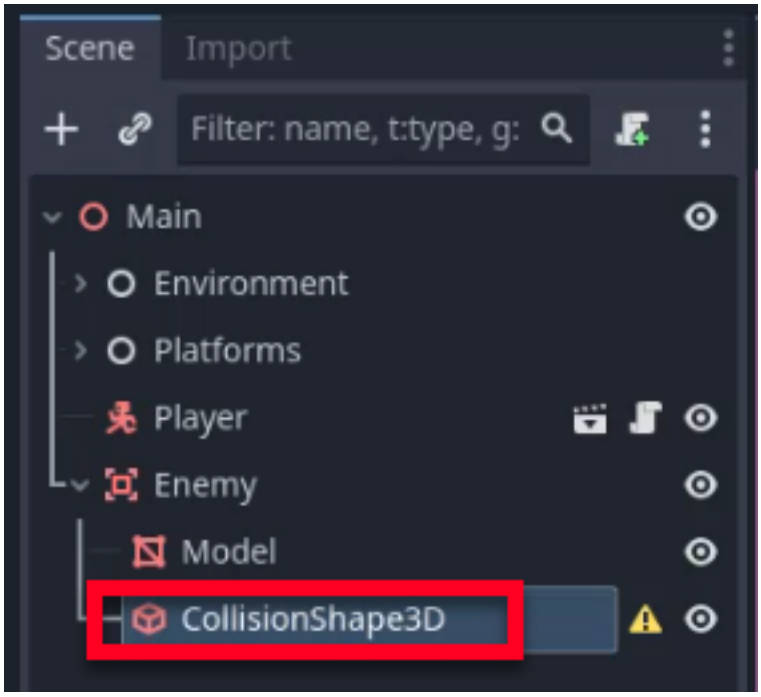

Inspector
Node
History
Model
Filter Properties
MeshInstance3D
Mesh
Skeleton
(1 change)
Surface Material Override
GeometryInstance3D
Geometry
Global Illumination
Visibility Range
VisualInstance3D
Layers
1 2 3 4 5 11 12 13 14 15
6 7 8 9 10 16 17 18 19 20
Sorting
Node3D
Transform
Position
x 0.0 m
y 0.0 m
z 0.0 m
Rotation
x 0.0 °
y 0.0 °
z 0.0 °
Scale

You should now see the saw blade centered in your scene, as shown below.

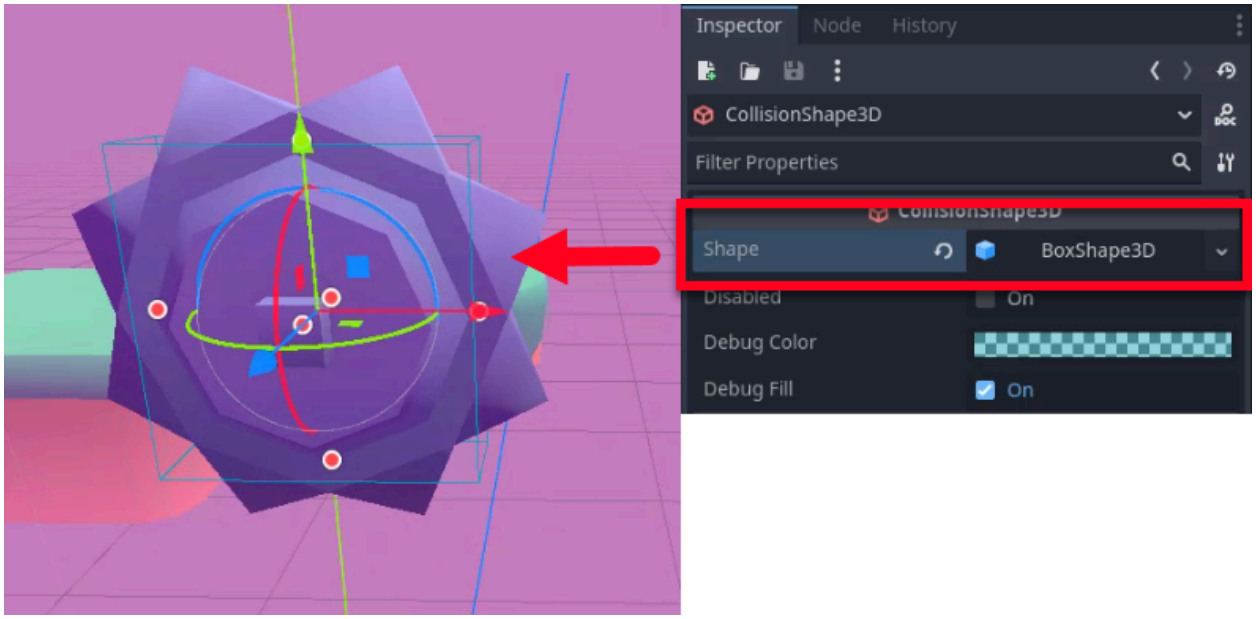


Adding a Collider

For the `Area3D` to detect collisions, it requires a collision shape. Right-click on the `Enemy` node and add a child node of type `CollisionShape3D`.



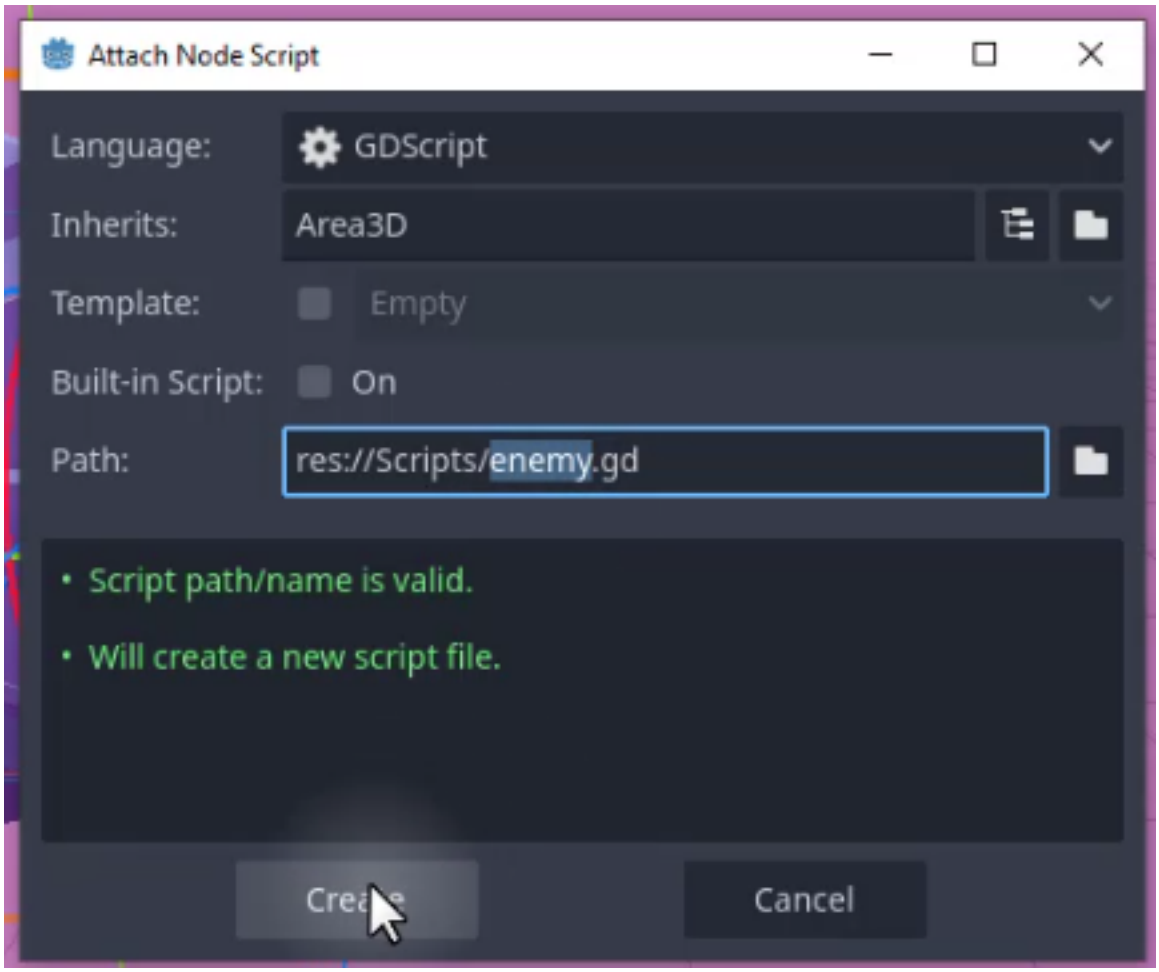
In the Inspector, set the **Shape** property of the `CollisionShape3D` to `BoxShape3D`. To fit the saw, adjust the size of the box so that it roughly matches the dimensions of the model. You can hold down the `Alt` key while dragging the handles in the viewport to resize the shape uniformly from the center.



Scripting Enemy Behavior

Now that our enemy scene is set up, we need to define its behavior. We want the enemy to move towards a target, return to its start position, and spin continuously.

To start, select the `Enemy` node and attach a new script. When the dialog appears, save it as `enemy.gd` in the `Scripts` folder.



Defining Variables

We will start by defining the variables that control the enemy's movement and rotation properties.

- `move_speed`: Controls how fast the enemy travels.
- `move_direction`: A vector defining the direction and distance of the patrol path.

- `spin_speed`: Controls the rotation speed in degrees per second.

Add the following code to your script:

```
extends Area3D

@export var move_speed : float = 2.0
@export var move_direction : Vector3
@export var spin_speed : float = 900.0
```

Next, we need to track the enemy's position. We will use `@onready` variables to calculate the start and target positions as soon as the node enters the scene tree.

- `start_pos`: Stores the initial global position of the enemy.
- `target_pos`: Stores the destination point (start position + move direction).
- `model`: A reference to the visual model node for rotation.

```
@onready var start_pos : Vector3 = global_position
@onready var target_pos : Vector3 = start_pos + move_direction
@onready var model = $Model
```

Implementing Movement

We will use the `_process` function to handle movement and rotation every frame (as our enemy doesn't rely on physics like the player).

To move the enemy, we use the `move_toward` method. This method moves a value (in this case, our position) towards a target by a specific amount (speed * delta).

Add the `_process` function to your script:

```
func _process (delta):
  global_position = global_position.move_toward(target_pos, move_speed
* delta)
```

Switching Targets

Currently, the enemy will move to the target and stop. To create a patrol loop, we need to check if the enemy has reached its destination and then swap the target.

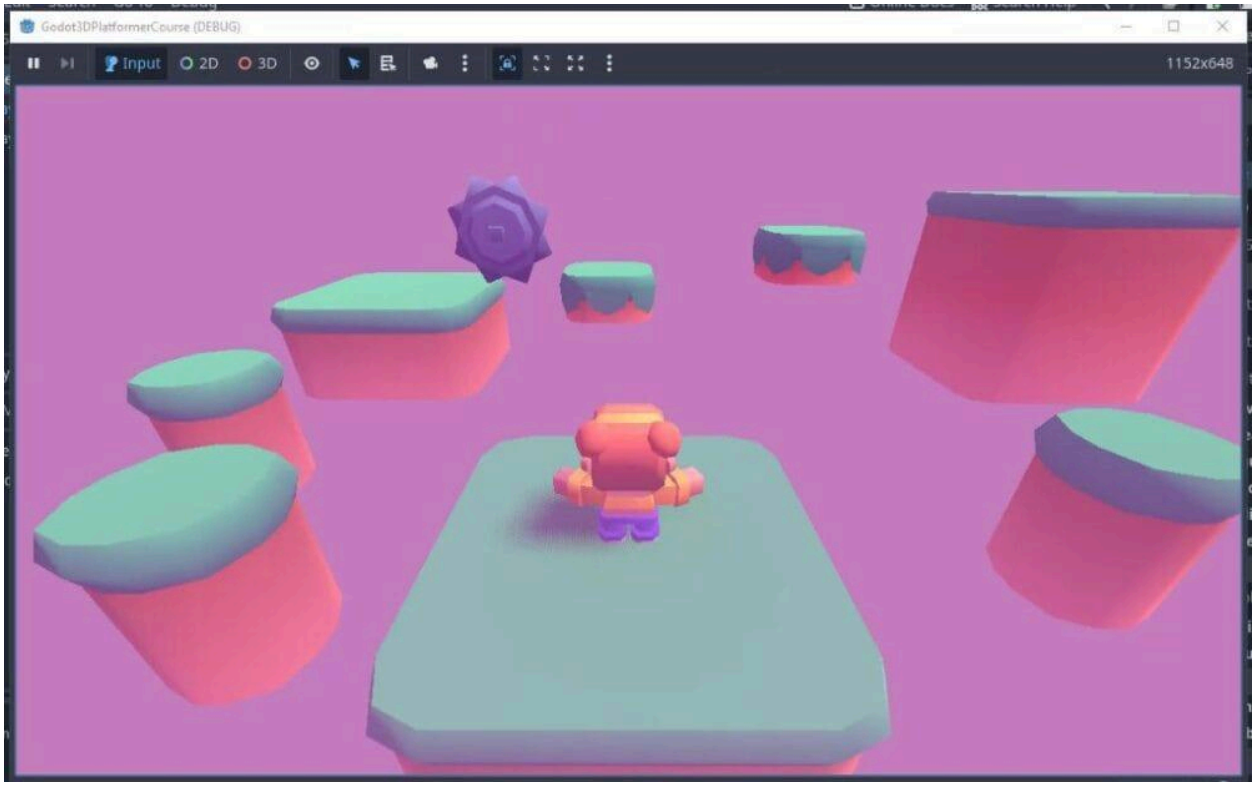
If the enemy is at the `start_pos`, we set the target to `start_pos + move_direction`. If it is at the end of the path, we set the target back to `start_pos`.

Update your `_process` function with this logic:

```
if global_position == start_pos:
    target_pos = start_pos + move_direction
elif global_position == start_pos + move_direction:
    target_pos = start_pos
```

You can test this now by running the scene. Before you run the game, make sure to set a movement vector in the `move_direction` variable we exported.

The enemy should now move back and forth between its starting point and the defined offset.



Rotating the Enemy

To make the saw blade look dangerous, we need it to spin. We will rotate the model along its Z-axis.

Add the rotation logic to the beginning of your `_process` function:

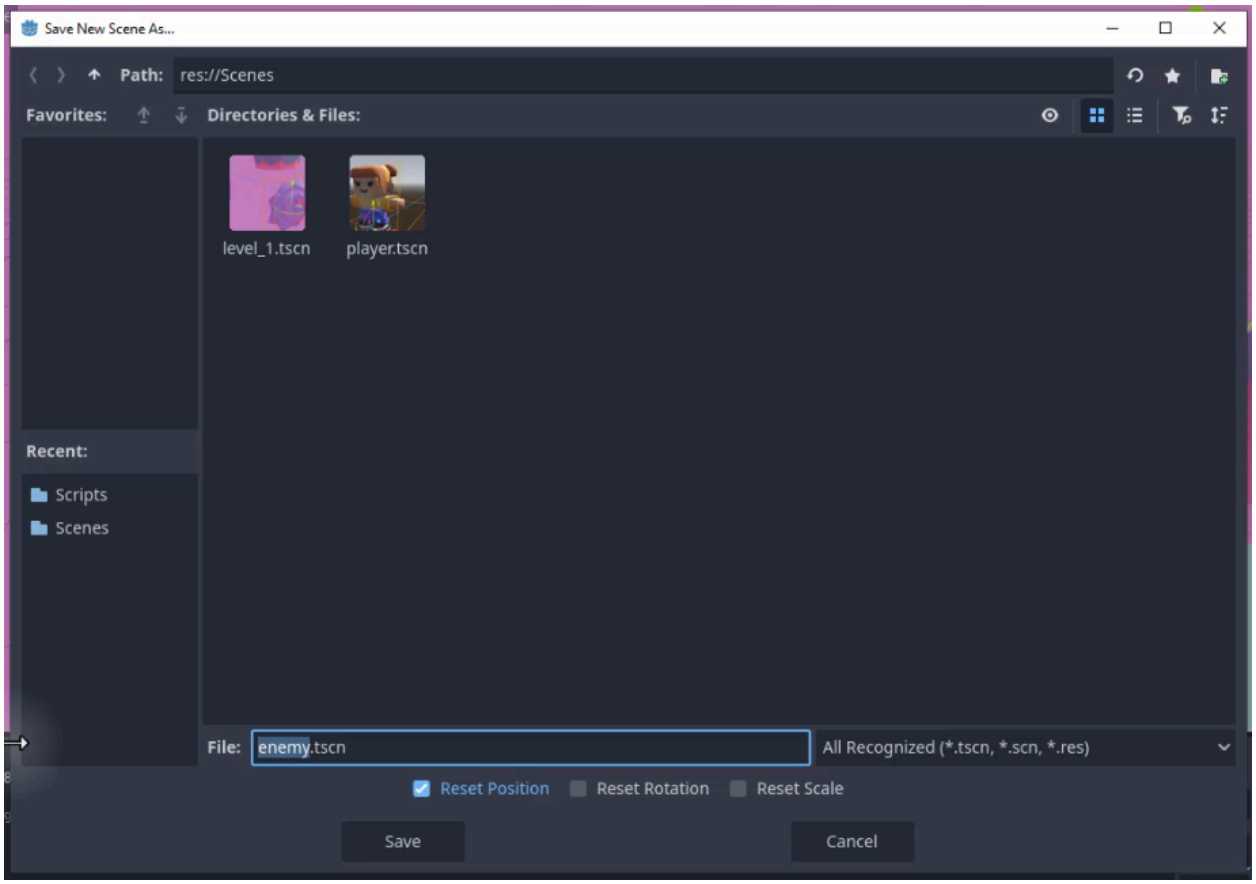
```
func _process (delta):
  model.rotation.z += deg_to_rad(spin_speed) * delta

  ...
```

Godot uses radians for rotation in code. We use `deg_to_rad` to convert our `spin_speed` into radians.

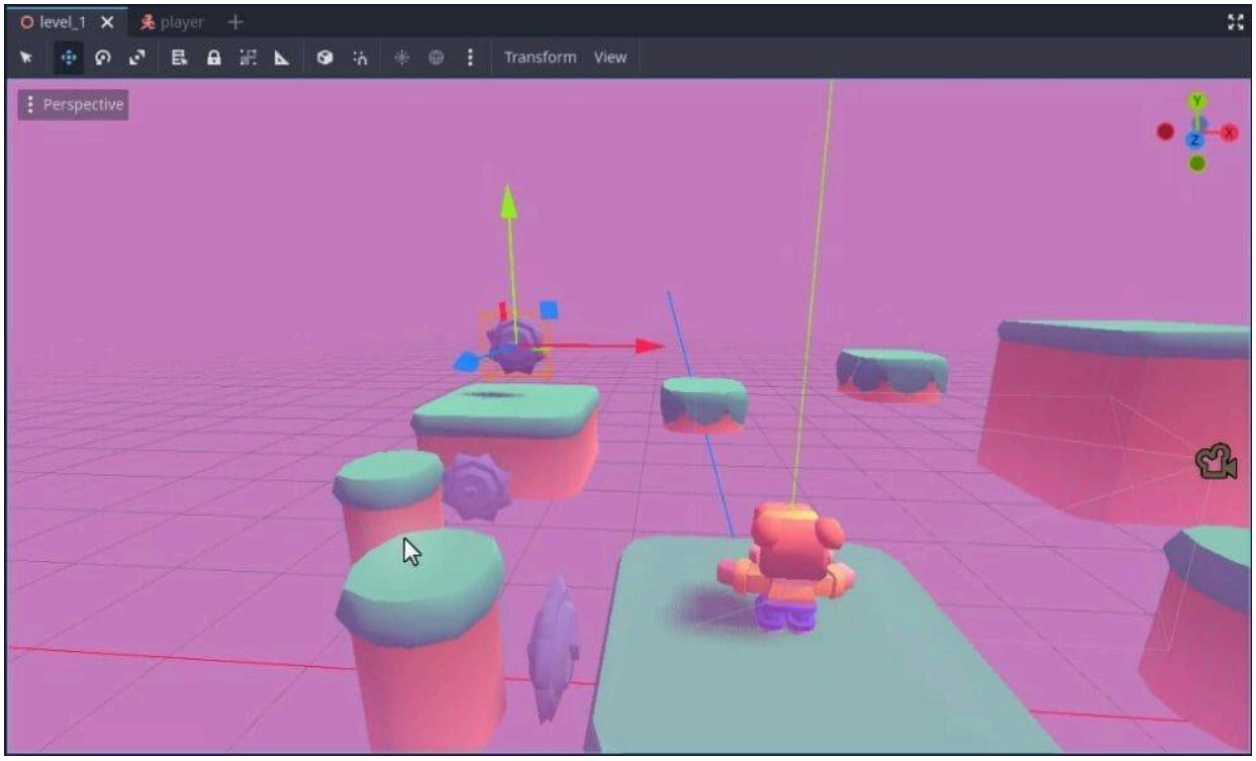
Saving the Enemy Scene

Now that the enemy logic is complete, save the `Enemy` node as a separate scene so we can reuse it throughout the level.

Right-click the `Enemy` node in the Scene dock, select **Save Branch as Scene**, and save it as `enemy.tscn` in the `Scenes` folder.



You can now duplicate the enemy in your level and set different `move_direction` values for each instance in the Inspector.

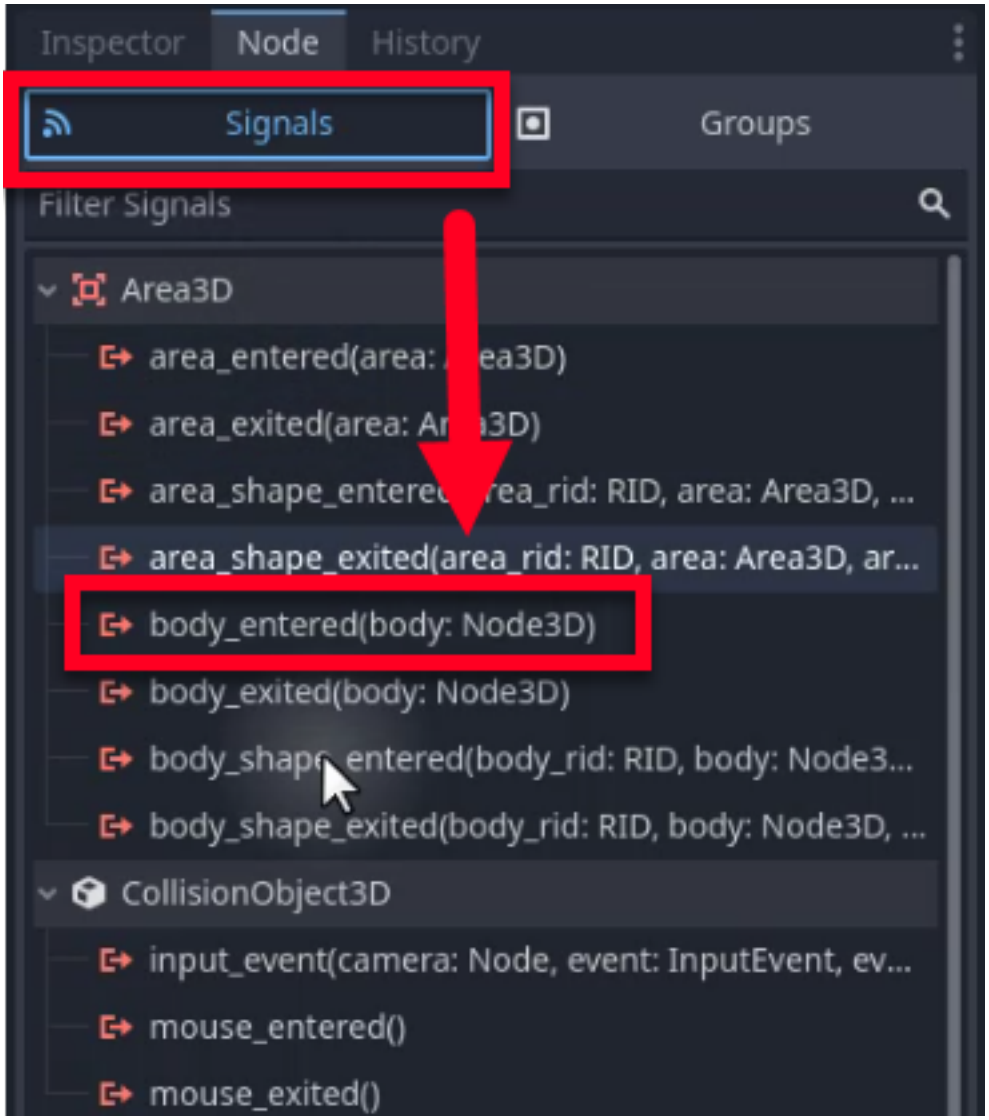


Detecting Collisions

The enemy moves and spins, but it does not yet interact with the player. We need to detect when the player enters the enemy's area.

Connecting the Signal

In order to detect the Player, we will use a signal made available to us by the node type we chose. First, select the `Enemy` node and go to the **Node** tab in the Inspector. To connect our collision handler, double-click the `body_entered` signal.

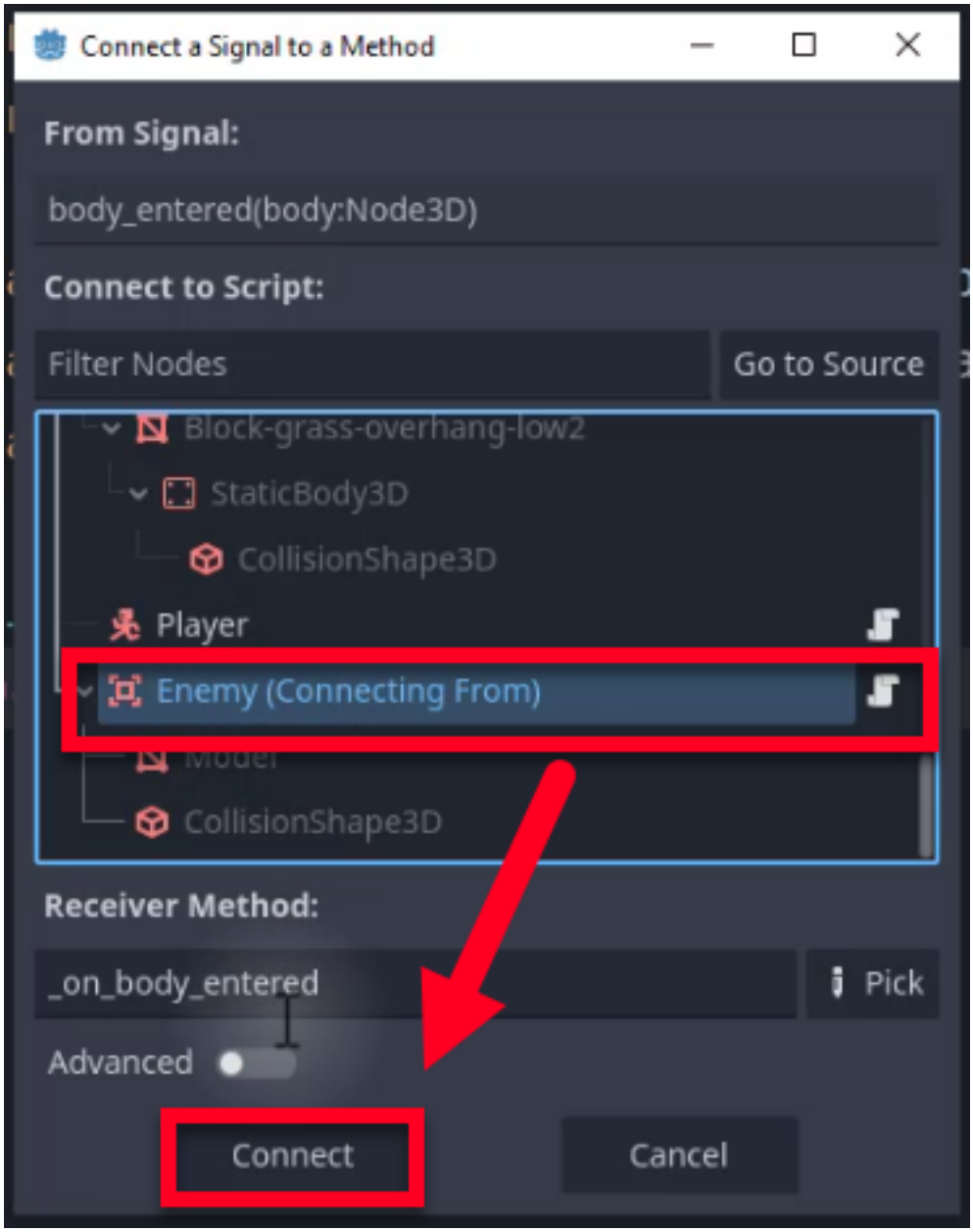


Connect this signal to the `Enemy` node itself. This will create a function called `_on_body_entered` in your `enemy.gd` script.

Godot Insight: Understanding signals

Signals are Godot's built-in implementation of the observer pattern. A node emits a signal when something happens (a body enters an area, a button is pressed, an animation finishes), and any other node that has connected to that signal receives a callback. This keeps your code decoupled: the Area3D does not need to know anything about the player; it simply announces "a body entered" and lets the connected function decide what to do. You can connect signals visually through the editor (as we just did) or in code with `signal_name.connect(callable)`.

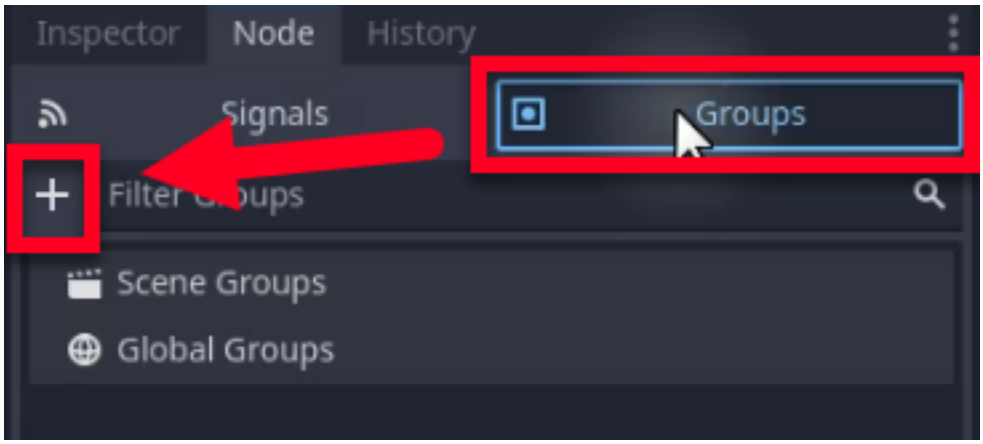
Learn more: Object signals (https://docs.godotengine.org/en/4.6/classes/class_object.html#signals)



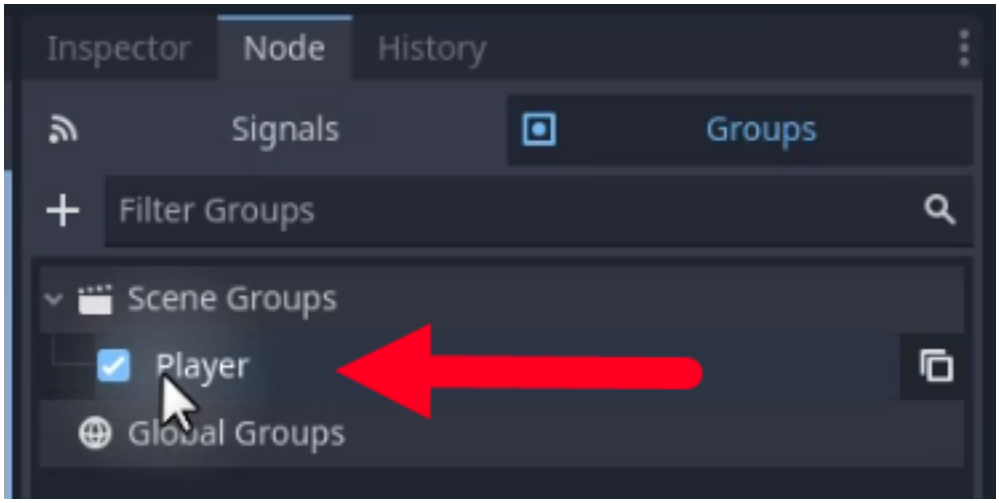
Using Groups

The `body_entered` signal fires for any physics body that enters the area, including platforms and other objects. We want the enemy to damage only the player. To achieve this, we will assign the player to a named group and then check group membership in our collision handler.

1. Open your `player.tscn` scene and select the root node.
2. In the **Node** tab, select **Groups**.
3. Add a new group named `Player`.



Before moving to the script, ensure the group is checked and active.



Handling the Collision

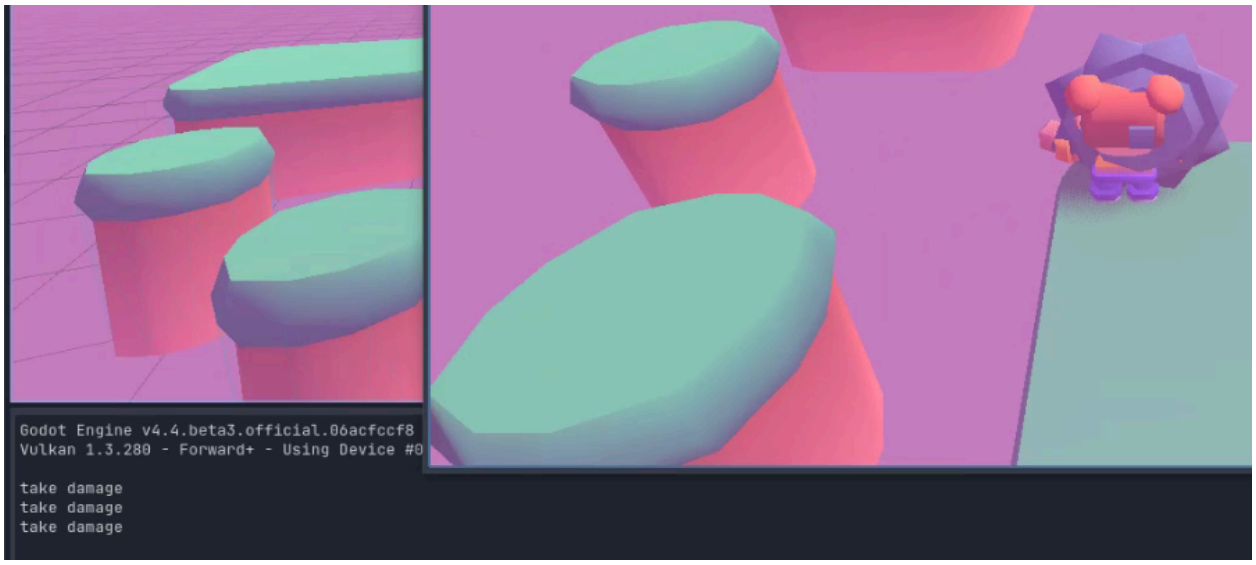
Back in `enemy.gd`, we can now check if the colliding body belongs to the "Player" group.

Update the `_on_body_entered` function as follows:

```gdscript
func _on_body_entered(body):
    if not body.is_in_group("Player"):
        return
```

```
    print("take damage")
```

If you run the game and walk into the enemy, you should see “take damage” printed in the Output console.



Implementing Health and Damage

Now that we can detect collisions, let’s implement the actual damage logic. We need to update the player script to track health and the enemy script to apply damage.

Updating the Player Controller

Let’s begin by opening `player_controller.gd`. We will add a health variable and a function to handle taking damage.

First, add the health variable at the top of the script (near the other variables):

```
@export var health : int = 3
```

Next, add the `take_damage` function. This function will reduce health and check for a “Game Over” condition.

```
func take_damage(amount : int):
    health -= amount
    if health <= 0:
        _game_over()
```

We also need to define what happens when the game is over. For now, we will just reload the scene. We will, however, revisit this section later once we add a menu scene.

```
func _game_over():
    get_tree().reload_current_scene()
```

Applying Damage from the Enemy

Next, return to `enemy.gd`. Instead of printing a message, we will now call the player's `take_damage` function.

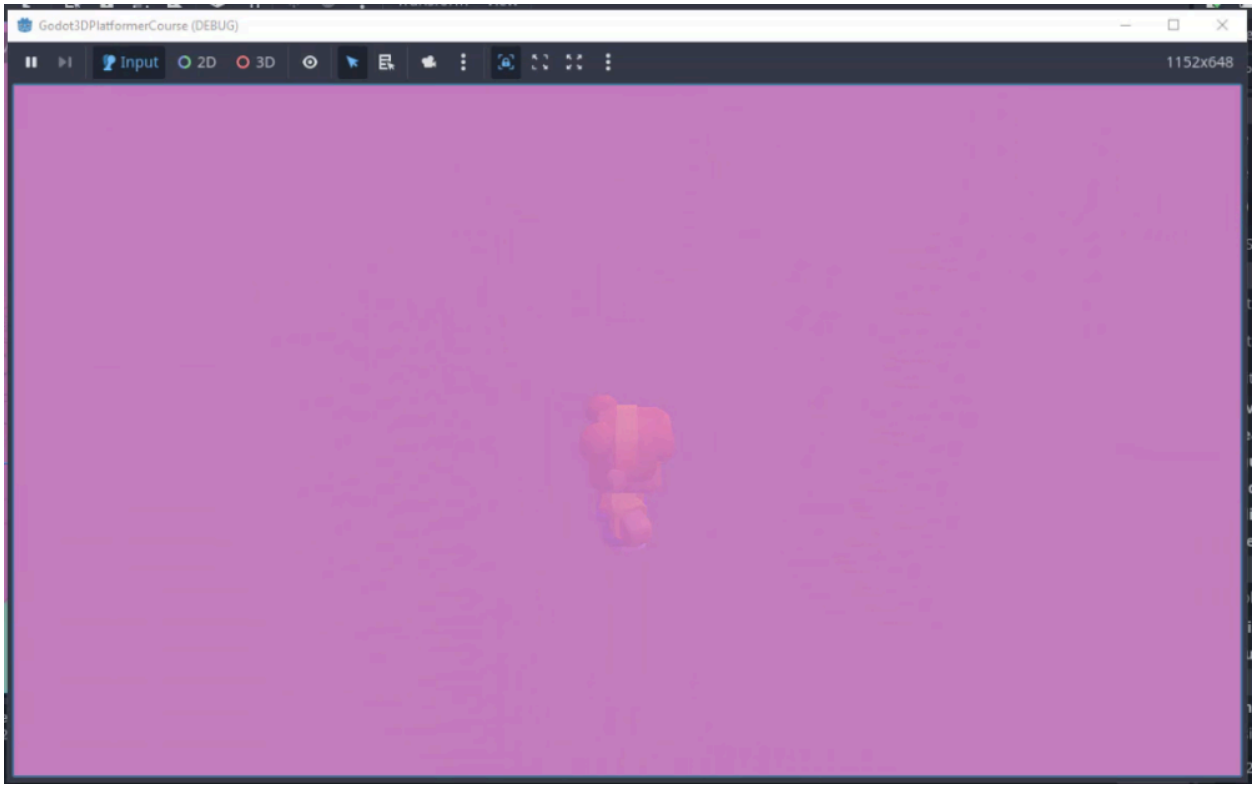
As shown below, update `_on_body_entered`:

```
func _on_body_entered(body):
    # Only the player takes damage from enemies
    if not body.is_in_group("Player"):
        return

    body.take_damage(1)
```

Now, when the player touches the enemy, their health will decrease. If they take three hits, the level will restart.

Handling Falling Off the Map

Finally, we should handle the case where the player falls off the platforms. Currently, they fall forever. We can treat this as an instant "Game Over."



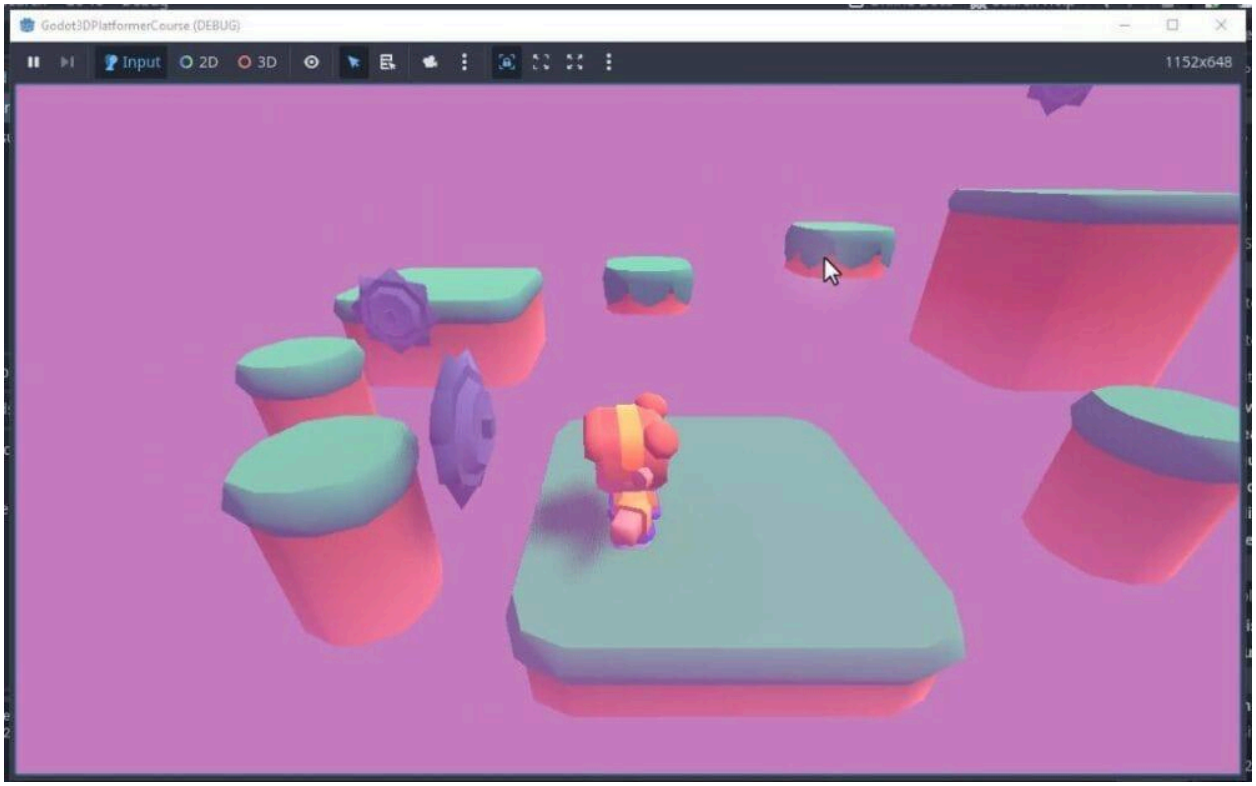
In `player_controller.gd`, add a check in the `_process` function (or create it if it doesn't exist) to monitor the player's Y position.

```gdscript
func _process(delta):
    if global_position.y < -5:
        _game_over()
```

This ensures that if the player falls below a height of -5 units, the level restarts immediately. Note that you may need to adjust this depending on the design of your level.

Conclusion

We have successfully implemented a dangerous enemy and a health system. The enemy patrols a set path, spins to indicate danger, and deals damage upon contact. The player also now has a limited number of lives and will restart the level if they run out of health or fall off the map.



In the next chapter, we will shift our focus from combat to rewards by implementing a coin collection and scoring system.

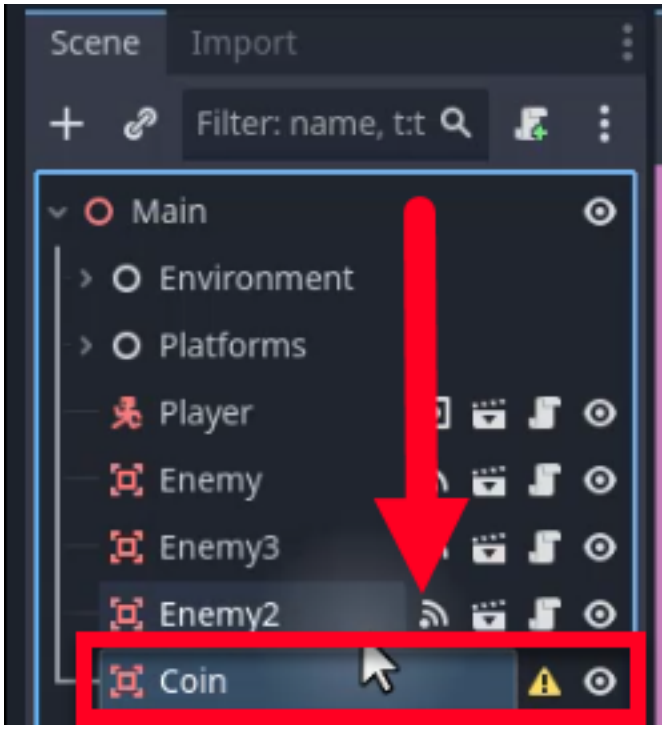
Collectibles and Scoring System

In this chapter, we will implement a collectible coin system to reward the player for exploration. We will create a coin object that rotates and bobs to attract attention, and then implement a persistent scoring system using Godot's Autoload feature. By the end of this chapter, players will be able to collect coins to increase their score, which will carry over between levels.

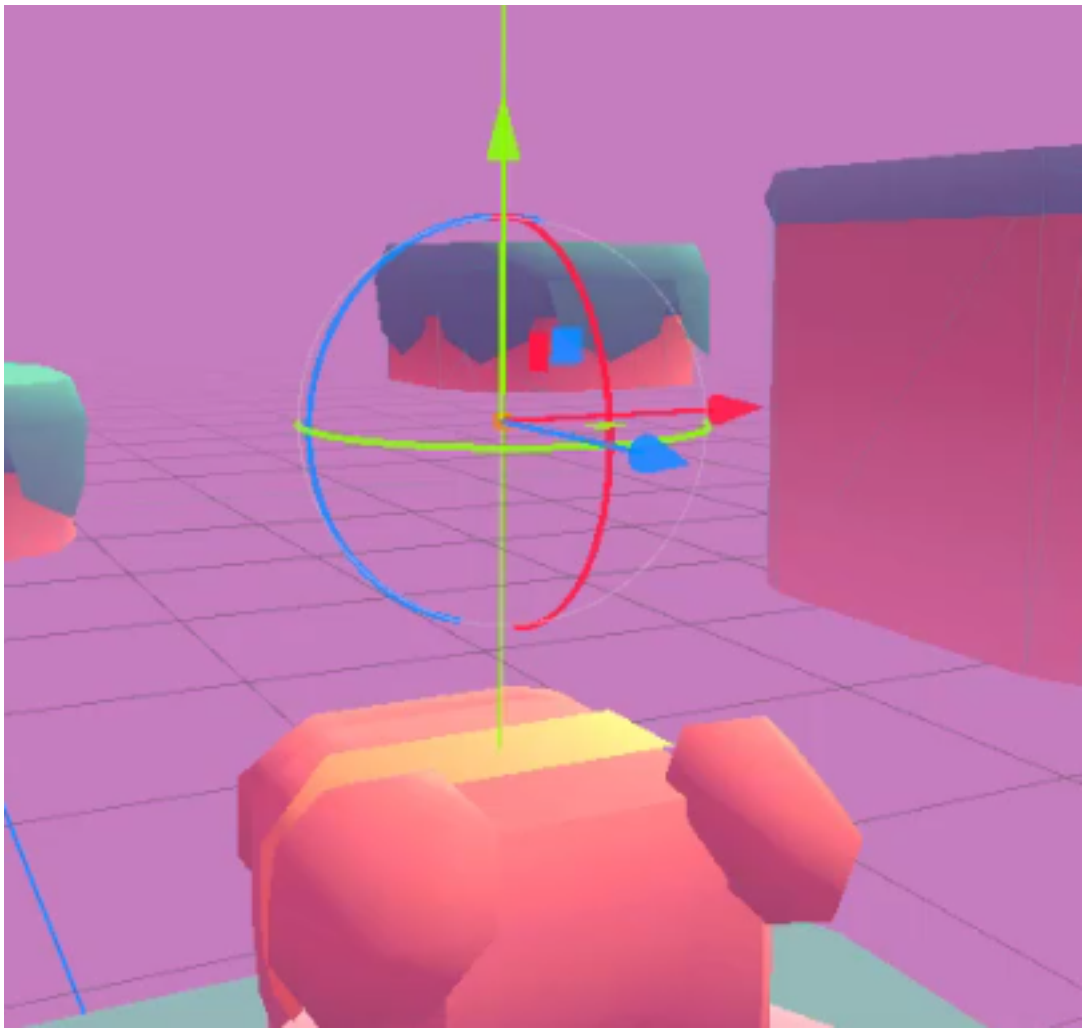
Creating the Coin

We will begin by creating the coin object. Similar to how we set up the enemy, the coin will use an `Area3D` node to detect when the player touches it without acting as a solid physical obstacle.

Create a new `Area3D` node in your scene and rename it to `Coin`.



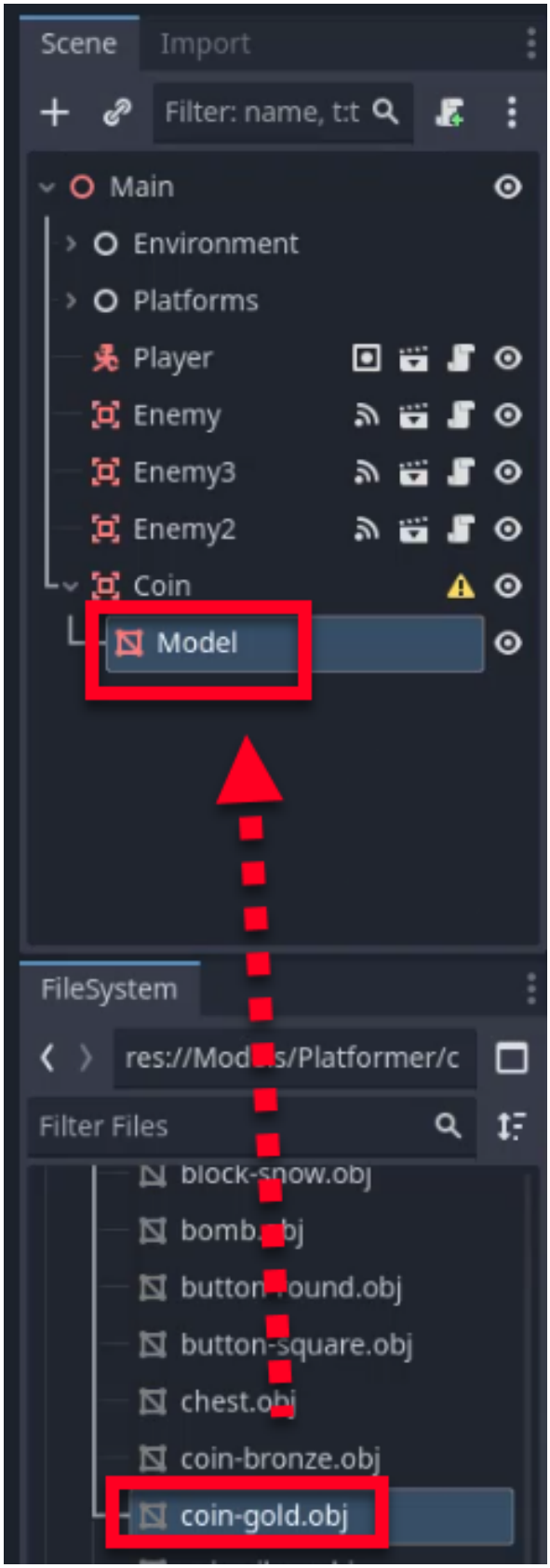
In the visual editor, move the `Coin` node up slightly so it is not sitting inside the floor, making it easier to work with.



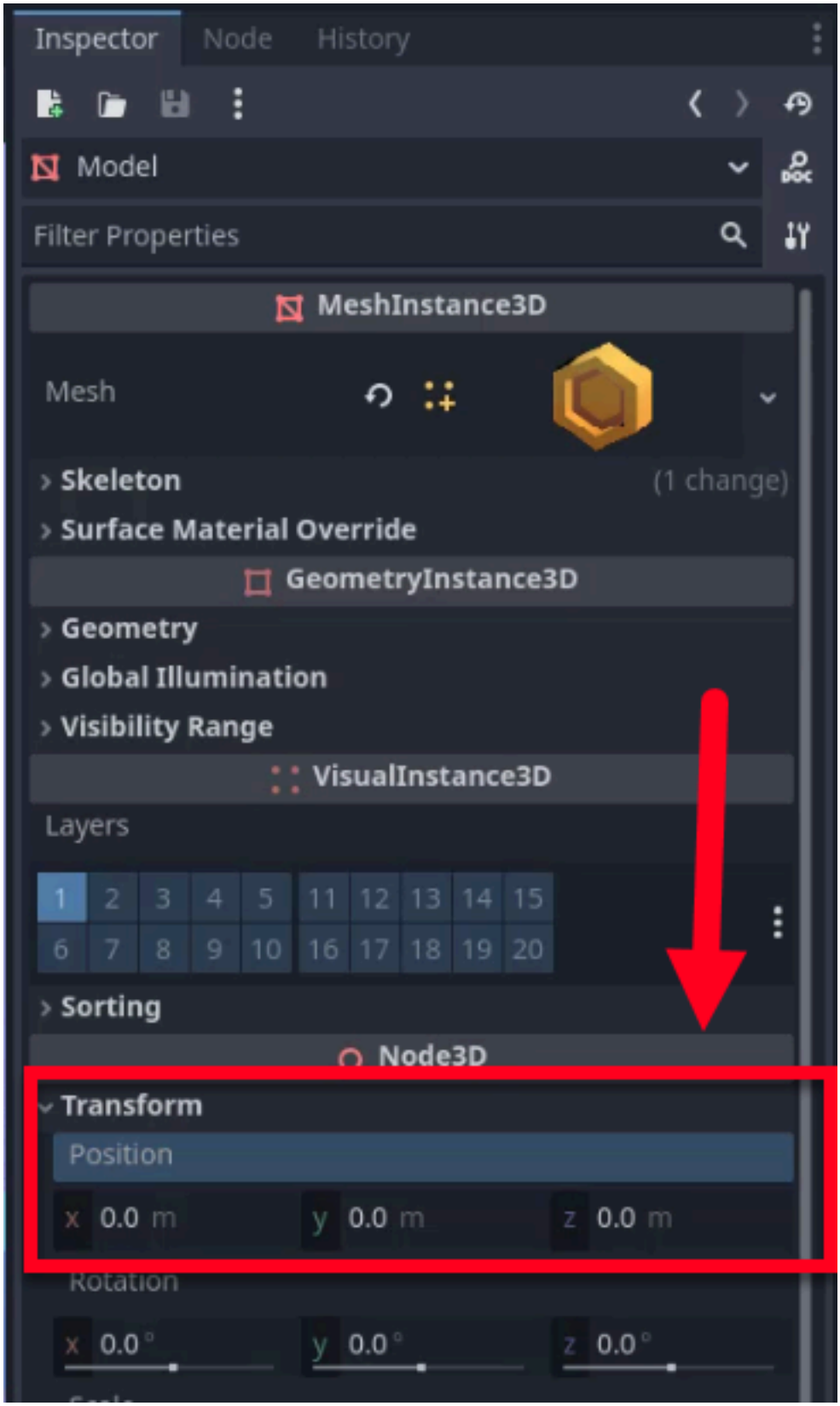
Adding the Visual Model

To give the coin a visual form, we will use a pre-made 3D model.

1. Start by navigating to the `Models` folder in the FileSystem dock, then to `Platformer`.
2. Locate the `coin-gold.obj` model and drag it into the scene as a child of the `Area3D` node.
3. Rename the model node to `Model` for clarity.


Scene
Import
Filter: name, t:t
Main
Environment
Platforms
Player
Enemy
Enemy3
Enemy2
Coin
Model
FileSystem
res://Mod s/Platformer/c
Filter Files
block-snow.obj
bomb bj
button ound.obj
button-square.obj
chest.obj
coin-bronze.obj
coin-gold.obj

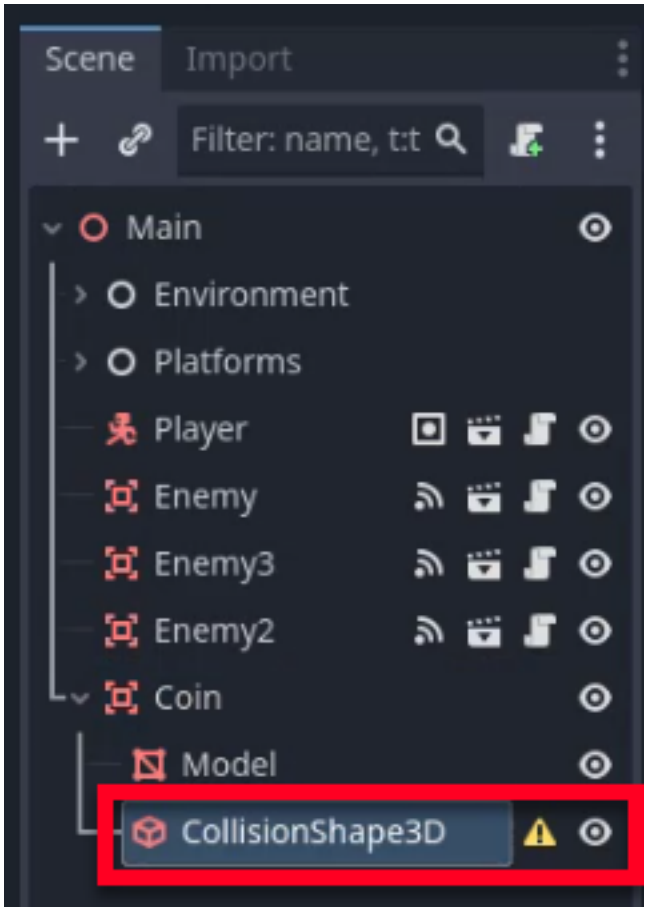
We need to ensure the model is centered relative to the parent node. To do so, select the `Model` node and set its **Position** to `(0, 0, 0)` in the Inspector.


Inspector
Node
History
Model
Filter Properties
MeshInstance3D
Mesh
Skeleton
(1 change)
Surface Material Override
GeometryInstance3D
Geometry
Global Illumination
Visibility Range
VisualInstance3D
Layers
1 2 3 4 5 11 12 13 14 15
6 7 8 9 10 16 17 18 19 20
Sorting
Node3D
Transform
Position
x 0.0 m
y 0.0 m
z 0.0 m
Rotation
x 0.0 °
y 0.0 °
z 0.0 °

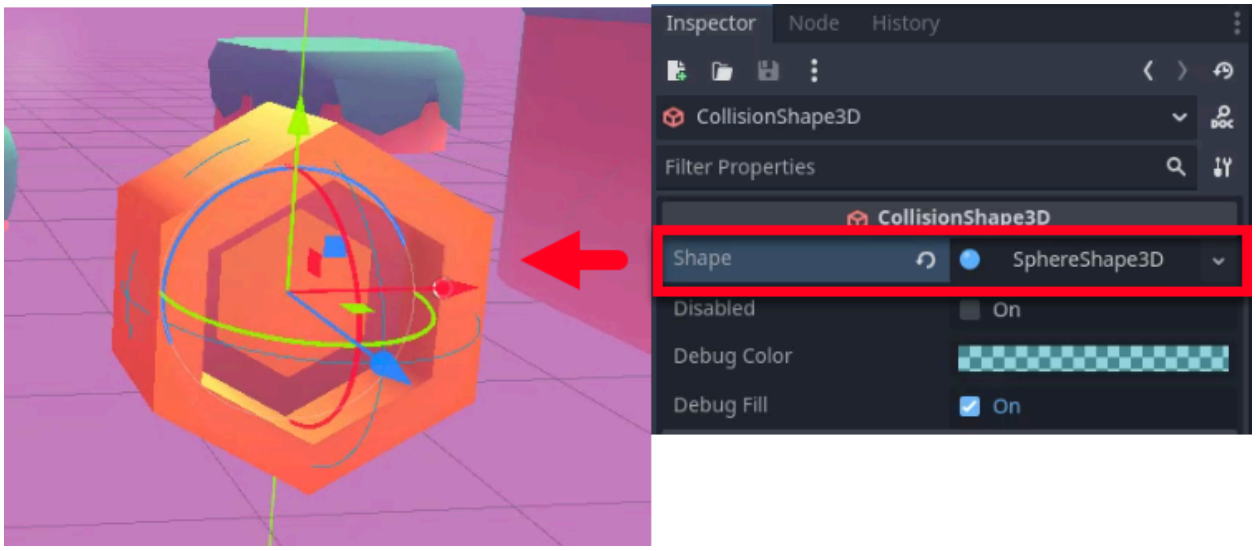
Adding a Collision Shape

For the `Area3D` to function, it requires a collision shape to define its boundaries.

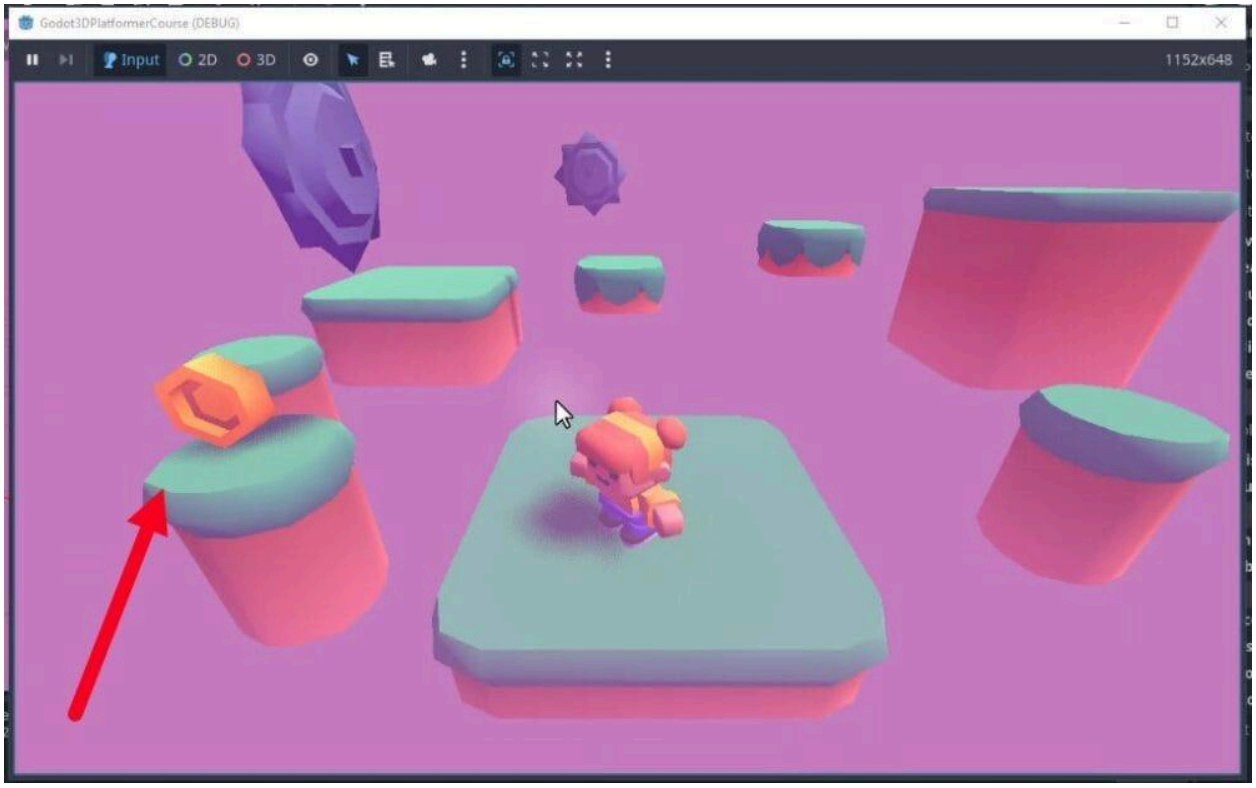
Right-click the `Coin` node and add a `CollisionShape3D` child node.



In the Inspector, create a new `SphereShape3D` resource for the **Shape** property. Then, resize the sphere so that it fully encompasses the coin model.



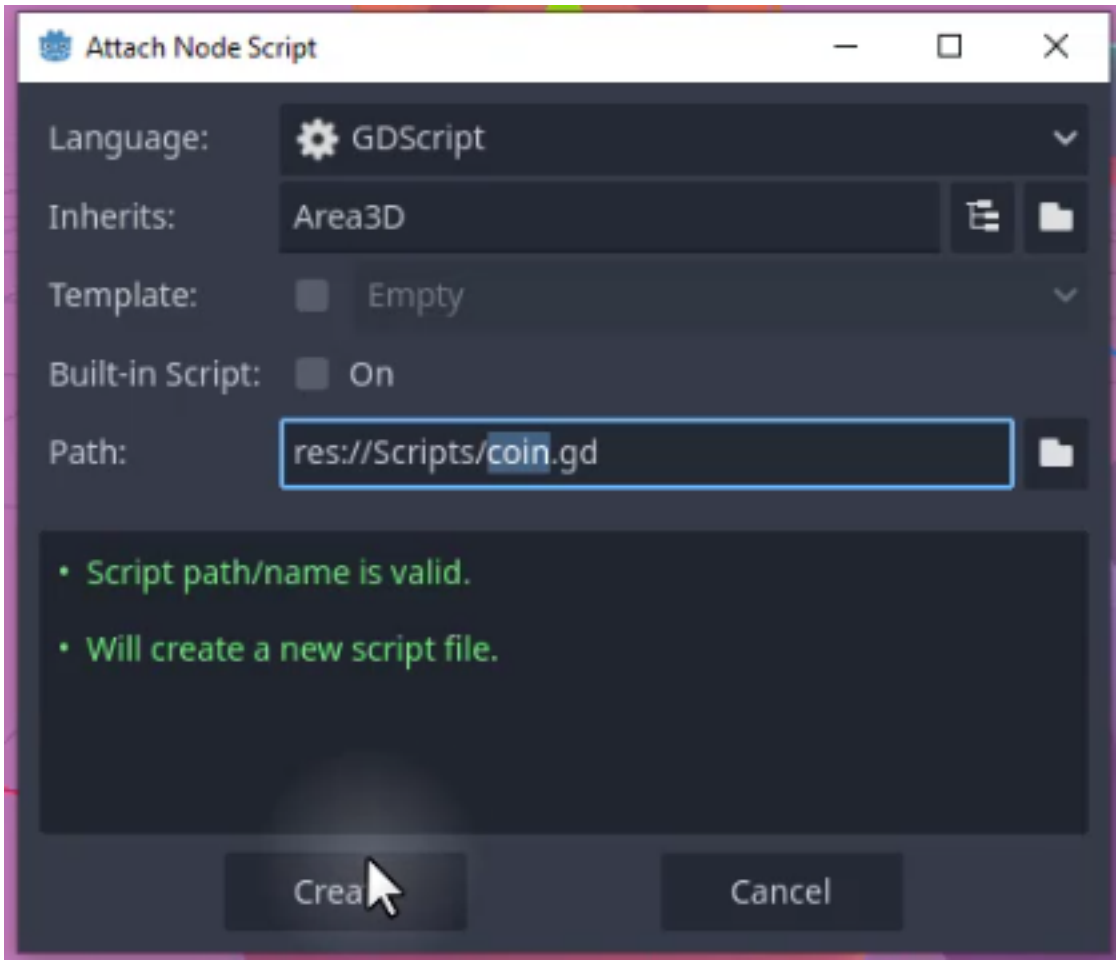
Once configured, position the coin in the scene where you want it to appear. This will allow us to test the collection mechanics in the next steps.



Scripting Coin Behavior

We need a script to handle the logic for collecting the coin. This script will detect when the player enters the coin's area and remove the coin from the scene.

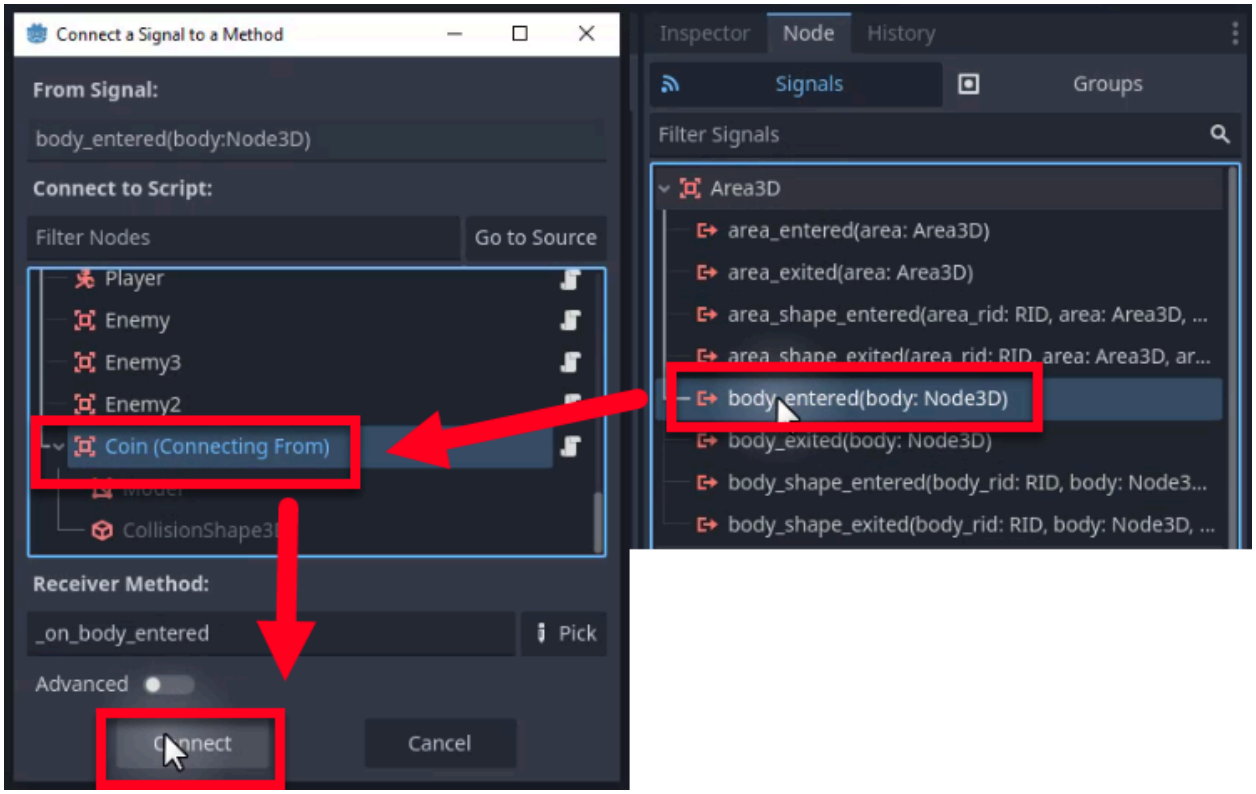
To begin, select the `Coin` node and attach a new script named `coin.gd`. Make sure to save to your `Scripts` folder.



Detecting the Player

To detect collisions, we will use the `body_entered` signal provided by the `Area3D` node.

To connect the signal, select the `Coin` node again and go to the **Node** tab (Signals) in the Inspector. Double-click the `body_entered` signal and connect it to the `coin.gd` script in the connection pop-up.



In the generated `_on_body_entered` function, we check if the body belongs to the "Player" group. If it does, we remove the coin using `queue_free()`.

```
extends Area3D

func _on_body_entered(body):
    if not body.is_in_group("Player"):
        return

    # Increase score (placeholder for now)
    queue_free()
```

If you run the game now, walking into the coin should cause it to disappear.

Visual Polish: Rotation and Bobbing

Static coins can be hard to notice and feel lifeless. A little visual flair goes a long way in catching the player's eye. We will add rotation and a bobbing motion using simple math, making the coins feel like they are floating and inviting the player to collect them.

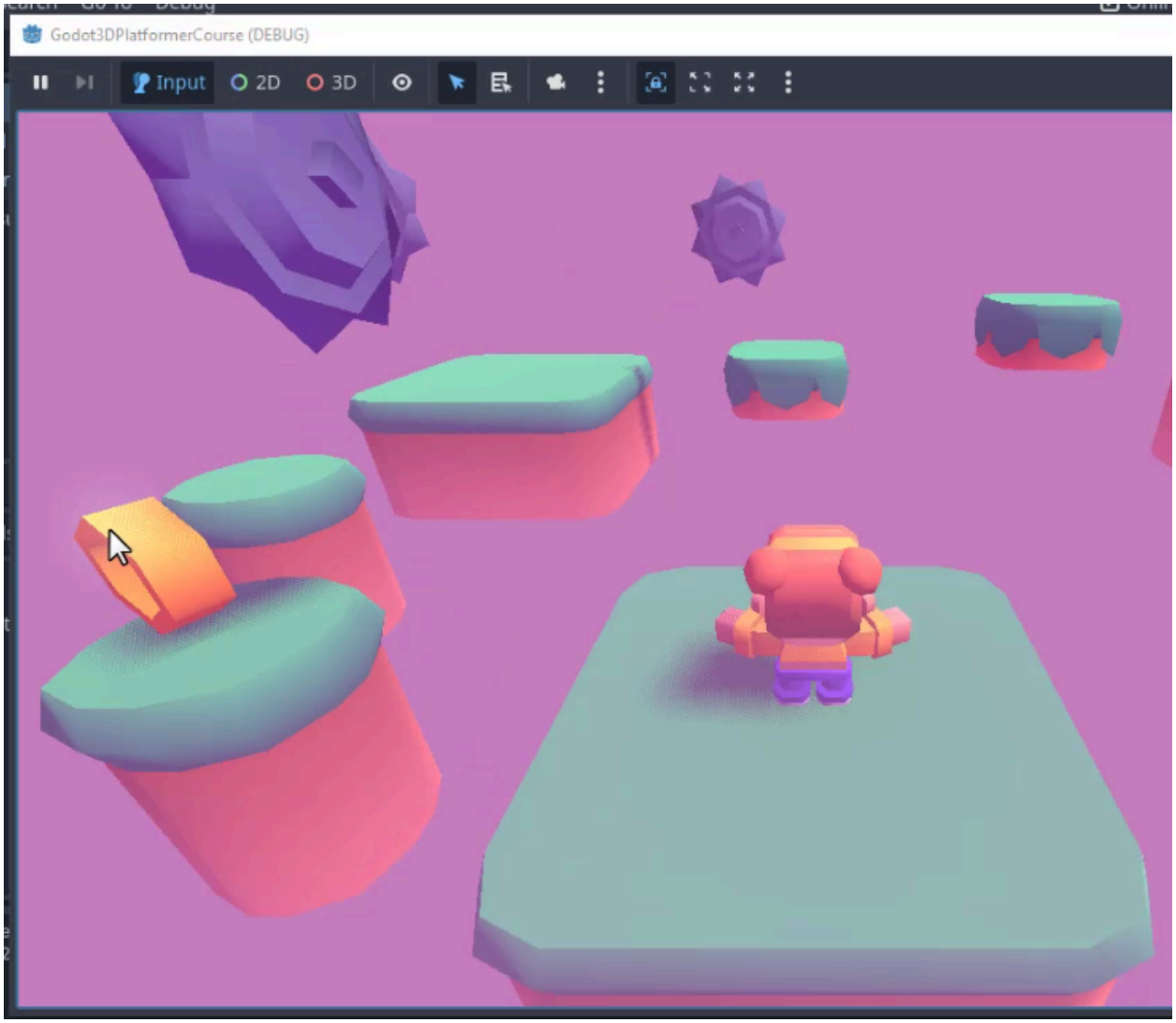
Making the Coin Rotate

First, let's make the coin spin. We will add a variable for rotation speed and update the rotation in the `_process` function.

Add the following code to `coin.gd`:

```
@export var rotate_speed : float = 180.0

func _process(delta):
    # Rotate the coin
    rotation.y += deg_to_rad(rotate_speed) * delta
```

The `deg_to_rad` function converts our speed (in degrees) to radians, which Godot uses for rotation calculations. If you test the game, you will already see an improvement in the coin's visual appeal.



Adding Bobbing Motion

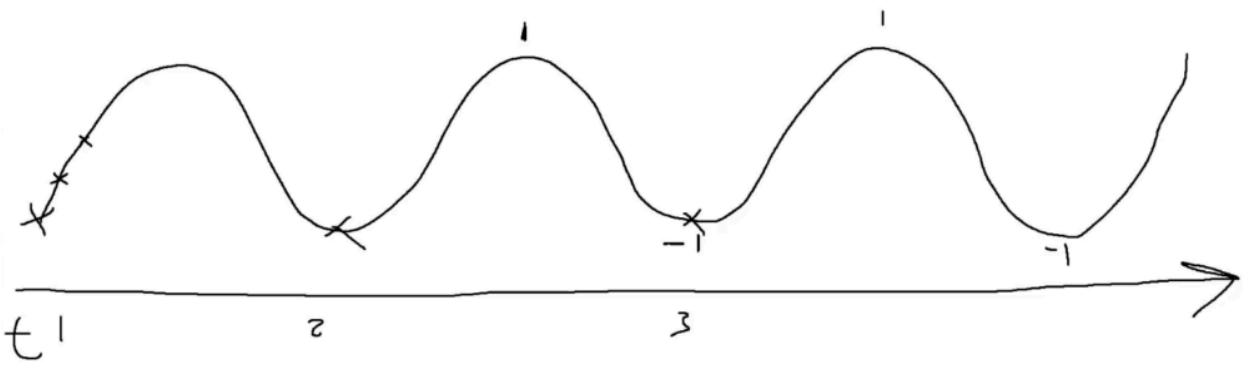
Next, we will add an up-and-down bobbing motion using a sine wave. This requires a few additional variables to control the height and speed of the bob, as well as the initial Y position.

First, add these variables to your script:

```
@export var bob_height : float = 0.2
@export var bob_speed : float = 5.0
@onready var start_y_pos : float = global_position.y
```

- `bob_height`: How high the coin moves up and down.
- `bob_speed`: How fast the coin completes a bobbing cycle.
- `start_y_pos`: The baseline Y position, captured when the game starts.

To create the motion, we use the `sin()` function, which oscillates between -1 and 1 over time.



We can map this sine wave to our coin's position. Update the `_process` function to include the bobbing logic:

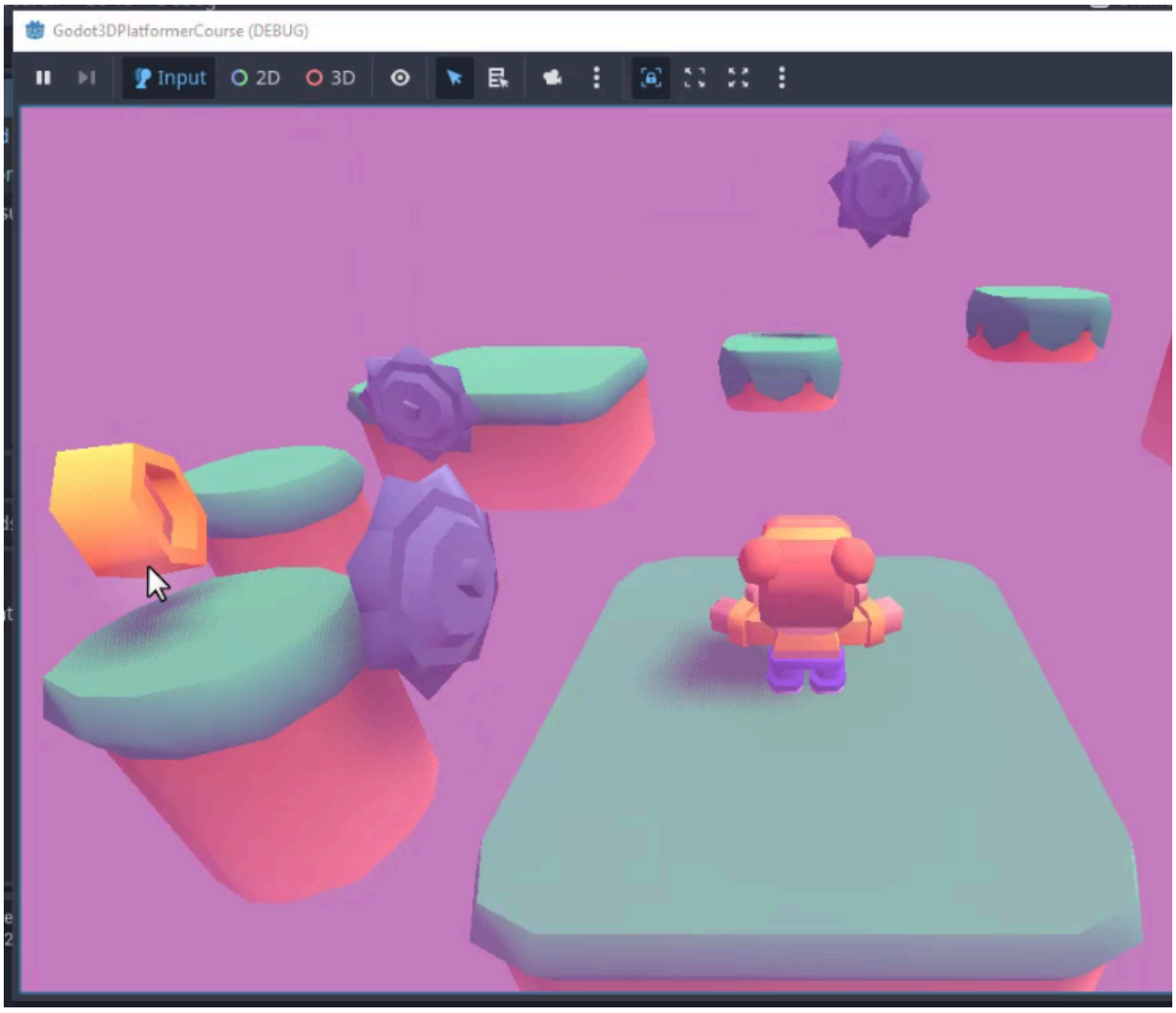
```
func _process(delta):
    # Spin the coin around the Y-axis for visual appeal
    rotation.y += deg_to_rad(rotate_speed) * delta

    # Use a sine wave mapped to 0..1 to bob above the starting position
    var time = Time.get_unix_time_from_system()
    var y_pos = (1 + sin(time * bob_speed)) / 2 * bob_height
    global_position.y = start_y_pos + y_pos
```

Here is a breakdown of the math:

1. `sin(time * bob_speed)` creates a value between -1 and 1.
2. Adding `1` shifts the range to 0 to 2.
3. Dividing by `2` normalizes the range to 0 to 1.
4. Multiplying by `bob_height` scales the motion to the desired height.

This ensures the coin always bobs *above* its starting position, preventing it from clipping into the platform below.

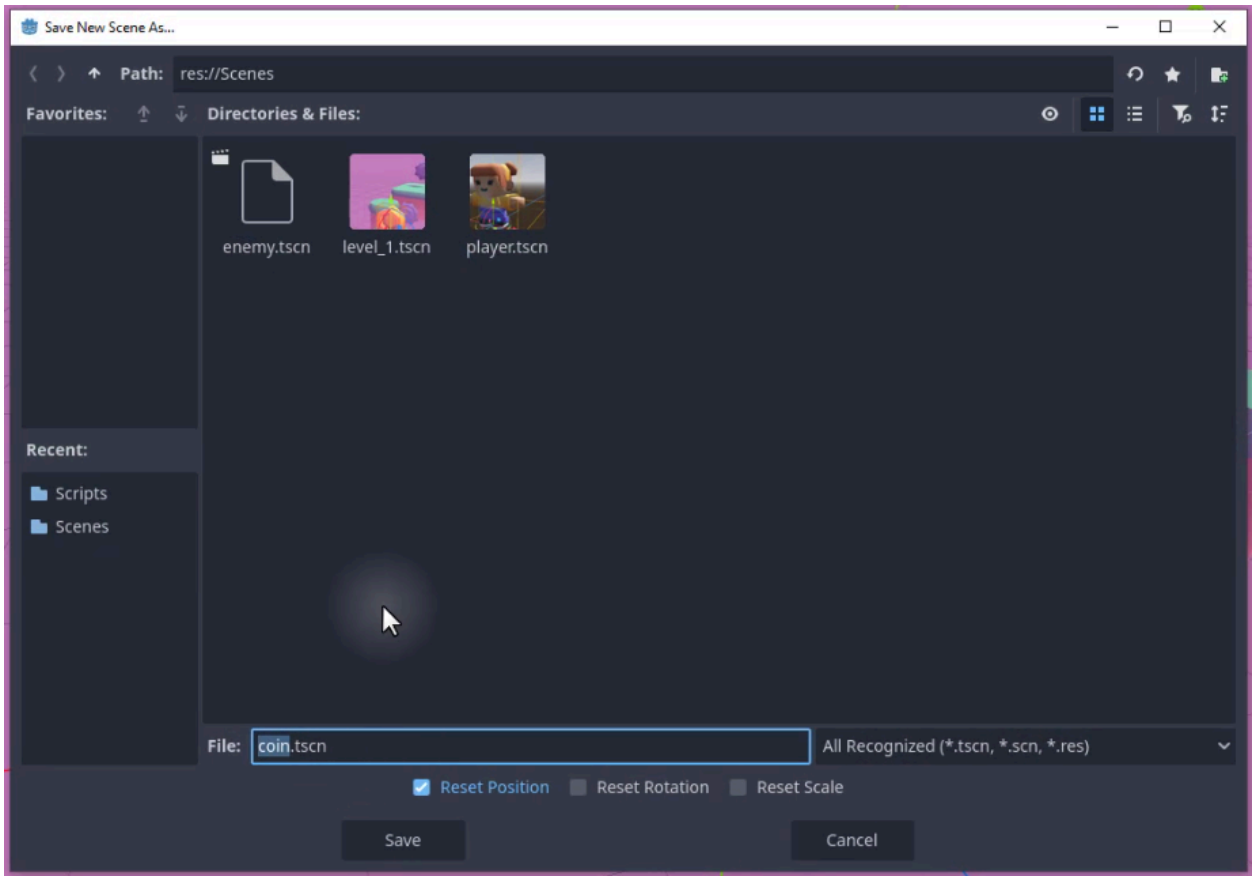
The coin should now be a lot "juicier" and stand out more to the player!



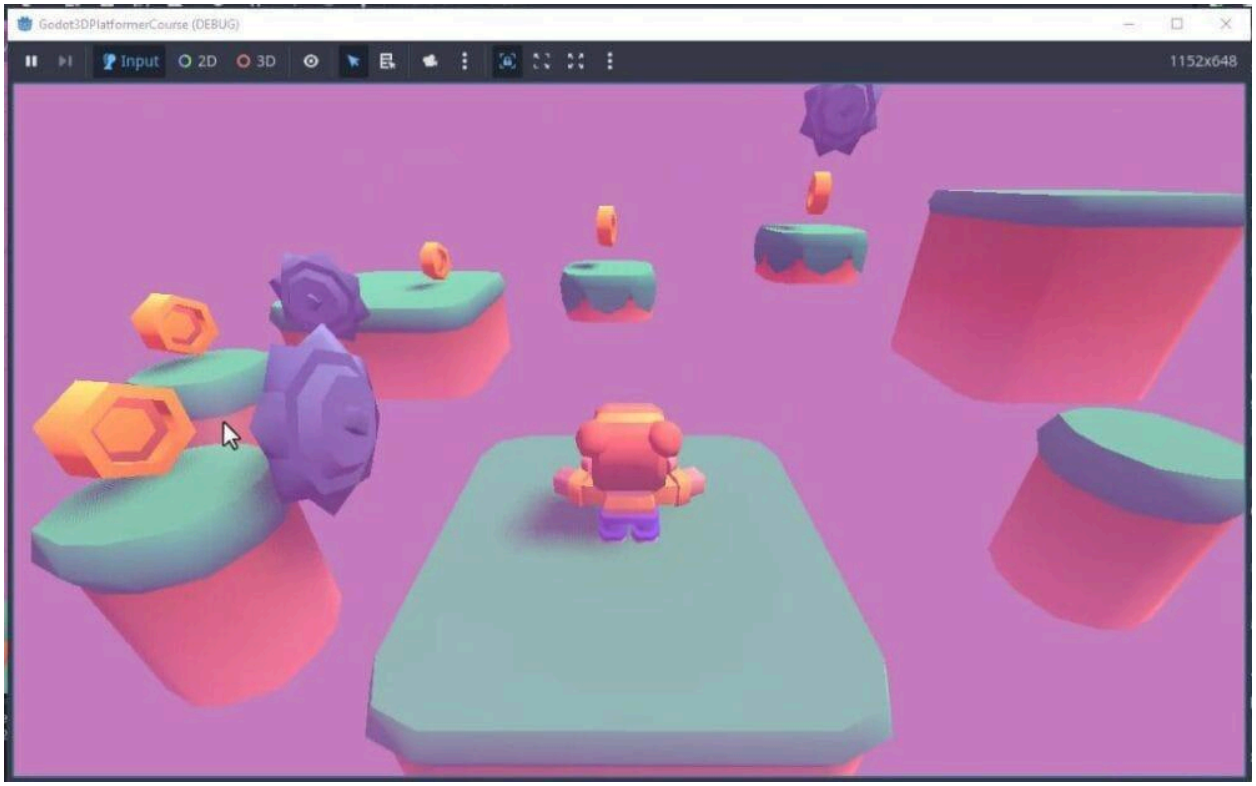
Saving and Placing Coins

Now that our coin is fully functional, we should save it as a reusable scene.

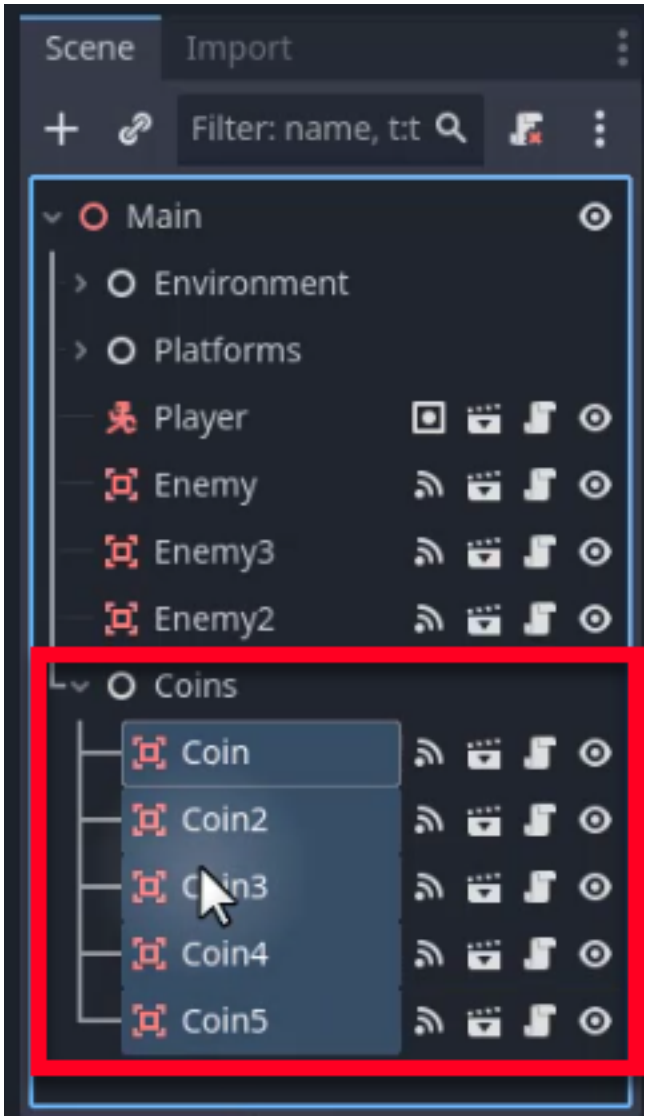
1. Right-click the `Coin` node in the Scene dock.
2. Select **Save Branch as Scene**.
3. Save it as `coin.tscn` in the `Scenes` folder.



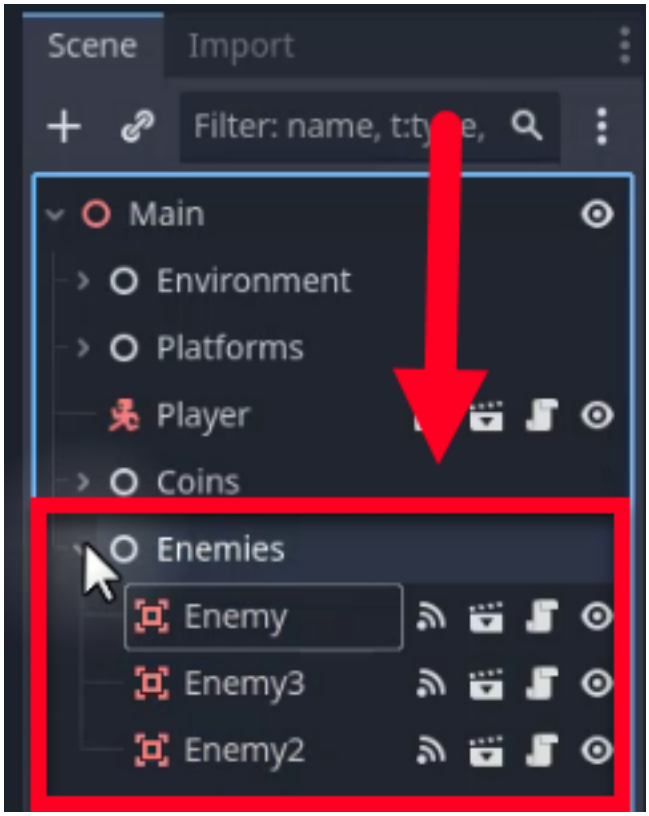
You can now duplicate the coin instance to place multiple coins around your level. Generally, you'll want to lay them out to guide the player to where the "end" of the level is (which we'll set up in a later chapter).



To keep the scene tree organized, create a generic `Node` named `Coins` and drag all your coin instances into it.



You can do the same for enemies to maintain a tidy hierarchy.



The Scoring System

With our coins placed, we need a system to track the score. Unlike local variables that reset when a scene reloads, we want the score to persist across levels. To achieve this, we will use a Godot feature called **Autoloads** (or Singletons).

Understanding Autoloads

Autoloads are scripts that Godot loads automatically when the game starts. They run independently of the current scene and remain active for the entire duration of the game session. This makes them ideal for storing global data like the player's score.

Godot Insight: Autoloads and the singleton pattern

In most programming contexts, a singleton is a class with exactly one instance that is accessible from anywhere. Godot's Autoload system provides a built-in way to create singletons: any script registered as an Autoload is loaded once at launch and stays alive across all scene changes.

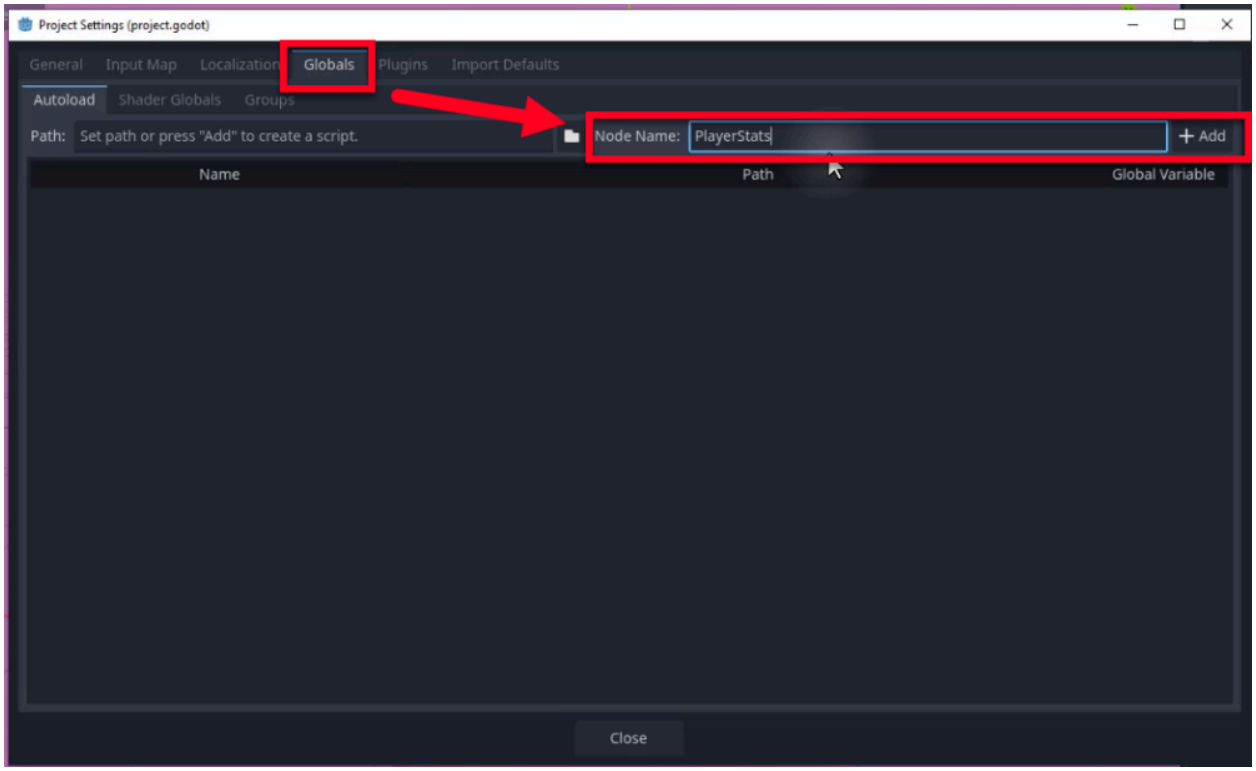
You access it by name (for example, `PlayerStats.score`) without needing a node reference. This is the recommended approach for game-wide data like scores, settings, and save-game state. Just be careful not to overuse autoloads for logic that belongs inside individual scenes; they work best as lightweight data holders or service providers.

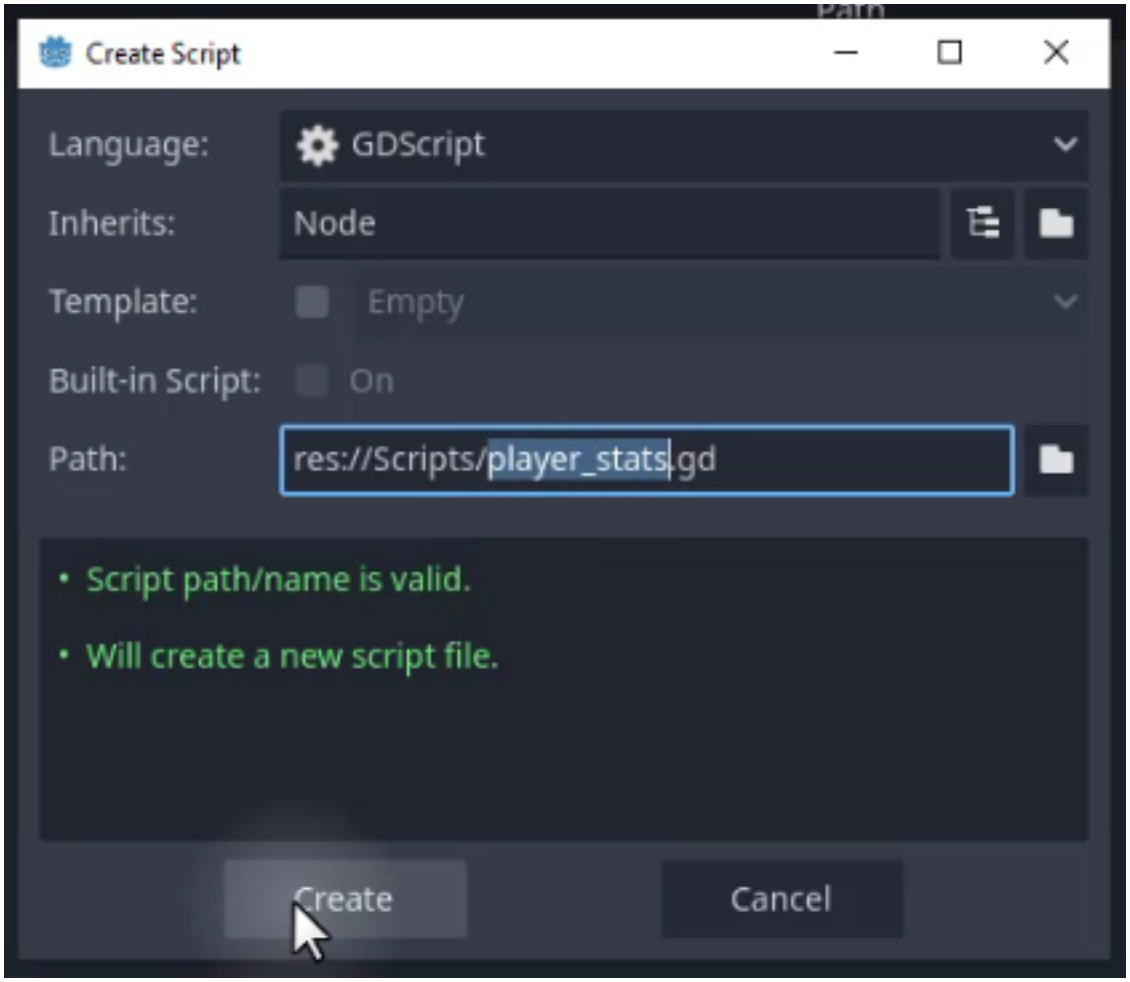
Learn more: Singletons (Autoload) (https://docs.godotengine.org/en/4.6/tutorials/scripting/singletons_autoload.html)

Creating the Autoload Script

To set up our scoring system:

1. Go to **Project** -> **Project Settings**.
2. Select the **Globals** tab.
3. In the **Node Name** field, enter `PlayerStats`.
4. Use the folder icon next to **Path** to create a new script. Save it as `player_stats.gd` in your `Scripts` folder.
5. Click **Add** to register the autoload.





This process will add the script to your Autoload's list, ready for scripting.

As such, all we need to do is open the newly created `player_stats.gd` script and add a variable to hold the score:

```
extends Node

var score : int = 0
```

Updating the Player Controller

We will now modify the `player_controller.gd` script to interact with this global score. Add a function to handle score increases.

```
func increase_score(amount: int):
    PlayerStats.score += amount
    print(PlayerStats.score)
```

By accessing `PlayerStats.score`, we are modifying the variable in the global singleton, ensuring the data persists even if the player dies or changes levels.

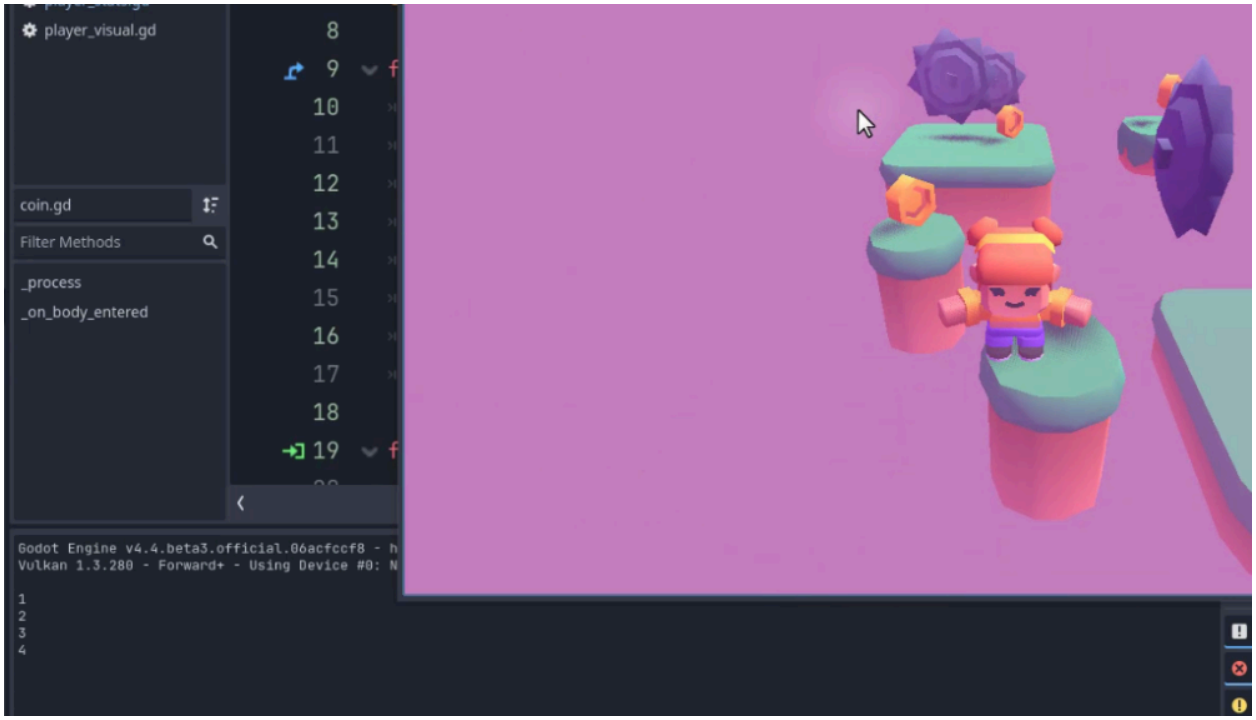
Connecting Coins to the Score

Finally, we must update the `coin.gd` script to call this new function when the coin is collected.

Modify the `_on_body_entered` function as below:

```
func _on_body_entered(body):
    # Only the player can collect coins
    if not body.is_in_group("Player"):
        return

    body.increase_score(1)
    queue_free()
```

Now, when you play the game and collect coins, you should see the score printing to the Output console.



Resetting the Score

Since the `PlayerStats` singleton persists for the entire game session, the score will not automatically reset when the player gets a "Game Over." We need to reset it manually.

Open `player_controller.gd` and update the `_game_over` function:

```
func _game_over():
    PlayerStats.score = 0
    # .. rest of code ..
```

This ensures that when the player restarts, they begin with a score of zero.

Conclusion

You have successfully implemented a system for collectibles and scoring. We created a dynamic coin object with visual flair and used Godot's Autoload feature to track the score persistently across the game. In the next chapter, we will take this a step further by displaying the score on the screen using a User Interface (UI).

User Interface Design

In this chapter, we will focus on adding essential UI elements to help keep track of the player's health and score. By the end of this tutorial, you will have a clear understanding of how to create and manage UI elements in Godot using the Control node system.

Setting Up the CanvasLayer

To begin, we need to create a **CanvasLayer**. This node is crucial for UI elements as it ensures that they remain fixed on the screen, independent of the camera's movement.

Navigate to your **PlayerScene** and create a new node called **CanvasLayer**.

Godot Insight: CanvasLayer and rendering layers

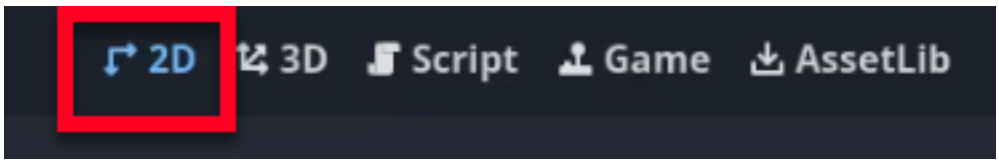
In a 3D game, everything you see is rendered through the camera. If you added UI nodes as regular children of the scene, they would exist in 3D space and move with the camera. CanvasLayer solves this by creating a separate 2D drawing layer that sits on top of (or behind) the 3D view. Its `layer` property controls the draw order: higher values draw in front. This is the standard way to build HUDs, menus, and overlays in Godot, keeping your UI cleanly separated from the game world.

Learn more: CanvasLayer (https://docs.godotengine.org/en/4.6/classes/class_canvaslayer.html)

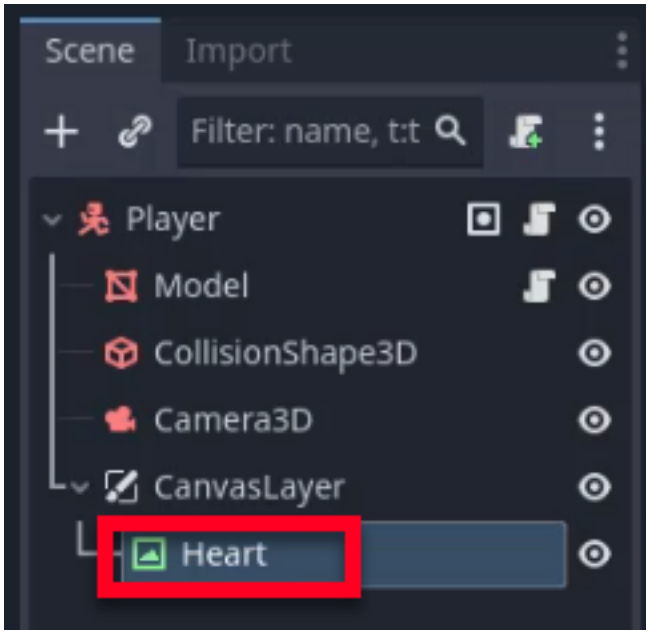
Adding Health Representations

Players need a clear, at-a-glance indicator of how much health they have left. We will add hearts to represent the player's health. These hearts will be displayed at the top left corner of the screen.

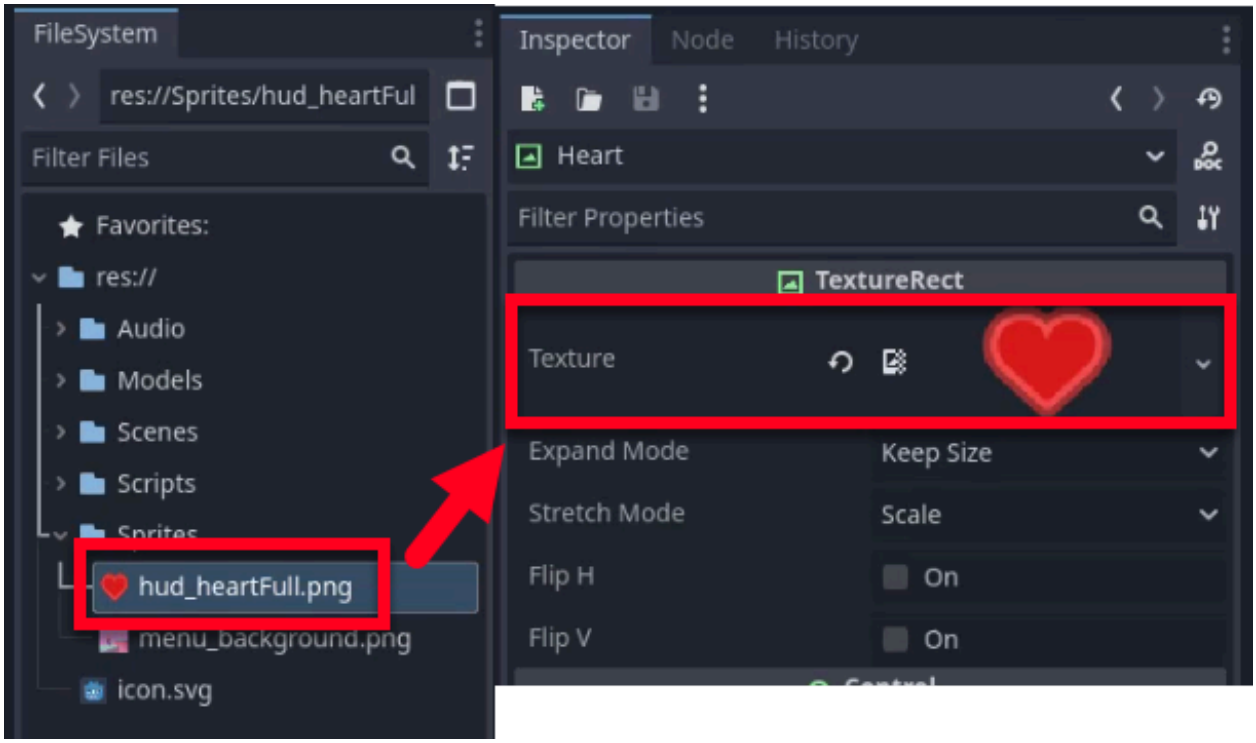
Since UI elements are 2D, first switch to **2D mode** in the editor toolbar for easier management and construction.



Next, right-click on the **CanvasLayer** and add a child node called **TextureRect**. Rename it to **Heart**.



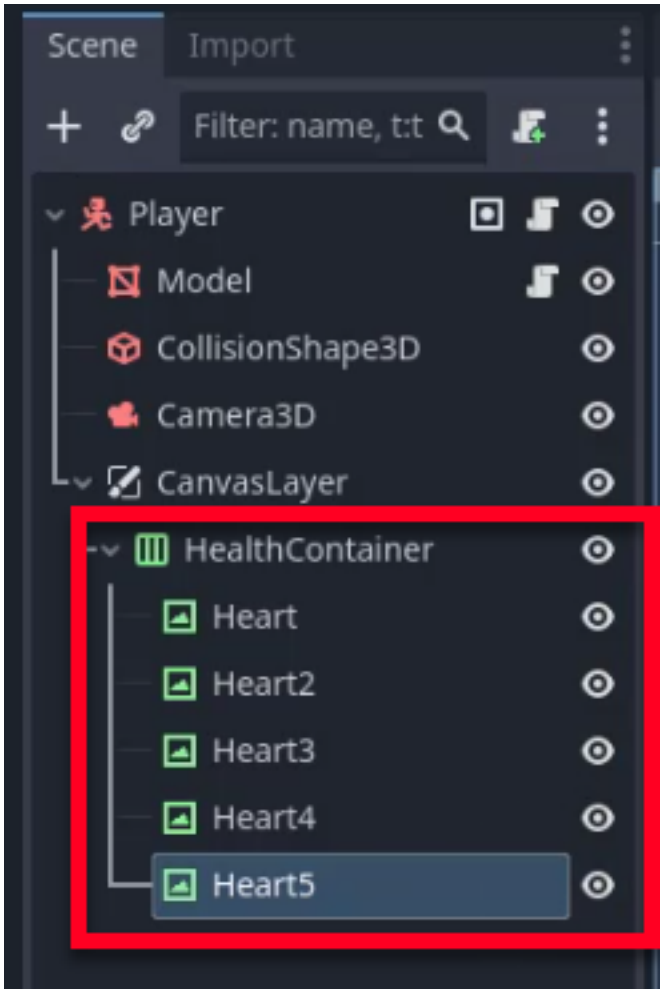
In the Inspector, set the **Texture** property to your heart sprite from the Sprites folder (e.g., **hud_heartFull**).



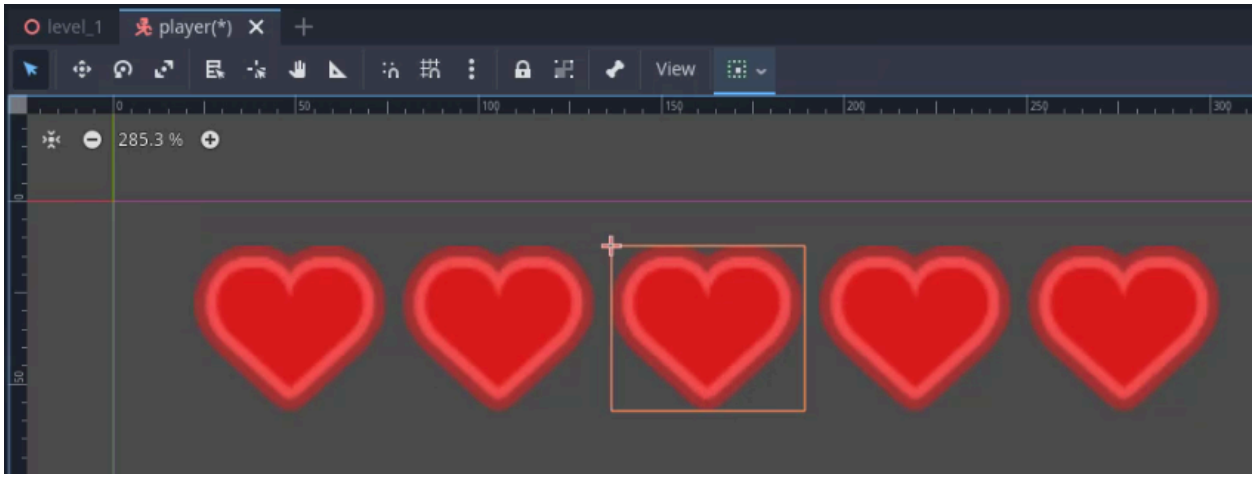
Using Horizontal Box Container

To manage multiple hearts efficiently, we will use a **Horizontal Box Container**. This node automatically arranges its child nodes horizontally, which is perfect for a row of health icons.

1. If you have any extra heart nodes from testing, delete all but one heart node.
2. Create a new node called **HBoxContainer** and rename it to **HealthContainer**.
3. Drag the remaining heart node into the **HealthContainer**.
4. Duplicate the heart node (`Ctrl` + `D`) to create multiple hearts.



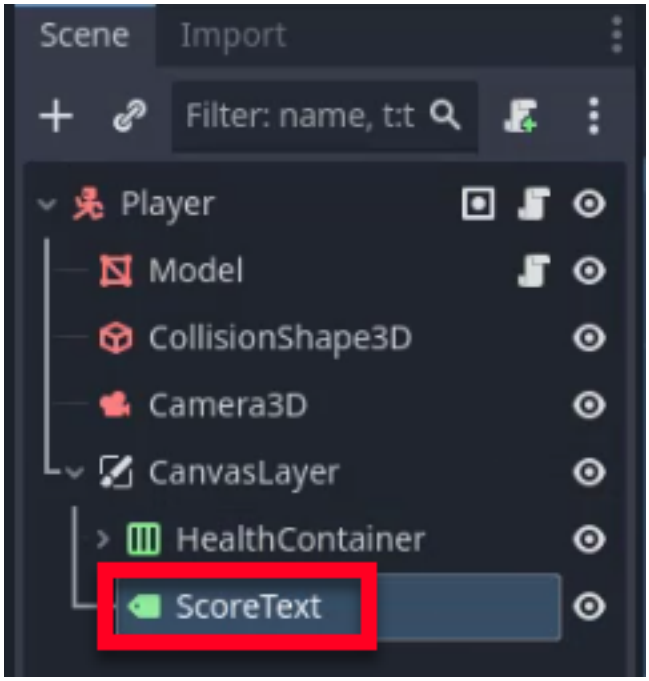
The **HBoxContainer** will automatically arrange them horizontally. Feel free to position the HBoxContainer to your desired location. We recommend giving it some spacing so the hearts aren't right on the edge of the screen.



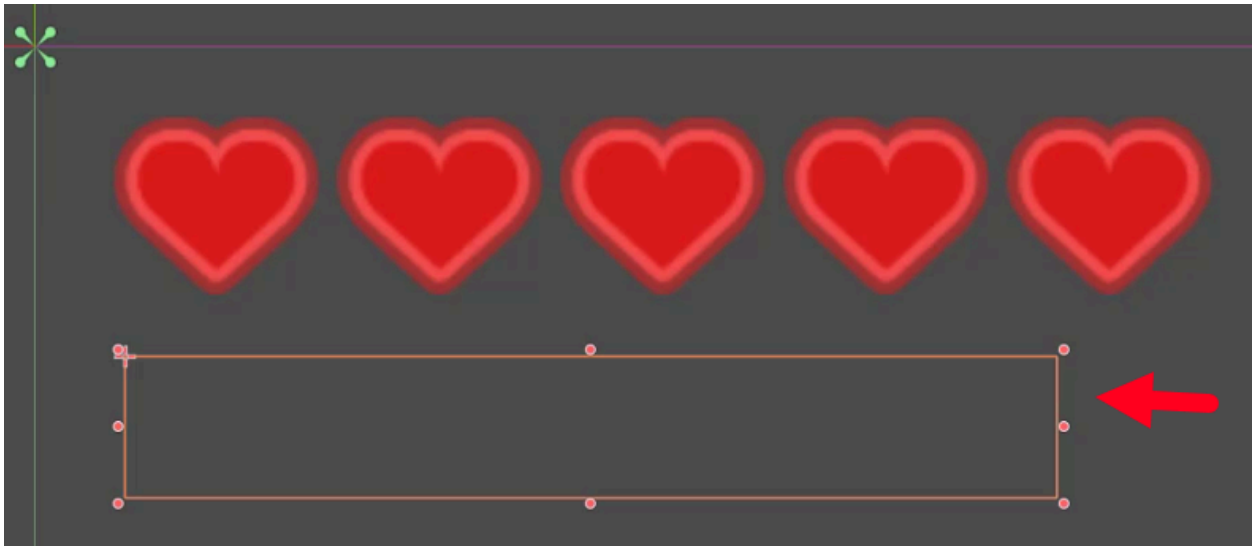
Adding Score Text

Below the hearts, we will add a text label to display the player's score.

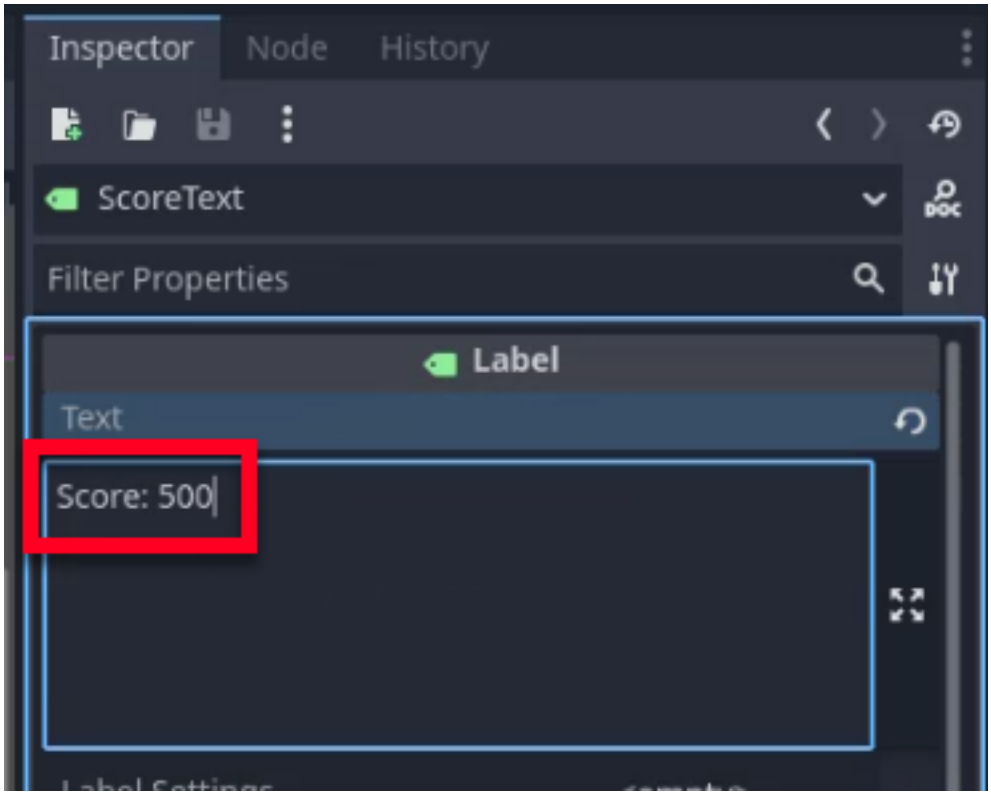
Create a new **Label** node outside the **HealthContainer** to avoid visual bugs. Rename the node to **ScoreText**.



Next, position and size the text box under your hearts in your desired location.



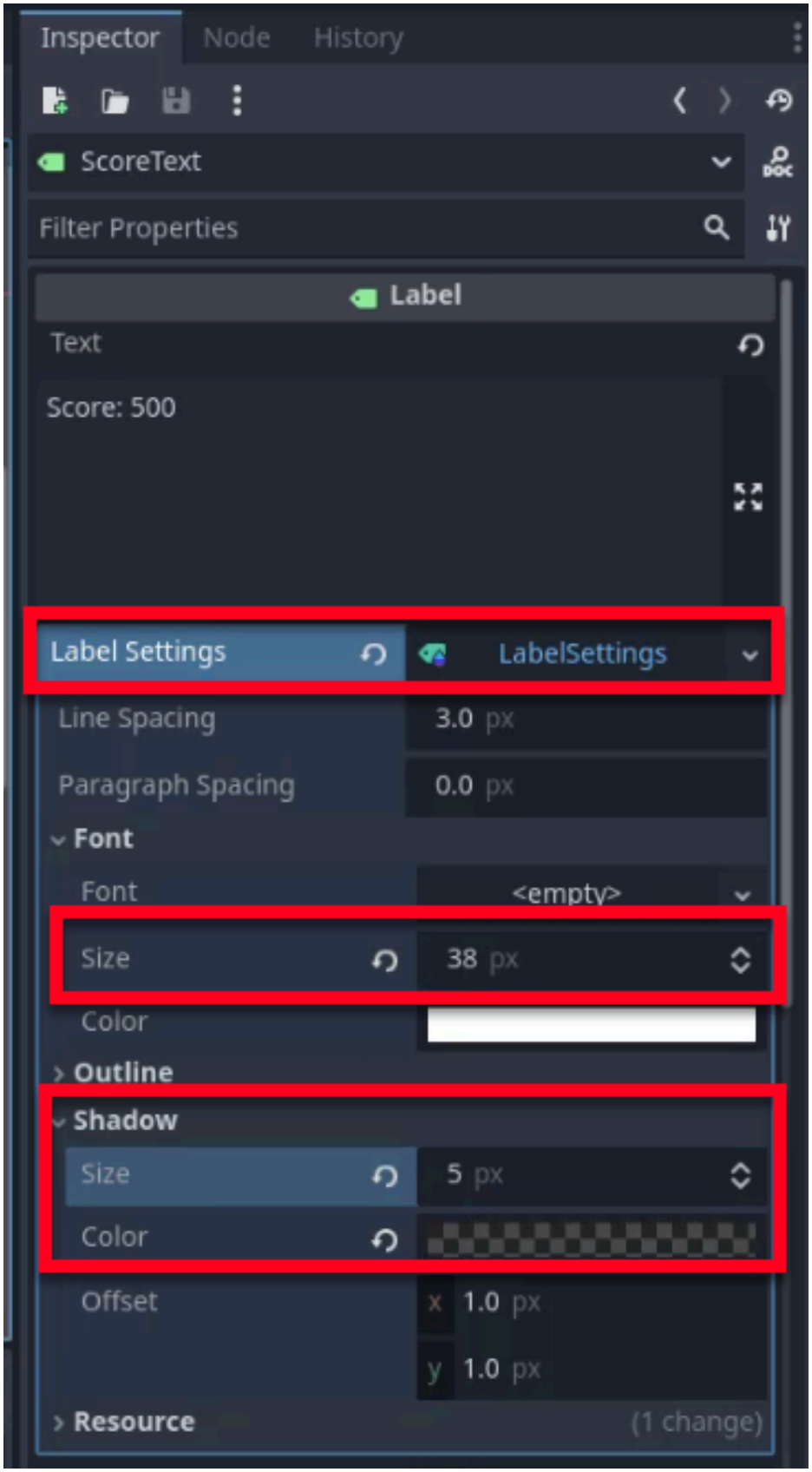
In the Inspector, set the text to something like "**Score: 500**" to see how it looks.



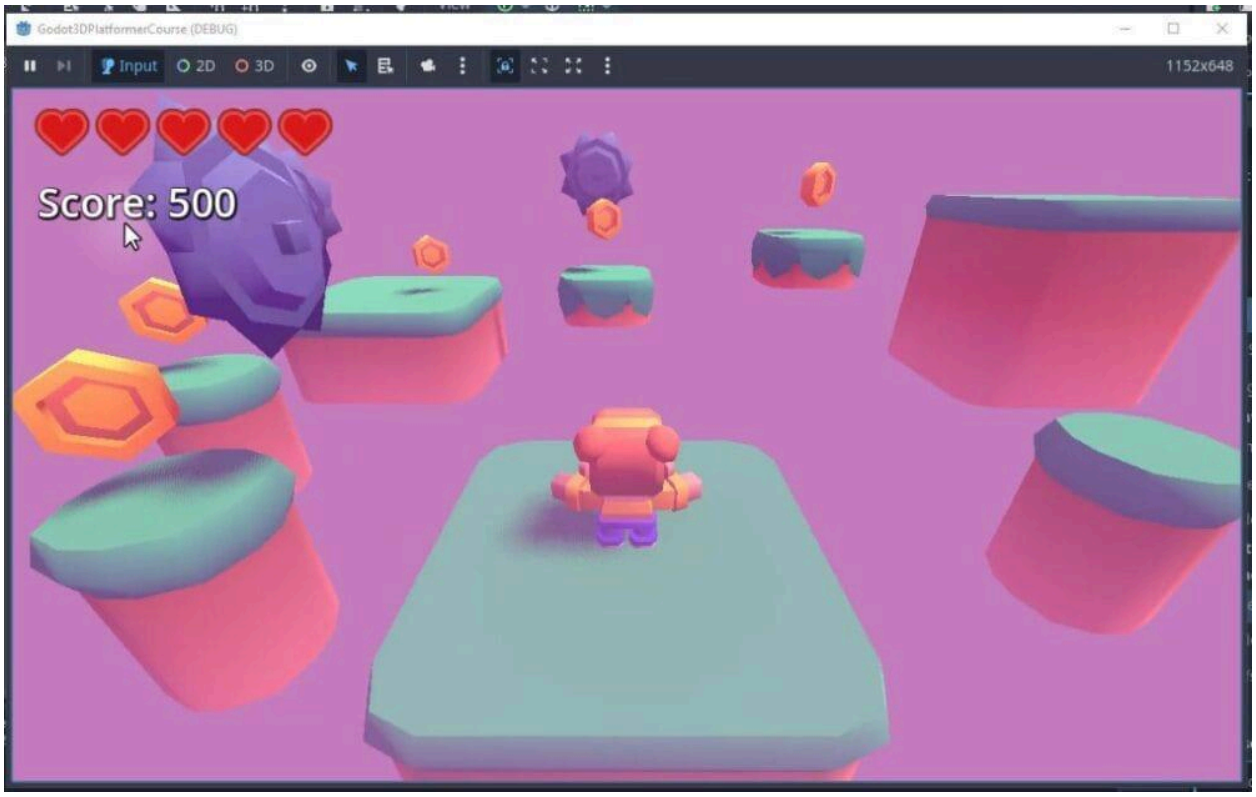
To improve visibility, we need to adjust the font size and add a shadow:

1. Create a new **Label Settings** resource in the Inspector.
2. Increase the font size (e.g., 38).

3. Add a shadow with a size of about 5 pixels and adjust the color for better contrast.


Inspector
Node
History
ScoreText
Filter Properties
Label
Text
Score: 500
Label Settings
LabelSettings
Line Spacing
3.0 px
Paragraph Spacing
0.0 px
Font
Font
<empty>
Size
38 px
Color
Outline
Shadow
Size
5 px
Color
Offset
x 1.0 px
y 1.0 px
Resource
(1 change)

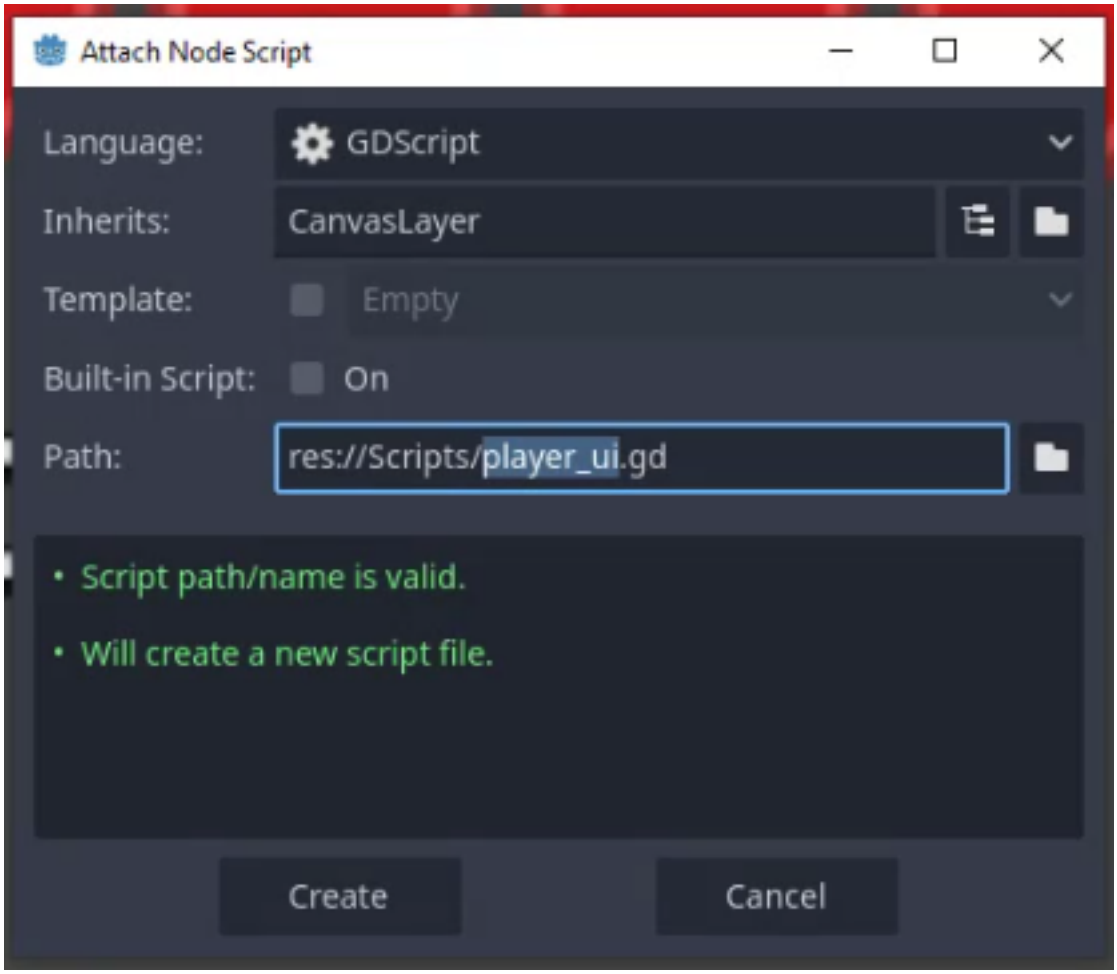
You should now be able to see your UI elements when testing the game. Feel free to tweak the settings further depending on your own project's look.



Scripting the UI

Now that our visual elements are in place, we need to make them interactive. We will create a script that handles updating the hearts and score text based on the player's actions.

To begin, select the **CanvasLayer**, attach a new script, and name it **player_ui.gd**. As usual, save the script in your Scripts folder.



Updating UI Elements Strategy

There are two primary ways to update UI elements based on player actions:

1. **Frame-by-Frame Update:** Check and update UI elements every frame. This method is feasible for a small number of UI elements but can impact performance as more elements are added.
2. **Event-Driven Update:** Update UI elements only when specific events occur, such as changes in the player's health or score. This method is more efficient and follows best practices by reducing dependencies between scripts.

We will use the **Event-Driven** approach using Signals. This keeps the UI script lightweight: it only runs code when something actually changes, rather than polling every frame.

Using Signals for Event-Driven Updates

In order to use our event-driven approach, we will define custom signals in our `player_controller.gd` script to notify the UI when the player's health or score changes. Open `player_controller.gd` and define these signals at the top of your script:

```
signal OnTakeDamage(hp: int)
signal OnUpdateScore(score: int)
```

> **Godot Insight: Event-driven UI with custom signals**
>
> Custom signals let you create your own events beyond the built-in ones like `body_entered`. By declaring `signal OnTakeDamage(hp: int)` in the player script, you define a broadcast channel that any other node can subscribe to. The UI connects to this signal once in `_ready()` and then passively waits for updates. This pattern keeps the player script unaware of the UI entirely: it simply emits the signal, and any listener (HUD, sound manager, analytics) can react independently. This decoupling makes your code easier to maintain and extend.

Emitting Signals

Next, we need to emit these signals at the appropriate times. For example, when the player takes damage or collects a coin, we will emit the respective signals.

Update your `take_damage` and `increase_score` functions in `player_controller.gd`:

```
func take_damage(amount: int):
    health -= amount
    OnTakeDamage.emit(health)

    if health <= 0:
        _game_over()

func increase_score(amount: int):
    PlayerStats.score += amount
    OnUpdateScore.emit(PlayerStats.score)
```

Listening to Signals

In our `player_ui.gd` script, we will create functions that will be called when these signals are emitted. These functions will update the UI elements accordingly.

Add the following empty functions to `player_ui.gd`:

```
func _update_hearts(health: int):
    pass

func _update_score_text(score: int):
    pass
```

To connect the signals to these functions, we will use the `connect` method in the `_ready` function of our player UI script.

```
func _ready():
    var player = get_parent()

    # Subscribe to the player's signals for event-driven UI updates
    player.OnTakeDamage.connect(_update_hearts)
    player.OnUpdateScore.connect(_update_score_text)
```

Now whenever these signals are emitted from the player, our UI update functions will trigger.

Initializing Variables

With our signals connected, we can now work on our update functions. We first need to initialize some variables to reference our UI elements. These variables will be used to update the UI elements when the signals are emitted. We need a reference to our health container, an array of all our hearts, and a reference to our score text label.

Add these variables to `player_ui.gd`:

```
@onready var health_container = $HealthContainer
var hearts: Array = []
```

```
@onready var score_text: Label = $ScoreText
```

Updating the Score Text

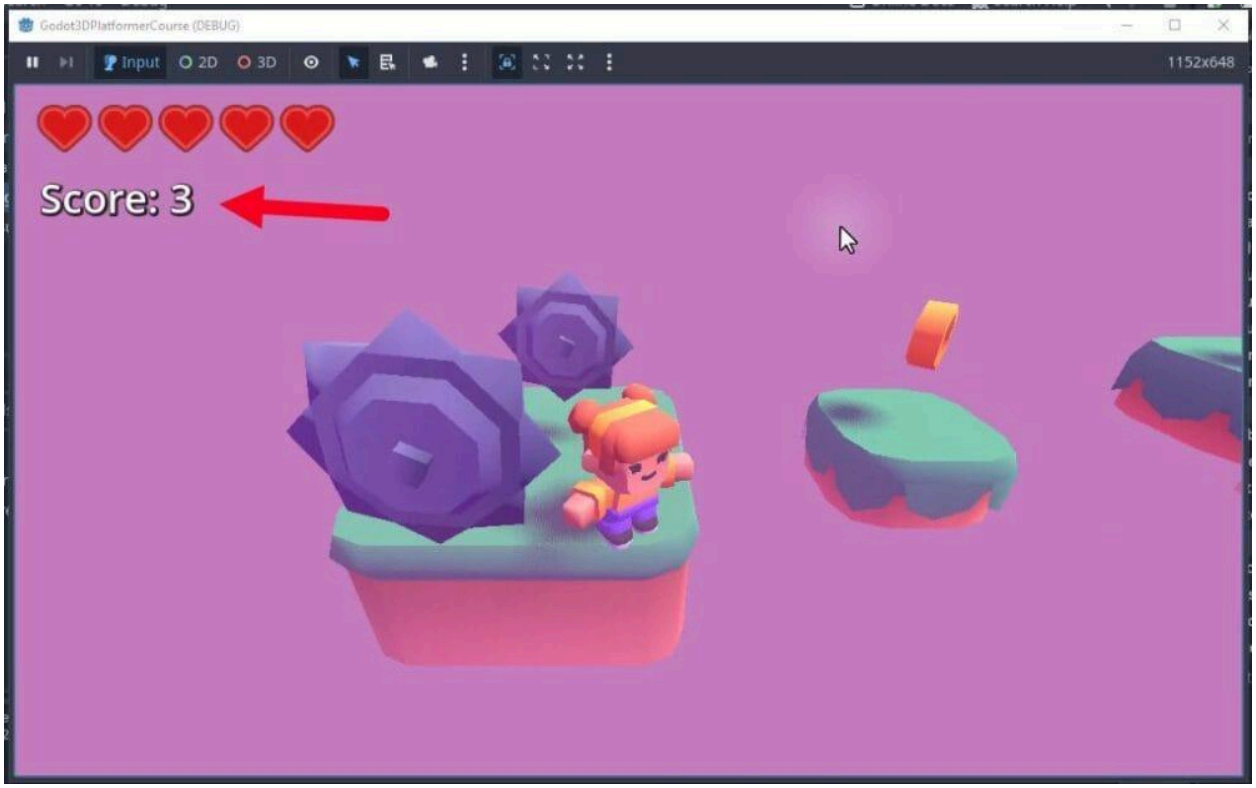
Let's start by updating the score text. This is a straightforward process where we update the text property of the score label whenever the score changes.

```
func _update_score_text(score: int):
    score_text.text = "Score: " + str(score)
```

To ensure we overwrite our default text when the game starts, we also want to call this function in the script's `_ready` function:

```
func _ready ():
  var player = get_parent()

  player.OnTakeDamage.connect(_update_hearts)
  player.OnUpdateScore.connect(_update_score_text)

  _update_score_text(PlayerStats.score)
```

If you test the game now, you should see the score text update accordingly, giving real-time feedback about our game.



Updating the Heart Icons

Next, we will focus on the heart icons. We need to access the heart nodes that are children of the `HealthContainer` node. These heart nodes will be stored in our `hearts` array.

In the `_ready` function, initialize the `hearts` array by retrieving all the child nodes of `HealthContainer`:

```
func _ready ():
  hearts = health_container.get_children()
  ...
```

Now, implement the `_update_hearts` function. This function will loop through each heart icon and set its visibility based on the player's health.

The logic is simple: we iterate through the `hearts` array. For each heart, we check if its index is less than the player's current health. If it is, the heart is visible; otherwise, it is hidden.

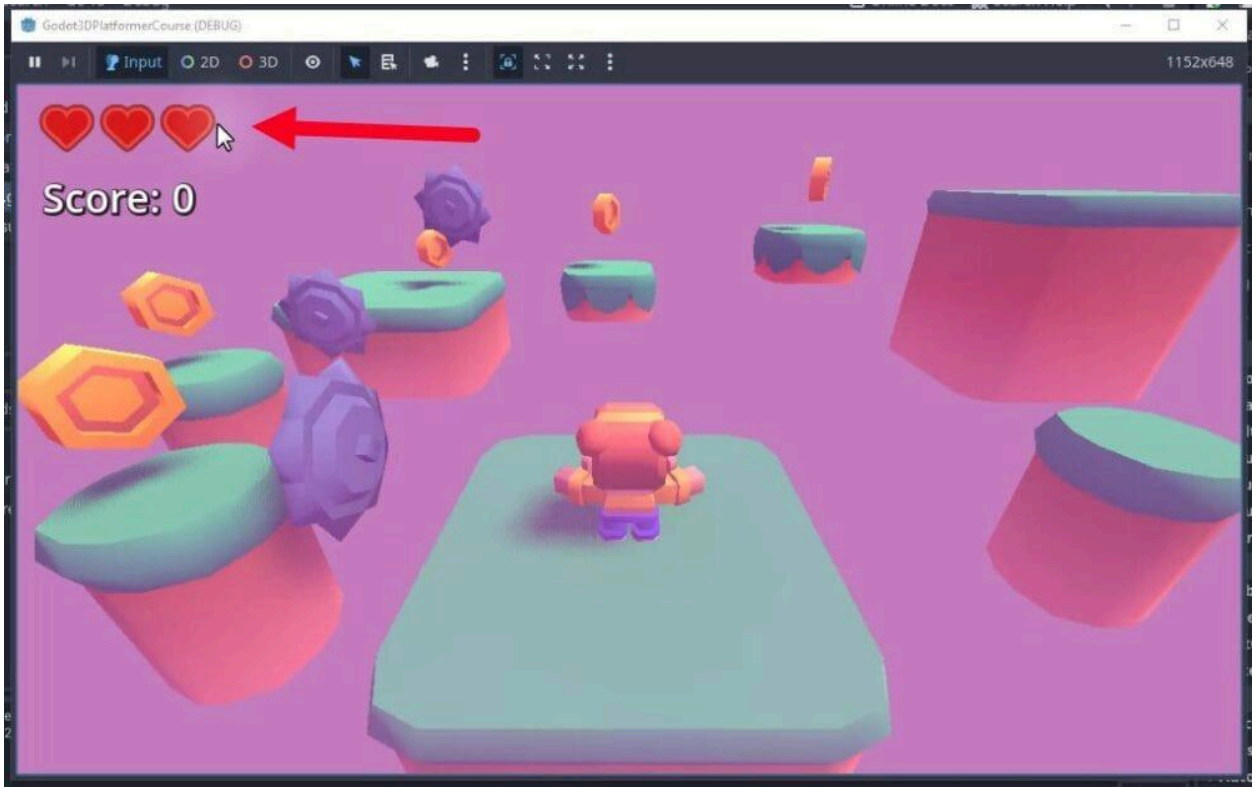
```
func _update_hearts(health: int):
    for i in len(hearts):
        hearts[i].visible = i < health
```

Like the score text, we need to ensure the heart icons are updated at the start of the game. Call the `_update_hearts` function in `_ready`:

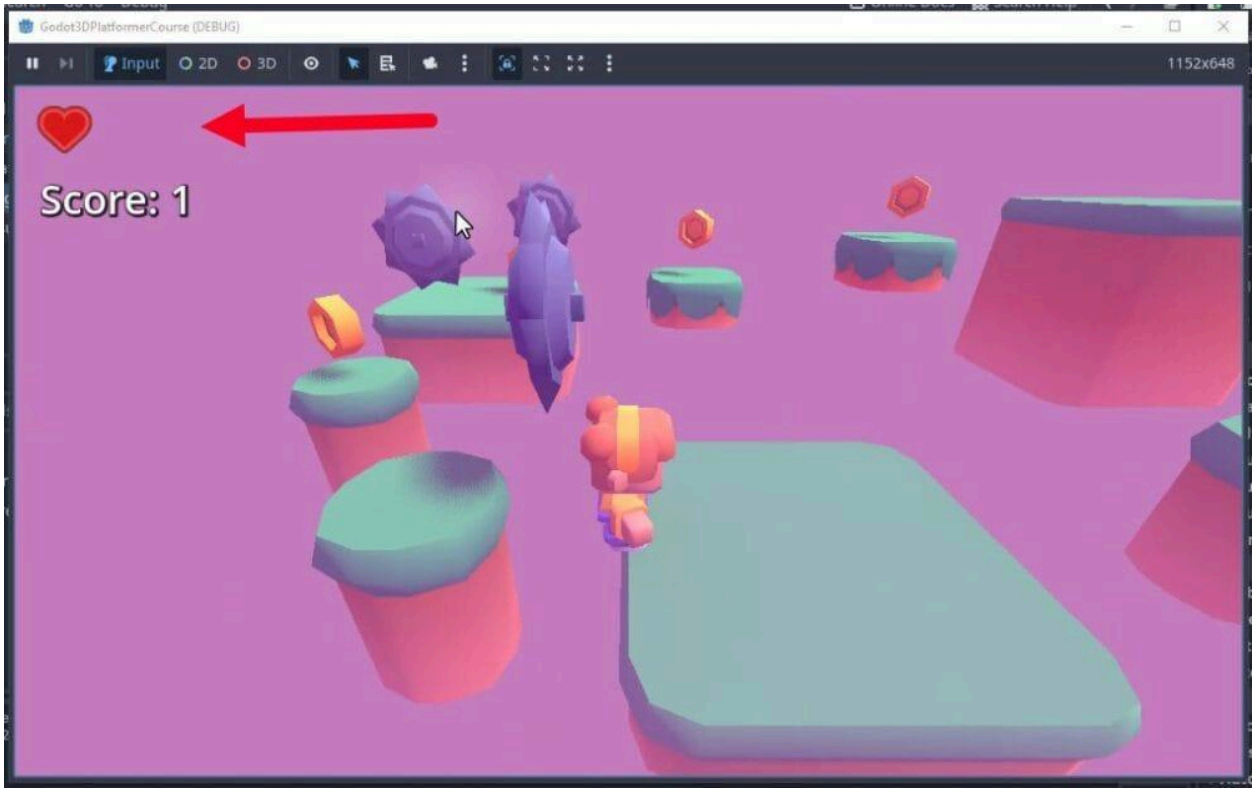
```
func _ready():
    ...
    _update_hearts(player.health)
```

Testing the Implementation

Finally, we will test our implementation to ensure that the heart icons are correctly displayed based on the player's health. When the game starts, the player should have the current full health number of hearts visible.



As the player takes damage, the number of visible hearts should decrease accordingly. Take damage to see the hearts disappear one by one until the player's health reaches zero.



Summary

In this chapter, we have learned how to set up and manage UI elements for displaying the player's health and score. We accessed the heart nodes, created a function to update their visibility based on the player's health, and connected the necessary signals to ensure the UI is updated in real-time. This knowledge will be valuable as you continue to develop and refine your game's user interface. These principles can also easily be applied to other projects.

In the next chapter, we will move on from our UI and work on our end flag, which will represent the goal the player is trying to reach in the level.

Level Management and Progression

In this chapter, we will implement the logic required to progress through the game. We will create an “End Flag” object that acts as a goal for the player. When the player reaches this flag, the game will automatically transition to the next level.

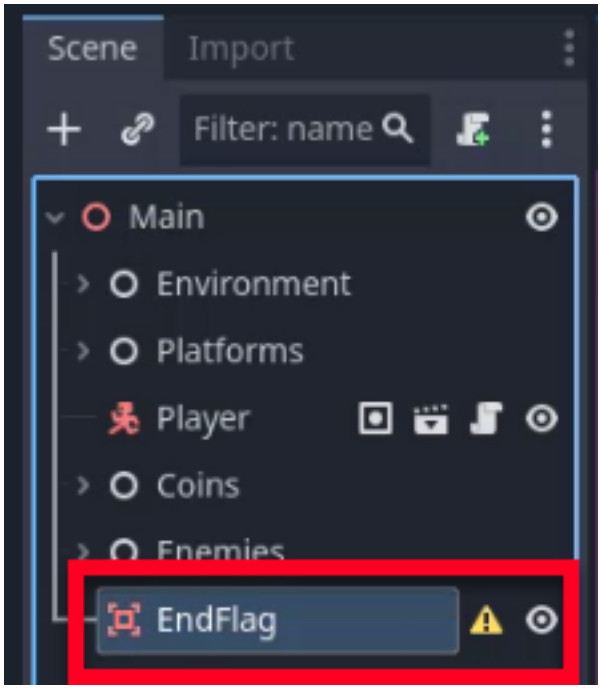
To test this functionality, we will also create a second level with a distinct visual style and link the two scenes together. By the end of this chapter, you will have a multi-level loop where the player can complete a stage and advance to the next challenge.

Creating the End Flag

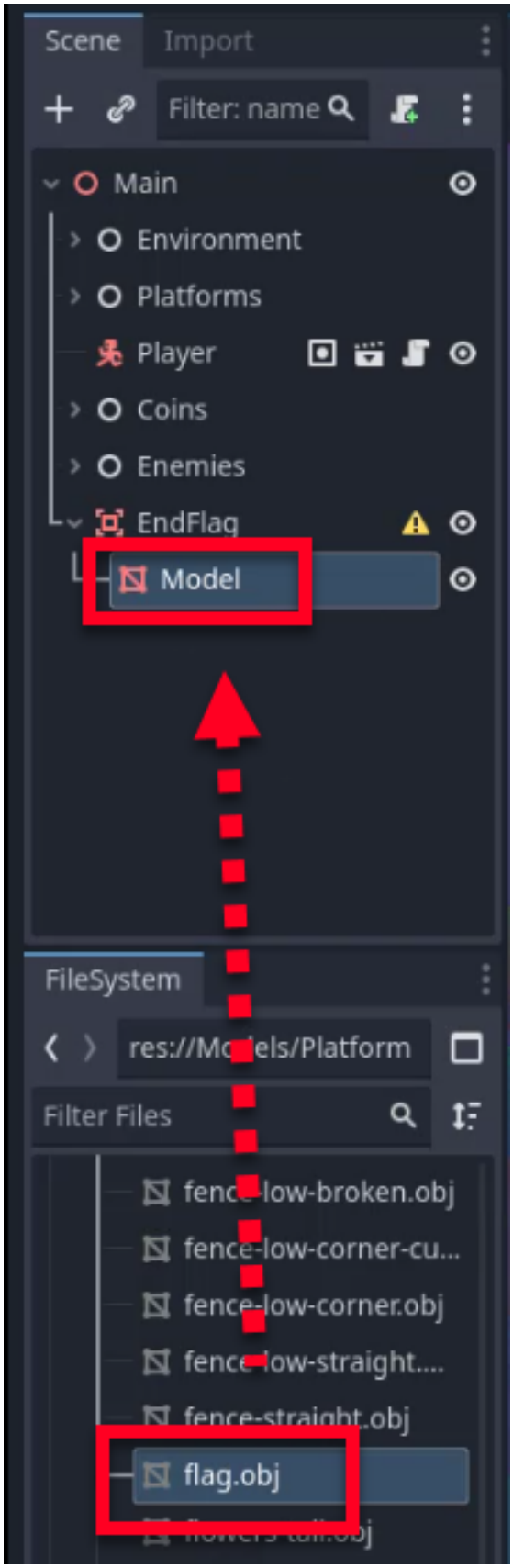
The End Flag shares many similarities with the coins and enemies we created earlier. It uses an `Area3D` node to detect when the player enters a specific zone, but instead of collecting an item or taking damage, it triggers a scene change.

Setting Up the Node Structure

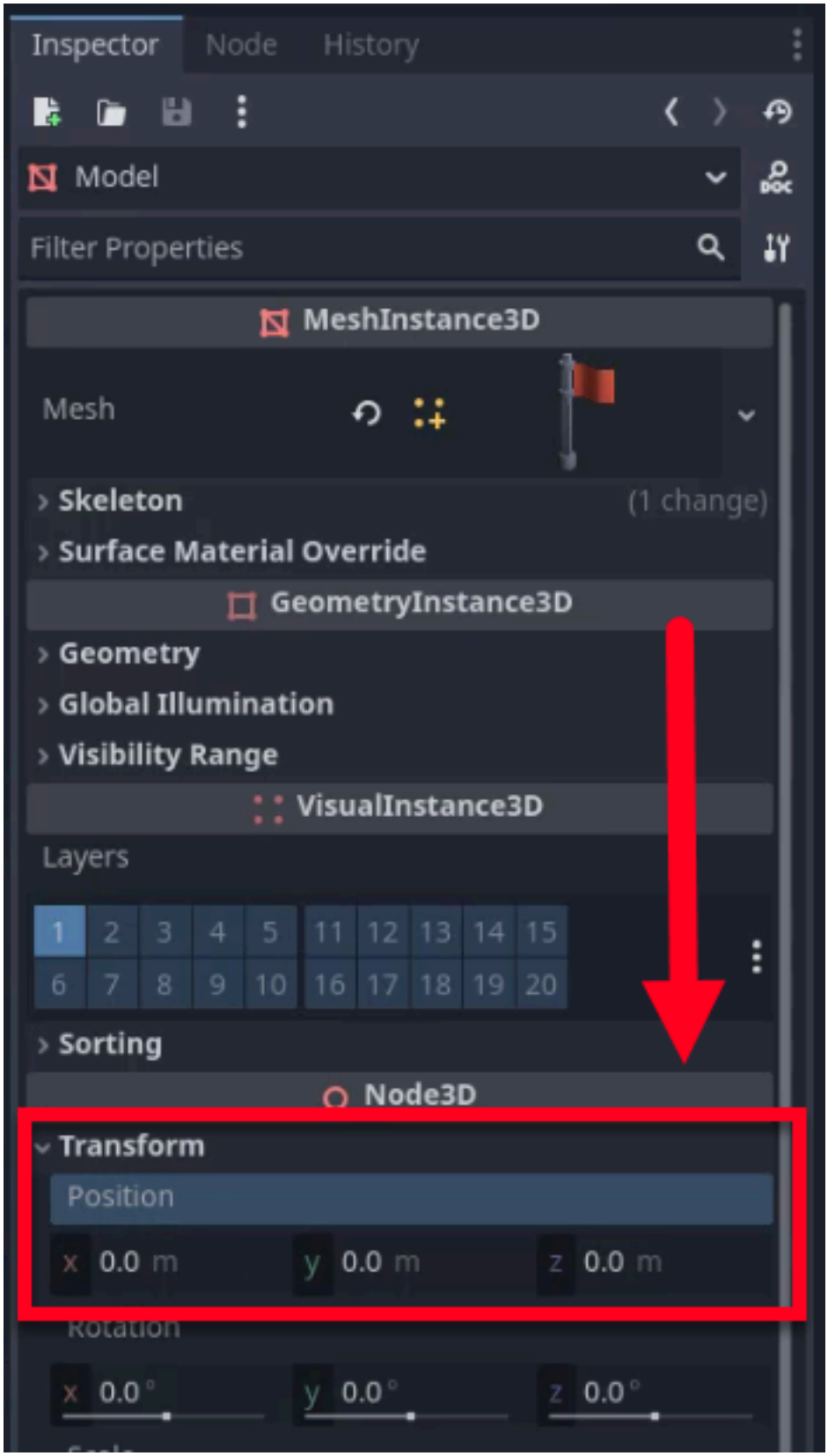
First, we need to create the basic node structure for our flag. Create a new **Area3D** node in your scene and rename the node to `EndFlag`.



Next, we need a visual representation. Navigate to your models folder and drag the 3D flag model (e.g., `flag.obj`) into the scene as a child of the `EndFlag` node. Rename the model node to `Model`.


Scene
Import
Filter: name
Main
Environment
Platforms
Player
Coins
Enemies
EndFlag
Model
FileSystem
res://Models/Platform
Filter Files
fence-low-broken.obj
fence-low-corner-cu...
fence-low-corner.obj
fence-low-straight....
fence-straight.obj
flag.obj

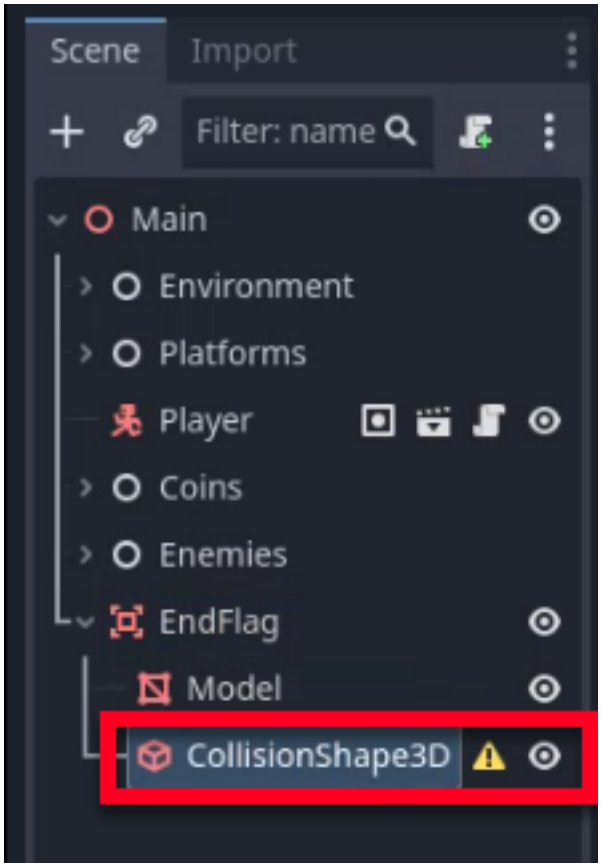
Next, ensure the model is centered relative to the parent node by setting its **Position** to `(0, 0, 0)` in the Inspector.


Inspector
Node
History
Model
Filter Properties
MeshInstance3D
Mesh
Skeleton
(1 change)
Surface Material Override
GeometryInstance3D
Geometry
Global Illumination
Visibility Range
VisualInstance3D
Layers
1 2 3 4 5 11 12 13 14 15
6 7 8 9 10 16 17 18 19 20
Sorting
Node3D
Transform
Position
x 0.0 m
y 0.0 m
z 0.0 m
Rotation
x 0.0 °
y 0.0 °
z 0.0 °

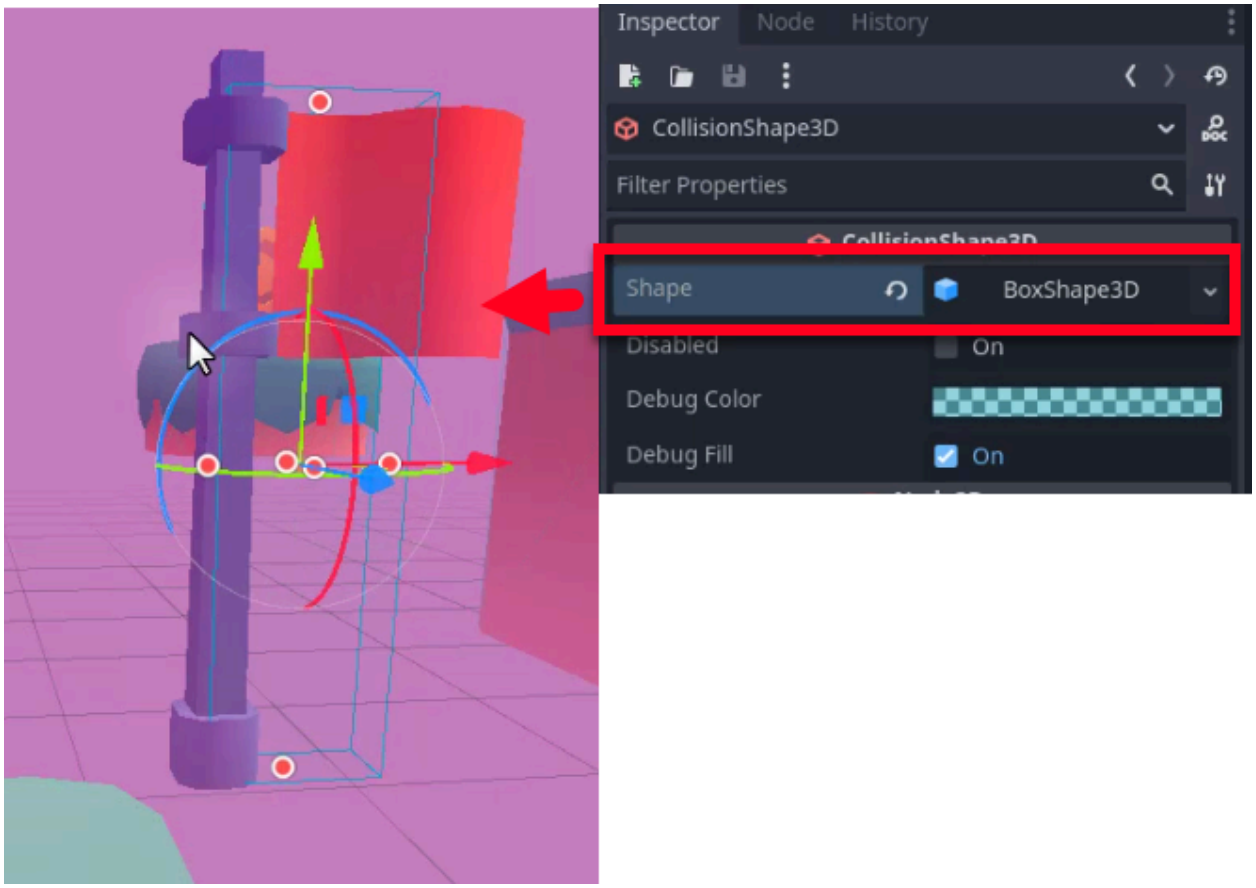
Adding a Collision Shape

To detect the player, we must add a collision shape to the Area3D.

1. First, attach a **CollisionShape3D** node to the `EndFlag` node.



2. In the Inspector, set the **Shape** to **BoxShape3D**.
3. Resize the box shape to approximate the size of the flag model. It does not need to be perfect; it just needs to define the zone where the player triggers the level completion.

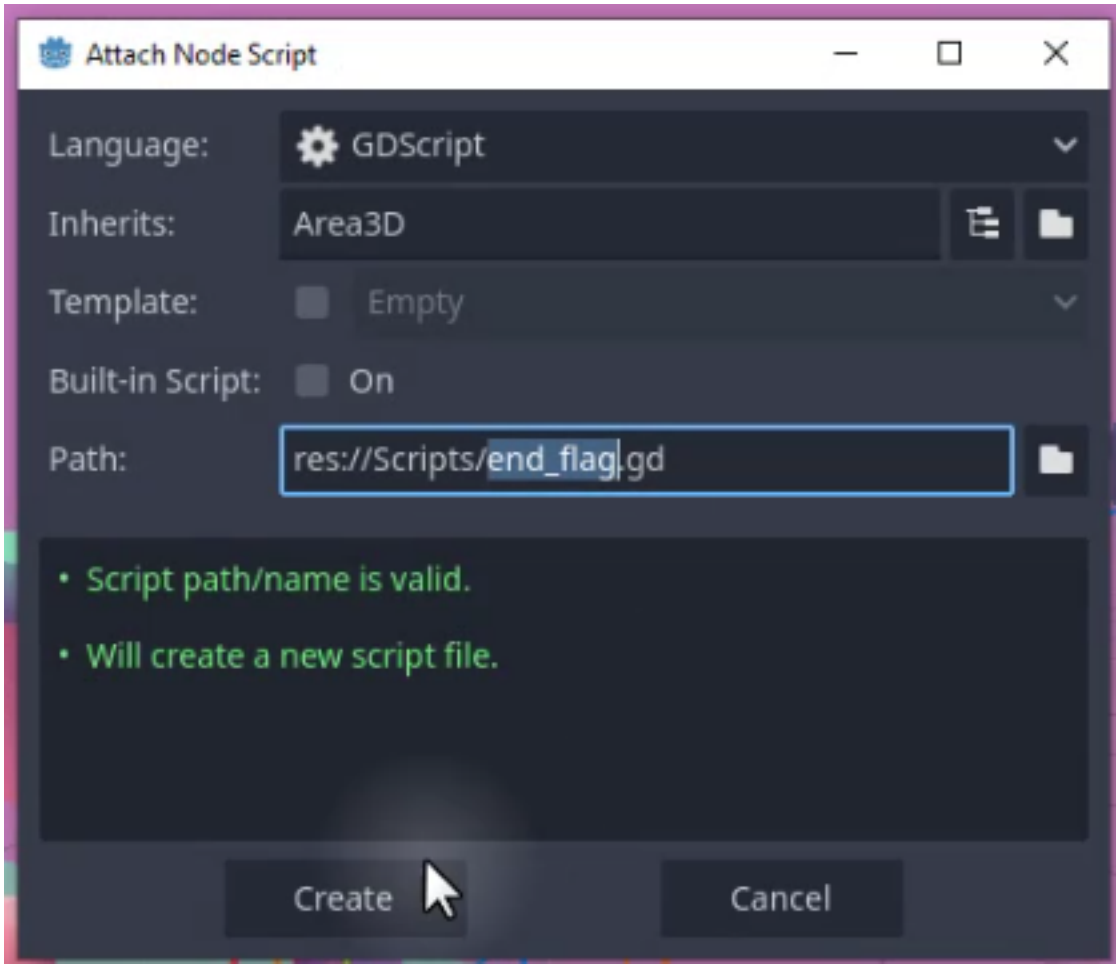


Scripting the Scene Transition

Now that the object is set up, we need to add the logic to handle the scene transition.

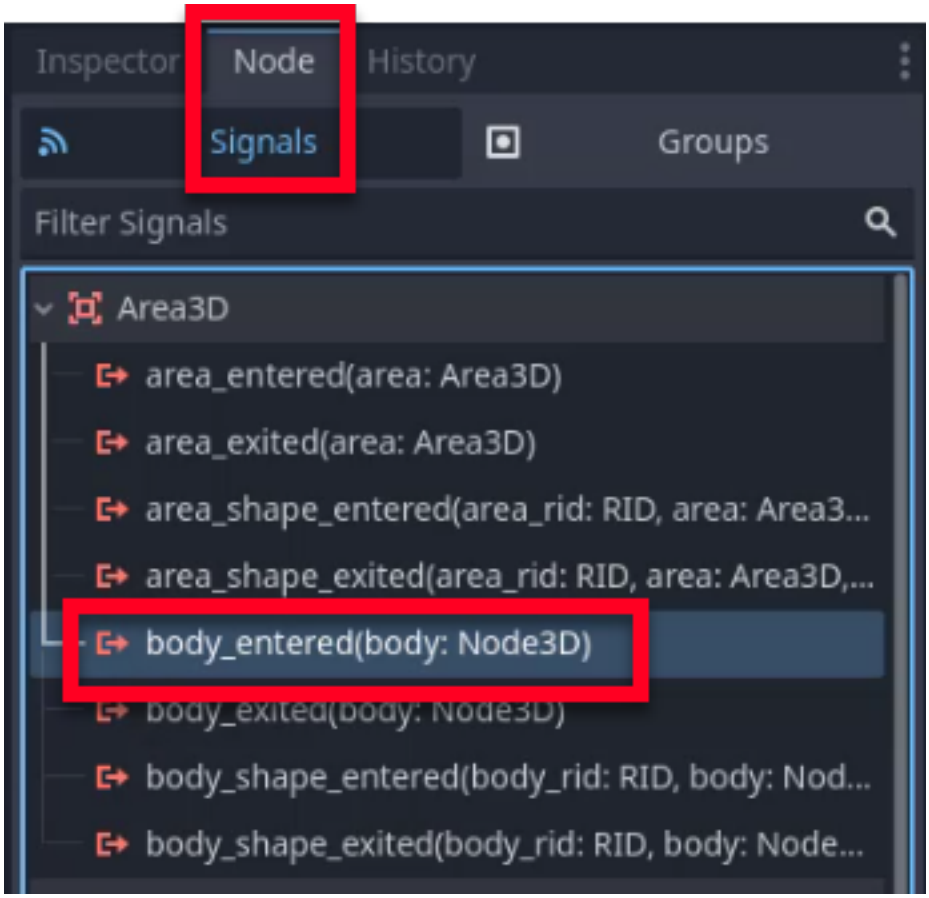
Creating the Script

To begin, attach a new script to the `EndFlag` node. Name it `end_flag.gd` and save it in your `Scripts` folder.

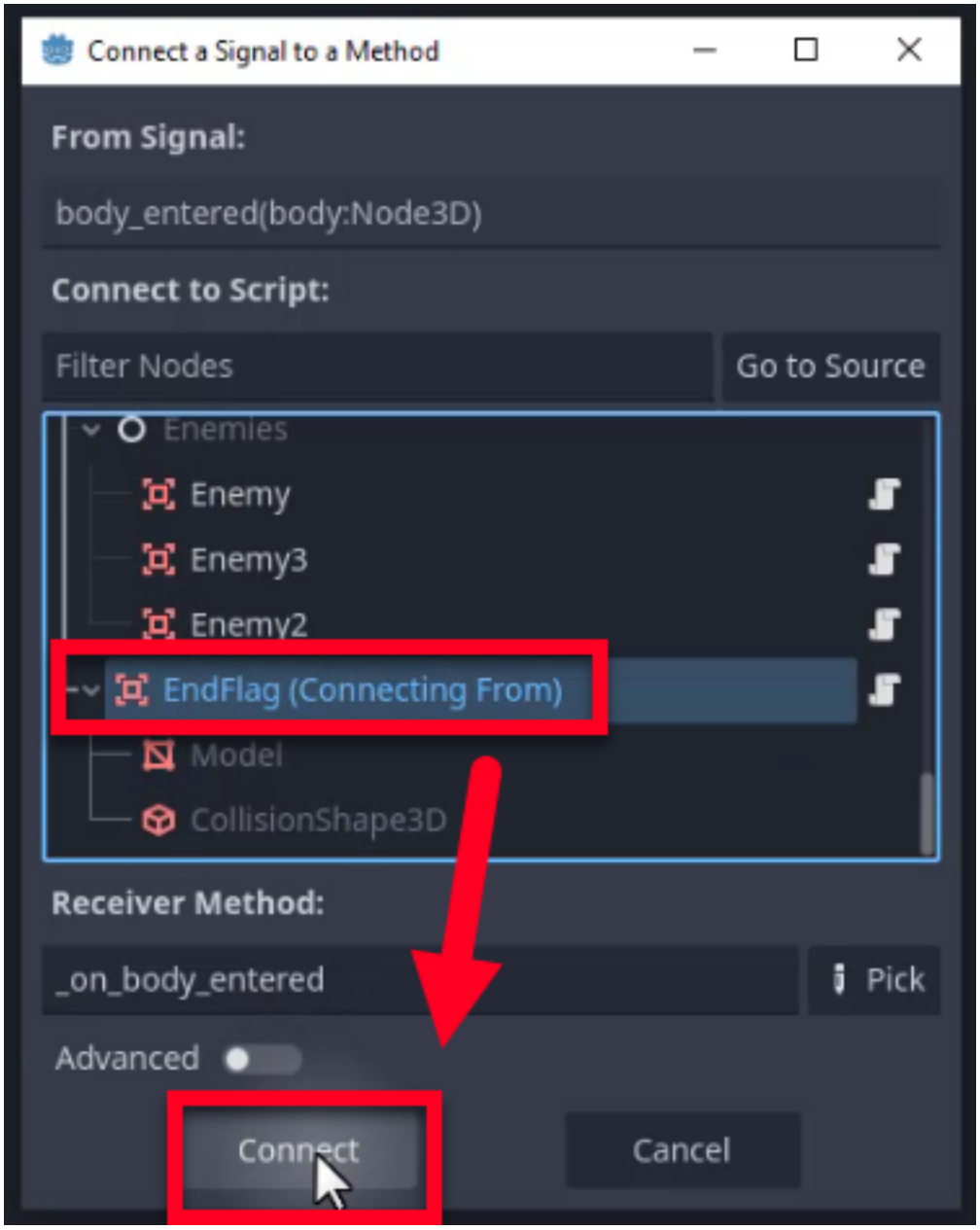


Connecting the Signal

Like the coins and enemies, we will use the `body_entered` signal to detect when the player touches the flag. To connect it, select the `EndFlag` node and go to the **Node** tab (Signals) in the Inspector. Double-click the `body_entered` signal to bring up the connection dialog.



In the connection dialog, connect it to the `end_flag.gd` script. As usual, this will create the `_on_body_entered` function.



Implementing the Logic

In the script, we need a way to define *which* scene to load next. We will use an exported `PackedScene` variable, allowing us to assign the next level directly in the Inspector for each instance of the flag.

Godot Insight: PackedScene and scene instancing

A PackedScene is a resource that stores an entire scene tree in a portable format. You can think of it as a blueprint: calling `instantiate()` on it creates a fresh copy of that scene's node tree, and `change_scene_to_packed()` replaces the entire current scene with it. By making our `scene_to_load` variable an exported PackedScene, we can drag any `.tscn` file from the FileSystem dock into the Inspector. This avoids hard-coding file paths in the script and makes the EndFlag reusable across levels with zero code changes.

Learn more: PackedScene (https://docs.godotengine.org/en/4.6/classes/class_packedscene.html)

Add the following code to `end_flag.gd`:

```
extends Area3D

@export var scene_to_load : PackedScene

func _on_body_entered(body):
    if not body.is_in_group("Player"):
        return

    get_tree().change_scene_to_packed(scene_to_load)
```

This script checks if the body entering the area is the player. If it is, it tells the scene tree to change to the scene stored in `scene_to_load`.

Saving as a Scene

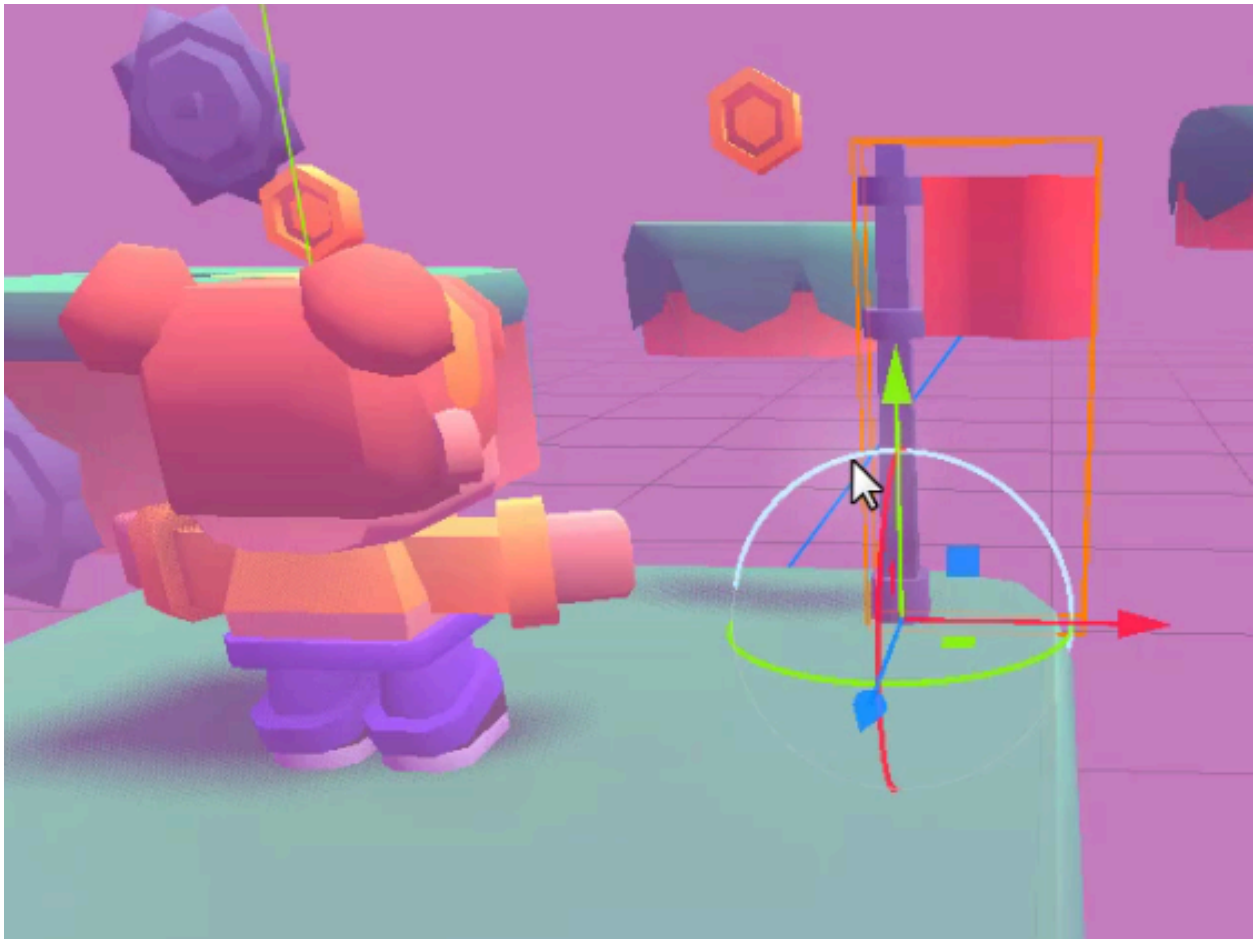
To reuse the End Flag across multiple levels, we need to save it as a scene.

1. Right-click the `EndFlag` node in the Scene dock.
2. Select **Save Branch as Scene**.
3. Save it as `end_flag.tscn` in the `Scenes` folder.

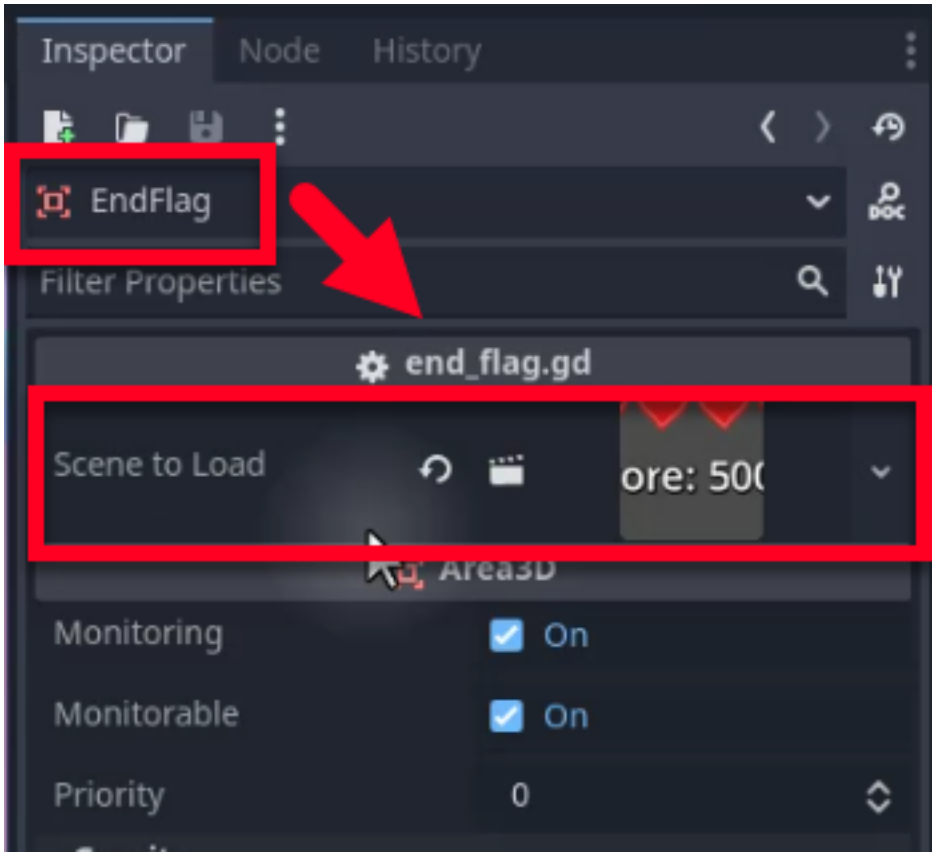

Scene
Import
Filter: name
Main
Environment
Platforms
Player
Coins
EndFlag
FileSystem
res://Scenes/level_1.ts
Filter Files
Favorites:
res://
Audio
Models
Scenes
coin.tscn
end_flag.tscn
level_1.tscn
player.tscn

Testing the Transition

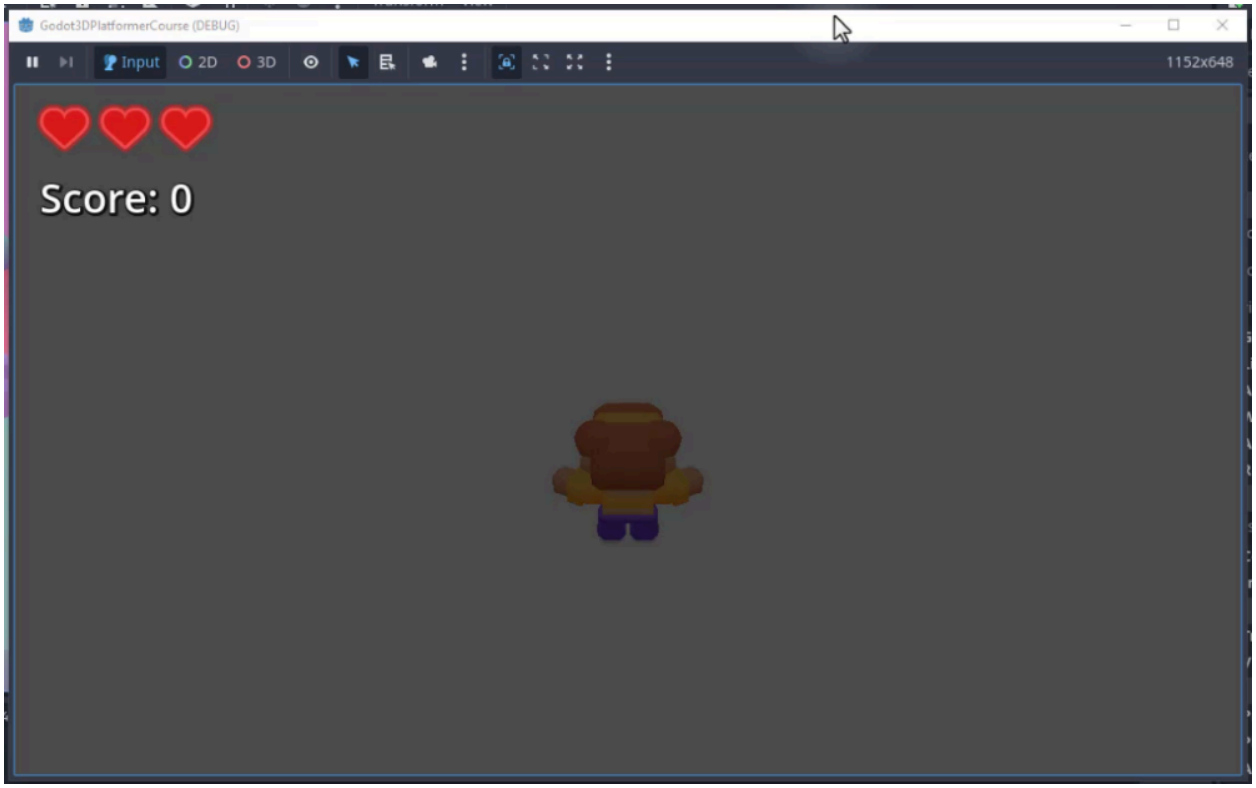
Before we build a full second level, let's verify that the flag works. To make testing easier, position the `EndFlag` near the player in your current scene.



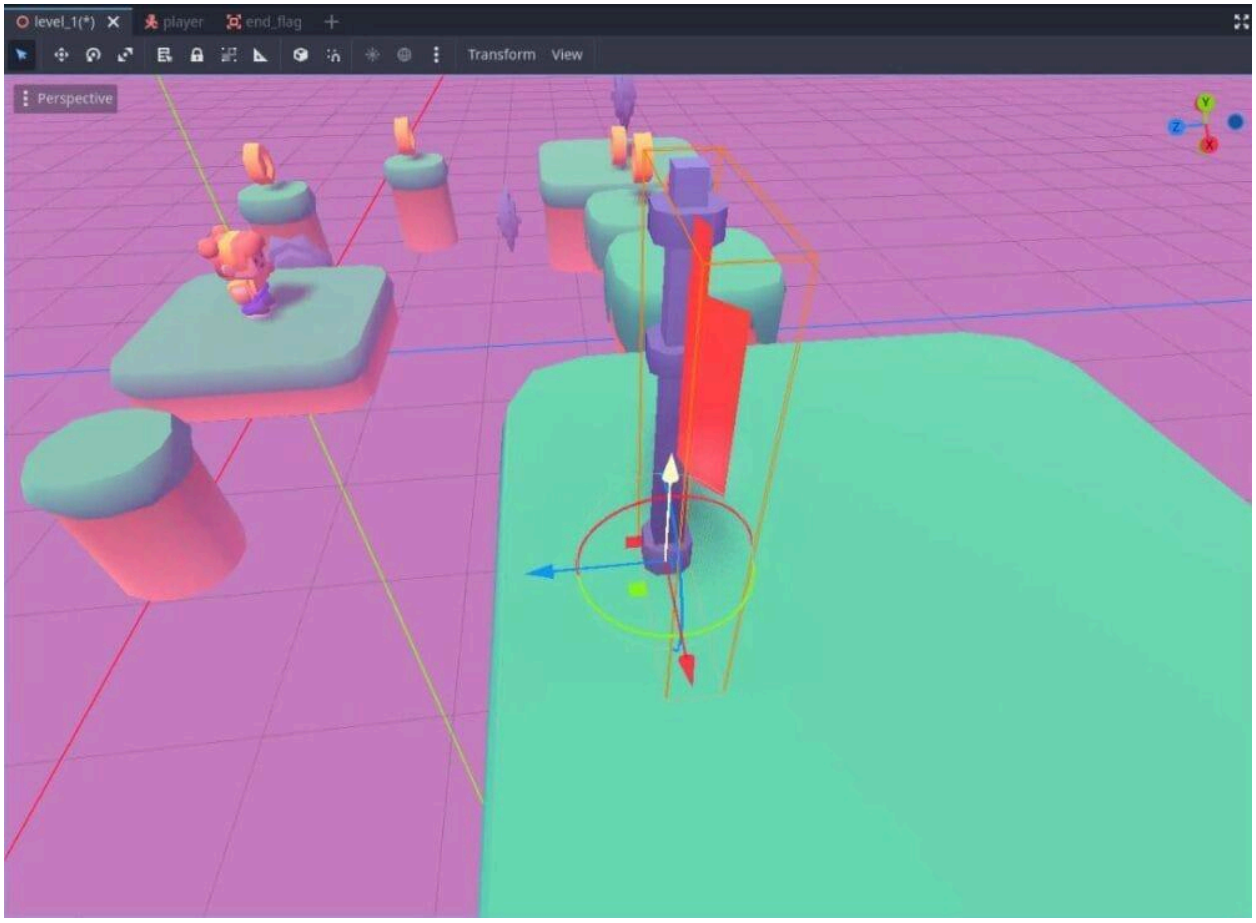
Select the `EndFlag` node. In the Inspector, you will see the **Scene To Load** property. Drag a different scene (e.g., `player.tscn` or a temporary test scene) into this property. Note that you cannot load the *current* scene (Level 1) directly in this manner without reloading, so using a different scene confirms the transition logic works.



Run the game and walk into the flag. If the game transitions to the assigned scene, the logic is correct.



Once tested, make sure to move the End Flag to the actual end of your level.

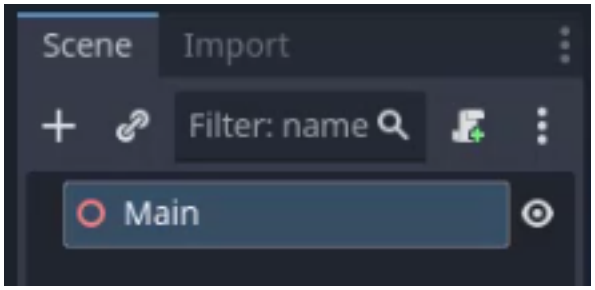


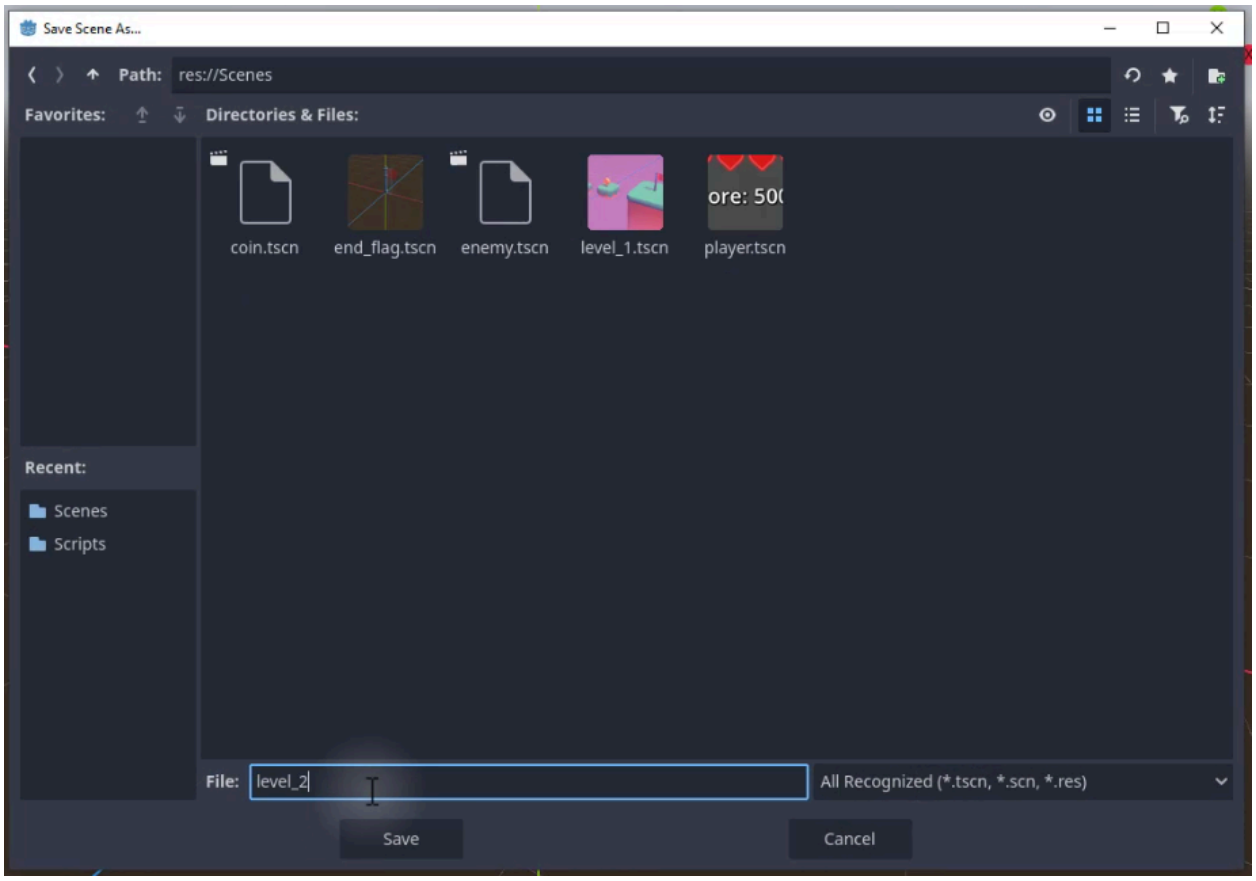
Creating Level 2

A single level makes for a short game. To make the experience feel like a real progression, we need a second level with its own distinct look and layout. The fastest way to get started is to duplicate our existing level and then customize it.

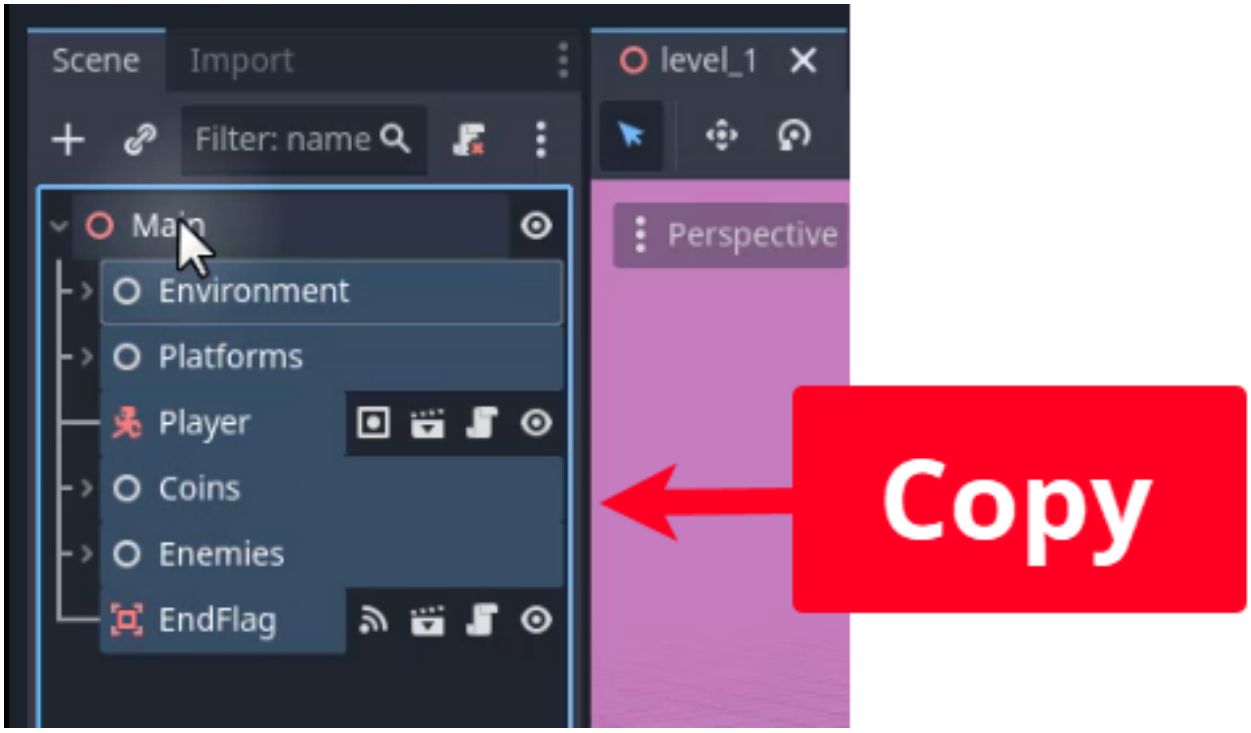
Duplicating Level 1

To begin, create a new **3D Scene** and rename the root node to `Main`. Then, save the scene as `level_2.tscn` in the `Scenes` folder.

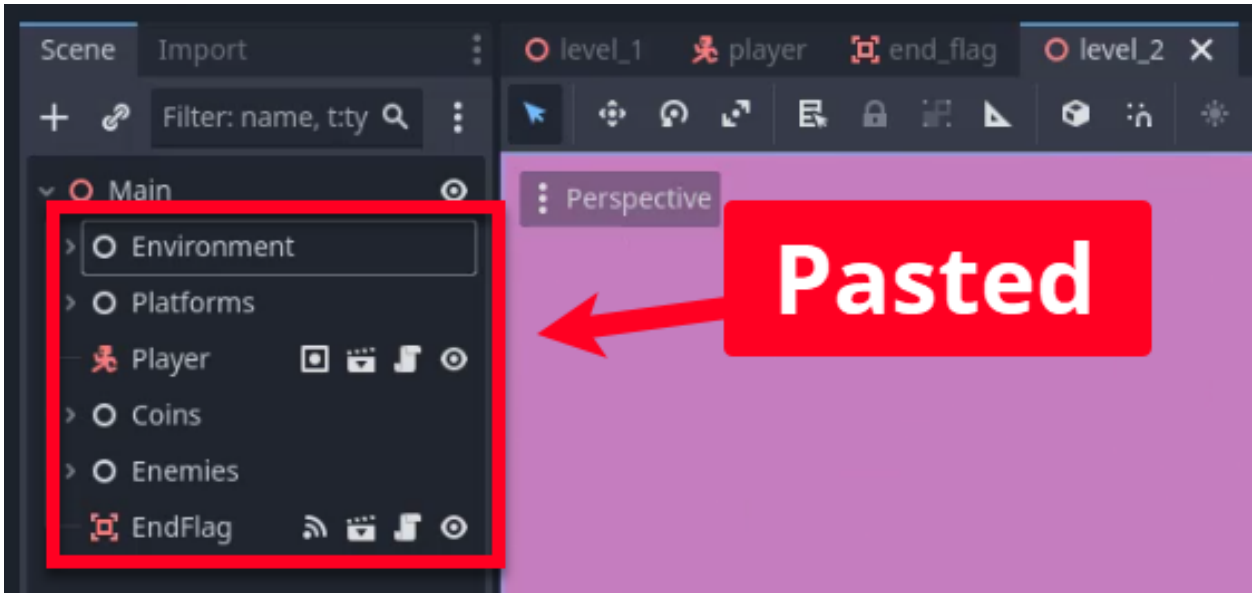




To save time, we will copy the contents of Level 1. Open `level_1.tscn` and select all child nodes of the `Main` node (Environment, Platforms, Player, Coins, Enemies, EndFlag). Copy them with `Ctrl+C`.



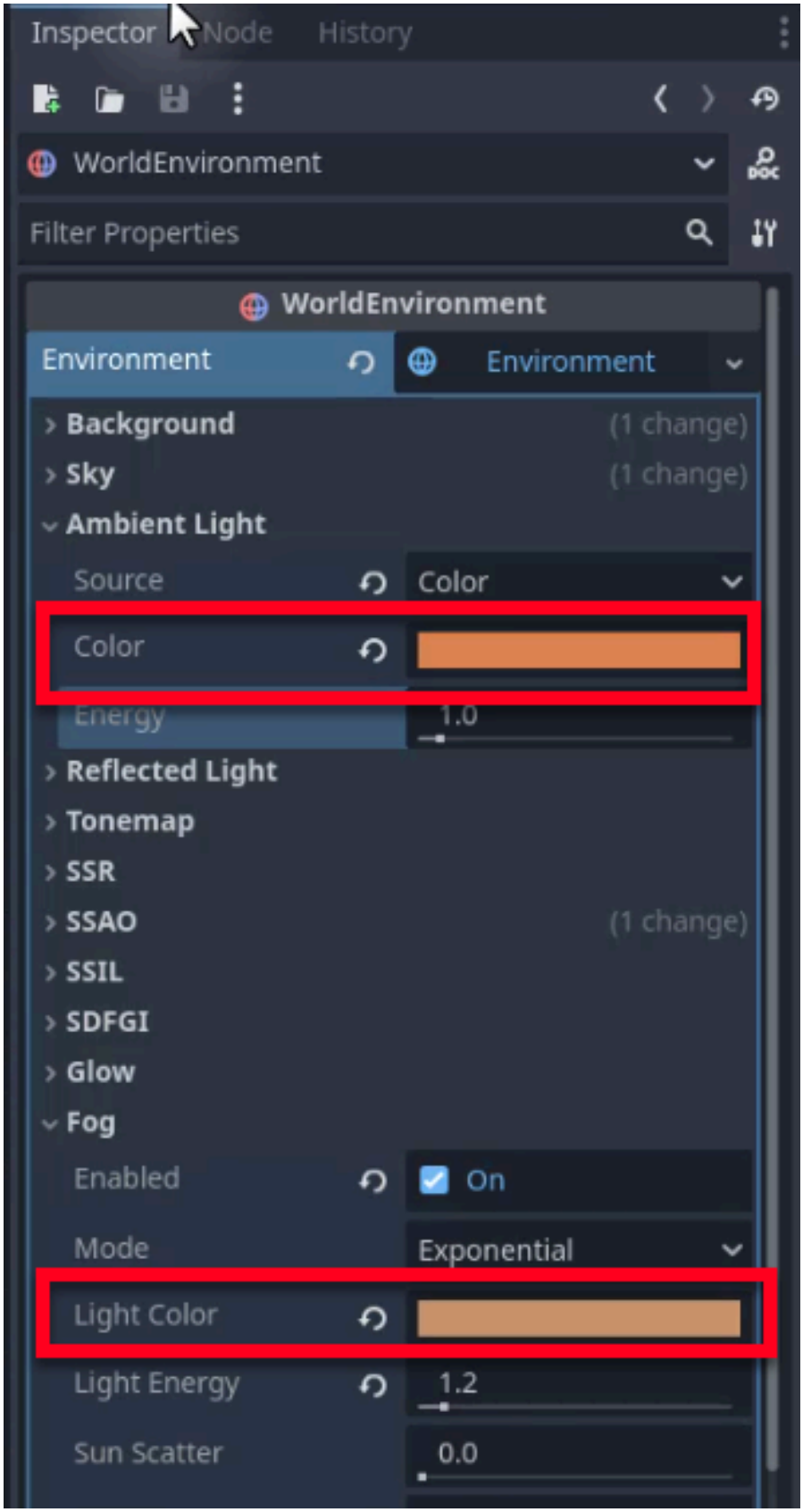
Then, go back to `level_2.tscn` and paste the nodes (`Ctrl+V`) under the `Main` root node.



Customizing the Visuals

To distinguish Level 2 from Level 1, let's change the environment settings to create a "desert" theme. Select the **WorldEnvironment** node in

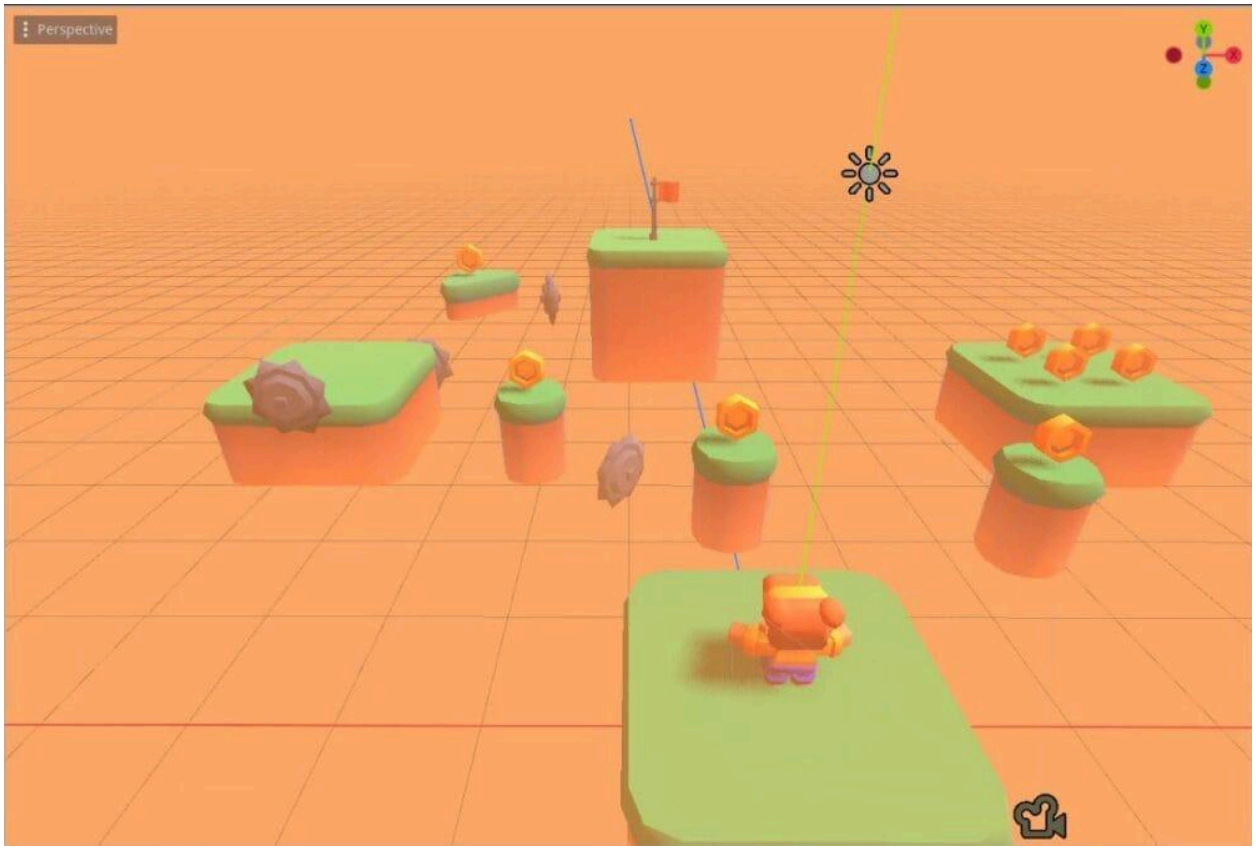
`level_2.tscn` and, in the Inspector, change the **Fog** -> **Light Color** to a yellow-orange hue and adjust the **Ambient Light** -> **Color** to match the new theme.


Inspector
Node
History
WorldEnvironment
Filter Properties
WorldEnvironment
Environment
Environment
Background
(1 change)
Sky
(1 change)
Ambient Light
Source
Color
Color
Energy
1.0
Reflected Light
Tonemap
SSR
SSAO
(1 change)
SSIL
SDFGI
Glow
Fog
Enabled
On
Mode
Exponential
Light Color
Light Energy
1.2
Sun Scatter
0.0

The scene should now look significantly different already.

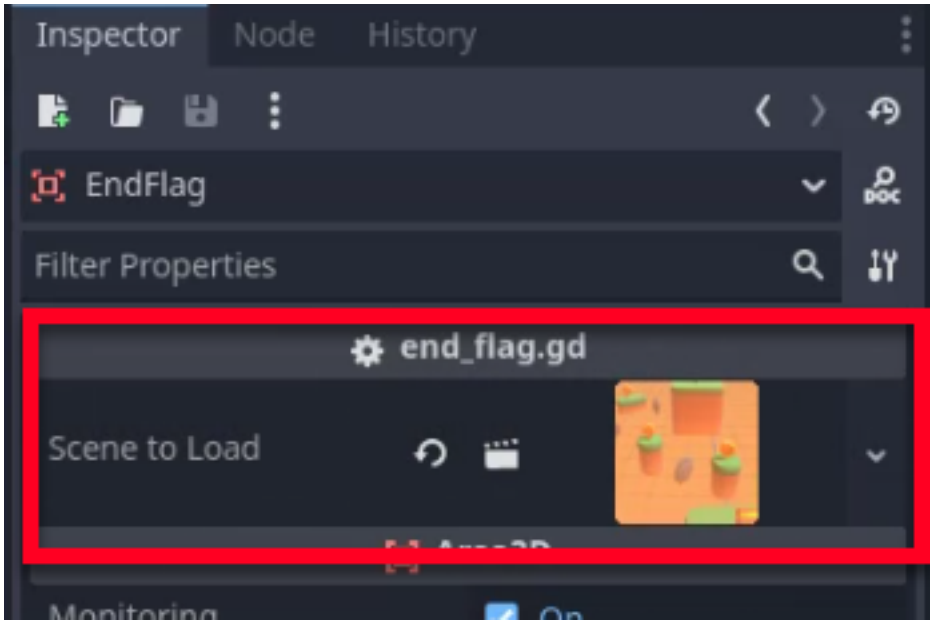
Adjusting the Layout

To complete our aesthetic change, rearrange the platforms, enemies, and coins to create a new challenge. You can move platforms to new positions, add more enemies, or change the path to the End Flag. There is no right or wrong - the goal is simply to make the level its own unique entity.



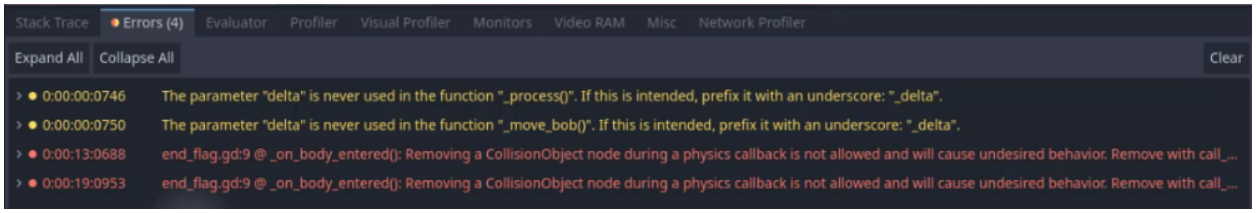
Connecting the Levels

Now that we have two distinct levels, we need to link them. In `level_1.tscn`, select the **EndFlag** node. Within the Inspector, drag `level_2.tscn` from the FileSystem into the **Scene To Load** property.



Fixing Transition Errors

If you tried to test now, you might encounter errors in the Debugger, or the game might crash.



This happens because the `body_entered` signal is triggered during the physics processing step. Attempting to delete or change the current scene while the physics engine is still calculating collisions for that frame can cause conflicts.

To fix this, we use `call_deferred`. This function tells Godot to wait until the end of the current frame (after all physics calculations are done) before executing the scene change.

As such, we will update the `end_flag.gd` script to use this function:

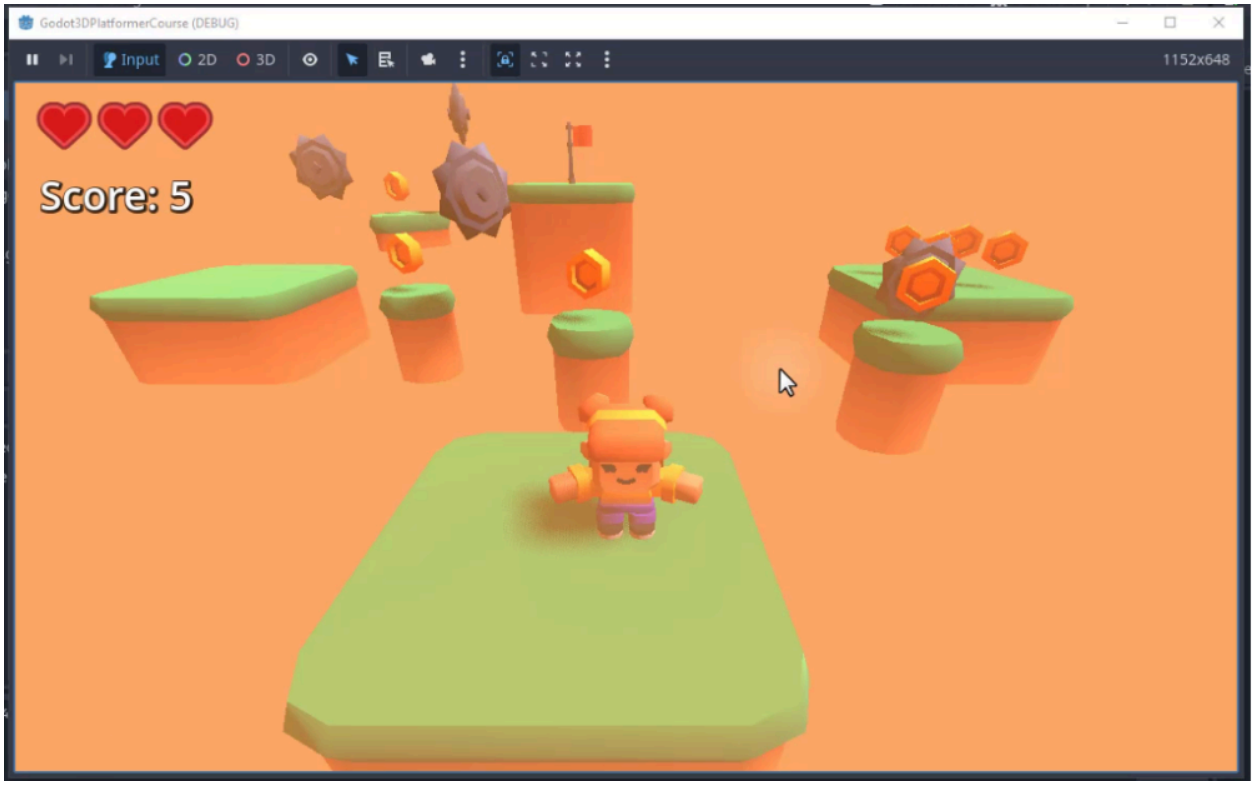
```
extends Area3D

@export var scene_to_load : PackedScene
```

```gdscript
func _on_body_entered(body):
    # Only the player triggers level transitions
    if not body.is_in_group("Player"):
        return

    # Defer the scene change to avoid modifying the tree mid-physics
step
    call_deferred("_load_new_scene")

func _load_new_scene():
    get_tree().change_scene_to_packed(scene_to_load)
```

By moving the scene change logic into a separate function (`_load_new_scene`) and calling it with `call_deferred`, we ensure a safe and smooth transition.

Final Testing

Play through Level 1, collect coins, avoid enemies, and reach the End Flag. The game should seamlessly transition to Level 2 without errors.



Congratulations! You have successfully implemented level progression. In the next chapter, we will create a main menu to serve as the starting point for your game.

Main Menu System

In this chapter, we will set up the main menu scene for our game. The menu scene is the first interface players will encounter when they launch the game. It will include a background image, a title, and two buttons: one for starting the game and another for quitting the application.

By the end of this chapter, you will have a fully functional menu that links into the game loop, allowing players to start playing, reach a game-over state, or complete the game and return to the menu.

Creating the Menu Scene

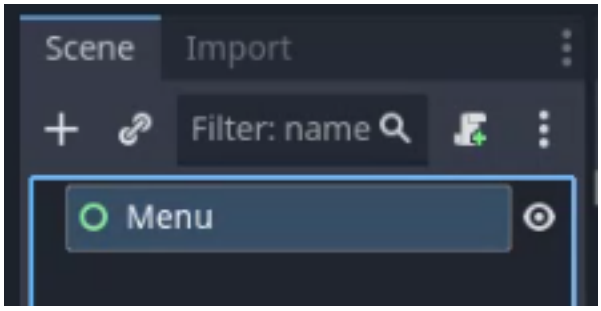
To begin, we need to create a new scene. Since this is a 2D interface overlay, we will use a **User Interface** node (Control node) as the root.

Godot Insight: Control nodes and anchors

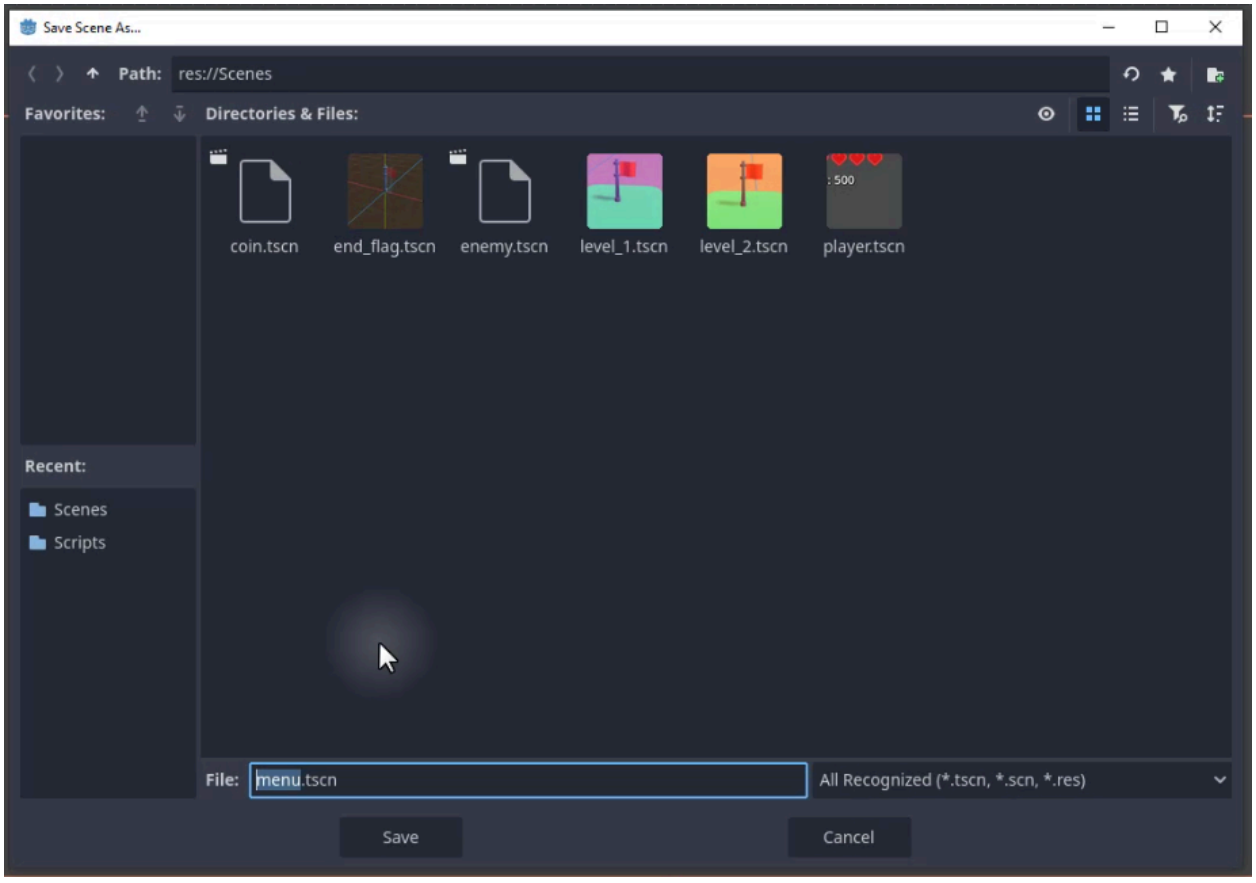
Every UI element in Godot inherits from the Control class. Control nodes use an anchor-and-offset system to position themselves relative to their parent. Anchors range from 0 (left/top edge) to 1 (right/bottom edge), so an anchor preset of "Full Rect" pins all four corners to the parent's edges, making the node stretch with the window. "Center" anchors pin the node to the parent's center point. This system lets you build responsive UIs that adapt to different screen sizes without manual repositioning. When you choose "User Interface" as a root node type, Godot creates a Control node that fills the entire viewport by default.

Learn more: Control (https://docs.godotengine.org/en/4.6/classes/class_control.html)

Let's create a new scene and select **User Interface**. Rename the root node to **Menu**.



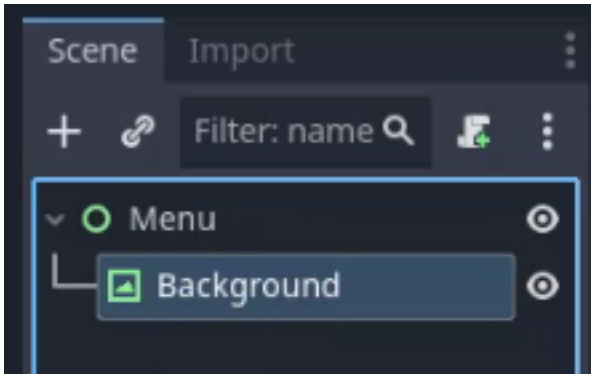
Save the scene to the **Scenes** folder as **menu.tscn**.



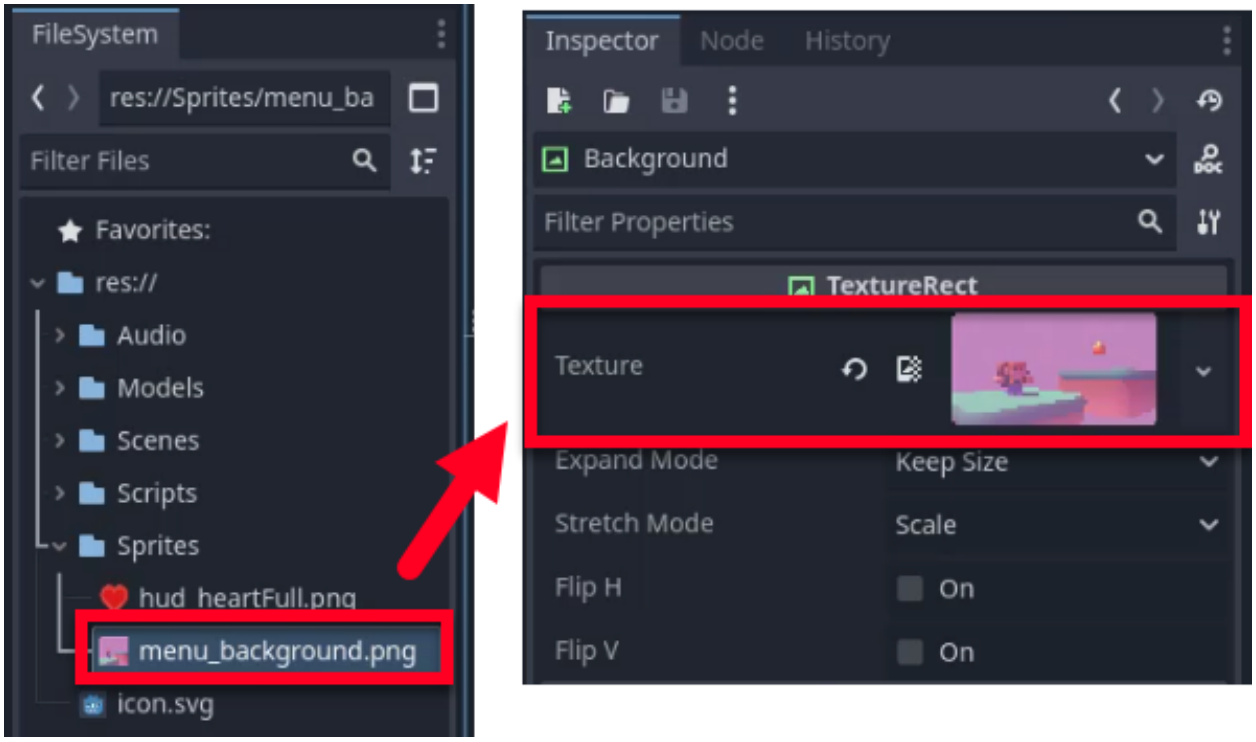
Adding a Background Image

A menu without a background looks bare and unfinished. We will use a **TextureRect** node, which is a control node designed specifically for displaying textures in a UI, to fill the screen with a background image.

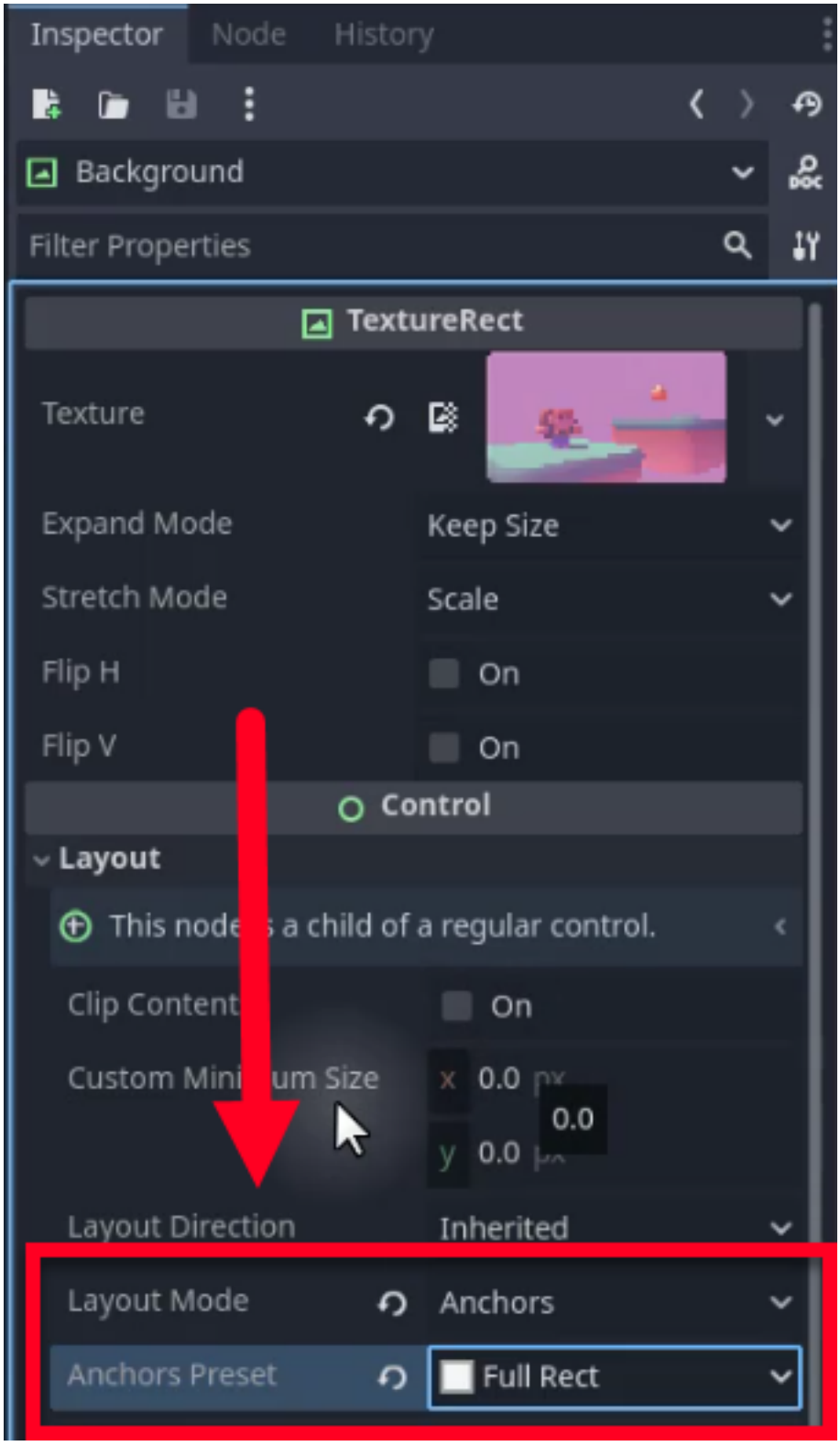
Add a **TextureRect** node as a child of the **Menu** node and rename it to **Background**.



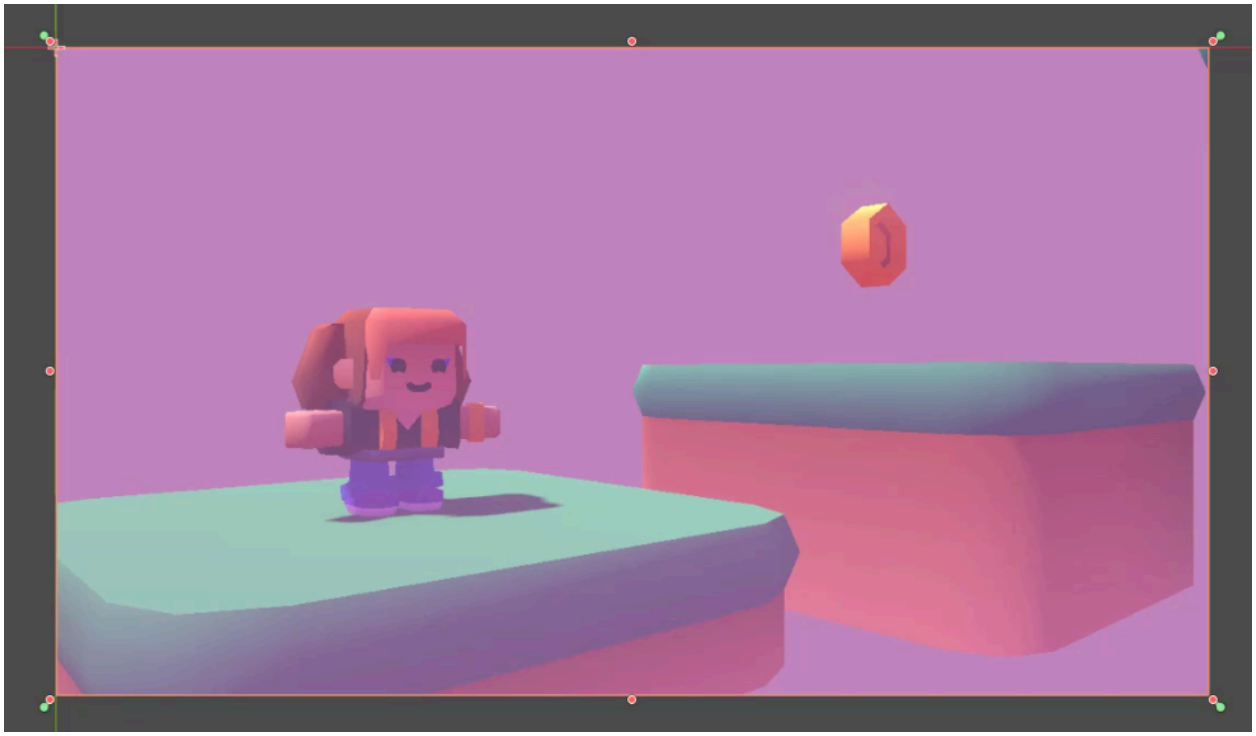
Next, we need to assign an image to this node. In the **FileSystem** dock, navigate to the **Sprites** folder and locate the **menu_background.png** file. Drag this file onto the **Texture** property of the **Background** node in the Inspector.



To ensure the background image covers the entire screen, regardless of the window size, we need to adjust its layout anchors. Select the **Background** node, then, in the Inspector, scroll down to the **Layout** section. Change the **Layout Mode** to **Anchors** and set the **Anchor Preset** to **Full Rect**.



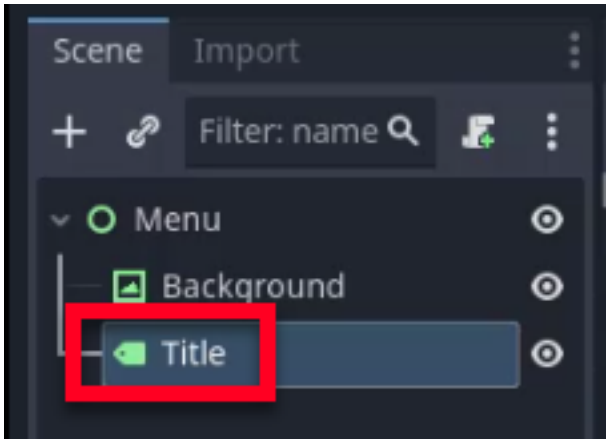
You should now have a background image that fills the scene.



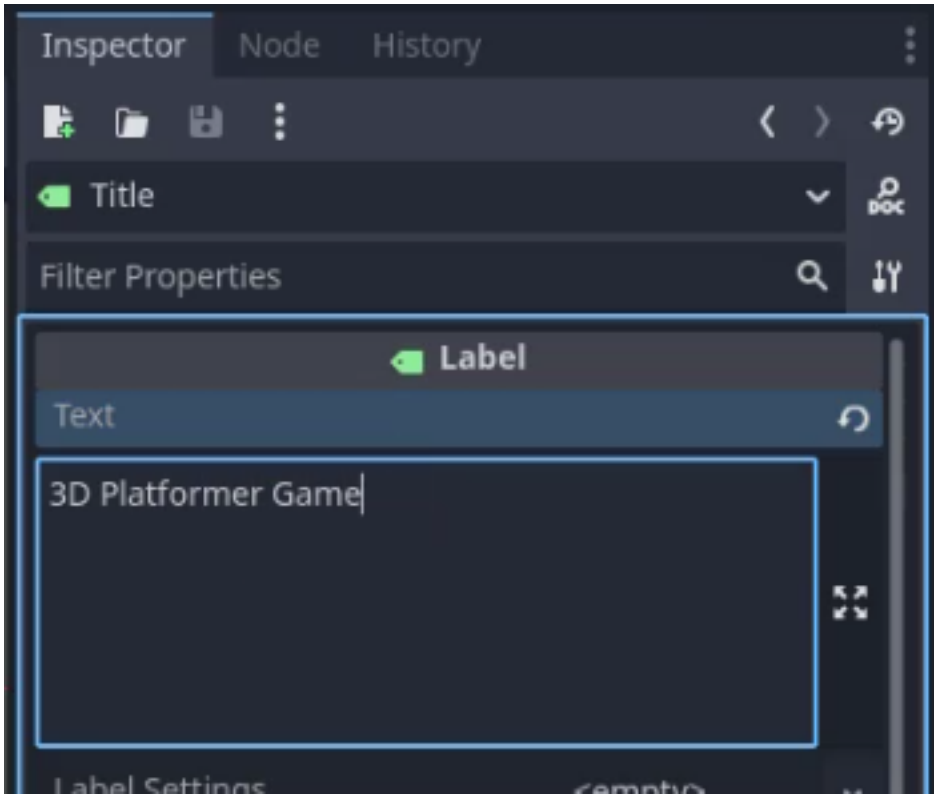
Adding the Title

Next, we will add a title to the menu.

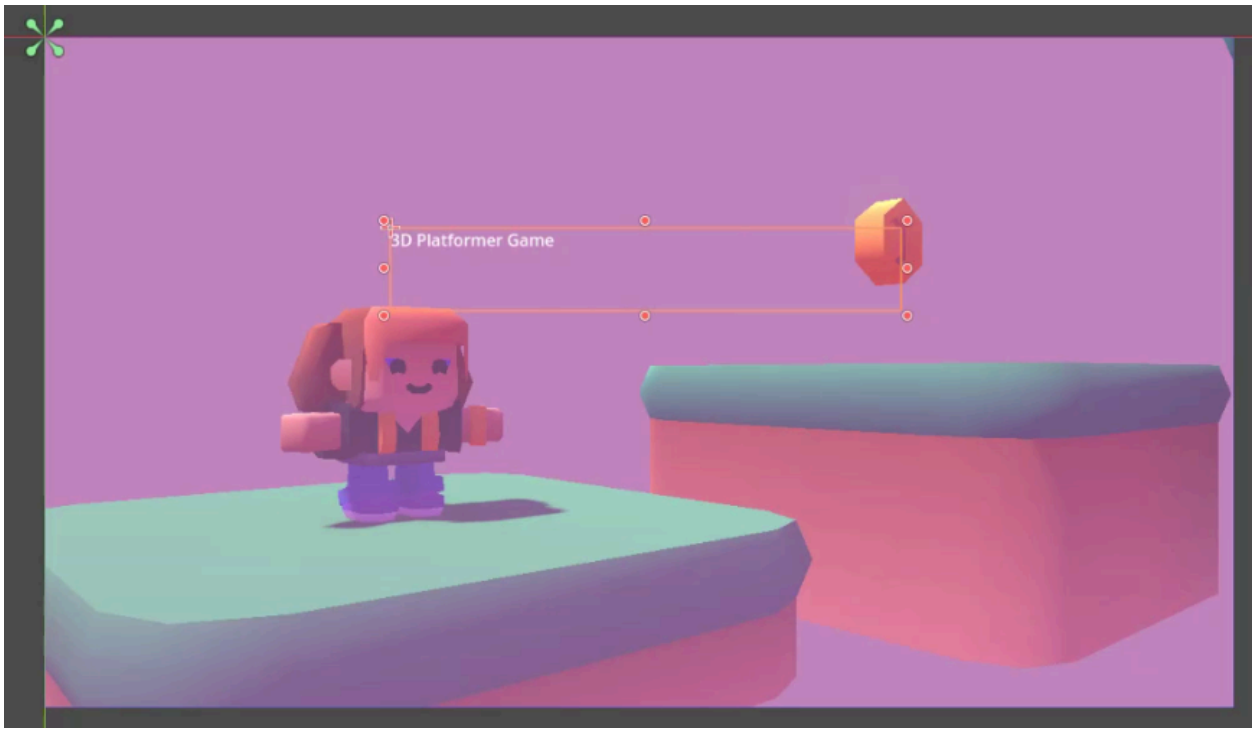
Add a **Label** node as a child of the **Menu** node and rename it to **Title**.



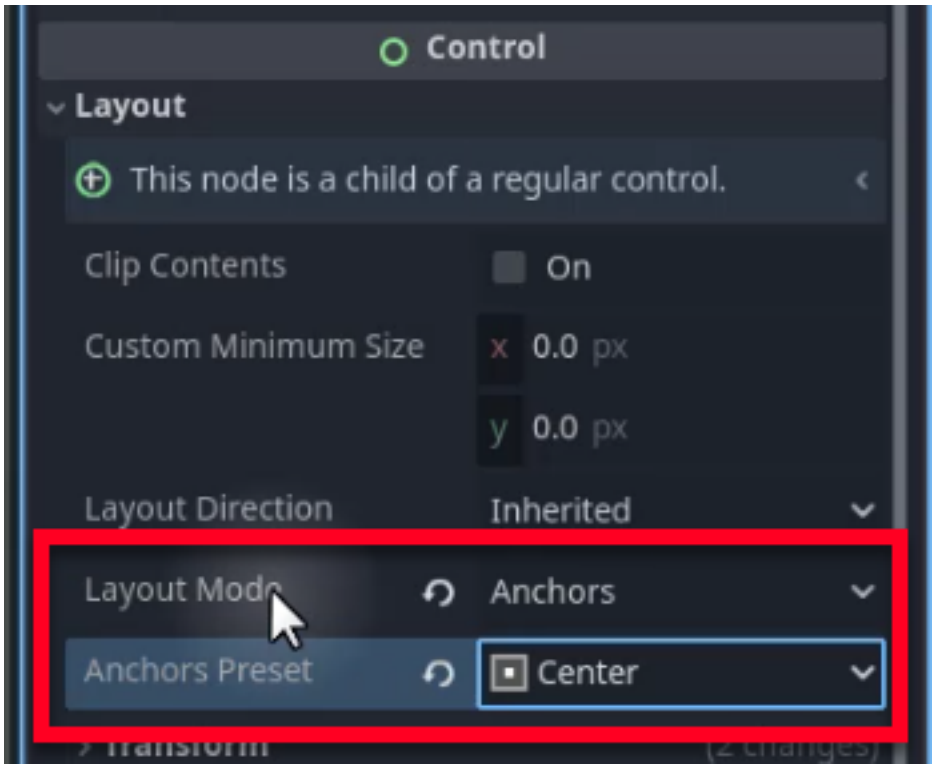
In the Inspector, let's set the **Text** property of the **Title** node to **"3D Platformer Game"** (or whatever you wish to call your game).



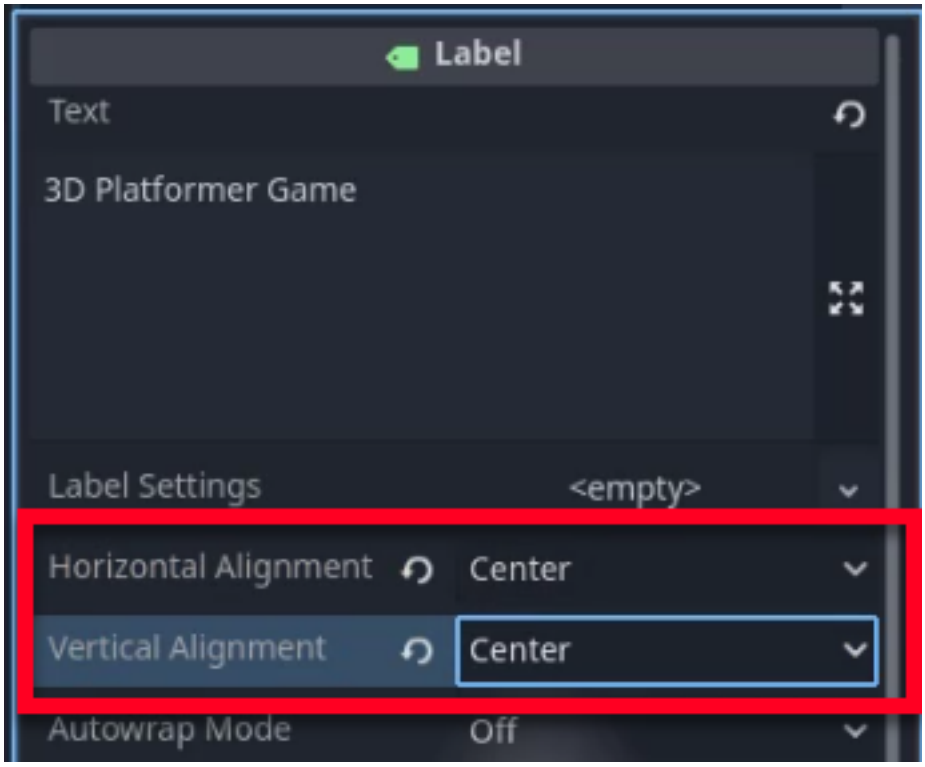
We want this title to remain centered at the top of the screen. As such, adjust the size of the label and move it to the center of the screen manually.



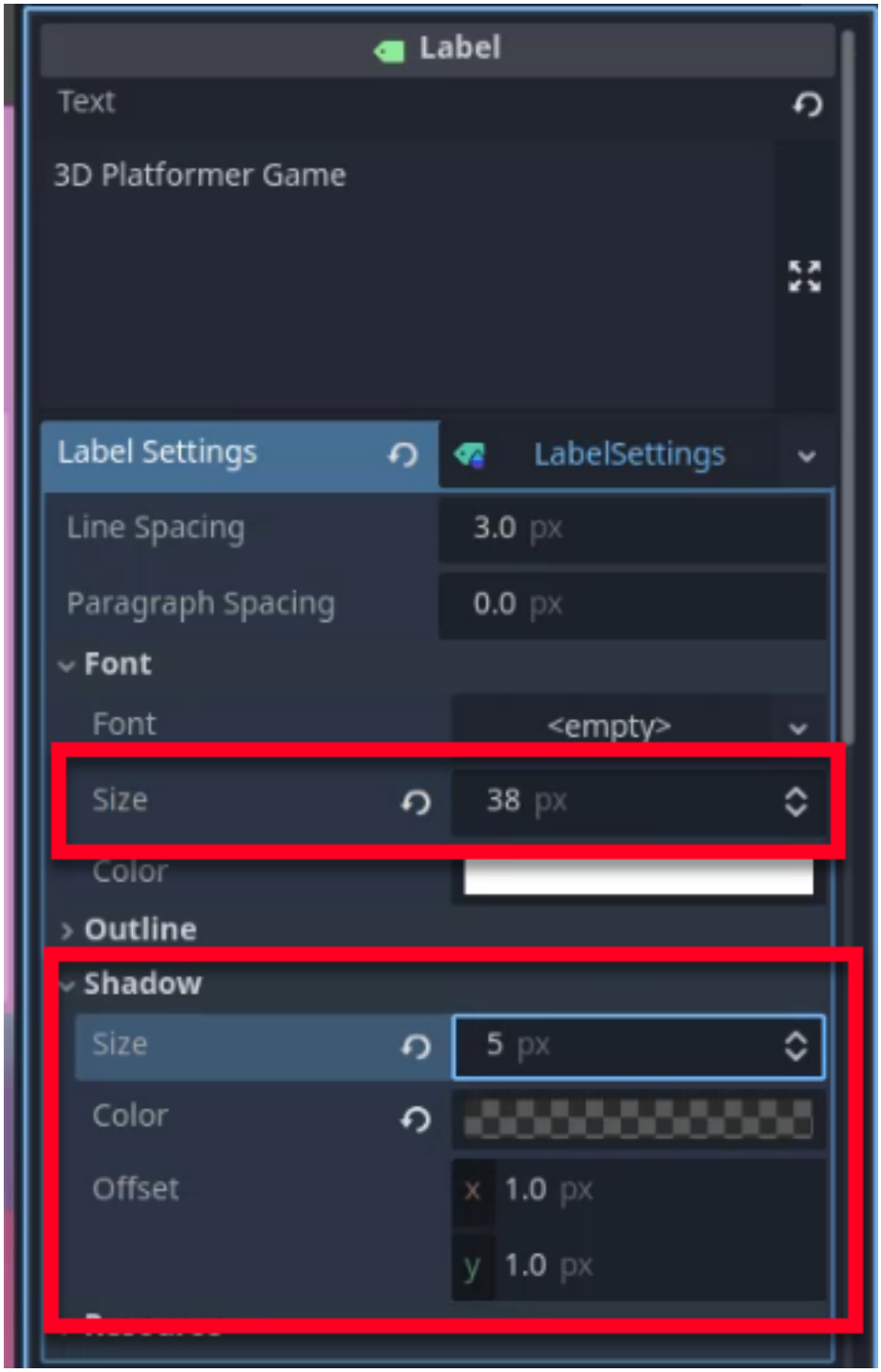
To ensure it stays centered when the screen resizes, set the **Anchor Preset** to **Center**.



Additionally, change the **Horizontal Alignment** and **Vertical Alignment** properties to **Center** to align the text within the label's bounding box.



To make the title stand out, we will adjust its font settings. In the Inspector, create a new **LabelSettings** resource. Increase the **Font Size** to **38** and add a **5px shadow** to make the text pop against the background.

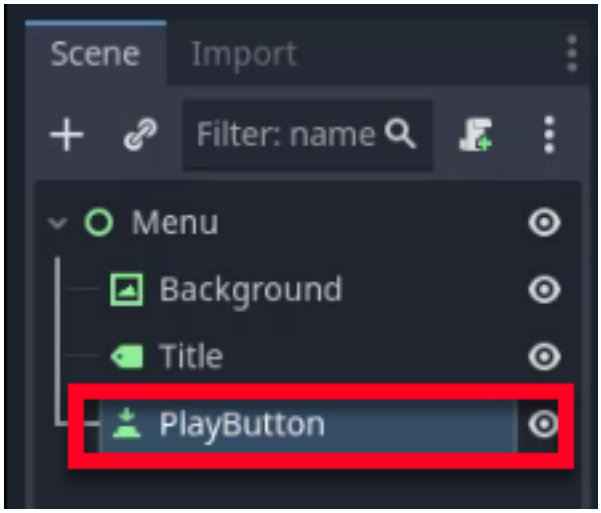


We should now have a prominent title displayed on the screen.

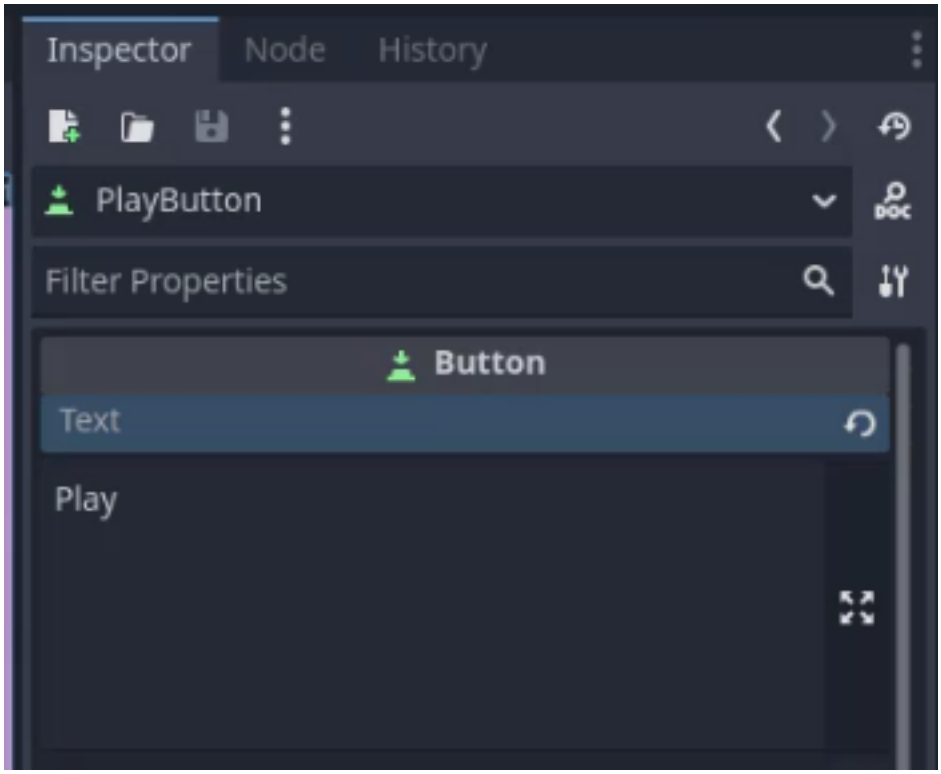
Adding the Play and Quit Buttons

We will now add the interactive elements to our menu: the Play and Quit buttons.

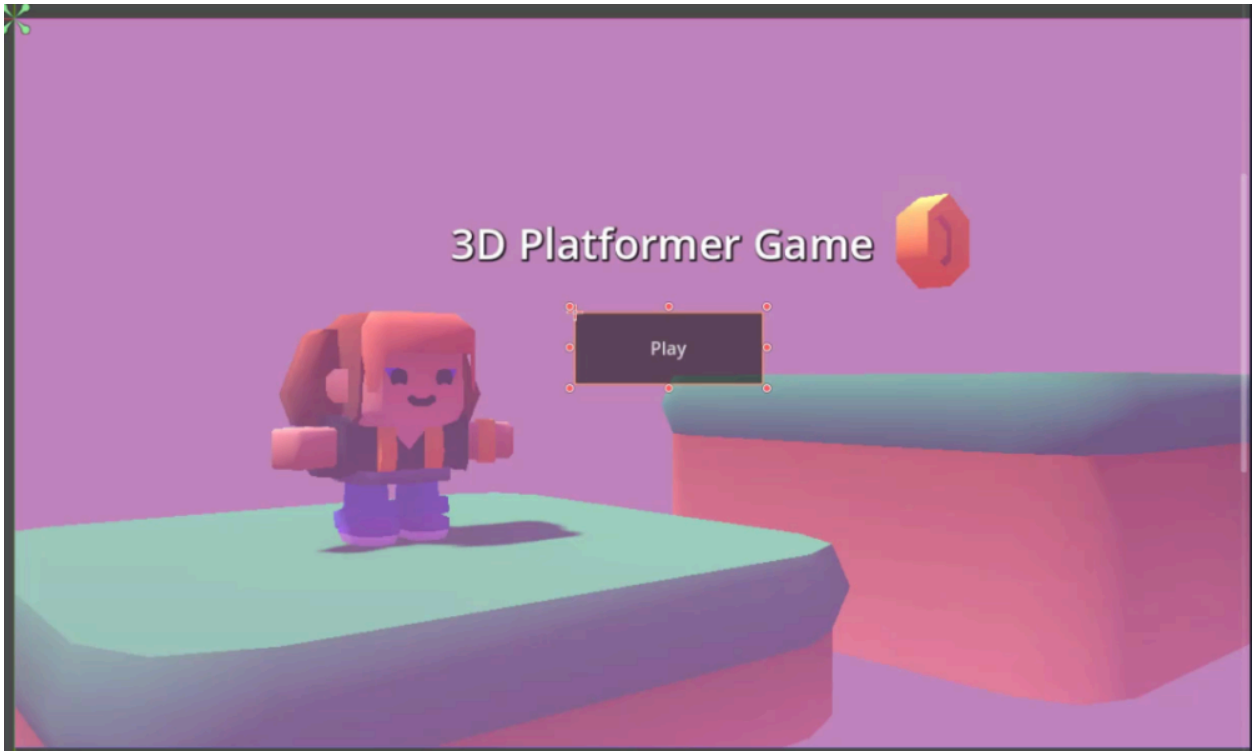
To begin, add a **Button** node as a child of the **Menu** node and rename it to **PlayButton**.



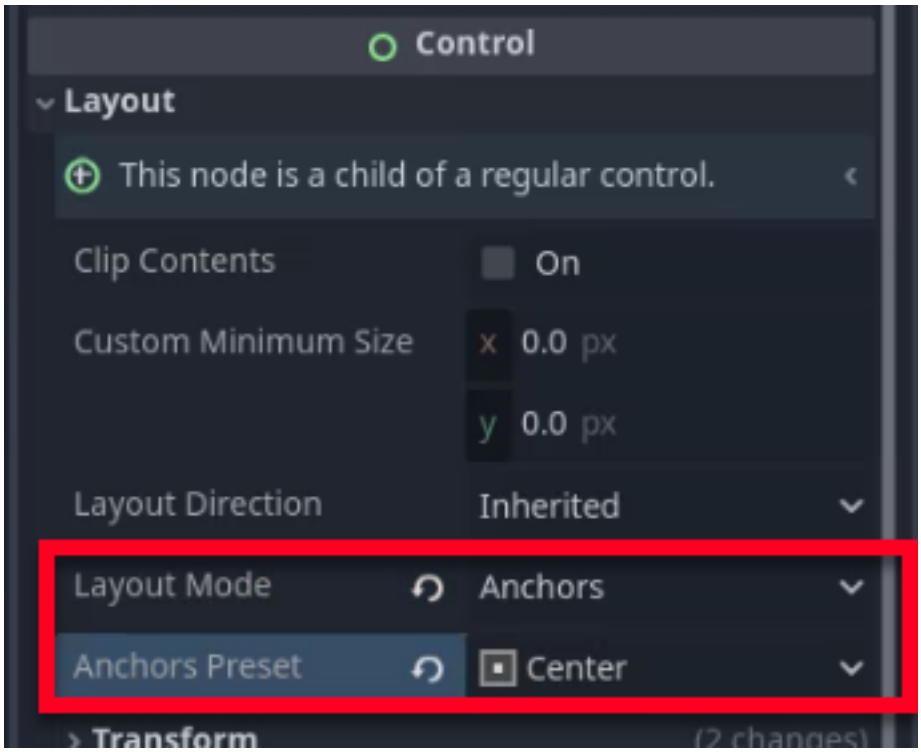
Set the **Text** property of the **PlayButton** node to **"Play"**.



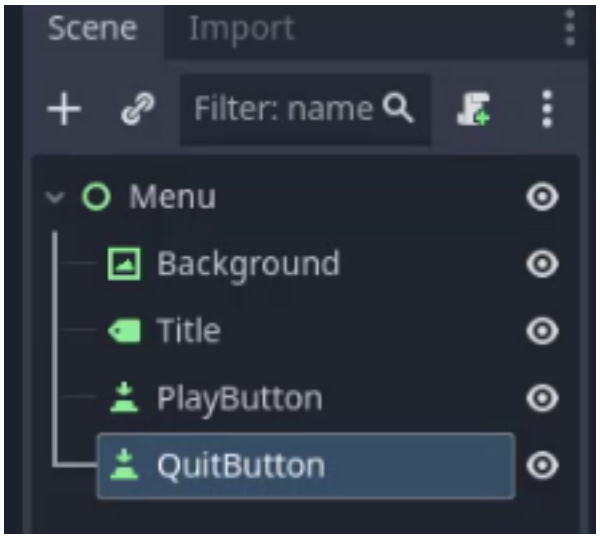
Next, adjust the size of the button and move it to the center of the screen, positioned below the title.



Like the title, set the **Anchor Preset** to **Center** to maintain its position relative to the screen center.



Now, to create the Quit button, simply duplicate the **PlayButton** node (`Ctrl + D`). Rename the duplicate to **QuitButton**.

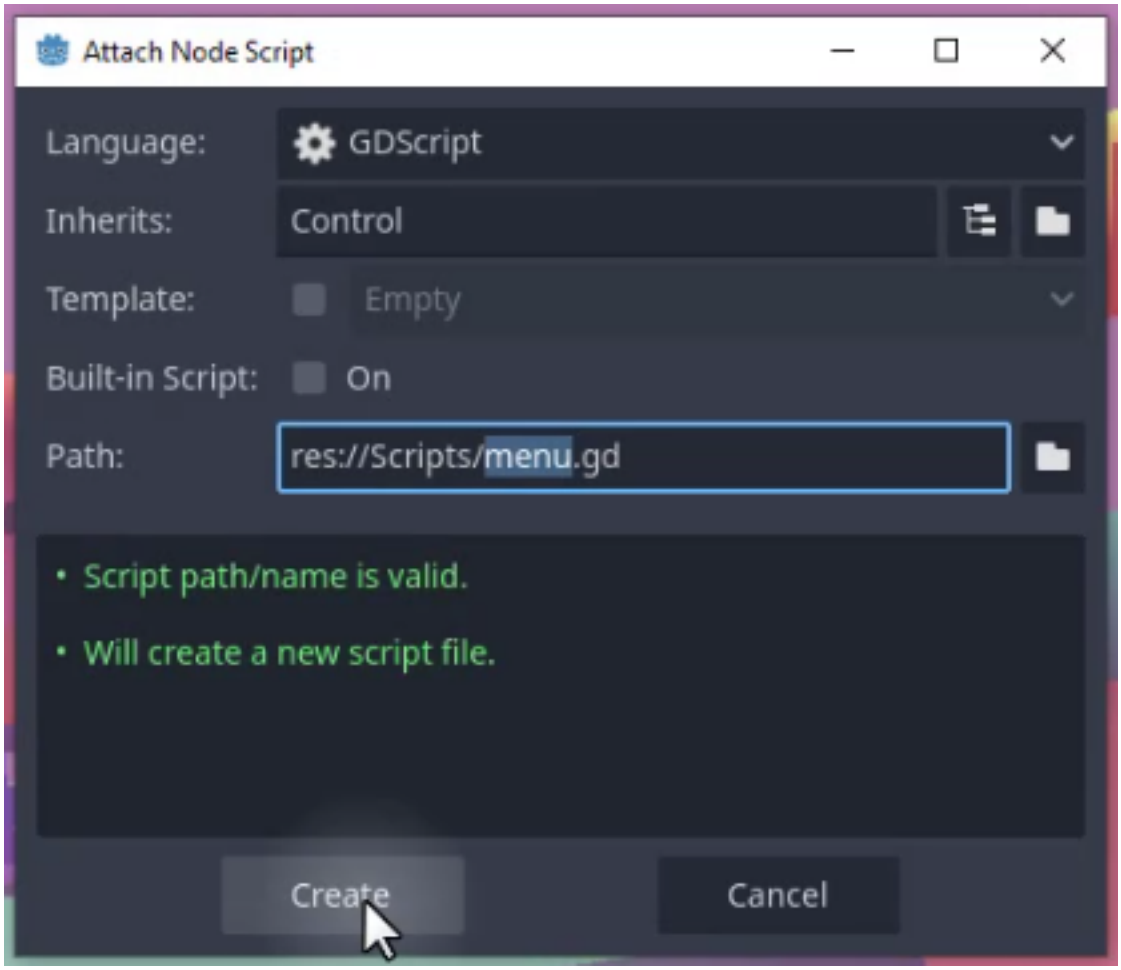


Next, set its Text property to “Quit”. Finally, move the **QuitButton** so it sits below the **PlayButton**. Your menu layout is now complete.

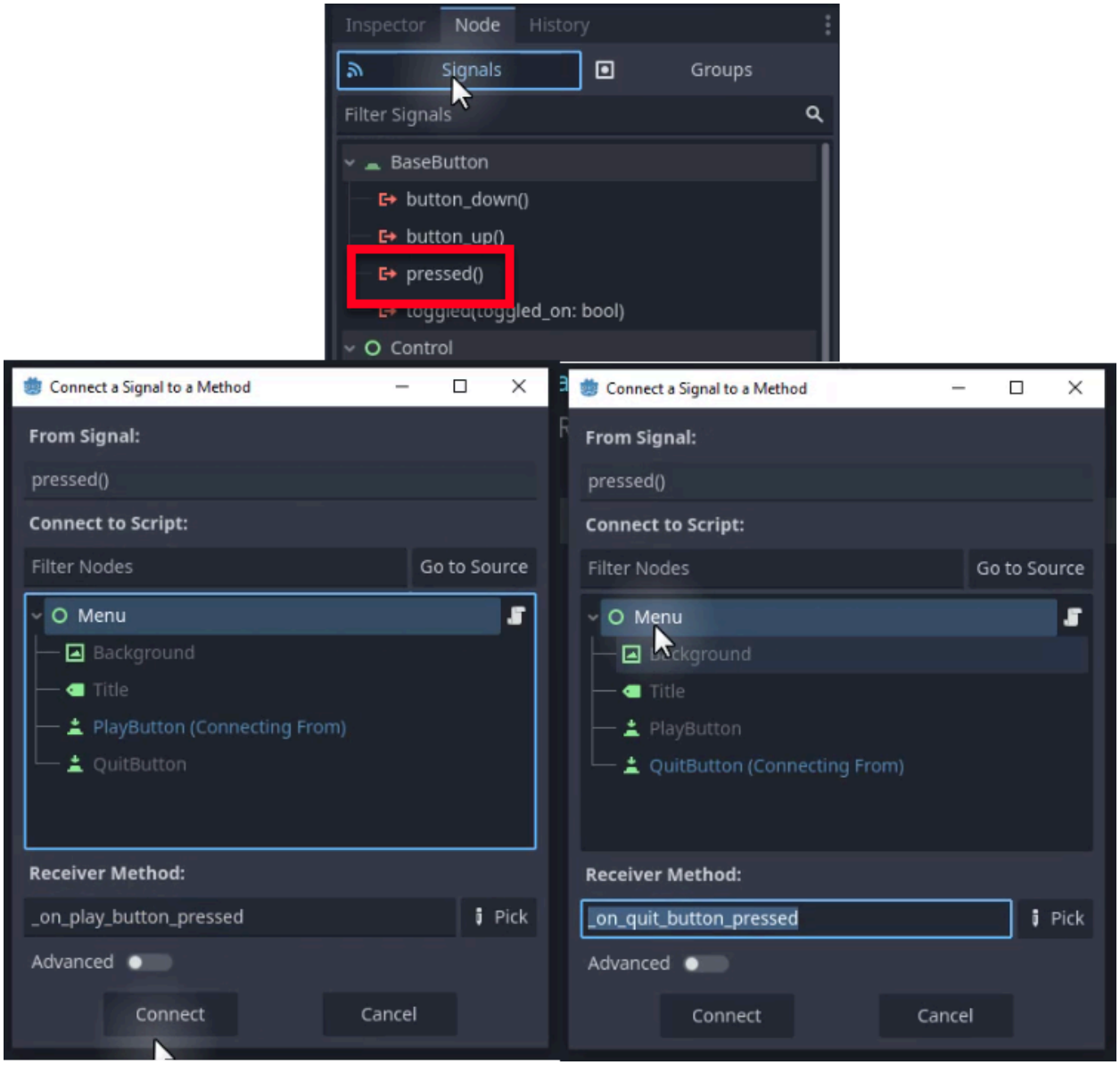
Connecting the Buttons to Scripts

Right now, our buttons look good but do nothing. To make them functional, we need to connect their `pressed` signals to a script. We will create a new script for the **Menu** node to handle these interactions.

To start, select the **Menu** node, create a new script named **menu.gd**, and save it to the **Scripts** folder.



Now, we need to connect the button signals to this script. Select the **PlayButton** and, in the Node tab (Signals), double-click the **pressed** signal. Connect it to the **menu.gd** script. Then repeat this process for the **QuitButton**.



In the **menu.gd** script, we should have had the signals automatically create functions for us. Add the following code to handle the button presses:

```
extends Control

func _on_play_button_pressed():
    # Reset the score before starting a new game
    PlayerStats.score = 0
    get_tree().change_scene_to_file("res://Scenes/level_1.tscn")

func _on_quit_button_pressed():
    get_tree().quit()
```

The play button resets the score (using the `PlayerStats` singleton we created earlier) and loads the first level. The quit button simply closes the application.

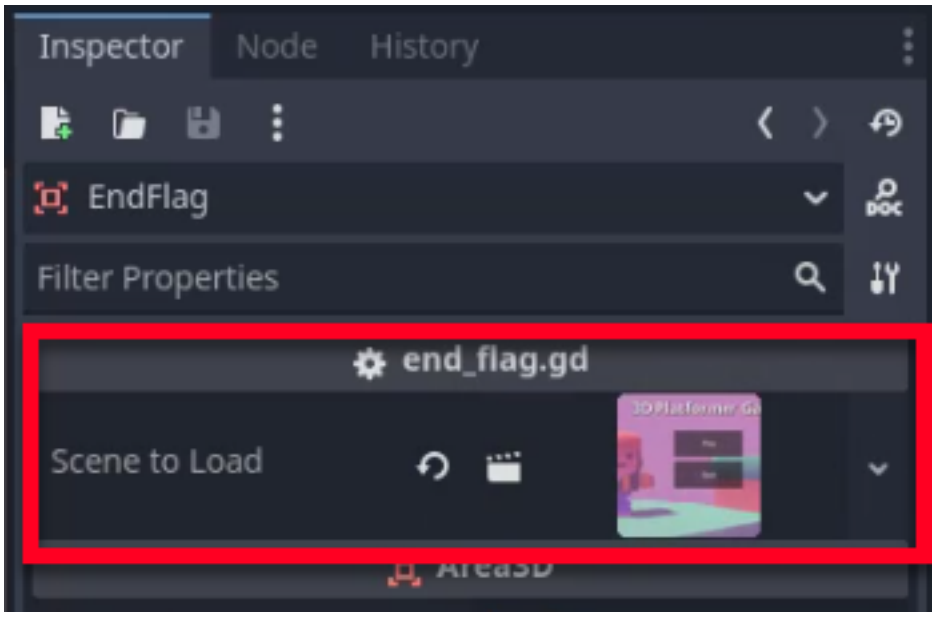
Handling Game Over and Level Completion

To ensure the game returns to the menu upon a "Game Over" or when the game is completed, we need to make a few adjustments to our existing scripts.

First, open the **player_controller.gd** script. Modify the `_game_over` function to load the menu scene instead of reloading the current level:

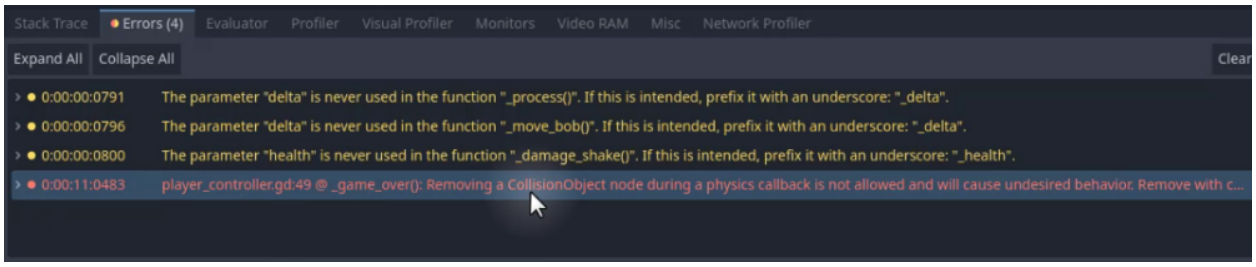
```
func _game_over ():
    PlayerStats.score = 0
    get_tree().change_scene_to_file("res://Scenes/menu.tscn")
```

Second, we need to handle the end of the game. In your final level (e.g., Level 2), select the **EndFlag** node. In the Inspector, set the **Scene To Load** property to the **Menu.tscn** scene. This creates a loop where completing the game returns the player to the main menu.



Fixing the Game Over Error

With the code as it stands, you may encounter an error when the player takes fatal damage. This occurs because the game attempts to change the scene (unload the player and camera) while the physics engine is still processing the collision that caused the damage. This is a problem we had previously with our level transitions.



To fix this, we use the `call_deferred` function. This tells Godot to wait until the end of the current frame execution before calling the `_game_over` function.

We need to modify the `take_damage` function in `player_controller.gd` as follows:

```
func take_damage(amount: int):
    health -= amount
    OnTakeDamage.emit(health)

    if health <= 0:
        # Defer to avoid changing the scene mid-physics callback
        call_deferred("_game_over")
```

With these steps, we have created a functional menu scene with play and quit buttons. The game loop is now complete: the player starts at the menu, plays through the levels, and returns to the menu upon winning or losing.

In the next chapter, we will add audio to our game to enhance the player experience.

Audio, Polish, and Level Design

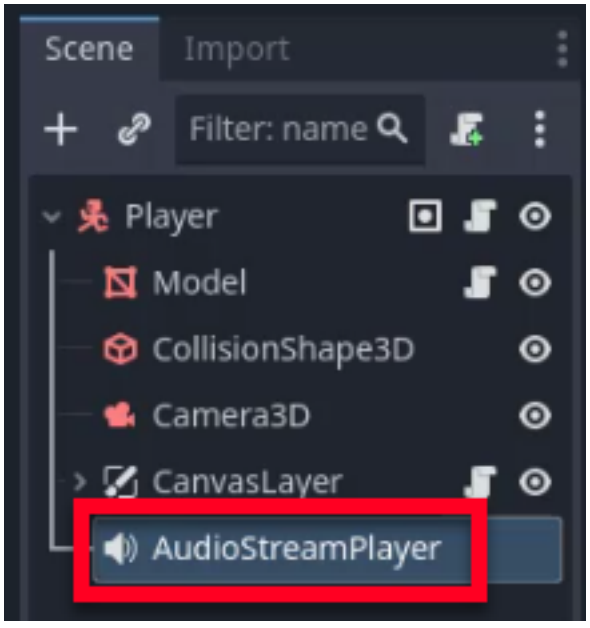
In this chapter, we will focus on polishing our game to make it feel more complete and professional. We will start by setting up an audio system to provide auditory feedback when the player collects coins or takes damage. Next, we will implement a camera shake effect to add visual impact to the damage mechanic. Finally, we will explore fundamental level design principles to help you create engaging and intuitive stages for your players.

Audio

Audio is a crucial element in game development, as it enhances the player's experience by providing immediate feedback and immersion. We will focus on playing sound effects for two key events: collecting a coin and taking damage.

Setting Up the AudioStreamPlayer Node

To begin, we need a node capable of playing sound effects. Let's open your `Player` scene and add a new node of type `AudioStreamPlayer`. This node will be responsible for playing our sound effects.



Configuring the Player Controller Script

Next, we need to modify the player controller script to handle audio playback. Open the `player_controller.gd` script. First, create a reference to the `AudioStreamPlayer` node so we can access it in our code:

```
@onready var audio : AudioStreamPlayer = $AudioStreamPlayer
```

We will preload the audio clips for the coin sound effect and the take damage sound effect. Preloading ensures that the sound effects are loaded into memory when the script is compiled, reducing any potential delay when the sounds are played during gameplay:

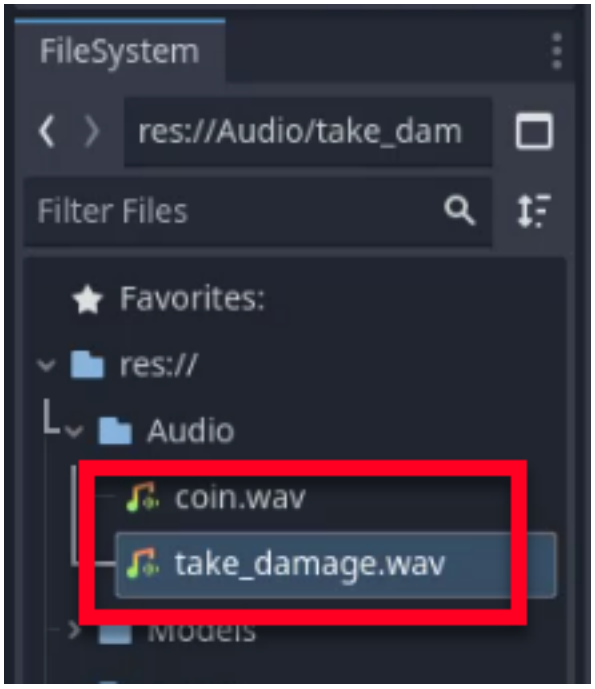
Godot Insight: preload vs load

GDScript offers two ways to load resources. `preload()` loads the resource at compile time, before the scene even starts. This guarantees zero delay when you use the resource, but the file path must be a constant string. `load()` loads the resource at runtime, which is useful when the path is dynamic (for example, loading a level by name), but it may cause a brief hitch if the file is large. For small, always-needed assets like sound effects, `preload` is the standard choice.

Learn more: AudioStreamPlayer (https://docs.godotengine.org/en/4.6/classes/class_audiostreamplayer.html)

```
var coin_sfx : AudioStream = preload("res://Audio/coin.wav")
var take_damage_sfx : AudioStream =
preload("res://Audio/take_damage.wav")
```

You can verify the location of these files in the FileSystem under the Audio folder, as shown below.



Creating the PlaySound Function

To keep our code organized, we will create a helper function called `_play_sound`. This function will take an `AudioStream` as a parameter, assign it to the player, and play it:

```
func _play_sound(sound : AudioStream):
    audio.stream = sound
    audio.play()
```

Integrating Sound Effects

Finally, we will integrate the sound effects into our existing gameplay logic.

In the `increase_score` function, add a call to `_play_sound` to play the coin sound effect whenever the score increases:

```
func increase_score (amount : int):
  PlayerStats.score += amount
  OnUpdateScore.emit(PlayerStats.score)
  _play_sound(coin_sfx)
```

In the `take_damage` function, add a call to play the damage sound effect when the player is hit:

```
func take_damage (amount : int):
  health -= amount
  OnTakeDamage.emit(health)
  _play_sound(take_damage_sfx)

  if health <= 0:
    _game_over()
```

Testing the Audio

After making these changes, press play to test the game. You should now hear the coin sound effect when collecting a coin and the take damage sound effect when colliding with an enemy.

By following these steps, you have successfully integrated audio into your game. In the next section, we will add more "game juice" by implementing a camera shake effect.

Camera Shake

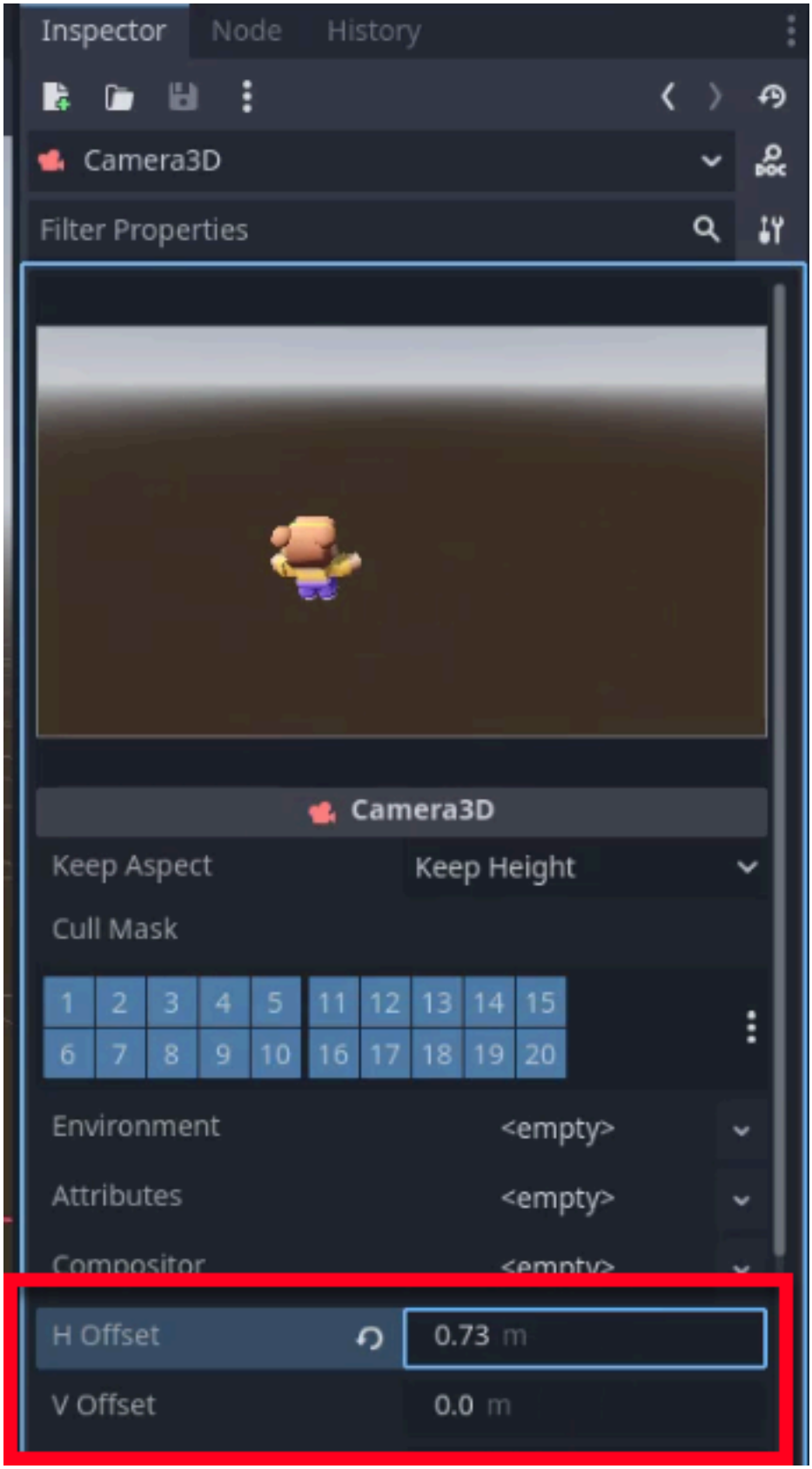
Sound effects provide auditory feedback, but players also respond strongly to visual feedback. A brief camera shake when the player takes damage makes the impact feel physical and urgent. We will achieve this by rapidly modifying the camera's offset properties.

Understanding Camera Offset

The camera offset properties allow us to shift the rendered view without moving the camera node itself:

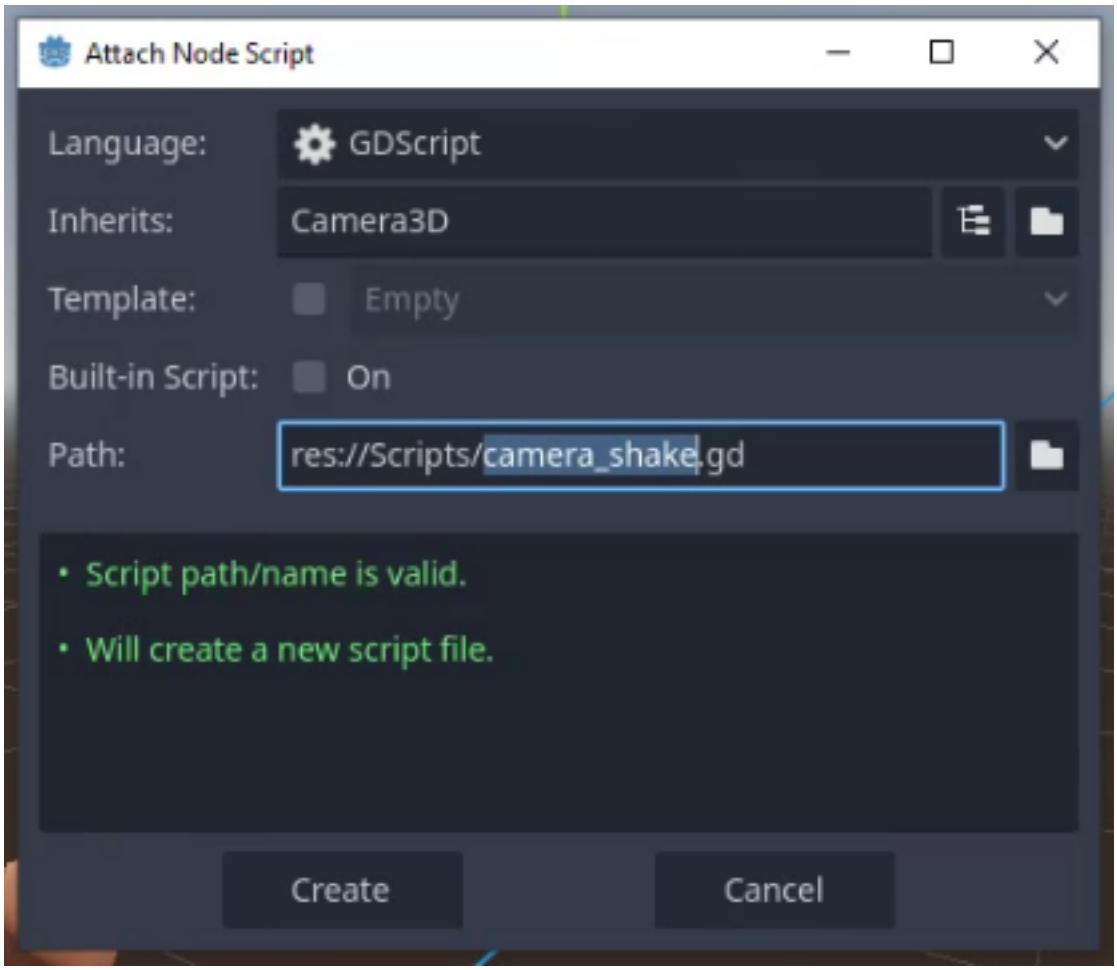
- **H Offset**: Moves the camera viewport horizontally.
- **V Offset**: Moves the camera viewport vertically.

By rapidly changing these values, we can create a shaking sensation.


Inspector
Node
History
Camera3D
Filter Properties
Camera3D
Keep Aspect
Keep Height
Cull Mask
1 2 3 4 5 11 12 13 14 15
6 7 8 9 10 16 17 18 19 20
Environment
<empty>
Attributes
<empty>
Compositor
<empty>
H Offset
0.73 m
V Offset
0.0 m

Setting Up the Camera Shake Script

To start, we need to select the Camera3D node in your scene and attach a new script. Save it as `camera_shake.gd` in the `scripts` folder. This script will manage the intensity of the shake and apply the offsets.



Creating the Intensity Variable

Inside the script, define an `intensity` variable. This float will control the magnitude of the shake:

```
extends Camera3D

var intensity : float = 0
```

Connecting the Damage Signal

Next, we need to connect the camera shake logic to the player's damage signal. In the `_ready` function, we access the parent node (the Player) and connect the `OnTakeDamage` signal to a new function called `_damage_shake`:

```
func _ready():
    get_parent().OnTakeDamage.connect(_damage_shake)
```

Implementing the Damage Shake Function

The `_damage_shake` function will set the initial intensity of the shake when the player takes damage:

```
func _damage_shake(health: int):
    intensity = 0.1
```

Reducing Intensity Over Time

In the `_process` function, we will gradually reduce the intensity of the shake over time using the `lerp` (linear interpolation) function. This ensures the shake fades out smoothly rather than stopping abruptly. We will also apply a random offset to the camera's position based on the current intensity:

```
func _process(delta):
    if intensity > 0:
        intensity = lerp(intensity, 0.0, delta * 10)
        var offset : Vector2 = _get_random_offset()
        h_offset = offset.x
        v_offset = offset.y
```

Generating Random Offsets

We need a helper function to generate the random values. The `_get_random_offset` function will return a random Vector2 based on the current intensity:

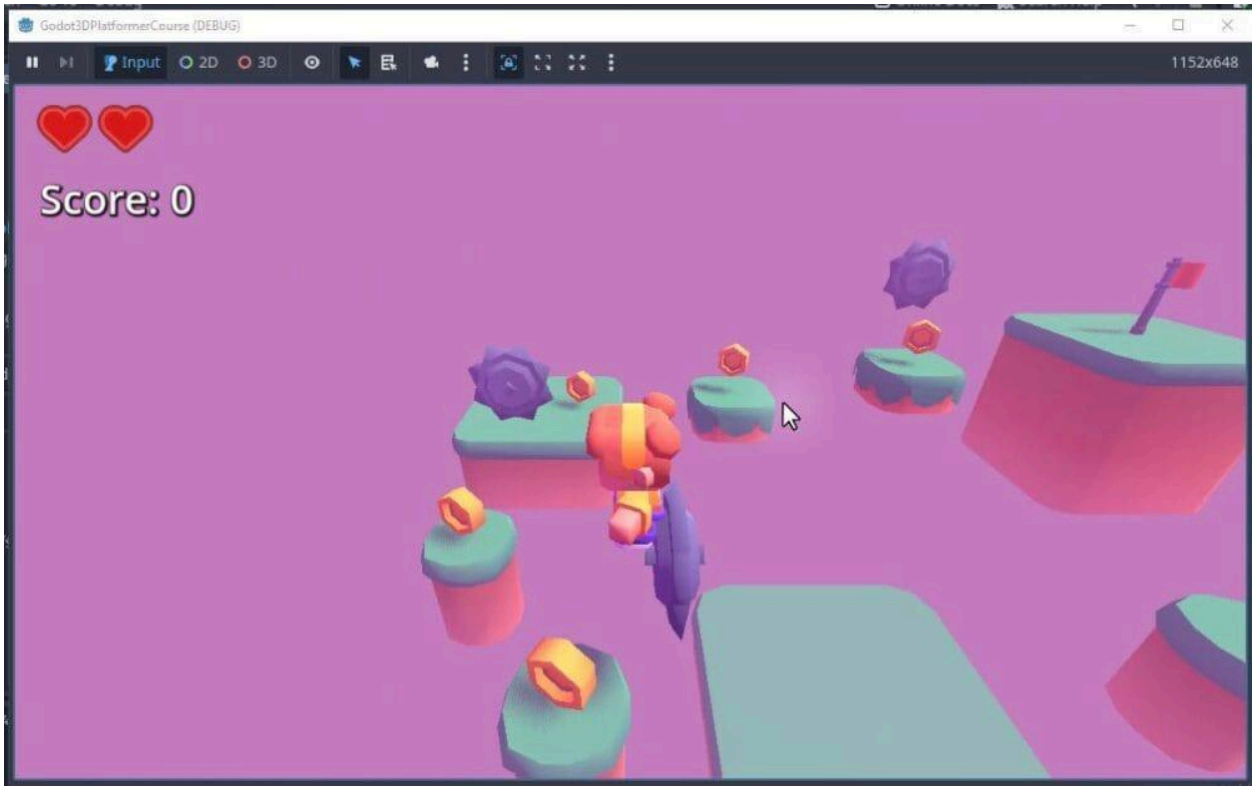
```
func _get_random_offset() -> Vector2:
    var x = randf_range(-intensity, intensity)
    var y = randf_range(-intensity, intensity)
    return Vector2(x, y)
```

Complete Camera Shake Script

Here is the complete `camera_shake.gd` script with all functions implemented:

```
extends Camera3D

var intensity : float = 0

func _ready():
    # Listen for the player's damage signal to trigger a shake
    get_parent().OnTakeDamage.connect(_damage_shake)

func _damage_shake(health: int):
    # Set the shake strength; higher values produce a more dramatic
effect
    intensity = 0.1

func _process(delta):
    if intensity > 0:
        # Smoothly decay the shake intensity each frame
        intensity = lerp(intensity, 0.0, delta * 10)
        # Apply a random viewport offset to simulate screen shake
        var offset : Vector2 = _get_random_offset()
        h_offset = offset.x
        v_offset = offset.y

# Returns a random 2D offset scaled by the current shake intensity
func _get_random_offset() -> Vector2:
    var x = randf_range(-intensity, intensity)
    var y = randf_range(-intensity, intensity)
    return Vector2(x, y)
```

With this script attached, you should observe a subtle camera shake whenever the player takes damage.



With these steps, you have successfully implemented a camera shake effect and ensured the game gives players even more feedback when they take damage.

Level Design

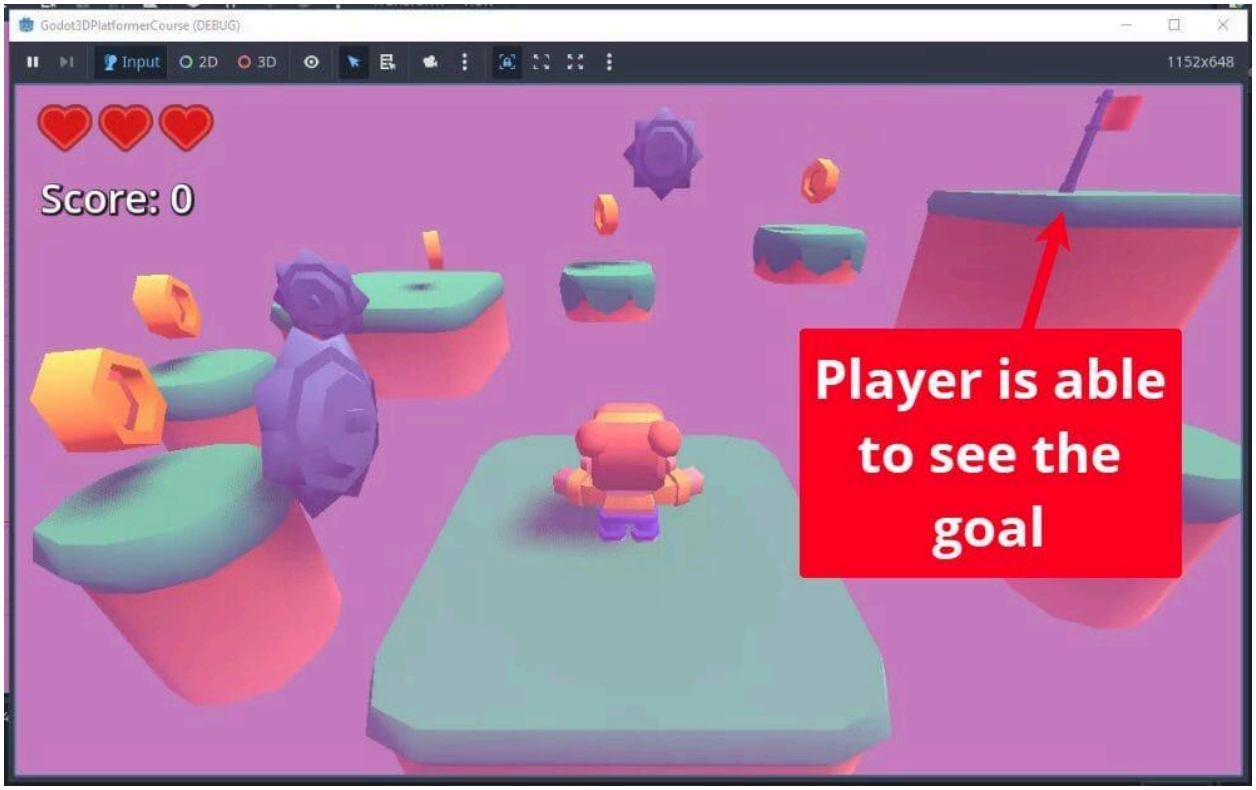
Now that we have our mechanics and polish in place, let's discuss how to use them effectively. Level design is the art of arranging your game elements to create an engaging experience. In this section, we will cover several techniques to help you create intuitive and fun levels.

Telegraphing the Objective

One of the fundamental aspects of level design is clearly communicating the objective to the player. This is often called "telegraphing."

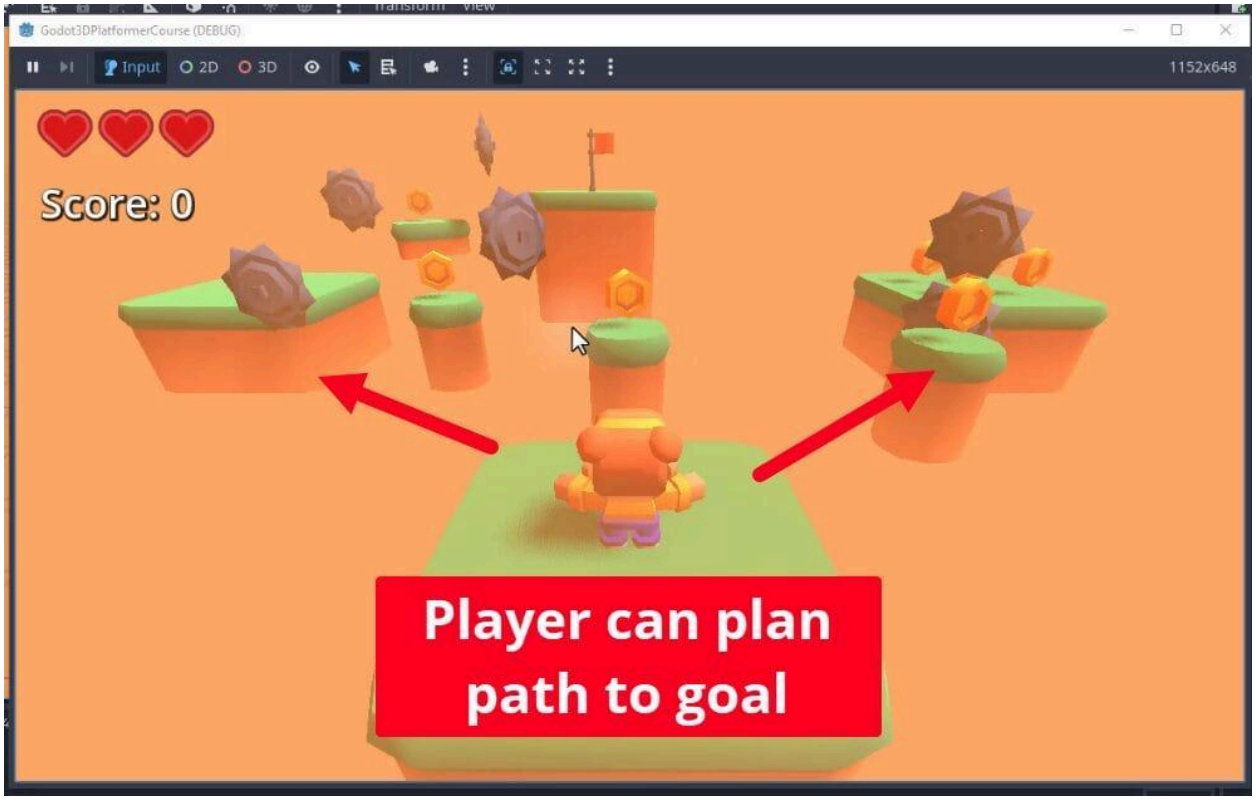
Make the end goal visible from the beginning.

By placing the end goal in a visible location right from the start, players immediately understand what they need to achieve.



Allow players to plan their path.

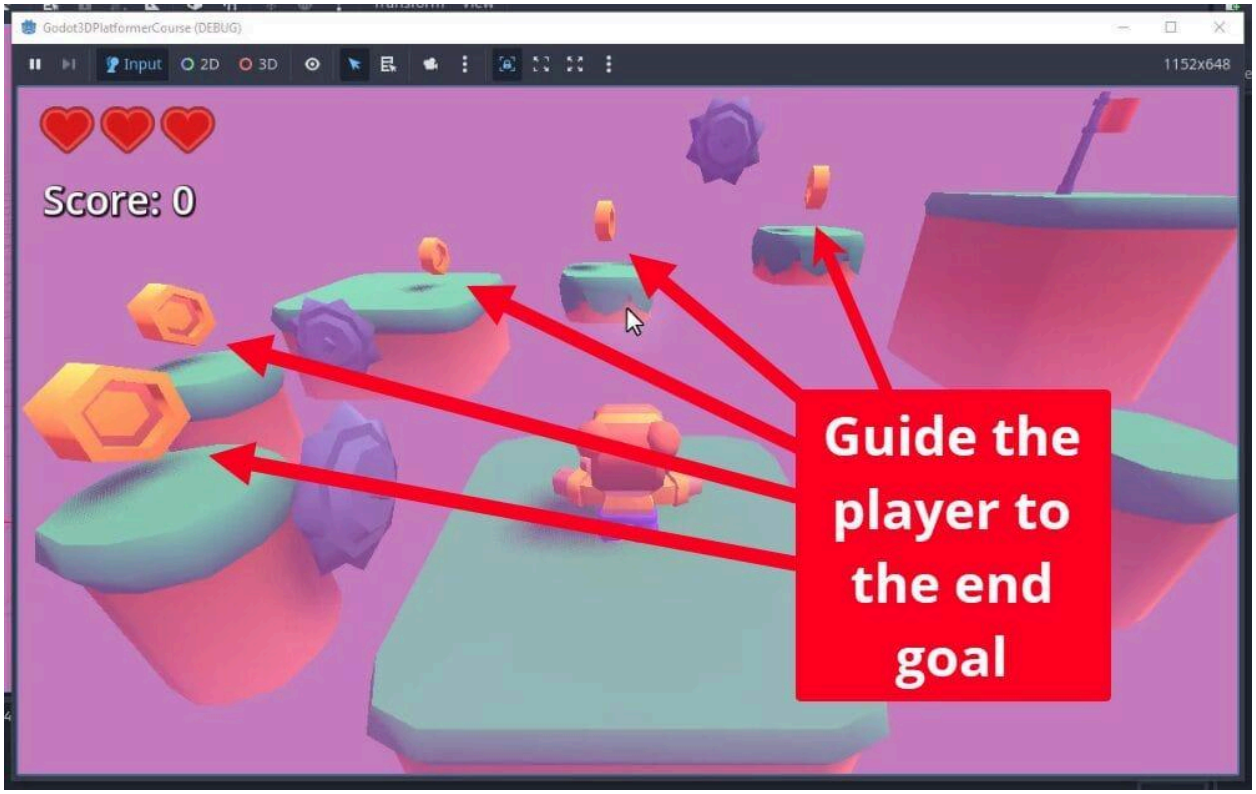
When players can see the goal and the obstacles between them and the finish line, they can plan their approach. This turns the level into a navigational puzzle.



Guiding Player Attention

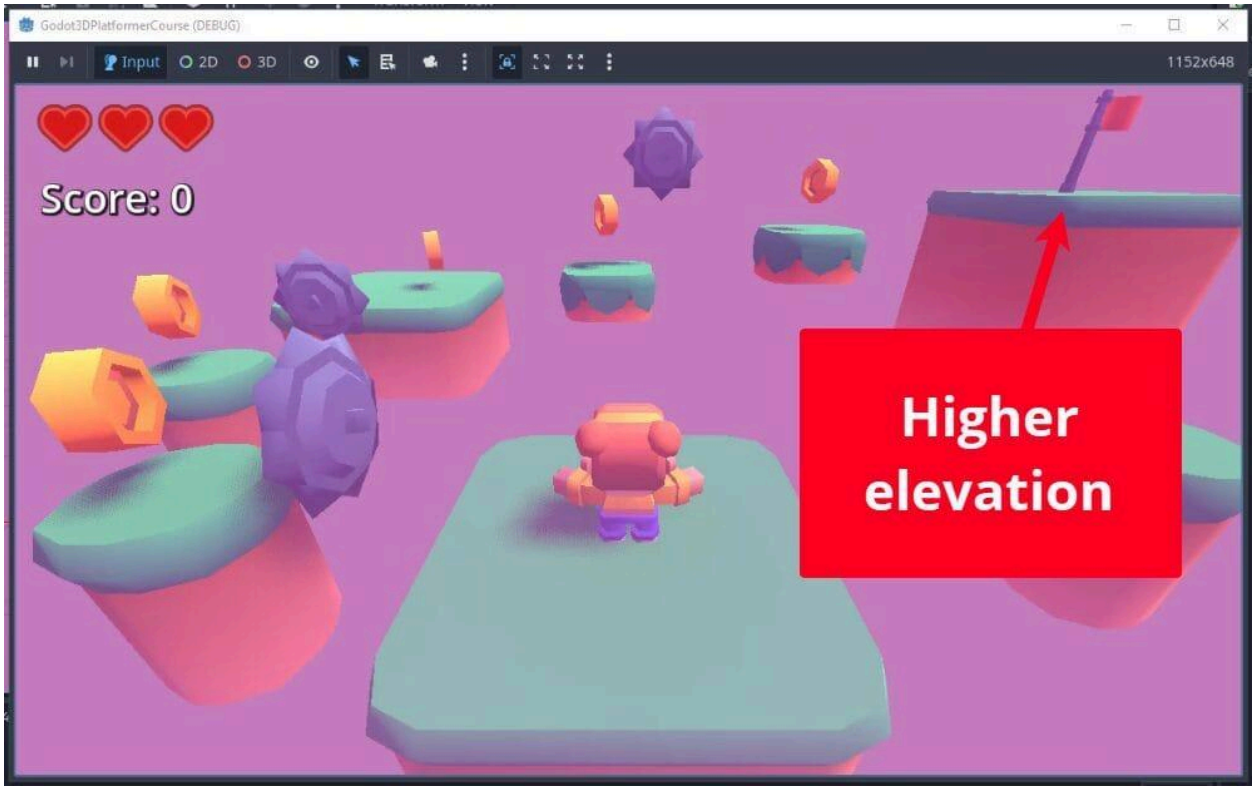
In levels with complex layouts or multiple pathways, you can use collectibles like coins to guide the player's attention. Coins act as "breadcrumbs," subtly leading players along the intended path or toward secret areas.

- Use coins to guide players along a path.
- Players are naturally drawn to collecting items, making this an effective way to direct movement without using arrows or signs.



Visibility and Storytelling

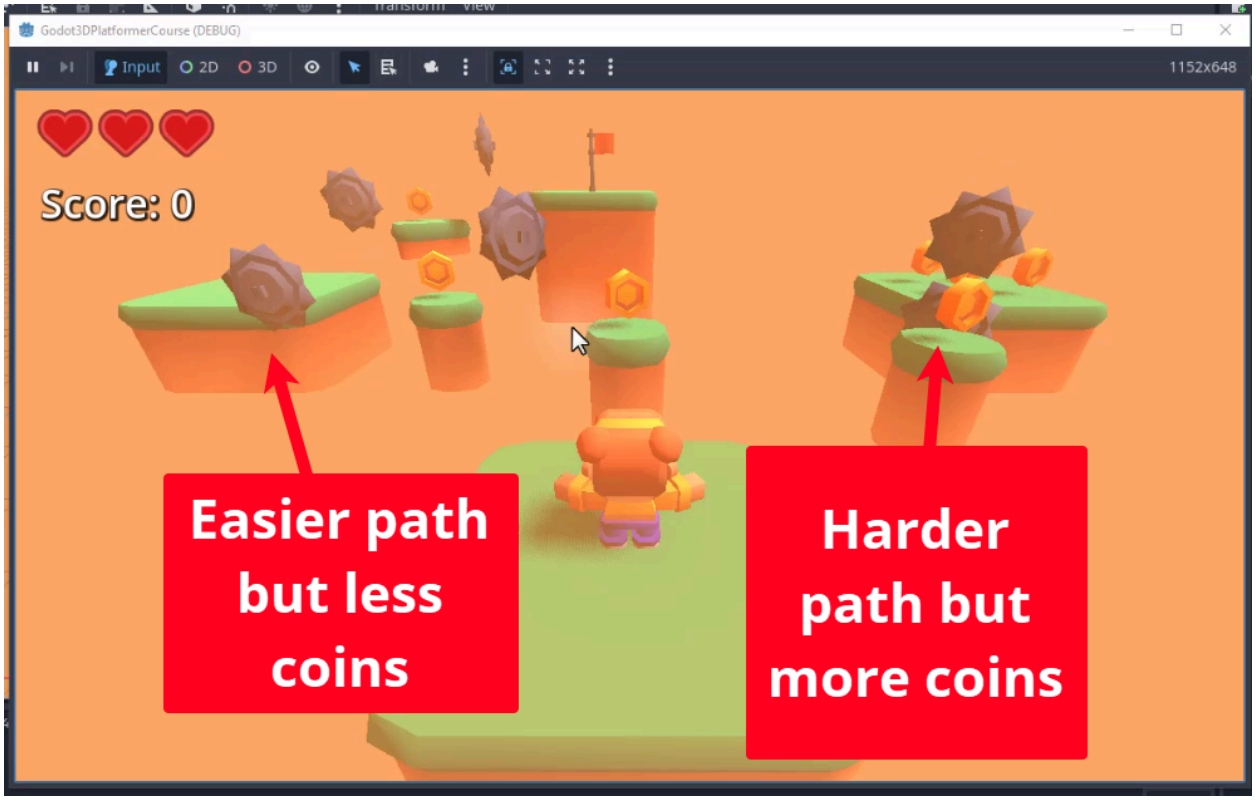
Placing the end goal at a higher elevation improves visibility and can serve as a storytelling device. Climbing upwards often signifies progress and achievement.



Providing Options and Risk vs. Reward

Giving players options adds depth to your levels. While a linear path is functional, providing branching paths with varying difficulties and rewards enhances engagement.

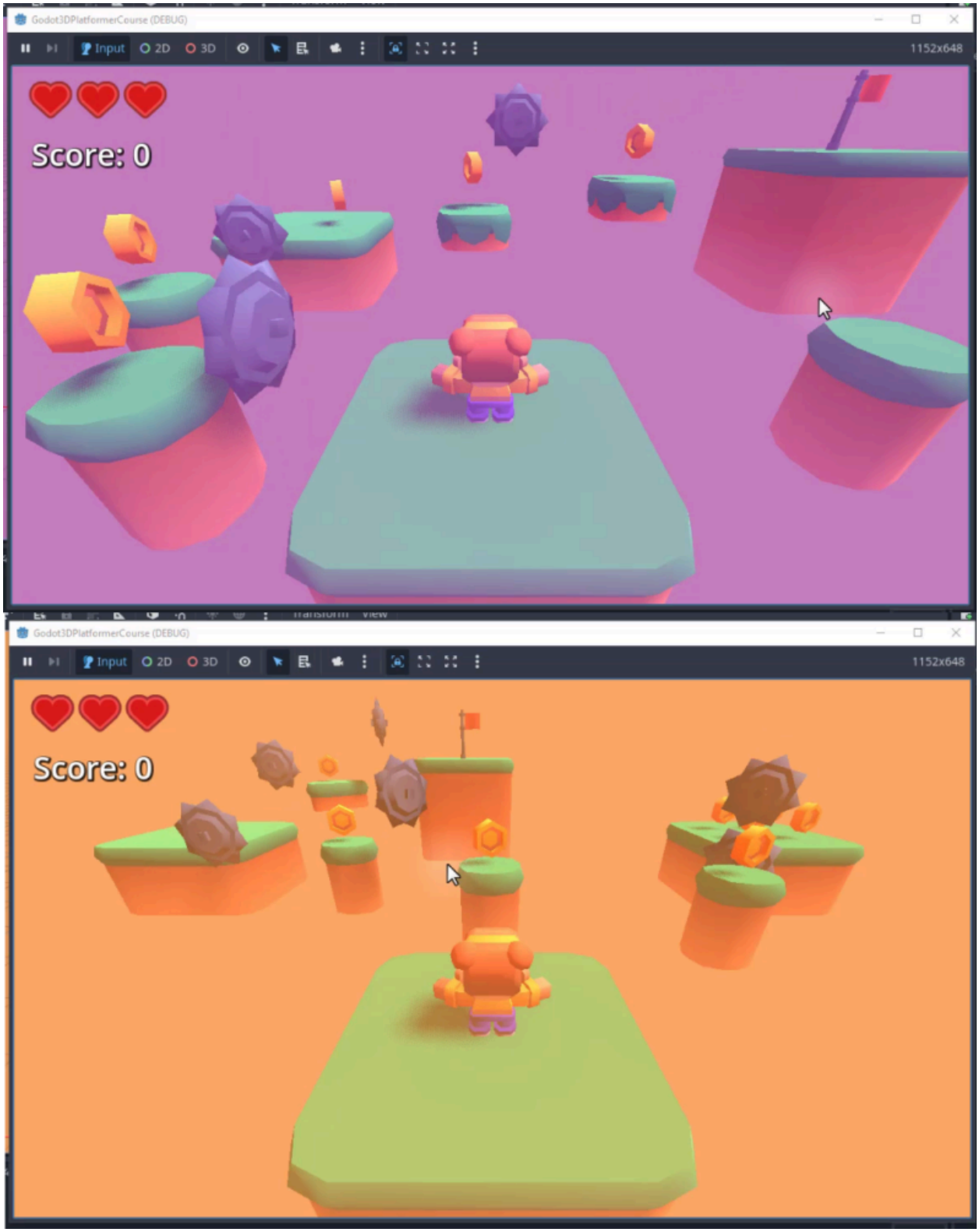
This also allows you to create scenarios where a harder path offers greater rewards (like more coins) or a shortcut, while an easier path offers fewer rewards.



Theming and Level Identity

Theming plays a significant role in level design. It contributes to the narrative and creates a unique identity for each stage. Even small changes in color palettes or environment settings (like fog color) can make levels feel distinct. They can also align with the narrative and aesthetic of your game.

For example, a sand-themed level could feature pyramids and warm lighting, while a fantasy level might feature floating islands and purple fog.


Godot3DPlatformerCourse (DEBUG)
Input
2D
3D
1152x648
Score: 0
Godot3DPlatformerCourse (DEBUG)
Input
2D
3D
1152x648
Score: 0

Conclusion

Level design is a vast field, but applying these basic techniques (telegraphing objectives, guiding attention, using elevation, offering choices, and applying themes) will help you create engaging and intuitive levels. The best way to improve is to create, test, gather feedback, and iterate on your designs.

This concludes the core development of our game project. In the next chapter, we will wrap up the project.

Closing Words

Congratulations on reaching the end of this book!

Finishing a technical book is an achievement in itself, but you have done more than just read. You have built a complete, playable 3D game from a blank project file. You have written code, debugged errors, designed levels, and polished the user experience. That takes real effort, and you should be proud of the result.

Many people dream of making games, but far fewer take the steps to actually build one. By completing this project, you have crossed the line from “aspiring developer” to “game developer.”

What You Accomplished

It is easy to lose sight of how much you have learned when you are focused on fixing the next bug or tweaking a specific game element. Let’s take a moment to look at the toolkit you have built. You did not just follow instructions; you mastered the fundamental systems that power Godot.

You mastered the Node and Scene system. You learned that everything in Godot is a node, and that complex behaviors arise from combining simple nodes into scenes. You understand how to compose these scenes to create modular, reusable game objects like enemies and collectibles.

You bridged the gap between player and game. Through the Input Map and GDScript, you learned how to translate physical button presses into digital movement. You handled character controllers, applied physics forces, and managed state to keep the gameplay feeling responsive.

You orchestrated game logic with Signals. You moved beyond writing code in a single script. By using signals, you decoupled your game systems. Your UI updates when the player takes damage, and your game over screen triggers when health reaches zero, all without these systems needing to know the internal details of one another.

You polished the experience. A game is more than just mechanics. You learned to use Control nodes to build responsive interfaces,

AudioStreamPlayers to provide feedback, and camera shakes to make interactions feel impactful.

Where to Go From Here

The end of this book is the beginning of your own journey. You now have a solid foundation, but game development is a vast field. Here are concrete steps you can take to continue growing.

Expand Your Project

The best way to solidify what you learned is to modify the game you just built. Try adding features that force you to stretch your skills:

- **Add a new enemy type:** Create an enemy that flies or shoots back. This will test your understanding of AI logic.
- **Create a high-score system:** Learn about Godot's `FileAccess` class to save and load data so the player's best score persists between sessions.
- **Add a "Boss" level:** Design a larger, more complex level that requires the player to use all their mechanics to succeed.

Explore the Documentation

We covered a lot of ground, but Godot has hundreds of nodes we did not touch. Make a habit of reading the documentation. If you are curious about how to make a 2D game, look up `CharacterBody2D`. If you want to build multiplayer games, read about the High-Level Multiplayer API.

See the Godot Documentation (https://docs.godotengine.org/en/stable/).

Join the Community

Godot is an open-source engine with a vibrant, helpful community (https://godotengine.org/community/). You do not have to learn in isolation.

- **Godot Discord:** A great place for quick questions and showing off progress.
- **Godot Forum:** Excellent for long-form discussions and finding solutions to complex problems.

- **Game Jams:** Consider joining a game jam (like the Godot Wild Jam). These are events where you make a game in a short time frame, usually a weekend. They are fantastic for learning how to scope a project and finish it. You can find tons of game jams listed on itch.io (https://itch.io/jams)

Continue Your Learning

If you prefer hands-on, project-based training in Godot and programming, explore Zenva Academy (https://academy.zenva.com). Our library offers more than 300 courses and over 40 Learning Pathways to take you from core fundamentals through to advanced topics.

Final Thoughts

Game development is a practice of constant problem-solving. There will be times when your code breaks, your physics glitch, or your design just isn't fun. This is normal. Every professional developer faces these challenges daily.

The difference now is that you have the tools to solve them. You know how to break a problem down into nodes, scripts, and signals.

Keep experimenting. Keep breaking things. Keep building. The skills you have gained here will serve you well no matter what you create next.

Good luck with your next project. We cannot wait to see what you build.

Full Source Code Reference

In this appendix, we provide the complete source code for the 3D Platformer project. This reference is intended to help you debug your own code, verify your implementation, or serve as a foundation for expanding the project with your own features.

If you encounter any issues while building the game, compare your scripts against these final versions to ensure your logic matches the completed project.

camera_shake.gd

File Path: `res://Scripts/camera_shake.gd`

This script extends the `Camera3D` class to implement a screen shake effect when the player takes damage. It listens for the `OnTakeDamage` signal from the parent node (the Player) and sets an intensity value. The `_process` function then applies a random offset to the camera's position based on this intensity, while simultaneously reducing the intensity over time using linear interpolation (`lerp`) to create a smooth fade-out effect.

```
extends Camera3D

var intensity : float = 0

func _ready ():
    # Listen for the player's damage signal to trigger a shake
    get_parent().OnTakeDamage.connect(_damage_shake)

func _damage_shake (_health : int):
    # Set the shake strength; higher values produce a more dramatic
effect
    intensity = 0.1

func _process (delta):
    if intensity > 0:
        # Smoothly decay the shake intensity each frame
        intensity = lerp(intensity, 0.0, delta * 10)
        # Apply a random viewport offset to simulate screen shake
        var offset : Vector2 = _get_random_offset()
```

```
        h_offset = offset.x
        v_offset = offset.y

# Returns a random 2D offset scaled by the current shake intensity
func _get_random_offset () -> Vector2:
    var x = randf_range(-intensity, intensity)
    var y = randf_range(-intensity, intensity)
    return Vector2(x, y)
```

coin.gd

File Path: `res://Scripts/coin.gd`

This script manages the behavior of the collectible coins. It handles the visual presentation by rotating the coin around its Y-axis and bobbing it up and down using a sine wave based on the system time. When a body belonging to the "Player" group enters the coin's area, the script increases the player's score and removes the coin from the scene.

```
extends Area3D

@export var rotate_speed : float = 180.0

@export var bob_height : float = 0.2
@export var bob_speed : float = 5.0
# Capture the starting Y so the bob oscillates above this baseline
@onready var start_y_pos : float = global_position.y

func _process (delta):
    # Spin the coin around the Y-axis for visual appeal
    rotation.y += deg_to_rad(rotate_speed) * delta

    # Use a sine wave mapped to 0..1 to bob above the starting position
    var time = Time.get_unix_time_from_system()
    var y_pos = (1 + sin(time * bob_speed)) / 2 * bob_height

    global_position.y = start_y_pos + y_pos

func _on_body_entered(body):
    # Only the player can collect coins
    if not body.is_in_group("Player"):
        return
```

```
    body.increase_score(1)
    queue_free()
```

end_flag.gd

File Path: `res://Scripts/end_flag.gd`

This script controls the level's end goal. It extends `Area3D` and detects when the player enters the flag's collision shape. Upon detection, it triggers a scene change to the `PackedScene` assigned to the `scene_to_load` variable. The scene change is called using `call_deferred` to ensure it happens safely after the current physics frame is processed.

```
extends Area3D

@export var scene_to_load : PackedScene

func _on_body_entered(body):
    # Only the player triggers level transitions
    if not body.is_in_group("Player"):
        return

    # Defer the scene change to avoid modifying the tree mid-physics
step
    call_deferred("_load_new_scene")

func _load_new_scene ():
    get_tree().change_scene_to_packed(scene_to_load)
```

enemy.gd

File Path: `res://Scripts/enemy.gd`

This script defines the behavior of the enemy saw blades. It moves the enemy back and forth between a starting position and a target position defined by `move_direction`. It also rotates the model continuously to simulate a spinning blade. If the player collides with the enemy, the player takes damage.

```
extends Area3D

@export var move_speed : float = 2.0
@export var move_direction : Vector3
@export var spin_speed : float = 900.0

@onready var start_pos : Vector3 = global_position
@onready var target_pos : Vector3 = start_pos + move_direction
@onready var model = $Model

func _process (delta):
    # Spin the saw blade around its Z-axis for a dangerous look
    model.rotation.z += deg_to_rad(spin_speed) * delta

    # Move toward the current target at a fixed speed
    global_position = global_position.move_toward(target_pos,
move_speed * delta)

    # Reverse direction when the enemy reaches either end of its patrol
path
    if global_position == start_pos:
        target_pos = start_pos + move_direction
    elif global_position == start_pos + move_direction:
        target_pos = start_pos

func _on_body_entered (body):
    # Only the player takes damage from enemies
    if not body.is_in_group("Player"):
        return

    body.take_damage(1)
```

menu.gd

File Path: `res://Scripts/menu.gd`

This script handles the user interface interactions for the main menu. It connects the “Play” button to a function that resets the score and loads the first level, and the “Quit” button to a function that closes the application.

```
extends Control

func _on_play_button_pressed():
```

```
    # Reset the score before starting a new game
    PlayerStats.score = 0
    get_tree().change_scene_to_file("res://Scenes/level_1.tscn")

func _on_quit_button_pressed():
    get_tree().quit()
```

player_controller.gd

File Path: `res://Scripts/player_controller.gd`

This is the core script for the player character. It extends `CharacterBody3D` and handles physics-based movement, including gravity, jumping, and directional input. It also manages the player's health and score, emitting signals (`OnTakeDamage`, `OnUpdateScore`) to update the UI. Additionally, it handles audio playback for interactions and manages the "Game Over" state if the player's health reaches zero or they fall off the map.

```
extends CharacterBody3D

# Custom signals to notify the UI without creating tight coupling
signal OnTakeDamage (hp : int)
signal OnUpdateScore (score : int)

@export var health : int = 3

@export var move_speed : float = 3.0
@export var jump_force : float = 8.0
@export var gravity : float = 20.0

@onready var camera : Camera3D = $Camera3D
@onready var audio : AudioStreamPlayer = $AudioStreamPlayer

# Preload audio so there is no loading delay during gameplay
var coin_sfx : AudioStream = preload("res://Audio/coin.wav")
var take_damage_sfx : AudioStream =
preload("res://Audio/take_damage.wav")

func _physics_process(delta):
    # Apply downward acceleration each physics frame
    velocity.y -= gravity * delta

    # Allow jumping only when standing on a surface
```

```
    if Input.is_action_pressed("jump") and is_on_floor():
        velocity.y = jump_force

    # Negate the vector because Godot's forward is -Z in 3D
    var move_input : Vector2 = -Input.get_vector("move_right",
"move_left", "move_back", "move_forward")
    var move_dir : Vector3 = Vector3(move_input.x, 0, move_input.y)

    velocity.x = move_dir.x * move_speed
    velocity.z = move_dir.z * move_speed

    move_and_slide()

func _process (_delta):
    # Treat falling off the map as an instant game over
    if global_position.y < -5:
        _game_over()

func take_damage (amount : int):
    health -= amount
    OnTakeDamage.emit(health)
    _play_sound(take_damage_sfx)

    if health <= 0:
        # Defer to avoid changing the scene mid-physics callback
        call_deferred("_game_over")

func _game_over ():
    PlayerStats.score = 0
    get_tree().change_scene_to_file("res://Scenes/menu.tscn")

func increase_score (amount : int):
    PlayerStats.score += amount
    OnUpdateScore.emit(PlayerStats.score)
    _play_sound(coin_sfx)

# Helper to swap and play a sound on the shared AudioStreamPlayer
func _play_sound (sound : AudioStream):
    audio.stream = sound
    audio.play()
```

player_stats.gd

File Path: `res://Scripts/player_stats.gd`

This script acts as a singleton (Autoload) to store persistent data. It contains a `score` variable that is maintained across scene changes, allowing the player's score to carry over from level to level.

```
extends Node

var score : int = 0
```

player_ui.gd

File Path: `res://Scripts/player_ui.gd`

This script manages the Heads-Up Display (HUD). It connects to the player's signals to update the visual representation of health (hearts) and the score text in real-time. It initializes the UI state in `_ready` and updates visibility and text whenever the corresponding signals are emitted.

```
extends CanvasLayer

@onready var health_container = $HealthContainer
var hearts : Array = []

@onready var score_text : Label = $ScoreText

func _ready ():
    # Cache all heart icons so we can toggle their visibility later
    hearts = health_container.get_children()

    var player = get_parent()

    # Subscribe to the player's signals for event-driven UI updates
    player.OnTakeDamage.connect(_update_hearts)
    player.OnUpdateScore.connect(_update_score_text)

    # Set the initial UI state to match the player's starting values
    _update_hearts(player.health)
    _update_score_text(PlayerStats.score)

# Show or hide each heart based on remaining health
func _update_hearts (health : int):
    for i in len(hearts):
```

```
        hearts[i].visible = i < health

func _update_score_text (score : int):
    score_text.text = "Score: " + str(score)
```

player_visual.gd

File Path: `res://Scripts/player_visual.gd`

This script handles the procedural animation of the player model. It rotates the mesh to face the direction of movement using `lerp_angle` for smooth transitions. It also applies a bobbing effect to the model's Y-scale when the player is moving on the floor, adding a sense of weight and motion to the character.

```
extends MeshInstance3D

@export var rotate_rate : float = 20.0
var target_y_rot : float = 0

@onready var player : CharacterBody3D = get_parent()

func _process (delta):
    _smooth_rotation(delta)
    _move_bob(delta)

# Gradually rotate the model to face the movement direction
func _smooth_rotation (delta):
    # Negate velocity to match the model's forward orientation
    var vel = -player.velocity

    if vel.x != 0 or vel.z != 0:
        target_y_rot = atan2(vel.x, vel.z)

    # lerp_angle handles the wraparound at 360 degrees smoothly
    rotation.y = lerp_angle(rotation.y, target_y_rot, rotate_rate *
delta)

# Subtly scale Y with a sine wave to simulate a walking bob
func _move_bob (_delta):
    var move_speed = player.velocity.length()

    # Only bob when the player is moving on the ground
```

```
if move_speed < 0.1 or not player.is_on_floor():
    scale.y = 1
    return

var time = Time.get_unix_time_from_system()
# Oscillate between 0.92 and 1.08 for a subtle squash-stretch
var y_scale = 1 + (sin(time * 30) * 0.08)
scale.y = y_scale
```